

RISC-V Mini Shell과 바이너리 실행: 프로세서 설계에서 운영체제까지

RISC-V 프로세서 설계와 Mini OS · 셸 모드 및 바이너리 실행

프로젝트	RISC-V/ (rtl, firmware, scripts, tb, constraints, docs)
대상 보드	AMD ZCU104 Evaluation Board · RISC-V soft core
Toolchain	RISC-V GNU Toolchain · RV32I / ILP32 · Vivado
생성일	2026-10-04
원본	docs/SHELL_BINARY_GUIDE.md (scripts/build_manual_pdf.py로 생성)

ZCU104의 교육용 단일 사이클 RISC-V 코어에서 직접 만든 프로그램을 파일로 저장하고 실행하는 과정을 다루는 교재입니다.

기준일: 2026-10-01. 저장소의 RTL, C/어셈블리, linker script, 빌드 스크립트와 실제 생성된 ELF를 근거로 설명합니다.

대상: 디지털 논리와 C의 기초를 배운 컴퓨터구조·운영체제 수강생. 각 장은 원리, 현재 구현, 관찰 방법을 연결합니다.

심화 확장판: CSR·MiniFS의 구조에 더해 GNU toolchain의 실제 출력, ELF·BIN·HEX의 대응 관계, 컴파일·링크 옵션과 현재 CPU·앱 ABI의 연결을 설명합니다. 표준 기능과 현재 구현의 차이를 구분합니다.

목차

1. 학습 목표와 전체 실행 경로	7
1.1 이 실습에서 실제로 일어나는 일	7
1.2 학습 후 설명할 수 있어야 하는 것	7
1.3 여기서 말하는 앱과 프로세스	8
1.4 원본 코드와 줄별 해설을 함께 읽는 방법	8
2. 최상위 회로와 SHELL_MODE의 설정	10
2.1 zcu104_top은 누가 호출하는가	10
2.2 동일한 이름을 사용하는 두 설정	10
2.3 make vivado-zcu104-shell의 전개	11
2.4 하드웨어 설정에 따른 메모리 선택	12
2.5 top에서 신호를 연결하는 실습	12
3. 바이너리 실행을 가능하게 하는 CPU와 메모리	15
3.1 단일 사이클 코어의 실행 경로	15
3.2 instruction port와 data port가 같은 RAM을 보는 이유	15
3.3 byte 주소와 word 인덱스	16
3.4 MMIO는 주소 디코딩이다	16
3.5 instruction cache가 생기면 추가되는 책임	18
3.6 현재 core의 architectural state와 내부 신호	18
3.7 RV32I와 46개 명령의 정확한 범위	18
3.8 register file과 ABI 이름	19
3.9 32비트 명령어의 공통 필드	20
3.10 R·I·S·B·U·J 형식	20
3.11 immediate 재구성과 부호 확장	20
3.12 ALU의 연산과 signed 해석	21
3.13 산술·논리 opcode의 해독	21
3.14 LOAD: 주소 계산에서 write-back까지	22
3.15 STORE: byte strobe와 데이터 복제	23
3.16 BRANCH: 조건과 target의 분리	25
3.17 JAL과 JALR: 함수 호출을 만드는 두 효과	25
3.18 LUI와 AUIPC: 큰 주소를 만드는 방법	26
3.19 CSR 명령 6종과 pseudo-instruction	26
3.20 구현된 CSR와 빠진 기능	27
3.21 decoder의 기본값과 write-back 선택	27
3.22 trap 우선순위와 정확한 부작용 억제	28
3.23 세 명령을 사이클별로 따라가기	29
3.24 timing, 합성 resource, 소프트웨어 비용	29
3.25 BNE 기계어를 비트 단위로 검사하기	30
3.26 load와 trap을 함께 보는 타이밍 표	30
3.27 CSR 이름과 세 가지 주소를 읽는 방법	31
3.28 mstatus와 MIE·MPIE의 정확한 의미	31
3.29 mie와 interrupt 종류별 허용 비트	33
3.30 mip와 pending 상태 및 timer의 연결	34
3.31 mtvec와 trap 진입 주소	35
3.32 mepc와 PC·ra·trap frame의 차이	36
3.33 mcause와 예외 번호·시스템 콜 번호	36

3.34 mscratch와 trap 진입의 임시 저장	37
3.35 mvendorid·marchid·mimpid·mhartid의 의미	38
3.36 이름은 알아 두되 현재 사용할 수 없는 CSR	38
3.37 CSR instruction이 RTL을 통과하는 순서	39
3.38 부팅·ECALL·timer·MRET를 값으로 연결하기	40
3.39 C inline assembly와 interrupt 상태 보존	40
3.40 CSR 파형과 증상에서 원인을 찾는 실습	41
4. 주소 공간과 세 가지 스택 개념	43
4.1 셀 이미지의 메모리 지도	43
4.2 부팅 스택과 셀 task 스택은 다르다	43
4.3 앱이 사용하는 스택	44
4.4 실행 이미지와 저장 파일의 수명 비교	44
4.5 static 배열이 실제 FPGA 스택이 되는 과정	45
4.6 sp와 저장 명령을 주소로 따라가기	46
4.7 slot에서 1을 빼는 이유	47
4.8 STACK_WORDS가 256인 이유와 크기 변경의 조건	47
5. 리셋에서 셀 프롬프트까지	49
5.1 구성 시 초기화와 CPU reset의 차이	49
5.2 boot.S의 초기화 순서	49
5.3 가짜 초기 trap frame으로 셀 시작하기	51
5.4 timer를 켜 뒤 나타나는 두 메시지	52
5.5 최초 task 시작의 PC·SP 변화표	52
5.6 entry와 task_exit의 주소를 프레임에 넣는 이유	53
5.7 태스크 함수가 return하면 task_exit으로 가는 과정	54
5.8 무한 루프인 task_exit의 목적과 한계	55
6. UART에서 명령행까지	56
6.1 serial bit, byte, command line의 세 단계	56
6.2 uart_rx와 uart_tx는 어디에 있는가	56
6.3 FIFO와 polling	57
6.4 한 줄을 만드는 코드	58
6.5 built-in command와 외부 앱	61
6.6 문자 C의 serial frame을 손으로 그리기	63
7. ECALL과 운영체제 서비스의 경계	64
7.1 시스템 콜 번호는 OS가 정한다	64
7.2 한 문자를 출력하는 앱 코드	64
7.3 하드웨어 trap 진입	65
7.4 왜 f->mepc += 4인가	65
7.5 파일 읽기 syscall의 인수·결과 추적	67
8. Trap frame과 셀 모드의 스케줄링	69
8.1 frame의 정확한 메모리 형식	69
8.2 저장·C 호출·복원	69
8.3 일반 OS와 셀 모드의 scheduler 차이	71
8.4 frame에 모든 CSR이 들어 있지는 않다	73
8.5 stack과 두 종류의 frame을 구분하기	73
8.6 trap 전후의 스택과 프레임 수명	74
8.7 panic에 들어가면 trap_handler가 끝나지 않는 이유	75
8.8 task_exit과 panic의 무한 루프가 다른 이유	75

8.9 trap_entry와 schedule은 누가 실행시키는가	76
8.10 C 함수의 반환 포인터가 어셈블리 복원으로 연결되는 과정	77
8.11 C handler라는 용어의 의미	78
8.12 스택 사용 용도별 실제 코드와 어어셈블 예	79
8.13 하나의 스택에서 사용량을 합산하는 예	81
9. MiniFS: 실행 파일을 보관하는 구조	82
9.1 RAM disk의 배치	82
9.2 file table entry	86
9.3 할당·읽기·삭제	91
9.4 60바이트 앱이 ls에서 72바이트인 이유	95
9.5 두 파일을 저장하고 삭제하는 전체 예	96
9.6 fs_u8과 세 종류의 디스크 포인터	96
9.7 superblock의 의미와 초기화 판단	97
9.8 변수와 함수 앞의 static 및 private과의 비교	98
9.9 fs_create는 이름을 가진 빈 파일을 만든다	99
9.10 emit은 함수 이름이 아니라 콜백 매개변수이다	100
10. C 프로그램에서 실행 파일까지	102
10.1 세 종류의 이미지 구분	102
10.2 예제 C 프로그램	102
10.3 make hello-app의 빌드 단계	104
10.4 compiler 옵션의 의미	113
10.5 ELF에서 raw binary로 바뀌면서 사라지는 것	119
11. 링커와 앱 ABI의 계약	120
11.1 ENTRY와 실제 첫 바이트는 별개다	120
11.2 왜 다른 주소에 복사하면 안 되는가	121
11.3 앱 시작 때 이미 준비된 것	122
11.4 .data와 .bss	122
11.5 앱 간 연결과 외부 symbol	123
11.6 하위 함수를 호출하는 앱의 stack 예제	123
11.7 커널 링커 스크립트 전체 읽기	124
11.8 아키텍처와 진입점 및 배치 문법	125
11.9 text와 data 및 bss의 줄별 해설	126
11.10 스택 심볼과 링크 검사 및 제거 section	127
12. UART 업로드 프로토콜과 APP1 형식	129
12.1 host와 board의 대화	129
12.2 왜 raw byte 대신 ASCII hex를 보내는가	130
12.3 두 가지 hex 표현은 바이트 순서가 다르다	130
12.4 셸의 업로드 임시 버퍼	131
12.5 APP1 header의 바이트 단위 해석	133
12.6 업로드 실패와 현재 프로토콜의 범위	133
13. run 명령의 검증·복사·호출	135
13.1 실제 동작 순서	135
13.2 파일 offset과 목적지 주소	136
13.3 왜 마지막 byte를 한 번 더 읽는가	136
13.4 함수 포인터 호출의 의미	137
13.5 반환 경로가 살아 있어야 하는 이유	138
13.6 잘못된 앱 파일에 대한 검증 경로	139

14. 60바이트 앱의 실제 명령어 추적	140
14.1 명령어와 문자열의 분리	140
14.2 첫 문자 H의 전체 이동	140
14.3 마지막 문자와 복귀	141
14.4 역어셈블에서 보이는 가짜 명령어	141
14.5 한 명령어를 손으로 해석하기	141
15. 실행 중 인터럽트와 세 종류의 복귀	143
15.1 앱 실행 중 timer가 발생하면	143
15.2 ECALL, 함수 호출, context switch 비교	143
15.3 timer 재설정과 interrupt masking	144
15.4 장시간 동작의 timer 제한	145
15.5 64비트 비교와 32비트 재설정의 불일치 예	145
16. ZCU104 실습 절차	147
16.1 호스트 명령과 rv> 명령 구분	147
16.2 처음부터 준비하는 순서	147
16.3 UART에 재접속하여 실행	148
16.4 이미 셸이 실행 중일 때 앱만 바꾸기	148
16.5 screen의 종료·분리·중지	149
16.6 증상에서 계층을 찾아가기	149
17. 검증 방법과 관측 가능한 증거	151
17.1 host artifact 검사	151
17.2 RTL 시뮬레이션이 확인하는 것	151
17.3 실제 보드 검증	152
17.4 현재 자동 검사의 범위	153
17.5 파형을 추가해 관찰할 신호	153
17.6 invariant를 이용한 검증 설계	154
18. 현재 구현의 한계와 확장 설계	155
18.1 M-mode 앱의 권한	155
18.2 앱 종료와 오류 처리	155
18.3 메모리 시스템의 제약	155
18.4 운영체제 기능별 다음 단계	156
18.5 파일 시스템과 프로토콜의 후속 과제	156
19. 수업 구성과 연습문제	158
19.1 4회 실습 수업 구성 예	158
19.2 기본 이해 문제	158
19.3 계산·코드 해석 문제	159
19.4 실험 문제	159
19.5 설계 문제	159
19.6 ISA와 데이터패스 심화 문제	160
19.7 CSR 이름과 trap 상태 심화 문제	160
19.8 스택과 종료 경로 심화 문제	161
19.9 호출 경계와 스택 사용량 확인 문제	161
20. 해설·용어·코드 찾아보기	163
20.1 기본 문제 해설	163
20.2 계산 문제 해설	163
20.3 설계 문제의 검토 기준	164

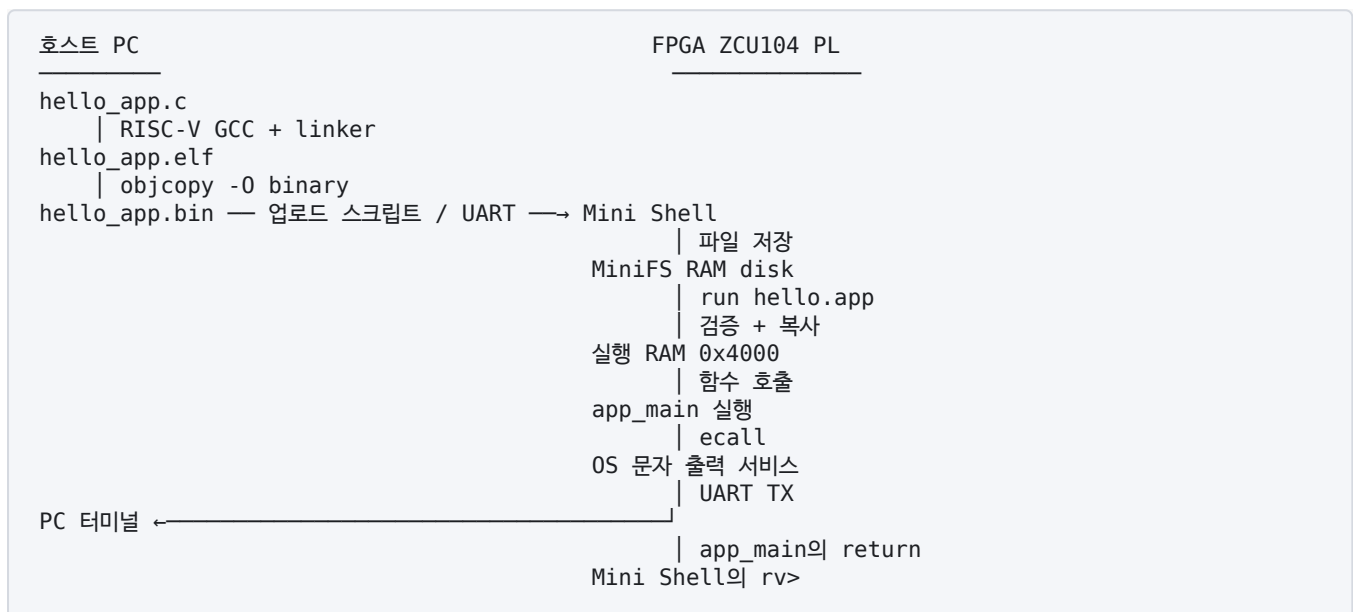
20.4 용어 정리	164
20.5 코드 찾아보기	165
20.6 참고 자료와 문서의 범위	166
20.7 ISA 심화 문제 해설	167
20.8 CSR 심화 문제 해설	167
20.9 스택과 종료 경로 심화 문제 해설	168
20.10 호출 경계와 스택 사용량 확인 문제 해설	168

1. 학습 목표와 전체 실행 경로

1.1 이 실습에서 실제로 일어나는 일

PC에서 `hello_app.c` 를 RISC-V 기계어로 컴파일하고, USB UART를 통해 FPGA에 전송합니다. FPGA에서 실행 중인 Mini Shell은 받은 바이트를 MiniFS 파일로 저장합니다. 사용자가 `run hello.app` 을 입력하면 셸은 파일을 읽고 검사한 뒤, 명령어를 가져올 수 있는 RAM으로 복사하고 그 시작 주소를 호출합니다. 프로그램은 운영체제의 문자 출력 서비스를 사용하고 셸로 돌아옵니다.

이 과정에서 프로세서가 수행하는 기본 동작은 여전히 명령어 인출, 디코딩, 레지스터 읽기, 연산, 메모리 접근, 다음 PC 선택입니다. 파일 이름이나 `run` 이라는 명령의 의미는 C 프로그램이 해석합니다. CPU는 `hello.app` 이라는 이름을 알지 못합니다. CPU가 직접 보는 것은 PC에 해당하는 32비트 명령어와 레지스터·메모리 값입니다.



위 실행 경로를 시간 순서로 나누면 서로 다른 두 준비 과정이 보입니다. 먼저 FPGA에 CPU 회로와 셸 펌웨어를 구성합니다. 그 후 CPU가 계속 실행되는 동안 앱 바이트를 전송합니다. 두 번째 작업은 기존 회로의 RAM 내용을 바꾸는 것이므로 새 앱을 시험할 때마다 논리 합성·배선을 반복할 필요가 없습니다. 하드웨어 플랫폼을 고정한 상태에서 소프트웨어를 바꾸는 경험이 이 실습의 출발점입니다.

예를 들어 문자열을 `Hello` 에서 `Welcome` 으로 바꾸는 일은 앱 변경입니다. 앱 창을 `0x4000` 에서 다른 주소로 옮기는 일은 loader·linker·메모리 배치의 공동 변경입니다. 곱셈 명령 `MUL` 을 직접 실행하고 싶다면 core의 ISA 확장 또는 소프트웨어 지원이 필요합니다. 변화의 종류를 먼저 분류하면 어떤 산출물을 다시 만들어야 하는지 판단할 수 있습니다.

1.2 학습 후 설명할 수 있어야 하는 것

학습자는 `SHELL_MODE` 가 언제 확정되는지, 프로그램의 파일 위치와 실행 주소가 왜 다른지, `ecall` 의 복귀 주소와 함수의 복귀 주소가 어떻게 구분되는지를 설명할 수 있어야 합니다. 또한 빈 UART 화면, 업로드 후 파일이 사라진 현상, 실행 파일 검증 실패를 계층별로 진단할 수 있어야 합니다.

전체 과정을 이해할 때 다음 네 가지 계약을 함께 보아야 합니다.

계약	약속하는 내용	현재 프로젝트의 예
ISA	명령어 인코딩과 실행 의미	LBU, JALR, ECALL, MRET
하드웨어 플랫폼	주소별 RAM·장치 동작	UART RX 데이터는 0x100000008
소프트웨어 ABI	레지스터, 스택, 서비스 호출 규칙	a7=1, a0=문자, ecall
실행 파일 형식	저장 바이트의 해석과 로딩 규칙	APP1 헤더, 고정 진입 주소 0x4000

ISA가 같다고 아무 운영체제용 실행 파일이나 동작하는 것은 아닙니다. 동일한 RISC-V 명령어로 만들어졌더라도 링크 주소, syscall 번호, 메모리 크기, 초기화 조건이 이 환경과 맞아야 합니다.

실습 기록에는 네 계약 중 어느 계약을 확인했는지 표시하면 좋습니다. objdump에서 JALR를 찾는 것은 ISA와 제어 흐름을 보는 것이고, ls의 72바이트는 파일 형식을 보는 것입니다. PC가 0x4000에 도달한 파형은 주소 배치와 loader를 함께 확인합니다. 각 관측을 연결해야 단순히 문자열이 우연히 출력된 경우와 의도한 경로로 실행된 경우를 구분할 수 있습니다.

권장 학습 순서는 먼저 주소 지도에 메모리와 장치를 표시하고, 명령 하나가 그 지도에 접근하는 과정을 익힌 뒤, 여러 명령으로 구성된 syscall·파일 복사·앱 실행을 추적하는 것입니다. 하드웨어를 처음 공부하는 독자는 3장의 ISA와 데이터패스를 충분히 읽고, C에 익숙한 독자는 같은 연산을 생성된 assembly와 대조하십시오.

1.3 여기서 말하는 앱과 프로세스

현재 앱은 Mini Shell이 호출하는 독립적으로 컴파일된 함수입니다. 앱에는 별도 PID, 주소 공간, 페이지 테이블, 사용자 모드, 전용 task slot이 없습니다. 셸과 앱은 같은 M-mode에서 같은 셸 task의 스택을 사용합니다.

따라서 이 프로젝트는 파일에서 코드를 읽어 실행하는 로더의 핵심을 학습하기에 적합합니다. 동시에 일반적인 OS의 프로세스 생성과 격리에 무엇이 더 필요한지도 비교할 수 있습니다. 본문에서 “앱 실행”은 이 구체적인 실행 모델을 뜻합니다.

함수 호출로 실행하더라도 앱은 별도 파일에서 가져왔으므로 작성·컴파일·배포 단위는 독립적입니다. 반면 실행 중 CPU 문맥의 소유자는 셸 task입니다. 이 둘을 각각 “프로그램 이미지”와 “실행 문맥”으로 나누어 생각하면, 파일을 여러 개 저장하는 기능과 여러 프로그램을 동시에 스케줄링하는 기능이 왜 다른지 이해할 수 있습니다.

현재 hello.app과 다른 앱 파일을 차례로 실행할 수는 있지만 두 앱을 같은 0x4000 실행 창에서 동시에 유지할 수는 없습니다. 다음 앱을 로딩하면 이전 이미지가 덮어써집니다. 정상 반환 후 이전 앱의 실행 상태를 다시 이어가는 suspend/resume 모델도 없습니다. 앱의 수명은 run의 호출부터 함수 반환까지입니다.

1.4 원본 코드와 줄별 해설을 함께 읽는 방법

이번 코드 해설판은 관련 개념 바로 뒤에 코드 해부 35개를 배치합니다. 발췌는 2026-10-02의 이 저장소 소스에서 가져왔으며, 새로운 구현이나 의사 코드로 바꾼 것이 아닙니다. 긴 주석과 빈줄은 생략하고 원본의 식·함수 이름·제어 순서는 유지했습니다. 여러 구간을 이어 보일 때는 생략 범위를 명시합니다.

코드 왼쪽 숫자는 원본 파일의 행 번호입니다. |는 번호와 코드를 구분하는 인쇄용 표시이며 소스 문법이 아닙니다. compiler에 그대로 붙여 넣지 말고 링크한 실제 파일을 사용하십시오. PDF에서 긴 원본 행이 두 줄로 접혀도 번호는 같은 논리적 코드 행을 가리킵니다. 각 표는 표시한 행마다 무엇을 읽고, 계산하고, 저장하며, 어느 조건에서 다음 경로를 선택하는지 설명합니다.

학습 경로	코드 해부	확인할 질문
회로 연결·주소 선택	2-A, 3-F	wire와 주소가 RAM·UART를 어떻게 선택하는가?
명령 실행·trap	3-A - 3-E	계산 중인 값과 edge에 반영되는 상태의 차이는?
부팅·최초 task	5-A - 5-B	SP·bss·mtvec·인공 frame을 누가 준비하는가?
UART·명령행	6-A - 6-E	byte가 문자열과 파일 명령으로 어떻게 바뀌는가?
syscall·context	7-A - 7-B, 8-A - 8-C	현재 register와 frame의 저장 값은 어떻게 다른가?
MiniFS	9-A - 9-F	entry·block·byte offset이 어떻게 주소가 되는가?
빌드·링커	10-A - 10-C, 11-A	source·ELF·BIN·HEX와 실행 주소의 관계는?
전송·실행	12-A - 12-B, 13-A - 13-B	언제 저장이 끝나고 PC가 앱으로 이동하는가?
timer·검증	15-A, 17-A	IRQ 해제 조건과 PASS의 검증 범위는?

읽는 순서는 **입력값 확인** → **각 행의 값 변화** → **최종 상태** → **실제 관찰 방법**입니다. 예를 들어 fs_read_at의 offset은 파일 내부 byte 거리이고, 계산한 off는 disk 내부 byte 거리이며, dst는 CPU 주소입니다. 비슷한 변수 이름이라도 기준점을 계속 적어 보십시오.

행 번호는 이 판의 관측 기준입니다. 이후 소스가 바뀌면 함수·모듈 이름으로 다시 찾고 새 행 번호와 비교해야 합니다. 코드 해설 추가는 RTL·firmware·프로토콜 변경이 아니며, 기존 보드 시험 기록과 이번 문서 검수의 범위도 구별합니다.

2. 최상위 회로와 SHELL_MODE의 설정

2.1 zcu104_top은 누가 호출하는가

Verilog의 module은 회로의 구조와 동작을 기술합니다. 다른 module 안에 인스턴스화하면 그 회로의 일부가 됩니다. 이 프로젝트의 최상위 module은 zcu104_top 이며, 합성 도구에 다음처럼 지정합니다.

```
synth_design -top zcu104_top -part xczu7ev-ffvc1156-2-e ...
```

Vivado는 이 모듈을 루트로 내부 연결을 해석하고 논리소자와 배선으로 구현합니다. **보드에서는 소프트웨어가 zcu104_top() 을 호출하는 것이 아닙니다. 구성된 회로가 클럭과 입력 신호에 따라 계속 동작합니다.** 시뮬레이션에서는 testbench가 설계 module을 인스턴스화하고 클럭·리셋·UART 입력을 만들어 줍니다.

현재 계층은 다음과 같습니다. 실제 합성 후에는 최적화로 계층 일부가 바뀔 수 있습니다.

```
zcu104_top
├─ 입력 클럭 버퍼 / 24분주 / CPU 클럭 버퍼
├─ reset 동기화 회로
├─ rv32_soc
│   ├── rv32_core
│   │   ├── rv32_regfile
│   │   ├── rv32_alu
│   │   └── rv32_csr
│   ├── unified instruction/data RAM
│   ├── MiniFS RAM disk
│   ├── UART MMIO와 RX FIFO
│   └── simple_timer
├─ uart_rx : serial_rx
├─ uart_tx : serial
└─ LED 관측 회로
```

보드의 300 MHz 입력을 24분주하여 CPU와 UART를 12.5 MHz로 구동합니다. UART RX/TX 모듈과 SoC가 같은 클럭을 사용하므로 이들 사이의 바이트 전달에는 별도 클럭 도메인 교차가 없습니다. 외부 UART 입력 자체는 비동기이므로 uart_rx 안에서 2단 레지스터로 동기화합니다. 현재 보드 구성은 PL 내부에서 동작하며 ARM PS 소프트웨어나 DDR 초기화를 사용하지 않습니다.

top의 입력·출력 포트 이름은 XDC의 get_ports 와 연결됩니다. 예를 들어 UART_RX 라는 논리 입력이 A20 pin에 배치되고, 내부 serial_rx 의 rx 로 이어집니다. 소스에 wire를 선언하는 것과 실제 package pin을 선택하는 것은 서로 다른 단계입니다. module 연결이 맞더라도 pin이나 I/O 전압 제약이 틀리면 보드에서는 정상 통신을 기대할 수 없습니다.

분주 회로는 0부터 11까지 세고 출력 클럭을 반전합니다. 반 주기에 입력 12클럭, 한 주기에 24클럭이 필요합니다. reset은 버튼으로 비동기 assert되고 4단 shift register를 통해 동기적으로 해제됩니다. 이 장의 회로 구조를 볼 때 clock buffer와 reset 동기화도 CPU가 안정적으로 명령어를 시작하기 위한 플랫폼의 일부로 보아야 합니다.

2.2 동일한 이름을 사용하는 두 설정

SHELL_MODE 는 C와 Verilog 양쪽에 등장하지만 설정 경로가 다릅니다.

위치	지정 방법	효력 시점	결과
C 전처리	GCC의 -DSHELL_MODE	펌웨어 컴파일	셀 함수, 앱 로더, 셀용 스케줄러 선택

위치	지정 방법	효력 시점	결과
Verilog parameter	Vivado의 <code>-generic SHELL_MODE=1</code>	elaboration·합성	32 KiB RAM과 <code>mini_shell.hex</code> 선택
빌드 환경 변수	<code>SHELL_BUILD=1</code>	Tcl 스크립트 실행	셀용 합성 옵션·출력 파일 선택

환경 변수 `SHELL_BUILD` 가 C 매크로나 Verilog parameter로 저절로 변환되는 것은 아닙니다. Makefile과 Tcl 코드가 두 빌드 경로를 연결합니다.

세 이름이 같거나 비슷하다고 하나의 전역 설정이라고 생각하면 빌드 오류를 놓치기 쉽습니다. C 매크로를 바꾸면 compiler가 포함하는 함수와 배열 크기가 달라집니다. Verilog parameter를 바꾸면 합성되는 메모리 용량과 초기화 이미지가 달라집니다. Tcl 환경 변수는 어느 명령행을 실행할지 고르는 host 측 정보이며, FPGA 안에 `SHELL_BUILD` 라는 실행 중 변수가 남는 것은 아닙니다.

학생은 Makefile에서 `-DSHELL_MODE`, Tcl에서 `-generic SHELL_MODE=1`, top에서 `MEM_WORDS` 를 각각 찾아 밑줄을 그어 보십시오. 세 지점이 하나의 build target으로 묶여 있다는 사실이 재현 가능한 시스템 빌드의 핵심입니다. 하나만 수정으로 바꾸고 나머지를 그대로 두면 firmware와 하드웨어의 계약이 어긋날 수 있습니다.

2.3 make vivado-zcu104-shell의 전개

관련 코드는 [Makefile](#)과 [build_zcu104.tcl](#)에 있습니다.

```
make vivado-zcu104-shell
├── make -B mini-shell
│   ├── GCC -DSHELL_MODE ... boot.S trap.S kernel.c minifs.c
│   ├── mini_shell.elf → mini_shell.bin
│   └── bin2hex.py → firmware/mini_shell.hex
└── SHELL_BUILD=1 vivado ... build_zcu104.tcl
    ├── read_verilog / read_xdc
    ├── synth_design ... -generic SHELL_MODE=1
    ├── 배치·배선·타이밍·DRC 검사
    └── build/zcu104_shell/zcu104_mini_shell.bit
```

`make -B`는 펌웨어를 강제로 재생성합니다. `.bit`에는 논리 구조뿐 아니라 합성에 사용한 메모리 초기값도 반영됩니다. 이후 C 파일을 수정해 `.hex`만 바꾸어도 이미 생성된 `.bit`의 내용은 바뀌지 않습니다.

Tcl의 분기 조건은 다음과 같습니다.

```
set shell [info exists ::env(SHELL_BUILD)]
```

이는 환경 변수의 존재 여부를 검사합니다. `SHELL_BUILD=0`으로 실행해도 변수가 존재하므로 셸 분기를 선택합니다. 현재 구현에서 일반 OS를 만들려면 이 변수가 없는 환경에서 일반 target을 사용해야 합니다. C 코드의 `#ifdef SHELL_MODE`도 정의 여부를 검사하므로 `-DSHELL_MODE=0`은 셸 코드를 포함합니다. Verilog의 조건식은 parameter의 숫자 값을 사용합니다.

합성은 RTL의 논리 기능을 FPGA resource로 바꾸고, 배치·배선은 그 resource의 위치와 연결을 정합니다. timing 분석은 register 사이의 경로가 12.5 MHz 클록의 제약을 만족하는지 평가합니다. DRC는 설정·배선 규칙을 검사합니다. 이들 검사를 통과한 것은 회로를 구성할 기반을 확인한 것이며, run의 파일 형식 검사나 C 코드의 의미까지 증명한 것은 아닙니다.

산출물의 갱신 관계를 추적하려면 source 수정 시각뿐 아니라 사용한 build target과 toolchain을 기록해야 합니다. 이 프로젝트의 .bit 에는 메모리 초기값이 들어 있으므로, 펌웨어 수정 후 기존 .bit 를 그대로 다운로드하는 것은 이전 프로그램을 다시 실행하는 결과가 됩니다. 반대로 앱 payload 변경은 이미 실행 중인 loader로 전송할 수 있습니다.

2.4 하드웨어 설정에 따른 메모리 선택

zcu104_top.v 에는 다음 선언과 연결이 있습니다.

```
parameter SHELL_MODE = 0,
parameter MEM_HEX = SHELL_MODE ? "firmware/mini_shell.hex"
                        : "firmware/mini_os.hex"

rv32_soc #(
    .MEM_WORDS(SHELL_MODE ? 8192 : 2048),
    .MEM_HEX(MEM_HEX)
) soc (...);
```

MEM_WORDS 의 단위는 32비트 word입니다. 따라서 셀 구성은 $8192 \times 4 = 32768$ 바이트이고 일반 OS 구성은 $2048 \times 4 = 8192$ 바이트입니다. 셀을 지원하기 위해 CPU에 새로운 명령어 디코더를 추가하는 것이 아니라, 기존 CPU를 더 큰 메모리와 다른 펌웨어로 구성합니다.

make program-zcu104-shell 의 SHELL_BUILD=1 은 이미 만들어진 셀용 .bit 를 선택하는 데 사용됩니다. 다운로드 시 parameter를 새로 변경하는 기능은 없습니다.

Verilog에서 parameter로 결정한 배열 크기는 실행 중 늘어나지 않습니다. 32 KiB 구성에서 앱이 더 많은 주소를 사용한다고 FPGA가 RAM을 추가로 할당하지 않습니다. CPU가 접근 가능한 주소를 decoder가 선택할 뿐입니다. 이후 4장의 linker 경계와 13장의 loader 크기 검사가 필요한 이유가 여기에 있습니다.

일반 OS 이미지와 셀 이미지의 차이는 메모리만이 아닙니다. C 쪽에서는 task 수, scheduler 분기, CRLF 출력, shell parser와 loader의 포함 여부도 달라집니다. 두 이미지는 같은 core를 사용하는 서로 다른 시스템 구성으로 이해해야 합니다. ELF의 symbol 목록을 비교하면 어느 C 함수가 실제로 포함됐는지도 확인할 수 있습니다.

2.5 top에서 신호를 연결하는 실습

module instance의 .포트이름(연결신호) 는 이름이 비슷한 변수를 선언하는 문법이 아니라 실제 회로 연결을 지정합니다. .uart_rx_data(rx_data) 에서 왼쪽은 rv32_soc가 정의한 입력 포트이고 오른쪽은 top 안의 wire입니다. 반대로 .data(rx_data) 는 uart_rx의 출력 포트를 같은 wire에 연결합니다. 두 인스턴스가 같은 이름의 wire를 통해 통신하는 것입니다.

top의 wire	보내는 쪽	받는 쪽	의미
cpu_clk	divider와 BUFG	SoC, RX, TX	공통 12.5 MHz clock
rst	reset synchronizer	SoC, RX, TX	내부 active-high reset
rx_data[7:0]	uart_rx.data	SoC.uart_rx_data	완성된 수신 byte
rx_valid	uart_rx.valid	SoC.uart_rx_valid	수신 byte 유효 pulse
data[7:0]	SoC.uart_tx_data	uart_tx.data	송신할 byte
valid	SoC.uart_tx_valid	uart_tx.valid	송신 요청 pulse
ready	uart_tx.ready	SoC.uart_tx_ready	송신기가 새 byte를 받을 수 있음

top의 wire	보내는 쪽	받는 쪽	의미
pc[31:0]	SoC.debug_pc	LED logic	내부 실행 위치의 일부 관측

RX의 valid와 TX의 valid는 역할이 다릅니다. RX valid는 외부에서 도착한 데이터 완료를 알리고, TX valid는 내부 software가 준비한 데이터를 보내 달라는 요청입니다. ready는 TX 쪽에만 있으며 현재 RX에는 외부 송신기를 멈추는 backpressure 선이 없습니다. 동일한 handshake 용어라도 방향과 수신 능력을 확인해야 합니다.

이 표를 보드 pin까지 확장하면 UART_RX → rx → rx_data → FIFO → MMIO와 MMIO → data/valid → tx → UART_TX의 두 경로를 그릴 수 있습니다. core가 pin의 serial bit를 직접 읽는 것이 아니라 SoC와 UART 회로가 단계적으로 표현을 바꾼다는 사실을 확인하는 실습입니다.

코드 해부 2-A. top에서 SoC와 UART를 실제로 연결하는 줄

출처: [rtl/zcu104_top.v](#), 원본 53 - 64행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

이 발췌는 parameter 선택과 UART 배선을 한 번에 보여 줍니다. 왼쪽 포트 이름과 오른쪽 wire를 구분하며 읽습니다.

```

53 | rv32_soc #(.MEM_WORDS(SHELL_MODE ? 8192 : 2048), .MEM_HEX(MEM_HEX)) soc (
54 |     .clk(cpu_clk), .rst(rst), .uart_tx_ready(ready),
55 |     .uart_tx_data(data), .uart_tx_valid(valid),
56 |     .uart_rx_data(rx_data), .uart_rx_valid(rx_valid), .debug_pc(pc)
57 | );
58 | uart_rx #(.CLOCK_HZ(12_500_000), .BAUD(115_200)) serial_rx (
59 |     .clk(cpu_clk), .rst(rst), .rx(UART_RX), .data(rx_data), .valid(rx_valid)
60 | );
61 | uart_tx #(.CLOCK_HZ(12_500_000), .BAUD(115_200)) serial (
62 |     .clk(cpu_clk), .rst(rst), .valid(valid), .data(data),
63 |     .ready(ready), .tx(UART_TX)
64 | );

```

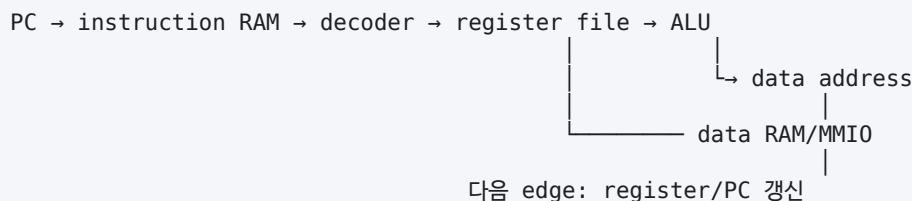
원본 행	한 줄씩 읽는 동작과 의미
53	8192 또는 2048개의 32비트 word를 가진 SoC 인스턴스 soc 를 만듭니다. SHELL_MODE는 합성 시 메모리 규모를 고르며 실행 중 바뀌는 C 변수가 아닙니다.
54	SoC에 CPU 클럭·리셋·TX ready를 연결합니다. ready 는 아래 TX 회로가 보내는 수신 가능 신호입니다.
55	SoC가 만들어 내는 송신 byte와 valid pulse를 data , valid wire로 내보냅니다.
56	RX 회로가 완성한 byte와 유효 신호를 SoC로 전달하고, 현재 PC는 관측용 wire로 받습니다.
57	앞에서 여러 줄에 걸쳐 작성한 연결·호출·표현식을 닫습니다.
58	별도의 UART 수신 회로를 인스턴스화합니다. 클럭 주파수와 baud는 수신기의 bit 시간 계산에 사용됩니다.
59	외부 직렬 핀은 rx 로, 완성된 8비트 값은 rx_data 로 연결됩니다. CPU가 읽는 단위는 이 byte입니다.
60	앞에서 여러 줄에 걸쳐 작성한 연결·호출·표현식을 닫습니다.
61	별도의 UART 송신 회로를 같은 클럭과 baud로 구성합니다.
62	SoC의 송신 요청·데이터를 TX에 연결합니다. 함수 호출이 아니라 두 회로 사이의 연결입니다.
63	TX의 ready를 SoC에 돌려주고 직렬 출력은 보드 포트에 보냅니다.
64	앞에서 여러 줄에 걸쳐 작성한 연결·호출·표현식을 닫습니다.

soc 와 UART 두 회로는 동시에 존재합니다. C가 UART MMIO에 쓰는 순간은 이 연결 위에서 byte 전달이 시작되는 한 사건이지, UART 모듈을 새로 호출하거나 생성하는 순간이 아닙니다.

3. 바이너리 실행을 가능하게 하는 CPU와 메모리

3.1 단일 사이클 코어의 실행 경로

현재 `rv32_core` 는 PC로 명령어를 읽고, 같은 사이클의 조합 논리에서 디코딩·레지스터 읽기·ALU 계산·데이터 읽기를 진행합니다. 다음 rising edge에서 PC와 필요한 레지스터·메모리 상태를 갱신합니다. **메모리가 같은 사이클 안에 값을 반환한다는 계약이 있기 때문에 가능한 구조입니다.**



UART 전송이나 파일 복사가 한 사이클에 끝난다는 뜻은 아닙니다. 각각 많은 명령어와 장치 대기로 이루어진 소프트웨어 작업입니다. 한 문자 출력도 ready 레지스터를 여러 번 읽는 반복문과 store 명령을 포함합니다.

한 사이클을 “계산 전 값이 register에 보관된 시점 → 조합 경로가 안정되는 구간 → 다음 edge에서 결과를 저장하는 시점”으로 나누면 이해하기 쉽습니다. 조합 논리에서 `next_pc` 가 여러 번 변할 수 있어도 최종적으로 edge에서 채택되는 값이 실행 결과입니다. 시뮬레이터의 delta cycle 동안 잠시 보이는 중간값과 실제 architectural commit을 구분해야 합니다.

현재 CPU에는 IF/ID, ID/EX 같은 pipeline register가 없습니다. 명령어 A의 계산 결과가 edge에서 저장된 뒤 명령어 B의 조합 계산이 시작됩니다. 그래서 forwarding이나 load-use stall 회로가 필요하지 않지만, 한 사이클의 길이는 가장 느린 명령의 조합 경로에 맞춰야 합니다. LW의 경우 instruction read부터 data read까지 같은 사이클에 들어갑니다.

3.2 instruction port와 data port가 같은 RAM을 보는 이유

`rv32_soc.v` 의 프로그램 메모리는 다음 배열입니다.

```
reg [31:0] mem [0:MEM_WORDS-1];
```

명령어 인출은 `mem[iaddr[15:2]]`, 데이터 접근은 `mem[daddr[15:2]]` 를 읽습니다. CPU 인터페이스에는 instruction/data 포트가 나뉘어 있지만 저장소는 같습니다. 이 구조를 Harvard 형태의 인터페이스를 가진 unified memory라고 설명할 수 있습니다.

앱 로더가 store 명령으로 `mem` 에 기계어 바이트를 쓴 뒤 PC를 해당 주소로 이동시키면, **instruction port가 그 바이트들을 명령어로 가져옵니다.** 파일이 실행으로 바뀌는 하드웨어상의 핵심이 이 연결입니다.

RAM disk 배열은 별도입니다. `0x80100000` 영역은 데이터 포트에서만 읽고 쓸 수 있으며, 현재 instruction fetch 경로는 이 배열을 선택하지 않습니다. 따라서 파일이 RAM disk에 있다고 그 주소로 바로 분기할 수는 없습니다. 실행 가능한 unified RAM으로 복사해야 합니다.

instruction 포트와 data 포트를 분리한 이유는 load/store를 실행하면서도 현재 명령어를 동시에 공급해야 하기 때문입니다. 하나의 storage를 두 조합 read 경로가 보는 구조이므로, 논리적으로 같은 메모리라도 read port 구현

비용이 발생합니다. FPGA에서 이런 read 형태는 동기식 block RAM의 기본 동작과 다르므로 단순히 배열이라고 모두 BRAM이 되는 것은 아닙니다.

프로그램 RAM에는 코드와 C 데이터가 함께 있습니다. 같은 32비트 값이라도 PC로 읽으면 decoder 입력이고 LW로 읽으면 일반 데이터입니다. CPU가 RAM 각 word에 “이것은 코드”라는 별도 태그를 붙이는 구조가 아닙니다. 현재는 실행 권한 bit도 없으므로 잘못된 분기로 데이터 영역을 실행하려 할 수 있습니다.

3.3 byte 주소와 word 인덱스

RISC-V의 주소는 byte 단위입니다. mem의 한 원소는 4바이트이므로 배열 인덱스는 주소를 4로 나눈 값입니다.

```
앱 주소 0x4000 / 4 = 0x1000 = 4096
따라서 앱 첫 명령어의 저장 위치는 mem[4096]
```

하위 두 비트는 word 안에서의 byte 위치입니다. 예를 들어 주소 0x4001의 byte는 mem[4096]의 비트 [15:8]에 있습니다. LBU는 해당 word를 읽고 offset만큼 오른쪽으로 이동한 뒤 8비트를 zero-extension합니다. SB는 byte write strobe로 필요한 lane만 갱신합니다.

```
// SB의 byte lane 선택
dwstrb_r = 4'b0001 << daddr_r[1:0];
```

이런 byte 연산 덕분에 파일 복사 코드가 임의의 바이트 수를 다룰 수 있습니다. 현재 구현은 자연 정렬을 전제로 하며 잘못 정렬된 word/halfword 접근을 일반적인 정렬 예외로 처리하지 않습니다.

가령 한 word가 0x44332211이면 낮은 주소부터 11 22 33 44가 저장됩니다. 주소 하위 비트가 10인 LBU는 word를 16비트 shift하여 0x33을 얻습니다. 같은 위치에 0xAA를 SB로 쓰면 strobe 0100이 선택되어 새 word는 0x44AA2211입니다. 인덱스 계산과 byte lane 선택을 따로 수행해야 인접 byte가 보존됩니다.

daddr[15:2]라는 slice 자체가 범위 검사 역할을 하는 것은 아닙니다. 우선 daddr < MEM_BYTES 인지 decoder가 검사하고, 선택된 경우에만 slice로 word를 고릅니다. 이 순서를 놓치면 주소의 상위 비트를 잘라 서로 다른 주소가 같은 RAM으로 alias되는 오류를 설명하지 못하게 됩니다.

3.4 MMIO는 주소 디코딩이다

데이터 포트 주소가 프로그램 RAM 범위이면 mem, RAM disk 범위이면 ramdisk, UART 주소이면 UART 상태·데이터 레지스터를 선택합니다. C의 포인터 역참조가 load/store 명령으로 바뀌고, 그 명령의 주소가 회로를 선택합니다.

이 프로젝트의 주소 선택은 memory-mapped I/O입니다. Unix의 mmap() 시스템 콜과는 구분해야 합니다. 현재 OS에는 가상메모리 매핑을 만드는 mmap() 서비스가 없습니다.

volatile 포인터는 compiler가 장치 접근을 일반 변수처럼 없애거나 재사용하지 않도록 하는 C 측 표현입니다. 실제 어떤 장치를 선택하는지는 RTL decoder가 결정합니다. volatile이 UART 회로를 생성하거나 메모리 보호를 설정하는 것은 아닙니다. 장치 register는 읽기 자체가 FIFO pop 같은 부작용을 가질 수 있어 일반 RAM 접근과 의미도 다릅니다.

현재 timer는 0x10001xxx page 전체를 선택하고 내부에서는 주소 bit 2로 두 register를 구분합니다. 따라서 문서상의 정식 주소 이외에도 alias가 존재합니다. firmware는 정식 word-aligned 주소만 사용해야 합니다. 엄격한 peripheral map이 필요하면 decoder의 허용 주소·접근 크기·쓰기 권한을 더 좁혀야 합니다.

코드 해부 3-F. 주소 디코더와 read mux의 줄별 연결

출처: [rtl/rv32_soc.v](https://github.com/riscv/riscv-isa-sim/blob/master/src/main/verilog/rv32_soc.v), 원본 57 - 60, 64 - 65, 98 - 106행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

57 | localparam MEM_BYTES = MEM_WORDS * 4;
58 | wire imem_sel = (iaddr < MEM_BYTES);
59 | wire dmem_sel = (daddr < MEM_BYTES);
60 | wire disk_sel = (daddr[31:13] == 19'h40080);
    ... (원본 61-63행 생략)
64 | wire [31:0] irdata = imem_sel ? mem[iaddr[15:2]] : 32'h0000_0013;
65 | reg [31:0] drdata;
    ... (원본 66-97행 생략)
98 | always @* begin
99 |     if (dmem_sel) drdata = mem[daddr[15:2]];
100 |     else if (disk_sel) drdata = ramdisk[daddr[12:2]];
101 |     else if (daddr == 32'h1000_0004) drdata = {31'd0, uart_tx_ready};
102 |     else if (daddr == 32'h1000_0008) drdata = rx_empty ? 32'd0 : {24'd0,
rx_fifo[rx_rd[5:0]]};
103 |     else if (daddr == 32'h1000_000c) drdata = {31'd0, !rx_empty};
104 |     else if (timer_sel) drdata = timer_rdata;
105 |     else drdata = 32'd0;
106 | end

```

원본 행	한 줄씩 읽는 동작과 의미
57	word 수를 4배하여 byte 단위 RAM 경계를 만듭니다. CPU 주소의 단위와 비교 단위를 일치시킵니다.
58	instruction 주소가 프로그램 RAM 안인지 검사합니다. RAM disk는 이 경로에 포함되지 않습니다.
59	data 주소도 같은 RAM 경계로 검사하지만, 아래 mux에는 별도 disk·UART·timer 선택지가 있습니다.
60	상위 19비트로 8 KiB disk 영역을 선택합니다. 하위 13비트는 이 장치 내부 offset입니다.
64	정상 fetch는 word index로 mem을 읽고, 범위 밖이면 NOP word를 반환합니다. access-fault를 발생시키는 구현은 아닙니다.
65	read mux의 조합 결과를 담은 Verilog 변수를 선언합니다. reg 자료형만으로 저장 register임을 뜻하지 않습니다.
98	입력 변화에 반응하는 조합 블록을 시작합니다.
99	프로그램 RAM을 고르면 byte 주소의 하위 두 비트를 제외한 word index로 읽습니다.
100	disk를 고르면 disk 내부 word를 읽습니다. byte 선택은 core의 LOAD 회로가 합니다.
101	TX ready 한 비트를 32비트 read 값으로 확장합니다.
102	RX FIFO가 비었으면 0, 아니면 가장 앞 byte를 반환합니다. 데이터 값 0과 입력 없음의 구분에는 다음 ready 주소가 필요합니다.
103	RX ready는 FIFO가 비어 있지 않다는 뜻입니다.
104	timer 영역이면 timer 모듈의 read 값을 연결합니다.
105	매핑되지 않은 나머지 주소는 0을 반환합니다. 일반 메모리를 새로 할당하지 않습니다.
106	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

같은 LW라도 주소가 어떤 분기를 선택하는지에 따라 RAM 읽기, 상태 확인, FIFO 소비가 됩니다. 포인터를 캐스팅하는 C 코드와 실제 장치 선택 회로가 만나는 지점입니다.

3.5 instruction cache가 생기면 추가되는 책임

현재는 cache와 prefetch pipeline이 없고 store와 fetch가 동일 RAM을 사용하므로, 로더의 복사가 끝난 뒤 새 코드를 가져올 수 있습니다. 다른 RISC-V 시스템까지 이 동작을 일반화하면 안 됩니다. 명령어 fetch와 데이터 store의 동기화에는 FENCE.I 를 사용하는 규약이 있습니다. 관련 의미는 [RISC-V Zifencei 명세](#)에 설명되어 있습니다.

현재 core는 FENCE 를 NOP로 처리하지만 FENCE.I 는 구현하지 않습니다. FENCE 와 FENCE.I 를 같은 명령어로 간주하면 안 됩니다. 나중에 instruction cache나 pipeline을 추가할 때는 로더와 명령어 동기화 구현을 함께 설계해야 합니다.

예를 들어 앱 A를 실행한 뒤 같은 RAM 위치에 앱 B를 복사했다고 가정하십시오. instruction cache가 A의 cache line을 보관한다면 data RAM은 B여도 CPU가 A의 명령을 계속 fetch할 수 있습니다. 이때 파일 checksum은 성공할 수 있으므로 파일 검증만으로 원인을 찾기 어렵습니다. data store의 완료와 instruction 관측의 일관성을 별도의 계약으로 다뤄야 합니다.

현재는 앞선 store가 edge에서 RAM에 반영되고 이후 fetch가 동일 배열을 읽습니다. 따라서 이 구현에 필요한 동기화는 구조적으로 단순합니다. pipeline·cache를 추가하는 시점에는 CPU 쪽 flush/invalidate 동작과 OS 쪽 실행 전 동기화 호출을 함께 추가해야 하며, 기존 소프트웨어가 그대로 안전하다고 가정하면 안 됩니다.

3.6 현재 core의 architectural state와 내부 신호

architectural state는 명령어 실행 결과로 프로그램이 관찰할 수 있는 상태입니다. 이 core에서는 PC, x0 - x31, 구현된 CSR, RAM과 MMIO 상태가 중심입니다. alu_a, load_shifted, next_pc 같은 값은 한 명령을 계산하기 위한 내부 조합 신호이며 별도의 프로그래머용 register가 아닙니다.

종류	실제 RTL 요소	갱신 시점 또는 성질
현재 명령 위치	reg [31:0] pc	rising edge
정수 register	regs[1:31], x0의 상수 read	rising edge write, 조합 read
trap 상태	mstatus_r, mepc_r 등	rising edge
연산 operand	alu_a, alu_b	decode에 따른 조합 값
write-back 요청	rd_we, rd_data, rd	edge에서 쓸 값의 조합 계산
다음 명령 후보	next_pc	기본 PC+4, branch/jump에서 변경
memory write 요청	dwstrb_r, dwdata_r	trap gating 전의 조합 요청

Verilog의 reg 라는 자료형 이름만 보고 모두 flip-flop이라고 판단하면 안 됩니다. always @* 에서 모든 경로에 값이 주어지는 reg 변수는 조합 회로가 됩니다. 실제 저장소인지 판단하려면 선언보다 어떤 always block에서 어떤 조건으로 갱신되는지를 보아야 합니다.

3.7 RV32I와 46개 명령의 정확한 범위

이 프로젝트는 RV32I 정수 연산을 기반으로 CSR와 MRET를 추가한 교육용 core입니다. 현재 decoder의 mnemonic 종류를 다음처럼 세면 46개입니다. pseudo-instruction과 syscall 서비스 번호는 이 수에 포함하지 않습니다.

계열	현재 명령	수
register 산술·논리	ADD SUB SLL SLT SLTU XOR SRL SRA OR AND	10

계열	현재 명령	수
immediate 산술·논리	ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI	9
load	LB LH LW LBU LHU	5
store	SB SH SW	3
branch	BEQ BNE BLT BGE BLTU BGEU	6
upper immediate	LUI AUIPC	2
jump	JAL JALR	2
memory order	FENCE, 이 구현에서는 NOP	1
환경 호출	ECALL	1
CSR	CSRRW CSRRS CSRRC CSRRWI CSRRSI CSRRCI	6
trap 복귀	MRET	1
합계		46

표준 RV32I의 40개 명령 중 EBREAK의 breakpoint 동작은 이 core에 없고 illegal instruction으로 처리됩니다. 정렬·접근 fault와 CSR 권한 검사에도 생략이 있습니다. 따라서 “46개가 있으므로 완전한 표준 호환성 검증을 마쳤다”는 결론은 성립하지 않습니다. 기본 ISA의 구분은 [RISC-V RV32I 명세](#), 위 표의 지원 여부는 현재 decoder가 근거입니다.

RV32의 32는 정수 register 폭을 뜻합니다. I는 기본 정수 ISA이며, M 곱셈·나눗셈, A 원자 연산, F/D 부동소수점, C 압축 명령, V vector는 구현하지 않았습니다. **프로그램의 문자열·파일·scheduler는 기본 연산을 반복해서 구현할 수 있으므로 전용 “파일 명령어”가 필요하지 않습니다.**

3.8 register file과 ABI 이름

register file은 source 두 개를 동시에 읽고 destination 하나를 edge에서 쓰는 2R1W 구조입니다. R-type의 add x5, x6, x7 은 x6와 x7을 조합으로 읽고 합을 x5에 저장합니다. destination 번호는 insn[11:7] 에서 이미 꺼내 rd_addr 로 연결하므로 opcode case 안에서 별도로 register 이름을 지정하지 않습니다.

register	ABI 이름	소프트웨어의 일반적 역할
x0	zero	읽으면 0, 쓰기는 버림
x1	ra	함수 복귀 주소
x2	sp	stack pointer
x3 / x4	gp / tp	global/thread pointer; 현재 runtime 사용 제한
x5 - x7	t0 - t2	임시 값
x8 - x9	s0 - s1	호출받은 함수가 보존; s0는 선택적 frame pointer
x10 - x17	a0 - a7	함수 인수·결과; Mini OS는 a7을 syscall 번호로 사용
x18 - x27	s2 - s11	호출받은 함수가 보존
x28 - x31	t3 - t6	임시 값

하드웨어 x1에 “return address만 저장해야 한다”는 제약은 없습니다. ra 와 sp 는 ABI의 약속입니다. addi sp, sp, -16 도 core 입장에서는 x2를 대상으로 하는 일반 덧셈입니다. 반면 x0는 RTL에서 실제로 특별 취급하여 읽기는

0, 쓰기는 무시합니다.

reset loop가 x1 - x31까지 0으로 만드는 것은 이 구현의 선택입니다. C runtime은 그 사실에만 기대지 않고 SP와 필요한 상태를 명시적으로 준비합니다. pipeline이 없으므로 이전 명령의 write edge가 지난 다음에는 새 값이 read port에 보이고, 별도의 forwarding 경로가 필요하지 않습니다.

3.9 32비트 명령어의 공통 필드

모든 명령은 4바이트 word로 fetch됩니다. decoder는 기본적으로 opcode를 보고 큰 계열을 선택하고, funct3·funct7로 같은 계열 안의 세부 연산을 선택합니다. 아래 표의 bit 번호는 memory byte 번호가 아니라 fetch된 32비트 word 안의 위치입니다.

필드	bit 위치	의미
opcode	[6:0]	LOAD, STORE, OP 등 큰 연산 계열
rd	[11:7]	destination register가 있는 형식에서 사용
funct3	[14:12]	load 크기·branch 조건·ALU 세부 기능 등
rs1	[19:15]	첫 source register 또는 CSR immediate
rs2	[24:20]	두 번째 source register가 있는 형식에서 사용
funct7	[31:25]	ADD/SUB, SRL/SRA 등의 구분

명령어에 해당 의미의 operand가 없는 경우에도 slice wire 자체는 존재합니다. 예를 들어 store의 [11:7]은 rd가 아니라 immediate의 일부입니다. decoder가 rd_we=0을 유지하므로 우연히 잘라낸 register 번호에 쓰지 않습니다. 필드 추출과 필드 사용 조건을 함께 이해해야 합니다.

3.10 R·I·S·B·U·J 형식

```
R: [ funct7 ][ rs2 ][ rs1 ][f3][ rd ][ opcode ]
I: [      imm[11:0]      ][ rs1 ][f3][ rd ][opcode]
S: [imm[11:5]][rs2][rs1][f3][imm[4:0]][opcode]
B: [imm[12|10:5]][rs2][rs1][f3][imm[4:1|11]][opcode]
U: [      imm[31:12]      ][rd][opcode]
J: [      imm[20|10:1|11|19:12]      ][rd][opcode]
```

도식의 칸 폭은 개념 표현이며 정확한 연결은 아래 식으로 확인합니다. R형은 두 register로 계산하는 연산, I형은 짧은 immediate·load·JALR, S형은 store, B형은 조건 branch, U형은 큰 상수의 상위 부분, J형은 PC-relative jump에 사용합니다.

S/B형에는 destination register가 없으므로 그 위치를 immediate에 사용합니다. B/J offset의 bit 0은 명령어 안에 저장하지 않고 0으로 붙입니다. offset이 2바이트 단위로 인코딩되더라도 현재 코어의 정상적인 instruction address는 4바이트 정렬이어야 합니다. 현재 정렬 fault를 구현하지 않은 사실과 ISA가 정한 정렬 조건은 별개입니다.

3.11 immediate 재구성과 부호 확장

현재 core의 helper function은 다음과 같은 의미입니다. sext는 최상위 부호 bit를 32비트 폭까지 반복한다는 표기입니다.

```

I = sext(insn[31:20])
S = sext({insn[31:25], insn[11:7]})
B = sext({insn[31], insn[7], insn[30:25], insn[11:8], 0})
U = {insn[31:12], 12'b0}
J = sext({insn[31], insn[19:12], insn[20], insn[30:21], 0})

```

I/S immediate는 -2048부터 2047까지 표현합니다. 12비트 0xFFC는 +4092가 아니라 -4이며, 32비트 부호 확장 결과는 0xFFFFFFF C입니다. `addi sp,sp,-16`과 `lw ra,12(sp)`가 같은 I형 immediate 기구를 사용하는 사례입니다.

B offset의 표현 범위는 -4096부터 +4094, J offset은 -1048576부터 +1048574이며 모두 2의 배수입니다. 실제 현재 프로그램의 분기 목적지는 4바이트 정렬 조건도 지켜야 합니다. U형은 하위 12비트를 0으로 만드는 것이고 CSR immediate는 5비트 값을 zero-extension한다는 점이 다릅니다.

3.12 ALU의 연산과 signed 해석

rv32_alu.v의 내부 operation 번호는 ISA opcode와 다릅니다. decoder가 ADD 계열인지 알아낸 뒤 ALU_ADD=0 같은 작은 제어값으로 ALU를 선택합니다. 입력은 두 개의 32비트 값, 출력은 32비트 결과이며 clock 없이 조합적으로 계산합니다.

입력/연산	결과	관찰점
0xFFFFFFFF + 1	0x00000000	32비트 밖 carry는 결과에서 버림
SLT(0xFFFFFFFF, 1)	1	signed에서는 -1 < 1
SLTU(0xFFFFFFFF, 1)	0	unsigned에서는 4294967295 > 1
SRL(0x80000000, 1)	0x40000000	위쪽을 0으로 채움
SRA(0x80000000, 1)	0xC0000000	부호 bit 1을 채움
SLL(1, 32)	1	register shift는 하위 5비트, 즉 0만 사용

덧셈 결과에 signed overflow가 있어도 별도의 arithmetic trap을 만들지 않습니다. 비교는 bit pattern의 해석을 명시적으로 선택합니다. RTL의 \$signed(a)가 필요한 이유는 동일한 wire의 값도 연산에 따라 signed와 unsigned 의미가 달라지기 때문입니다. 별도의 조건 flag register는 없고 branch나 SLT 명령이 직접 비교합니다.

3.13 산술·논리 opcode의 해독

register 연산의 opcode는 0110011, immediate 연산은 0010011입니다. 아래는 현재 case 문을 읽기 위한 간단한 표입니다.

funct3	OP의 funct7=0000000	OP의 funct7=0100000	OP-IMM의 대응
000	ADD	SUB	ADDI
001	SLL	미지원	SLLI, 상위 bit 검증
010	SLT	미지원	SLTI
011	SLTU	미지원	SLTIU
100	XOR	미지원	XORI

funct3	OP의 funct7=0000000	OP의 funct7=0100000	OP-IMM의 대응
101	SRL	SRA	SRLI/SRAI, 상위 bit로 구분
110	OR	미지원	ORI
111	AND	미지원	ANDI

예를 들어 `add a0,a1,a2` 에서 `rd=x10, rs1=x11, rs2=x12`입니다. opcode와 funct 조합으로 `ALU_ADD`를 선택하면 `alu_y=x11+x12, rd_data=alu_y, rd_we=1` 이 됩니다. 다음 edge에서 `x10`만 바뀌고 `PC`는 4 증가합니다.

`ANDI`의 immediate도 부호 확장됩니다. `andi a0,a0,255`는 하위 8비트 mask지만 `andi a0,a0,-1`은 모든 bit를 유지합니다. `SLTIU`는 immediate를 부호 확장한 뒤 unsigned 비교하므로 “U가 붙으면 immediate도 zero-extension”이라고 생각하면 잘못됩니다.

3.14 LOAD: 주소 계산에서 write-back까지

`LOAD` opcode는 `0000011`입니다. 유효 주소는 `rs1_data + imm_i`이고, SoC가 해당 aligned word를 조합으로 반환하면 core가 크기·offset에 맞는 값을 골라 `rd`에 씁니다.

funct3	명령	확장 방식
000	LB	8비트 부호 확장
001	LH	16비트 부호 확장
010	LW	32비트 word 그대로
100	LBU	8비트 zero-extension
101	LHU	16비트 zero-extension

`dmem_rdata=0x807F02FF`, 주소 `offset=00`이면 `LB` 결과는 `0xFFFFFFFF`, `LBU`는 `0x000000FF`입니다. `offset=30`이면 선택 byte는 `0x80`이므로 `LB`는 `0xFFFFFFFF80`, `LBU`는 `0x00000080`입니다. 파일 payload는 unsigned byte들의 열이므로 loader의 byte sum에는 `LBU` 의미가 적합합니다.

`LW`는 word 정렬을 전제로 별도 shift를 하지 않습니다. `LH/LHU`는 `offset 0` 또는 `2`에서 사용해야 한 word 안의 두 byte가 올바르게 선택됩니다. 자연 정렬을 위반하면 현재 하드웨어가 자동으로 두 word를 조합해 주지 않습니다. compiler가 정렬된 C object를 다루도록 주소·자료형을 설계해야 합니다.

코드 해부 3-A. LOAD 한 줄씩: 주소에서 byte 선택과 확장까지

출처: [rtl/rv32_core.v](#), 원본 202 - 214행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

202 | 7'b0000011: begin
203 |     daddr_r=rs1_data+imm_i(insn); rd_we=1'b1;
204 |     load_shifted=dmem_rdata >> (8*daddr_r[1:0]);
205 |     case (funct3)
206 |         3'b000: rd_data={{24{load_shifted[7]}},load_shifted[7:0]};
207 |         3'b001: rd_data={{16{load_shifted[15]}},load_shifted[15:0]};
208 |         3'b010: rd_data=dmem_rdata;
209 |         3'b100: rd_data={24'd0,load_shifted[7:0]};
210 |         3'b101: rd_data={16'd0,load_shifted[15:0]};
211 |         default: begin illegal=1'b1; rd_we=1'b0; end
212 |     endcase
213 | end

```

원본 행	한 줄씩 읽는 동작과 의미
202	LOAD 계열 opcode를 선택합니다. 같은 opcode 안에서도 funct3가 접근 크기와 부호 확장을 구분합니다.
203	rs1 값에 부호 확장한 immediate를 더해 byte 주소를 만듭니다. rd_we는 나중 clock edge에 결과를 저장하라는 요청이며, 이 줄에서 register가 즉시 바뀌지는 않습니다.
204	주소 하위 두 비트를 0·8·16·24의 shift 양으로 바꿉니다. SoC가 준 aligned word의 원하는 byte를 하위 8비트로 이동시킵니다.
205	명령의 funct3를 보고 최종 rd_data를 선택합니다.
206	LB입니다. 선택한 byte의 bit 7을 24번 복제하여 음수의 부호를 보존합니다.
207	LH입니다. 선택한 halfword의 bit 15를 16번 복제합니다. 자연 정렬이라는 전제가 필요합니다.
208	LW입니다. 정렬된 32비트 word를 그대로 반환합니다.
209	LBU입니다. 선택 byte 위에 24개의 0을 붙입니다. 0xFF는 -1이 아닌 255가 됩니다.
210	LHU입니다. 선택 halfword 위에 16개의 0을 붙입니다.
211	정의되지 않은 funct3이면 illegal을 세우고 일반 register 쓰기 요청을 끕니다.
212	앞의 case 선택을 닫습니다. 각 분기는 같은 decode 단계의 대안입니다.
213	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

예를 들어 `dmem_rdata=0x44332211`, `daddr_r[1:0]=2` 이면 shift 결과는 `0x00004433`이고 LBU 결과는 `0x33`입니다. 주소 계산, word 공급, byte 선택, edge의 write-back을 서로 다른 단계로 따라가 보십시오.

3.15 STORE: byte strobe와 데이터 복제

STORE opcode는 `0100011` 이고 주소는 `rs1+imm_s` 입니다. `rs2` 는 저장할 데이터이며 destination register는 없습니다. RAM write는 조합 계산 측시가 아니라 clock edge에 발생합니다.

명령	주소 offset	dwstrb	저장 데이터 구성
SB	0 / 1 / 2 / 3	0001 / 0010 / 0100 / 1000	rs2 하위 byte를 4번 복제
SH	0 / 2	0011 / 1100	rs2 하위 halfword를 2번 복제
SW	0	1111	rs2의 32비트 전체

주소 offset=2에서 `sb a0,2(t0)` 를 실행하고 `a0`의 하위 byte가 `0x5A` 이면 write data는 `0x5A5A5A5A`, strobe는 `0100` 입니다. 실제 RAM은 lane 2만 기록하므로 다른 세 byte는 그대로입니다. 데이터를 네 위치에 복제해 둔 덕분에 별도의 32비트 가변 left shift 없이 lane을 고를 수 있습니다.

SH 역시 하위 16비트를 양쪽 halfword에 복제합니다. 잘못 정렬된 offset=1의 SH도 현재 decoder에서 자동으로 trap되지 않으므로 firmware가 정렬 조건을 지켜야 합니다. “어떤 bit 조합에 RTL 값이 나오는가”와 “ISA와 프로그램이 사용해도 되는 접근인가”를 구분하십시오.

코드 해부 3-B. STORE 한 줄씩: 데이터 복제와 byte enable

출처: [rtl/rv32_core.v](#), 원본 216 - 224행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

216 | 7'b0100011: begin
217 |     daddr_r=rs1_data+imm_s(insn);
218 |     case (funct3)
219 |         3'b000: begin dwstrb_r=4'b0001 << daddr_r[1:0]; dwdata_r={4{rs2_data[7:0]}}; end
220 |         3'b001: begin dwstrb_r=4'b0011 << daddr_r[1:0]; dwdata_r={2{rs2_data[15:0]}}; end
221 |         3'b010: begin dwstrb_r=4'b1111; dwdata_r=rs2_data; end
222 |         default: illegal=1'b1;
223 |     endcase
224 | end

```

원본 행	한 줄씩 읽는 동작과 의미
216	STORE opcode에 들어갑니다. STORE는 일반 destination register를 만들지 않습니다.
217	rs1와 S immediate로 byte 주소를 계산합니다. 저장할 값은 주소 계산에 쓰인 rs1이 아니라 rs2에서 옵니다.
218	funct3로 SB·SH·SW를 선택합니다.
219	SB는 주소 offset만큼 0001을 이동하여 한 lane만 켵니다. 낮은 byte를 네 번 복제해 어느 lane이 선택되어도 같은 byte가 준비되게 합니다.
220	SH는 두 lane을 켜고 낮은 16비트를 두 번 복제합니다. 이 구현에서 정상 offset은 0 또는 2입니다.
221	SW는 네 lane을 모두 켜고 rs2 전체를 전달합니다. 정상 주소는 4바이트 정렬입니다.
222	미지원 크기는 illegal입니다. 뒤의 공통 illegal 처리와 trap gating이 실제 쓰기를 억제합니다.
223	앞의 case 선택을 담습니다. 각 분기는 같은 decode 단계의 대안입니다.
224	앞에서 연 블록을 담습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

주소 offset=2, rs2의 낮은 byte=0xAA라면 dwstrb=0100, dwdata=0xAAAAAAAA 입니다. 실제 word 갱신은 다음 코드의 세 번째 lane에만 일어납니다.

코드 해부 3-C. SoC에서 STORE 요청이 RAM에 반영되는 줄

출처: [rtl/rv32_soc.v](#), 원본 120 - 125행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

120 | if (dmem_sel && (!dwstrb)) begin
121 |     if (dwstrb[0]) mem[daddr[15:2]][7:0] <= dwdata[7:0];
122 |     if (dwstrb[1]) mem[daddr[15:2]][15:8] <= dwdata[15:8];
123 |     if (dwstrb[2]) mem[daddr[15:2]][23:16] <= dwdata[23:16];
124 |     if (dwstrb[3]) mem[daddr[15:2]][31:24] <= dwdata[31:24];
125 | end

```

원본 행	한 줄씩 읽는 동작과 의미
120	주소가 프로그램 RAM에 속하고 write strobe 중 하나라도 1일 때만 RAM write를 진행합니다. reduction OR <code>!dwstrb</code> 는 네 비트의 논리합입니다.
121	lane 0이 켜졌으면 word의 최하위 byte만 갱신합니다. little-endian에서 가장 낮은 byte 주소입니다.
122	lane 1은 두 번째 byte를 갱신합니다. 켜져 있으면 기존 RAM bit가 유지됩니다.
123	lane 2는 세 번째 byte를 갱신합니다. 위 SB 예제에서 실제로 선택되는 줄입니다.

원본 행	한 줄씩 읽는 동작과 의미
124	lane 3은 최상위 byte를 갱신합니다. SW에서는 네 줄이 같은 edge에 모두 적용됩니다.
125	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

이 발췌는 `always @(posedge clk)` 안에 있습니다. `<=` 는 clock edge의 저장 동작을 표현합니다. RAM이 0x44332211이면 위 SB 이후 0x44AA2211이 되며 다른 세 byte는 보존됩니다.

3.16 BRANCH: 조건과 target의 분리

BRANCH opcode는 1100011 입니다. target은 현재 PC에 B immediate를 더한 값이고, 조건이 거짓이면 기본값 PC+4를 사용합니다. branch 자체는 rd를 쓰지 않습니다.

funct3	명령	비교
000 / 001	BEQ / BNE	같음 / 다름
100 / 101	BLT / BGE	signed 작음 / 크거나 같음
110 / 111	BLTU / BGEU	unsigned 작음 / 크거나 같음

앱의 `bne a0,zero,0x400C` 는 PC 0x401C 에서 -16을 더합니다. 다음 순차 주소 0x4020 을 기준으로 계산하면 -20이라는 틀린 offset이 나옵니다. branch immediate의 기준은 해당 branch의 PC입니다. target 계산과 조건 비교를 각각 확인하면 off-by-four 오류를 쉽게 찾을 수 있습니다.

이 core에는 MIPS의 branch delay slot이 없습니다. 조건이 성립하면 다음 fetch부터 target의 명령을 가져오며 PC+4의 명령을 의무적으로 한 번 실행하지 않습니다. pipeline이 없으므로 branch flush할 중간 pipeline register도 없습니다.

3.17 JAL과 JALR: 함수 호출을 만드는 두 효과

JAL은 `rd=PC+4` 와 `PC=PC+J immediate` 를 함께 수행합니다. JALR은 `rd=PC+4` 와 `PC=(rs1+I immediate)&0xFFFFFFF` 를 수행합니다. 하나의 명령이 복귀 주소 기록과 제어 이동을 동시에 수행하는 것입니다.

호출 직전: PC=0x0A18, a5=0x4000
명령어 : jalr ra,0(a5)
edge 이후: ra=0x0A1C, PC=0x4000

rd 필드는 모든 JAL/JALR 명령에 인코딩되어 있으므로 RTL opcode branch에서는 `rd_we=1`, `rd_data=pc+4` 만 지정해도 됩니다. 실제 destination은 공통 wire `rd=insn[11:7]` 입니다. `jal zero,label` 은 return address를 버리는 jump이며, `jalr zero,0(ra)` 는 return입니다.

JALR는 bit 0만 0으로 만듭니다. 따라서 target이 자동으로 4바이트 정렬된다고 단정할 수 없습니다. 현재 core는 instruction-address-misaligned 예외를 구현하지 않았으므로 잘못된 bit 1은 software가 피해야 합니다. 앱 entry와 trap vector를 4바이트 정렬하는 linker 규칙이 이 전제를 지켜 줍니다.

3.18 LUI와 AUIPC: 큰 주소를 만드는 방법

LUI는 20비트 immediate 뒤에 12개의 0을 붙여 rd에 씁니다. `lui a5,0x4`의 결과가 `0x4000`인 이유입니다. AUIPC는 같은 형태의 값을 현재 PC에 더합니다. 하나는 상수 생성, 다른 하나는 PC-relative 주소 생성에 적합합니다.

12비트 ADDI immediate는 signed이므로 큰 주소를 상·하위로 나눌 때 보정이 필요할 수 있습니다. 주소 `0x12345ABC`를 만들려면 다음과 같이 표현할 수 있습니다.

```
lui t0,0x12346
addi t0,t0,-1348      # -1348 = -0x544
```

`0x12346000 - 0x544 = 0x12345ABC`입니다. 하위 12비트 ABC의 sign bit가 1이므로 상위 부분에 carry 보정이 필요합니다. `%hi`, `%lo`, `la`와 linker relocation은 이런 주소 분할을 처리합니다. 주소를 무조건 상위 20bit와 하위 12bit의 양수로 나누면 오류가 납니다.

현재 `call` pseudo-instruction은 relaxation을 끈 빌드에서 보통 AUIPC+JALR 조합으로 나타납니다. linker가 목적지까지의 거리를 반영하며, 앱의 고정 함수 포인터 호출은 앞에서 본 LUI+JALR 형태로 나타납니다. 같은 “호출”도 주소를 얻는 방식에 따라 앞부분 명령이 달라집니다.

3.19 CSR 명령 6종과 pseudo-instruction

CSR 명령은 SYSTEM opcode 안에서 funct3로 구분하며, CSR 주소는 `[31:20]`입니다. rs1 field를 register 번호로 쓰는 형태와 5비트 immediate로 쓰는 형태가 있습니다.

명령	rd에 반환하는 값	CSR에 요청하는 값
CSRRW	이전 CSR 값	rs1 값
CSRRS	이전 CSR 값	이전 값 OR rs1
CSRRC	이전 CSR 값	이전 값 AND NOT rs1
CSRRWI	이전 CSR 값	zero-extended zimm
CSRRSI	이전 CSR 값	이전 값 OR zimm
CSRRCI	이전 CSR 값	이전 값 AND NOT zimm

CSRRS/CSRRC는 rs1 field가 x0일 때 쓰기를 생략합니다. 일반 register의 내용이 우연히 0인 것과 인코딩이 x0인 것은 구분됩니다. immediate set/clear는 zimm=0이면 쓰기를 생략합니다. CSRRW/CSRRWI는 값을 교체하는 명령이므로 0도 유효한 쓰기입니다. 표준 의미와 예외는 [Zicsr 명세](#)를 참고하고 실제 write mask는 `rv32_csr.v`를 확인하십시오.

```
csrr t0,mcause      # csrrs t0,mcause,zero
csrw mie,t0         # csrrw zero,mie,t0
csrsi mstatus,8     # csrrsi zero,mstatus,8
```

이 pseudo-instruction들을 각각 새 하드웨어 명령으로 세지 않습니다. CSR은 memory-mapped register와 주소 공간도 다릅니다. mcause의 12비트 CSR 번호 `0x342`를 RAM 주소 `0x342`로 load하는 것은 CSR 읽기가 아닙니다.

3.20 구현된 CSR와 빠진 기능

CSR는 **Control and Status Register**, 즉 제어·상태 레지스터입니다. 제어는 interrupt를 허용하거나 handler 위치를 지정하는 것이고, 상태는 trap 원인이나 복귀 위치를 기록하는 것입니다. 먼저 현재 읽을 수 있는 11개 CSR 주소를 전체 지도처럼 살펴봅니다. 아래의 영문 이름은 약어를 풀어 읽기 위한 이름이며, `mcause`는 표준 절 제목에서 Machine Cause라고도 부릅니다.

주소·이름	영문 이름	현재 RTL의 동작
0x300 <code>mstatus</code>	Machine Status Register	MIE bit 3, MPIE bit 7만 갱신
0x304 <code>mie</code>	Machine Interrupt-Enable Register	MTIE bit 7만 저장
0x305 <code>mtvec</code>	Machine Trap-Vector Base-Address Register	하위 2bit를 0으로 저장, direct mode
0x340 <code>mscratch</code>	Machine Scratch Register	32비트 임시 값; 현재 trap entry에서는 미사용
0x341 <code>mepc</code>	Machine Exception Program Counter	trap PC 저장, software write와 복귀 출력은 4바이트 정렬
0x342 <code>mcause</code>	Machine Trap Cause Register	마지막 trap 원인 저장
0x344 <code>mip</code>	Machine Interrupt-Pending Register	timer_irq에서 MTIP bit 7을 조합 생성
0xF11 <code>mvendorid</code>	Machine Vendor ID Register	읽기 값 0, 쓰기 무시
0xF12 <code>marchid</code>	Machine Architecture ID Register	읽기 값 0, 쓰기 무시
0xF13 <code>mimpid</code>	Machine Implementation ID Register	읽기 값 0, 쓰기 무시
0xF14 <code>mhartid</code>	Machine Hardware Thread ID Register	유일한 hart의 ID 0, 쓰기 무시

이 표의 주소·명칭은 [RISC-V CSR 목록](#), 마지막 열은 [현재 CSR RTL](#)을 근거로 합니다. 각 이름의 의미와 사용 예는 3.27 - 3.40절에서 설명합니다. 11개 주소를 읽는다고 11개의 독립된 32비트 저장 배열을 구현했다는 뜻은 아닙니다. 실제 저장 상태는 6개이고, `mip`는 입력 신호에서, ID CSR 4개는 상수에서 읽기 값을 만듭니다.

현재 core는 `misa`, `mtval`, `mcycle`, `minstret`, `satp`, PMP CSR 등을 제공하지 않습니다. 인식되지 않는 CSR 주소는 `csr_valid=0`이 되어 illegal instruction으로 처리됩니다. `mip`와 ID CSR처럼 읽기는 가능하지만 write case가 없는 대상에 대한 쓰기는 무시되며, 완전한 표준 read-only CSR 위반 trap 검사도 구현하지 않았습니다.

M-mode만 사용하므로 MPP를 포함한 일반적인 privilege 전환 상태를 관리하지 않습니다. `mret`가 더 낮은 privilege로 내려가는 기능도 현재 사용·구현되지 않습니다. 이 표는 현재 OS가 기대하는 최소 CSR 계약이며, 범용 OS를 올릴 수 있는 전체 privileged 구현 목록이 아닙니다.

3.21 decoder의 기본값과 write-back 선택

`always @*` 첫 부분은 `next_pc=pc+4`, `rd_we=0`, `dwstrb_r=0`, `csr_we=0` 같은 기본값을 지정합니다. 이후 명령 계열별로 필요한 부분만 덮어씁니다. 이 구조는 값을 할당하지 않은 경로에서 이전 값을 유지하는 latch가 생기지 않도록 하는 데 중요합니다.

명령 계열	rd_data의 출처	memory write	next_pc 후보
ALU	<code>alu_y</code>	없음	PC+4
LUI/AUIPC	U immediate 또는 PC+U	없음	PC+4

명령 계열	rd_data의 출처	memory write	next_pc 후보
LOAD	word/byte 선택과 확장 결과	없음	PC+4
STORE	rd write 없음	byte strobe	PC+4
BRANCH	rd write 없음	없음	PC+4 또는 branch target
JAL/JALR	PC+4	없음	jump target
CSR	이전 CSR 값	RAM write 없음	PC+4
MRET	rd write 없음	없음	mepc

주소 계산이나 PC+4가 모두 `rv32_alu` 인스턴스를 통과하는 것은 아닙니다. 현재 RTL은 `pc+4`, `rs1+imm`, branch 비교 등을 core 안의 표현식으로도 작성합니다. 합성기는 이 표현식들을 조합 회로로 구현·최적화합니다. “ALU가 하나 선언되어 있으니 FPGA 안에 덧셈 회로도 반드시 하나뿐”이라고 해석하면 안 됩니다.

3.22 trap 우선순위와 정확한 부작용 억제

core의 `take_trap`은 pending timer interrupt 또는 동기 exception에서 발생합니다. 둘이 같은 사이클에 보이면 현재 RTL은 timer를 우선 선택합니다. PC 갱신은 `reset`, `trap`, 일반 `next_pc` 순서입니다. 이 구현에서 어느 명령을 다시 실행할지 이해하려면 이 우선순위를 보아야 합니다.

```
assign dmem_wstrb = take_trap ? 4'b0000 : dwstrb_r;
// register write: rd_we && !take_trap
// CSR write:      csr_we && !take_trap
```

store가 decode된 사이클에 timer trap이 선택되면 write strobe를 0으로 만들어 store를 아직 실행하지 않은 것으로 유지합니다. `mepc`에 현재 PC를 저장하고 나중에 돌아와 store를 한 번 수행합니다. 만약 store를 먼저 실행하고 같은 PC로 돌아오면 UART 같은 부작용 장치에 두 번 출력하는 오류가 생길 수 있습니다.

RX 데이터 load에는 읽기 부작용인 FIFO pop도 있습니다. 그래서 SoC의 `rx_pop`은 `!core_trap` 조건을 포함합니다. register write와 memory write만 막고 pop을 허용하면, trap에서 돌아와 같은 load를 실행할 때 다음 문자를 읽게 되어 한 바이트가 사라질 수 있습니다. precise한 중단을 만들 때 read side effect도 고려해야 한다는 사례입니다.

코드 해부 3-D. 다음 PC를 실제로 선택하는 순차 회로

출처: [rtl/rv32_core.v](#), 원본 308 - 312행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
308 | always @(posedge clk) begin
309 |     if (rst) pc <= RESET_PC;
310 |     else if (take_trap) pc <= csr_mtvec;
311 |     else pc <= next_pc;
312 | end
```

원본 행	한 줄씩 읽는 동작과 의미
308	PC 저장 register를 rising edge마다 갱신하는 순차 블록입니다. decoder의 조합 계산과 구분합니다.
309	reset이면 다른 조건보다 먼저 RESET_PC를 선택합니다.
310	reset이 아니면서 trap이 선택되면 mtvec로 이동합니다. 이미 계산된 branch target이나 MRET target보다 우선합니다.
311	그 외에는 decoder가 계산한 next_pc를 채택합니다. 순차 실행·branch·JALR·MRET가 이 후보를 공유합니다.

원본 행	한 줄씩 읽는 동작과 의미
312	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

PC 선택만 바뀌어서는 precise trap이 되지 않습니다. 같은 edge에 register write, CSR write, memory strobe와 UART RX pop도 억제되어야 중단된 명령을 복귀 후 한 번 실행할 수 있습니다.

3.23 세 명령을 사이클별로 따라가기

다음은 interrupt가 없는 상황의 개념 추적입니다. t0는 정렬된 RAM 주소, t1은 저장할 값이며 instruction은 별도 위치에 있다고 가정합니다.

```
sw    t1,0(t0)
lw    t2,0(t0)
addi  t2,t2,1
```

사이클	조합 계산	마지막 edge의 변경
N	SW decode, daddr=t0, dwddata=t1, strobe=1111	RAM[t0]=t1, PC+4
N+1	LW decode, 갱신된 RAM 읽기, rd_data=t1	t2=t1, PC+4
N+2	ADDI decode, 새 t2 읽기, ALU=t1+1	t2=t1+1, PC+4

단일 사이클 설계에서는 이전 명령이 끝난 후 다음 명령을 계산하므로 위 의존성에 별도 hazard stall을 넣지 않습니다. 반면 pipeline에서는 동시에 여러 명령이 진행되기 때문에 동일한 코드에서 forwarding이나 load-use 제어가 필요합니다. ISA 결과는 같고 내부 실행 구조가 달라지는 것입니다.

같은 방법으로 ecall 사이클을 추적하면 일반 write 대신 mepc/mcause/mstatus가 바뀌고 다음 PC가 mtvec가 됩니다. 그 다음 사이클부터 trap.S의 첫 명령을 수행합니다. trap frame 전체가 하드웨어에서 한 번에 저장되는 것이 아니라 여러 SW 명령으로 시간에 걸쳐 저장됩니다.

3.24 timing, 합성 resource, 소프트웨어 비용

12.5 MHz의 clock period는 80 ns입니다. 단일 사이클 CPU의 허용 주기는 instruction memory·decoder·register read·연산·data read·write-back mux의 최장 경로와 register setup, clock 관련 여유를 포함해 정해집니다. 클록을 단순히 높이면 컴파일된 C 코드가 더 빨라지는 것이 아니라 timing 위반 가능성이 생깁니다.

이론상 정상 명령은 한 클록에 하나씩 완료되지만 프로그램 실행 시간을 instruction 수만으로 단순 계산하기도 어렵습니다. UART ready를 기다리는 loop, syscall의 register 저장·복원, timer handler가 추가 명령을 실행합니다. 앱 자체의 static instruction이 9개라는 사실과 전체 동적 실행 명령 수는 크게 다릅니다.

LUT RAM과 register·배선 사용량은 메모리 크기와 read port에 따라 변합니다. 실제 FPGA 자원 수와 최대 동작 주파수는 합성·배선 보고서로 확인해야 합니다. RTL의 줄 수나 ISA 명령 수가 resource 수를 직접 결정하지는 않습니다. 향후 BRAM을 쓰는 multi-cycle 구조는 명령당 cycle이 늘어도 자원과 clock 측면에서 이점이 있을 수 있습니다.

3.25 BNE 기계어를 비트 단위로 검사하기

앱의 실제 `bne a0,zero,0x400C` 는 PC `0x401C` 에 있으며 machine word는 `0xFE0518E3` 입니다. 먼저 operand와 상대 거리를 정합니다.

```
rs1 = a0 = x10 = 010102
rs2 = zero = x0 = 000002
funct3 = 0012 (BNE)
opcode = 11000112
offset = 0x400C - 0x401C = -16
13비트 offset 표현 = 0x1FF0
```

offset을 인코딩하는 field들은 연속된 한 덩어리가 아닙니다. 아래처럼 나누어 배치합니다.

instruction 위치	넣는 값	이 예제
bit 31	offset bit 12	1
bits 30:25	offset bits 10:5	111111
bits 24:20	rs2	00000
bits 19:15	rs1	01010
bits 14:12	funct3	001
bits 11:8	offset bits 4:1	1000
bit 7	offset bit 11	1
bits 6:0	opcode	1100011

조합하면 `1 111111 00000 01010 001 1000 1 1100011` 이 되고 16진수로 `FE0518E3` 입니다. little-endian memory의 네 byte는 `E3 18 05 FE` 입니다. 다시 core의 `imm_b()` 순서로 모으면 부호 확장된 -16이 복구됩니다. assembler와 decoder가 같은 field 계약을 반대 방향으로 사용하는 것입니다.

여기서 offset의 bit 0은 항상 0으로 복구되지만 bit 1은 별도로 표현됩니다. 따라서 인코딩 가능한 모든 B target이 현재 4바이트 instruction 정렬을 만족하는 것은 아닙니다. assembler·linker가 정상 코드 label로 branch를 생성하고, hardware의 미구현 정렬 예외 한계를 알고 프로그램을 작성해야 합니다.

3.26 load와 trap을 함께 보는 타이밍 표

RX FIFO의 앞 원소가 문자 A이고 다음 원소가 B라고 가정합니다. CPU는 `RX_DATA` 주소에 LW를 수행하려 합니다. 아래 두 상황은 instruction은 같아도 edge에서의 결과가 다릅니다.

관찰	interrupt 없음	timer trap이 같은 edge에서 선택됨
조합 주소	<code>0x10000008</code>	<code>0x10000008</code>
조합 read data	A의 값 <code>0x41</code>	A의 값 <code>0x41</code> 이 보일 수 있음
<code>rd_we && !take_trap</code>	1	0
<code>rx_pop</code>	1	0
edge의 rd	A로 갱신	이전 값 유지
FIFO 다음 상태	B가 앞 원소	A가 계속 앞 원소

관찰	interrupt 없음	timer trap이 같은 edge에서 선택됨
다음 PC	PC+4	mtvec

trap 상황에서 data wire에 A가 보였다는 이유만으로 A를 읽었다고 판단하면 안 됩니다. architectural 결과는 register write와 peripheral side effect가 실제로 commit됐는지로 결정됩니다. handler 후 같은 LW로 돌아오면 그때 A를 정상 소비합니다.

이 사고방식은 나중에 ready/valid memory bus나 pipeline에도 이어집니다. 요청이 계산된 상태, 장치가 응답한 상태, instruction이 완료된 상태를 구분해야 합니다. 현재 단일 사이클에서는 이 단계들이 가까이 붙어 있어도 trap을 넣는 순간 차이를 명확하게 다뤄야 합니다.

3.27 CSR 이름과 세 가지 주소를 읽는 방법

CSR 이름 앞의 m은 Machine, 즉 M-mode와 관련된 제어 상태라는 뜻입니다. memory의 약자도 아니고, 곱셈·나눗셈 명령을 뜻하는 ISA의 M extension도 아닙니다. 이 core에는 M-mode만 있으므로 앱·셀·OS가 모두 같은 CSR에 접근할 수 있습니다. ecall을 사용한다는 사실만으로 앱의 직접 CSR 접근이 금지되지는 않습니다.

status는 상태, ie는 Interrupt Enable, ip는 Interrupt Pending, tvec는 Trap Vector, epc는 Exception Program Counter입니다. 따라서 mie는 “어떤 interrupt를 허용하는가”, mip는 “어떤 interrupt 요청이 있는가”, mepc는 “trap 때 어느 instruction에 있었는가”를 나타냅니다. 이름을 해석하면 기능별 관계를 기억하기 쉽습니다.

다음 예에는 서로 다른 번호 체계가 동시에 등장합니다. 숫자가 같거나 비슷해 보여도 접근 방법이 다릅니다.

구분	예	접근 명령과 의미
일반 register 번호	x5, ABI 이름 t0	ADD·LW 등의 rd/rs1/rs2가 고르는 operand
CSR 주소	mtvec = 0x305 라는 CSR 번호	csrr·csw의 12비트 CSR field
CSR에 저장된 값	mtvec의 내용 0x00000040	trap 때 instruction fetch를 시작할 메모리 주소
MMIO 주소	0x10001004	SW로 timer의 compare 값을 쓰는 data 주소

csrw mtvec, t0는 t0의 값을 CSR 번호 0x305에 저장합니다. RAM 주소 0x305에 SW하는 것이 아닙니다. mtvec에 0x40을 넣었다면 trap 때 PC가 0x40이 될 뿐, CSR의 번호가 0x40으로 바뀌는 것도 아닙니다. 12비트 CSR 번호 공간은 4096개 위치를 표현하지만 현재 decoder는 그중 표에 있는 11개만 인식합니다.

표준의 CSR 주소에는 접근 속성도 표현됩니다. [11:10]=11은 read-only 분류이고 [9:8]=11은 machine 수준을 나타냅니다. 예를 들어 0xF14는 M-mode read-only CSR 번호입니다. 이 주소 규칙은 CSR 주소 매핑 명세를 읽는 단서이지, 현재 RTL이 자동으로 권한 검사를 해 준다는 뜻은 아닙니다. 현재 core에는 privilege 비교 회로와 ID CSR 쓰기 위반 trap이 없습니다.

3.28 mstatus와 MIE·MPIE의 정확한 의미

mstatus의 영문 이름은 Machine Status Register입니다. 현재 core의 동작 상태 중 “지금 machine interrupt를 받을 수 있는가”와 “trap 직전에는 받을 수 있었는가”를 보존합니다. CSR 주소는 0x300이며, rv32_csr_v의 mstatus_r가 그 저장 장소입니다.

비트	영문 이름	현재 core에서의 의미
MIE, bit 3	Machine Interrupt Enable	1이면 machine interrupt의 전역 허용 조건 충족
MPIE, bit 7	Machine Previous Interrupt Enable	trap 직전 MIE를 보관하고 MRET 때 복원
나머지	현재 구현되지 않음	reset 값 0을 유지하며 쓰기로 설정되지 않음

bit 번호는 0부터 쉼니다. 그러므로 MIE의 mask는 $1u \ll 3 = 0x08$, MPIE의 mask는 $1u \ll 7 = 0x80$ 이고 두 비트의 mask 합은 $0x88$ 입니다. 이 RTL의 software write는 다음 식으로 동작합니다.

```
mstatus_r <= (mstatus_r & ~32'h0000_0088) |
              (write_data & 32'h0000_0088);
```

낮은 두 자리만 보면 $0x00$ 은 $MIE=0 \cdot MPIE=0$, $0x08$ 은 $1 \cdot 0$, $0x80$ 은 $0 \cdot 1$, $0x88$ 은 $1 \cdot 1$ 입니다. 이 mask는 **현재 구현의 mask**입니다. 다른 RISC-V CPU에서 mstatus의 나머지 비트가 모두 무의미하다고 일반화하면 안 됩니다.

trap entry에서는 $MPIE \leftarrow$ 이전 MIE, $MIE \leftarrow 0$ 이 같은 clock edge에 반영됩니다. RTL의 nonblocking assignment인 $\leq$ 는 오른쪽을 갱신 전 값으로 평가하므로 MIE를 0으로 만드는 문장이 있어도 MPIE에는 이전 MIE가 저장됩니다. MRET에서는 $MIE \leftarrow MPIE$, $MPIE \leftarrow 1$ 이 됩니다. 다음 표는 handler가 mstatus를 따로 쓰지 않는 경우입니다.

시점	원래 interrupt 허용	원래 interrupt 금지
trap 직전 MIE	1	0
trap 직후 MIE, MPIE	0, 1	0, 0
MRET 직후 MIE, MPIE	1, 1	0, 1

두 번째 열은 timer interrupt나 ECALL에 적용될 수 있습니다. 세 번째 열은 $MIE=0$ 이어도 발생할 수 있는 ECALL 같은 **동기 예외**를 생각하면 됩니다. MRET가 MIE를 무조건 1로 만드는 것은 아닙니다. 저장된 MPIE가 0이면 복귀 후에도 interrupt는 금지됩니다.

특히 $MIE=0$ 은 “모든 trap 금지”가 아닙니다. 현재 회로에서 MIE는 timer 허용 식에만 들어갑니다. handler 안의 illegal instruction이나 ECALL은 여전히 trap을 일으켜 $mepc \cdot mcause \cdot MPIE$ 를 덮어쓸 수 있습니다. 현재 trap.S가 이런 중첩 예외를 안전하게 복구하는 범용 구조는 아닙니다.

표준에는 **MPP, Machine Previous Privilege**처럼 복귀할 privilege와 관계되는 상태도 있습니다. 이 core는 MPP와 privilege 전환을 구현하지 않습니다. MPP 위치가 0으로 읽혀도 U-mode가 존재한다는 뜻이 아니며 MRET 후에도 계속 M-mode입니다. 표준 상태와 복귀 규칙은 [Machine-Level ISA](#), 위 mask와 예제 값은 현재 RTL을 기준으로 구분해야 합니다.

코드 해부 3-E. CSR의 trap 진입과 MRET 상태 갱신

출처: [rtl/rv32_csr.v](#), 원본 98 - 108행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

98 | if (trap_enter) begin
99 |     mepc_r      <= trap_pc;
100 |     mcause_r    <= trap_cause;
101 |     mstatus_r[7] <= mstatus_r[3];
102 |     mstatus_r[3] <= 1'b0;
105 | end else if (mret) begin
106 |     mstatus_r[3] <= mstatus_r[7];
107 |     mstatus_r[7] <= 1'b1;
108 | end

```

원본 행	한 줄씩 읽는 동작과 의미
98	trap_enter가 참인 edge의 상태 갱신을 시작합니다. software CSR write보다 뒤에서 같은 상태를 갱신하는 우선순위를 확인합니다.
99	현재 중단된 instruction 주소를 mepc에 저장합니다. ECALL에서도 자동 PC+4가 아닙니다.
100	선택된 trap 원인을 mcause에 기록합니다. timer라면 0x80000007, M-mode ECALL이면 11입니다.
101	갱신 전 MIE를 MPIE에 복사합니다. nonblocking assignment이므로 다음 줄의 MIE=0에 영향을 받지 않습니다.
102	handler 동안 machine interrupt 전역 허용을 끕니다. 동기 예외까지 금지하는 뜻은 아닙니다.
105	동시에 trap 진입이 없는 경우에만 MRET 분기를 선택합니다. 이 분기는 trap보다 낮은 우선순위입니다.
106	이전 허용 상태인 MPIE를 MIE로 복원합니다. MIE를 항상 1로 만드는 명령이 아닙니다.
107	복귀 처리에서 MPIE를 1로 설정합니다.
108	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

이 블록은 PC를 직접 쓰지 않습니다. CSR은 복귀 상태를 준비하고 core는 `csr_mepc`를 `next_pc`로 선택합니다. `mstatus`의 변화와 PC의 변화를 두 모듈에서 연결해 읽어야 MRET의 전체 효과가 보입니다.

3.29 mie와 interrupt 종류별 허용 비트

`mie`는 **Machine Interrupt-Enable Register**, CSR 주소 0x304입니다. `mstatus.MIE`와 철자가 비슷하지만 같은 대상이 아닙니다. 소문자 `mie`는 여러 interrupt 종류의 허용 비트를 담은 CSR 전체이고, 대문자 `MIE`는 `mstatus` 안의 한 비트입니다.

현재 구현하는 비트는 **MTIE, Machine Timer Interrupt Enable**, bit 7뿐입니다. `mie_r <= write_data & 0x80`이므로 `csr_w mie, t0`에서 `t0=0x80`이면 `MTIE=1`, `t0=00`이면 `MTIE=00`이 됩니다. `t0=0x08`을 써도 `MTIE`는 켜지지 않습니다. `0x08`은 `mstatus.MIE`에 사용하는 mask입니다.

이름	무엇을 선택하는가	현재 구현
<code>mstatus.MIE</code>	machine interrupt의 전역 허용	bit 3
<code>mie.MTIE</code>	Machine Timer Interrupt Enable	bit 7
<code>mie.MSIE</code>	Machine Software Interrupt Enable	미구현
<code>mie.MEIE</code>	Machine External Interrupt Enable	미구현

MSIE의 software interrupt를 ECALL과 혼동하지 않아야 합니다. ECALL은 instruction이 일으키는 동기 예외이고, software interrupt는 별도의 pending 원인을 통해 전달하는 비동기 interrupt입니다. 현재의 UART RX 역시 MEIE로 처리하는 외부 interrupt가 아니라 FIFO를 software가 polling하는 경로입니다.

timer interrupt의 현재 허용 조건은 다음 한 줄입니다. 한 항이라도 0이면 timer로 trap에 들어가지 않습니다.

```
irq_pending = mstatus_r[3] && mie_r[7] && timer_irq;
```

kernel_main() 은 csr_write_mie(0x80u) 다음 csr_set_mstatus_mie() 를 호출하여 두 허용 조건을 차례로 설정합니다. 이때 timer comparator도 이미 미래의 tick으로 설정되어 있어야 합니다. MIE와 MTIE는 요청을 만드는 장치가 아니라 이미 있는 요청을 CPU가 받아들일지를 정합니다.

MTIE를 보존적 set 연산으로 커려면 다음처럼 register operand를 사용합니다.

```
li    t0, 0x80
csrs mie, t0          # CSRRS x0,mie,t0: MTIE set
csrsi mstatus, 8      # CSRRSI x0,mstatus,8: MIE set
```

CSR immediate의 숫자는 비트 번호가 아니라 5비트 mask 값입니다. 따라서 csrsi mstatus, 8은 bit 3을 켜고, csrsi mie, 7은 bit 7을 켜는 코드가 아닙니다. csrsi mie, 0x80 도 immediate 범위 0 - 31을 넘으므로 사용할 수 없습니다. 위 예제는 0x80을 일반 register에 넣어 그 값을 mask로 전달합니다.

3.30 mip와 pending 상태 및 timer의 연결

mip는 Machine Interrupt-Pending Register, CSR 주소 0x344 입니다. pending은 “CPU가 아직 handler로 처리하지 않았거나, interrupt를 요구하는 조건이 여전히 남아 있다”는 뜻으로 읽을 수 있습니다. 현재 회로에서는 MTIP, Machine Timer Interrupt Pending, bit 7이 timer의 level을 그대로 나타냅니다.

mip를 저장하는 mip_r는 없습니다. read mux가 timer_irq ? 0x80 : 0을 반환합니다. 따라서 mstatus.MIE나 mie.MTIE를 0으로 바꾸더라도 timer_irq가 1이면 mip는 계속 0x80입니다. 반면 이 RTL의 irq_pending wire는 전역·종류별 enable까지 모두 적용한 “지금 trap을 받아들일 조건”입니다. 같은 pending이라는 단어를 써도 두 값의 생성 식은 다릅니다.

MIE	MTIE	timer_irq	mip 읽기	core의 irq_pending
0	0	1	0x80	0
0	1	1	0x80	0
1	0	1	0x80	0
1	1	0	0x00	0
1	1	1	0x80	1

이 관계를 hardware 경로로 읽으면 다음과 같습니다. mtime은 machine time counter, mtimecmp는 machine time compare 값이며, 이 프로젝트에서는 CSR가 아니라 [simple timer.v](#)의 MMIO 주변장치 상태입니다.

```
mtime >= mtimecmp — timer_irq ——— mip.MTIP
                        |
                        AND mie.MTIE
                        AND mstatus.MIE — irq_pending
```

`csrw mip, zero`는 이 RTL에서 무시되므로 timer 요청을 해제하지 못합니다. trap에 진입해 MIE가 0이 되어도 comparator가 참이면 MTIP는 여전히 1입니다. OS가 `MTIMECMP_L0 = MTIME_L0 + TICK_CYCLES`를 실행하여 compare를 미래로 옮겨야 조건이 거짓이 됩니다. 이 정상 재설정 설명은 15.4 - 15.5절의 32비트 wrap 제한에 걸리지 않는 구간을 전제로 합니다.

MRET 자체도 MTIP를 지우지 않습니다. 원인을 제거하지 않고 MRET로 MIE를 복원하면 다음 실행 경계에서 다시 timer trap을 받을 수 있습니다. 그리고 pending bit 하나는 이벤트 큐나 누적 횟수 counter가 아닙니다. MIE를 오래 꺼 두었다고 밀린 tick 수만큼의 요청이 별도로 쌓이는 구조는 아닙니다.

3.31 mtvec와 trap 진입 주소

mtvec의 영문 이름은 Machine Trap-Vector Base-Address Register입니다. trap이 발생했을 때 CPU가 어디에서 handler instruction을 가져와야 하는지를 정합니다. CSR 주소는 `0x305`이며, RAM에 함수 이름을 저장하는 것이 아니라 linker가 결정한 주소 값을 보관합니다.

표준의 구성은 상위 `[31:2]`의 BASE와 하위 `[1:0]`의 MODE로 나뉘어 읽습니다. 실제 byte 단위 base address는 `mtvec & ~3u`입니다. Direct에서는 모든 trap이 BASE로 가고, Vectored에서는 interrupt가 BASE에 cause 번호의 4배를 더한 위치로 갑니다. 동기 예외는 Vectored에서도 BASE로 갑니다. 이 구분은 [mtvec 명세](#)를 읽기 위한 배경입니다.

현재 core는 Direct만 구현합니다. `mtvec_r <= {write_data[31:2], 2'b00}`으로 MODE를 항상 0으로 저장하며, core는 trap 종류와 무관하게 `pc <= csr_mtvec`를 선택합니다. 예를 들어 `0x41`을 써도 `0x40`으로 읽히며 Vectored로 전환되지 않습니다. trap-entry PC를 원인에 따라 계산하는 회로가 없기 때문입니다.

설정 예	ECALL 진입 PC	timer cause 7의 진입 PC
현재 Direct, mtvec=0x40	0x40	0x40
표준 Vectored를 지원하는 별도 CPU, BASE=0x40·MODE=1	0x40	0x40+4×7=0x5C

둘째 행은 현재 보드에서 관측되는 결과가 아니라 비교용 계산입니다. 또한 vector 위치에는 보통 실행할 instruction을 배치합니다. `mtvec=0x40`이라고 해서 CPU가 RAM `[0x40]`에서 함수 포인터를 읽어 다시 점프하는 것은 아닙니다. CPU는 `PC=0x40`에서 곧바로 instruction을 fetch합니다.

현재 부팅 코드는 다음처럼 설정합니다. `trap_entry`의 구체적 주소 `0x40`은 현재 생성물의 값이고, 앞으로 code 배치가 바뀌면 symbol 주소도 바뀔 수 있으므로 소스에는 symbol을 사용합니다.

```
la    t0, trap_entry
csrw  mtvec, t0
```

reset PC와 mtvec도 구분해야 합니다. reset은 core의 `RESET_PC`로 시작하고, 그 뒤 boot code가 mtvec를 초기화합니다. reset 직후 `mtvec=0`인 상태에서 ECALL을 잘못 실행하면 자동으로 trap.S를 찾아가는 것이 아니라 주소 0으로 이동합니다. 그래서 timer enable과 앱 실행보다 trap vector 설정이 앞서야 합니다.

3.32 mepc와 PC·ra·trap frame의 차이

mepc 는 Machine Exception Program Counter, CSR 주소 0x341 입니다. Exception이라는 이름이 들어 있지만 이 프로젝트에서 timer interrupt와 동기 예외 모두의 복귀 위치를 보관합니다. 평소 매 instruction마다 증가하는 현재 PC와는 별개의 register입니다.

core는 trap entry에서 trap_pc=pc 를 CSR block에 전달합니다. 따라서 ECALL이면 ECALL instruction 자신의 주소를, timer이면 아직 실행 결과가 반영되지 않은 현재 instruction의 주소를 저장합니다. “항상 다음 instruction의 PC+4가 hardware에서 저장된다”는 설명은 이 core에 맞지 않습니다.

위치	언제 바뀌는가	복귀에서의 역할
core의 pc	instruction 또는 trap을 선택한 edge	지금 fetch할 위치
CSR의 mepc_r	trap entry 또는 CSR write	다음 MRET가 참조할 복귀 후보
RAM의 frame.mepc	trap.S 저장, C 수정	task별로 보관하는 복귀 후보의 사본
일반 register ra=x1	JAL/JALR 및 software 복원	일반 함수 RET의 복귀 주소

예를 들어 앱의 0x4014에서 ECALL이 발생하면 CSR mepc=0x4014입니다. trap.S는 이 값을 frame offset 0에 SW합니다. C handler의 f->mepc += 4 는 RAM에 있는 사본을 0x4018로 바꿉니다. 그 순간 CSR mepc나 현재 PC가 바뀌는 것은 아닙니다. 복원부의 lw t0,0(sp) 와 csrw mepc,t0 를 거친 후 MRET가 PC를 0x4018로 만듭니다.

timer 경로는 frame.mepc에 4를 더하지 않습니다. 예를 들어 PC=0x400C의 LBU를 timer가 가로챘다면 해당 LBU의 register write는 억제되었으므로 복귀 후 같은 LBU를 실행해야 합니다. timer에도 일괄적으로 +4하면 instruction 하나를 건너뛰는 오류가 됩니다. illegal instruction은 현 OS가 panic 처리하므로 무조건 +4하고 계속 실행하지 않습니다.

정상 프로그램의 instruction 주소는 4바이트 정렬입니다. software가 mepc에 쓸 때는 하위 두 비트를 0으로 만들고, MRET용 출력도 하위 두 비트를 0으로 만듭니다. 다만 trap 저장 경로는 mepc_r <= trap_pc 로 원래 PC를 받습니다. 비정렬 PC의 예외 처리를 구현하지 않은 현재 core에서 잘못된 target까지 정렬 예외로 진단해 준다고 이해하면 안 됩니다.

마지막으로 MRET는 stack에서 PC를 직접 pop하는 명령이 아닙니다. RAM frame을 읽고 mepc에 쓰는 것은 trap.S의 책임이고, MRET는 이미 준비된 mepc와 MPIE를 사용하는 instruction입니다. scheduler가 다른 frame을 반환하면 복원되는 mepc도 다른 task의 값이 되어 context switch가 완성됩니다.

3.33 mcause와 예외 번호·시스템 콜 번호

mcause 는 Machine Cause Register, 역할까지 풀어 쓰면 Machine Trap Cause Register입니다. 주소는 0x342 이고, 가장 최근에 선택한 trap의 원인을 mcause_r 에 저장합니다. 현재 RV32 core에서는 bit 31이 interrupt 여부이며 [30:0] 이 원인 번호입니다.

```
uint32_t is_interrupt = cause >> 31;
uint32_t cause_code   = cause & 0x7fffffffu;
```

현재 core의 값	bit 31	번호	kernel.c의 해석
0x00000002	0	2	Illegal Instruction: 지원하지 않는 명령·CSR 접근 등
0x0000000B	0	11	Environment Call from M-mode: ECALL 서비스 요청

현재 core의 값	bit 31	번호	kernel.c의 해석
0x80000007	1	7	Machine Timer Interrupt: tick 갱신과 스케줄링

0x80000007 은 큰 양의 timer 번호가 아니라 interrupt 표지 0x80000000 과 원인 번호 7을 합한 값입니다. C에서는 uint32_t로 읽어 bit를 해석하면 부호와 혼동하지 않습니다. 또한 같은 번호라도 interrupt bit가 다른 의미입니다. 하위 번호만 보고 7이라고 판정하는 코드는 전체 원인 분류가 아닙니다.

특히 mcause=11 과 a7=1 은 전혀 다른 층의 번호입니다. 11은 CPU가 “M-mode ECALL이었다”고 기록하는 architectural cause이고, 1은 OS가 “SYS_PUTC 서비스를 요청한다”고 약속한 데이터입니다. SYS_PUTC·SYS_YIELD·SYS_FS_READ 모두 ECALL을 쓰면 mcause는 똑같이 11입니다. handler가 frame의 a7을 추가로 읽어 서비스를 고릅니다.

mcause는 요청 상태가 아니라 최근 trap의 기록입니다. timer를 재설정하여 mip.MTIP가 0이 되어도 mcause에는 0x80000007이 남을 수 있습니다. MRET 역시 mcause를 0으로 초기화하지 않습니다. software write나 다음 trap entry가 있을 때 값이 바뀝니다. 그래서 mcause만 보고 “지금 interrupt가 pending이다”라고 판단하면 안 됩니다.

현재 RTL은 mcause software write도 허용하지만 값을 0으로 쓴다고 주변장치가 acknowledge되는 것은 아닙니다. 또한 이 core에서 timer와 ECALL decode가 같은 사이클에 겹치면 irq_pending이 우선하여 먼저 0x80000007을 기록합니다. timer 복귀 후 그 PC의 ECALL이 다시 선택될 때 비로소 11로 바뀝니다. 이 우선순위는 [현재 core의 선택식](#)을 근거로 한 설명입니다.

3.34 mscratch와 trap 진입의 임시 저장

mscratch는 Machine Scratch Register, 주소 0x340 입니다. scratch는 계산 중 잠시 메모해 두는 임시 공간이라는 뜻입니다. 현재 RTL에서는 32비트 전체를 읽고 쓸 수 있으며 reset 값은 0입니다. 이름에 scratch가 들어 있다고 hardware가 trap 때 자동으로 stack pointer를 넣어 주지는 않습니다.

다음은 CSR 교환 instruction의 동작을 확인하기 위한 독립 예제입니다. 현재 trap handler가 mscratch를 사용하지 않는 조건에서, 끝난 뒤 t0=0x12345678, mscratch=0xABCD가 됩니다.

```
li    t0, 0x12345678
csw   mscratch, t0
li    t0, 0x0000abcd
csrrw t0, mscratch, t0
```

마지막 instruction은 이전 mscratch를 t0에 반환하면서 이전 t0를 mscratch에 씁니다. rd와 rs1이 같은 t0여도 operand를 먼저 읽고 edge에서 결과를 반영하므로 교환이 성립합니다. CSR instruction이 일반 register와 특수 상태 사이를 연결한다는 점을 보여 주는 예입니다.

전용 kernel stack을 사용하는 다른 trap 설계에서는 software가 미리 mscratch에 kernel SP를 넣고 csrrw sp, mscratch, sp로 교환하는 방법을 사용할 수 있습니다. 그러면 sp는 준비된 kernel stack을 가리키고 mscratch에는 원래 SP가 남습니다. 이것은 추가 설계 예이며 현재 trap.S의 동작이 아닙니다.

현재 trap.S는 중단된 task의 stack에 바로 128바이트 frame을 만듭니다. 여기에 교환 한 줄만 삽입하면 mscratch 초기값 0을 SP로 가져오거나 기존 frame 구조를 깨뜨릴 수 있습니다. 전용 stack 방식에는 초기 pointer 설정, 중첩 trap 구분, 이전 SP 보존, 복귀 시 교환까지 별도 계약이 필요합니다. CSR가 존재하는 것과 OS가 그 용도를 구현한 것은 구분해야 합니다.

3.35 mvendorid·marchid·mimpid·mhartid의 의미

ID는 **Identifier**, 즉 식별자입니다. 이 CSR들은 interrupt를 제어하기보다 “어떤 실행 엔진인가”를 software가 구분하는 용도로 이해하면 됩니다. 현재 RTL은 네 주소를 인식하지만 모두 상수 0을 반환합니다.

CSR	영문 이름을 풀어 읽기	현재 값 0의 해석
mvendorid	Machine Vendor Identifier	이 core에 vendor 식별 정보를 제공하지 않음
marchid	Machine Architecture Identifier	이 core에 architecture 식별 정보를 할당하지 않음
mimpid	Machine Implementation Identifier	구현 버전·revision 식별 정보를 제공하지 않음
mhartid	Machine Hardware Thread Identifier	현재 하나뿐인 hart를 0으로 식별

hart는 **Hardware Thread**입니다. 여기서는 독립된 PC·register·CSR 상태를 가지고 instruction을 실행하는 hardware 실행 단위로 생각하면 됩니다. 이 프로젝트의 RISC-V hart는 하나이고, scheduler가 그 위에서 software task를 번갈아 수행합니다. 따라서 task 번호 cur가 바뀌어도 mhartid는 변하지 않습니다.

ZCU104의 FPGA vendor가 AMD라고 해서 이 soft-core의 mvendorid에 AMD의 식별 값이 자동으로 들어가지는 않습니다. FPGA의 소자 식별 정보와 그 위에 Verilog로 만든 RISC-V core의 CSR 응답은 별개의 설계입니다. marchid 역시 문자열 RV32I를 반환하는 레지스터가 아닙니다.

이름의 ID가 같아도 쓰임은 다릅니다. vendor는 공급 주체, architecture는 설계 식별, implementation은 그 구현의 revision, hart는 여러 실행 엔진 사이의 구분입니다. 현재의 상수 0 읽기로 알 수 있는 범위를 넘어 제품·버전 정보를 추론해서는 안 됩니다. ID 명칭의 기준은 [공식 CSR 목록](#), 상수 반환의 근거는 rv32_csr.v의 read case입니다.

3.36 이름은 알아 두되 현재 사용할 수 없는 CSR

다른 RISC-V OS나 매뉴얼을 읽으면 아래 이름들도 자주 만납니다. 표는 **추가 학습용 명칭 지도**이며 지원 목록이 아닙니다. 현재 rv32_csr.v에는 아래 주소의 read case가 없으므로 이를 CSR instruction으로 접근하면 illegal instruction 예외가 됩니다.

이름	영문 이름	기능을 이해하는 핵심어
misa	Machine Instruction Set Architecture Register	ISA·extension 정보
mtval	Machine Trap Value Register	trap 원인을 보충하는 값
medeleg	Machine Exception Delegation Register	동기 예외 처리의 위임
mideleg	Machine Interrupt Delegation Register	interrupt 처리의 위임
mcycle	Machine Cycle Counter	실행 cycle 계수
minstret	Machine Instructions-Retired Counter	완료된 instruction 계수
mcounteren	Machine Counter-Enable Register	낮은 privilege의 counter 접근 허용
satp	Supervisor Address Translation and Protection	supervisor 주소 변환 설정
pmpcfgN	Physical Memory Protection Configuration	물리 메모리 보호 설정
pmpaddrN	Physical Memory Protection Address	물리 메모리 보호 영역 주소

위 이름은 [CSR 주소·기능 목록](#)에 대응합니다. N은 register 묶음의 번호를 나타내는 표기입니다. mcounteren을 counter 자체의 증가 on/off라고 읽지 않고 **접근 허용**으로 읽어야 한다는 점에도 주의하십시오.

현재 프로그램이 compiler의 `-march=rv32i`를 사용한다고 해서 hardware의 misa 읽기가 구현되는 것은 아닙니다. `csrr t0,misa`로 자동 탐색하려는 범용 startup code를 가져오면 지금은 cause 2로 멈출 수 있습니다. 특히 현재 misa는 “읽으면 0을 돌려주는 CSR”가 아니라 주소가 인식되지 않는 CSR입니다. ID CSR 4개의 상수 0 응답과 구분해야 합니다.

mtval이 없으므로 현재 panic은 mcause와 저장 PC를 이용합니다. 잘못된 메모리 주소나 instruction word를 mtval에서 읽어 출력하는 일반적인 진단 code는 그대로 쓸 수 없습니다. mcycle·minstret도 없으므로 14장에서 센 애플 명령 시도 수를 hardware 성능 counter의 측정 결과라고 부르지 않습니다.

satp·PMP 설정과 U/S-mode를 추가하는 작업은 이름 몇 개와 32비트 register를 더하는 것으로 끝나지 않습니다. 주소 변환·보호 검사·권한 전환·예외 전달까지 연결해야 합니다. 현재 M-mode 앱이 CSR와 kernel 메모리를 직접 바꿀 수 있다는 한계를 18장의 격리 설계와 연결해서 이해하십시오.

3.37 CSR instruction이 RTL을 통과하는 순서

CSR가 별도의 register라는 설명만으로는 assembly 한 줄이 회로를 어떻게 움직이는지 충분히 보이지 않습니다. `csrsi mstatus,8`을 예로 들면 instruction의 12비트 CSR field는 0x300, funct3는 CSRRSI를 뜻하는 110, zimm은 01000, rd는 x0입니다.

단계	현재 RTL의 값 또는 동작
instruction decode	SYSTEM opcode, funct3=110
CSR 선택	<code>read_addr=insn[31:20]=0x300</code>
조합 읽기	<code>csr_rdata=mstatus_r</code> , <code>csr_valid=1</code>
새 값 계산	<code>csr_wdata</code> 는 이전 <code>csr_rdata</code> 와 0x08의 bitwise OR
write 요청	zimm이 0이 아니므로 <code>csr_we=1</code>
edge 반영	trap이 없으면 mstatus write mask를 적용
일반 register 결과	rd=x0이므로 이전 CSR 값의 반환은 버림

초기 `mstatus=0x80`이면 새 값은 0x88입니다. 기존 MPIE는 보존하고 MIE만 컵니다. 반면 `csrwi mstatus,8`은 set이 아니라 교체 요청이므로 `mstatus=0x08`이 되어 MPIE를 지웁니다. 이런 차이 때문에 이름 마지막의 S(Set), C(Clear), W(Write)를 구분해야 합니다. immediate 형태의 I는 Integer가 아니라 Immediate를 구분하는 접미 문자입니다.

`csrr t0,mcause`는 CSRRS `t0,mcause,x0`로 인코딩됩니다. read case에서 이전 mcause를 내보내고, rs1 field가 x0라서 CSR write는 생략하고, clock edge에서 t0에 그 값을 씁니다. `csrw mie,t0`는 CSRRW `x0,mie,t0`이므로 반대로 t0를 CSR에 보내고 이전 값의 일반 register 반환은 버립니다. 정식 instruction 의미는 [Zicsr 명세](#)를 참고하십시오.

표준 CSRRW/CSRRWI에서 rd=x0은 CSR 읽기 자체를 생략하는 조건이기도 합니다. 현재 구현은 CSR read mux를 항상 조합식으로 평가하지만, 구현된 CSR 읽기에 별도의 부작용이 없으므로 이 예제에서는 차이가 드러나지 않습니다. 향후 read-to-clear 같은 custom CSR를 추가하면 단순히 반환만 버리는 것과 읽기 자체를 하지 않는 것의 차이를 반드시 설계해야 합니다.

timer가 같은 사이클에 선택되면 `csr_we && !take_trap`이 0이 되어 해당 software CSR write는 실행되지 않습니다. 대신 trap entry가 mepc·mcause·mstatus를 갱신합니다. 복귀 후 원래 CSR instruction을 다시 실행하는

구조입니다. mstatus.MIE가 0인 상태에서 1로 켜는 instruction은 먼저 완료될 수 있고, 이미 pending인 timer는 그 다음 실행 경계에서 받아들여집니다.

3.38 부팅·ECALL·timer·MRET를 값으로 연결하기

이제 CSR들을 각각 외우지 않고 하나의 실행 기록으로 연결합니다. 아래 예는 현재 firmware의 순서를 따르며, ECALL 중에는 timer가 새로 발생하지 않고, trap handler가 mstatus를 직접 수정하지 않으며, 주소가 모두 정상 정렬되어 있다고 가정합니다.

순서	일어나는 일	주요 상태
1	reset	mstatus=0, mie=0, mtvec=0
2	boot.S가 trap_entry 설정	mtvec=0x40, 아직 MIE=0
3	kernel이 timer를 미래로 예약	timer_irq=0, mip=0
4	MTIE와 MIE를 차례로 설정	mie=0x80, mstatus=0x08
5	최초 timer trap	mcause=0x80000007, mstatus=0x80
6	handler가 timer 재설정·셀 frame 선택	mip=0, 복원 mepc=셀 entry
7	MRET로 셀 시작	PC=셀 entry, mstatus=0x88
8	이후 앱의 0x4014 ECALL	mepc=0x4014, mcause=11, mstatus=0x80
9	handler가 frame PC를 4 증가	frame.mepc=0x4018, CSR mepc는 아직 0x4014
10	trap.S의 CSR 복원과 MRET	PC=0x4018, mstatus=0x88, mcause=11 유지

표에서 mtvec는 진입 위치, mepc는 복귀 위치, mcause는 이유, mstatus는 허용 상태, mie는 종류별 허용, mip는 장치 요청이라는 역할이 분리됩니다. 한 register가 이 정보를 모두 담는 것이 아닙니다. task stack의 frame은 CSR 일부와 일반 register를 software가 보존하는 RAM 구조입니다.

이후 timer가 PC=0x400C에서 발생하면 mepc=0x400C, mcause=0x80000007, MIE=0·MPIE=1이 됩니다. timer handler가 재예약하고 같은 frame을 복원하면 MRET 후 PC도 0x400C입니다. ECALL 경로의 +4와 비교하면 왜 trap 종류에 따라 OS의 복귀 정책이 다른지 확인할 수 있습니다.

현재 SHELL_MODE에서는 스케줄러가 셀 task를 계속 선택하므로 timer가 발생했다고 앱마다 별도 프로세스로 전환되는 것은 아닙니다. 앱은 셀 task의 함수 호출 안에서 실행합니다. 마지막 ret는 ra를 사용하여 셀의 shell_run() 다음 code로 돌아갑니다. 이 마지막 반환은 MRET가 아니며 mepc를 최종 앱 종료 주소로 사용하는 것도 아닙니다.

3.39 C inline assembly와 interrupt 상태 보존

현재 kernel의 csr_read_mcause()는 C 지역변수에 CSR 값을 가져오는 얇은 연결 함수입니다. C 문법만으로 특수 register를 지정할 수 없으므로 GCC inline assembly를 사용합니다.

```
static inline uint32_t csr_read_mcause(void)
{
    uint32_t v;
    __asm__ volatile ("csrr %0, mcause" : "=r"(v));
    return v;
}
```

%0는 첫 operand 자리, "=r"(v)는 결과를 일반 register로 받아 C 변수 v에 연결하라는 뜻입니다. assembler에 %0라는 실제 RISC-V register가 있는 것은 아닙니다. compiler가 선택한 a0나 t0 등의 이름으로 치환됩니다. static inline은 해당 번역 단위에서 사용할 작은 함수라는 뜻이며 항상 call instruction이 사라짐을 보장하는 표현은 아닙니다.

__asm__ volatile은 compiler가 assembly의 효과를 임의로 불필요하다고 제거하지 않도록 하는 데 필요합니다. 다만 volatile만으로 주변 C 메모리 접근의 모든 재배치가 차단되지는 않습니다. interrupt를 잠시 막아 C 자료구조를 보호하려는 helper라면 compiler에 메모리 경계도 알려야 합니다. 다음은 수업용 추가 예제이며 현재 kernel에 새로 삽입한 code는 아닙니다.

```
static inline uint32_t irq_save(void)
{
    uint32_t old;
    __asm__ volatile ("csrrci %0, mstatus, 8"
                      : "=r"(old) : : "memory");
    return old;
}
```

irq_save()는 한 CSRRCI instruction으로 이전 mstatus를 받고 MIE만 지웁니다. 반환값 old는 이전 MIE가 켜져 있었는지 판단하는 데 사용합니다. 대응하는 복원 함수는 다음과 같습니다.

```
static inline void irq_restore(uint32_t old)
{
    if (old & 8u)
        __asm__ volatile ("csrsi mstatus, 8" : : : "memory");
    else
        __asm__ volatile ("csrci mstatus, 8" : : : "memory");
}
```

irq_restore()는 이전 MIE만 복원하므로 MPIE 등 다른 상태를 전체 값으로 덮어쓰지 않습니다. 처음부터 MIE=0인 호출자도 있을 수 있어 끝에서 무조건 csrsi mstatus, 8을 실행하는 방법은 올바른 상태 복원이 아닙니다. 사용한다면 save가 반환한 값을 같은 논리적 구간의 restore에 넘겨야 합니다.

"memory"는 compiler의 메모리 재배치에 대한 장벽이지 RISC-V FENCE instruction을 자동 생성하는 요청이 아닙니다. operand 제약과 volatile·memory clobber의 의미는 [GCC Extended Asm 문서](#)를 참고하십시오. 또한 이 예는 한 hart에서 timer와 C 임계 구간의 관계를 설명하기 위한 것입니다. ECALL·illegal instruction까지 막지 않으며, 여러 hart·DMA·cache가 있는 시스템의 동기화 전체를 대신하지 않습니다. 현재 shell에 이 helper를 넣지 않아도 기존 실행 기능은 그대로입니다.

3.40 CSR 파형과 증상에서 원인을 찾는 실습

CSR를 관찰할 때는 “wire에서 지금 보이는 값”과 “clock edge 후 저장된 값”을 구분합니다. tb_shell의 dut 아래에서 core instance 이름은 cpu, CSR instance 이름은 csr입니다. 아래 표는 VCD에 포함하면 유용한 관찰 대상 안내이며 기존 testbench가 이 신호를 모두 이미 dump한다는 뜻은 아닙니다.

관찰 대상	확인할 질문
dut.cpu.pc, dut.cpu.take_trap	어느 PC에서 trap이 선택됐는가?
dut.cpu.csr.mstatus_r	bit 3이 내려갈 때 bit 7은 이전 값을 받았는가?
dut.cpu.csr.mie_r	timer 종류별 허용 mask가 0x80인가?
dut.cpu.csr.timer_irq	enable을 끄더라도 요청 level은 남는가?

관찰 대상	확인할 질문
<code>dut.cpu.csr.irq_pending</code>	MIE·MTIE·timer_irq의 AND와 같은가?
<code>dut.cpu.csr.mtvec_r</code> , <code>mepc_r</code> , <code>mcause_r</code>	진입 주소·복귀 주소·원인이 서로 맞는가?
<code>dut.timer.mtime</code> , <code>dut.timer.mtimecmp</code>	재설정 후 comparator가 거짓이 되는가?

`read_data`는 `read_addr`가 선택한 CSR 하나의 mux 결과입니다. instruction이 바뀌면 `read_addr`도 바뀌므로, mux 출력만 계속 보면서 이를 항상 `mcause` 값이라고 해석하면 안 됩니다. 저장 상태를 관찰하려면 해당 `_r` 신호를 보거나, CSRR가 실제로 그 주소를 읽는 구간인지 함께 확인해야 합니다.

timer trap edge 직후에는 PC가 `mtvec`로 바뀌고 `mepc`에는 직전 PC가 남으며 MIE=0이 됩니다. 그 뒤 trap.S의 여러 SW가 frame을 만듭니다. 즉 CSR 저장과 RAM context 저장은 서로 다른 단계입니다. 복귀 때도 `csrw mepc, t0`가 먼저 값을 설정하고, 나중의 MRET edge에서 PC가 이동하는 순서를 찾아보십시오.

증상	먼저 구분할 CSR·회로 상태
timer가 전혀 실행되지 않음	MIE=1인가, MTIE=1인가, MTIP가 실제 1인가?
handler에서 나오자마자 다시 진입	compare 재설정 후에도 MTIP가 1인가?
같은 ECALL 주소가 반복됨	frame.mepc +4와 CSR mepc 복원이 둘 다 수행됐는가?
첫 ECALL 후 주소 0으로 감	boot.S가 mtvec를 설정했는가?
새로 가져온 startup에서 cause 2	misa·mtval 등 미지원 CSR를 읽었는가?
MRET 뒤 timer가 계속 꺼져 있음	MPIE가 0이었는가, handler가 status를 덮어썼는가?

관찰을 위해 새로운 CSRR를 삽입하면 그 instruction 자체가 실행 시간과 PC 배치를 바꿉니다. 특히 미지원 CSR를 무작정 읽어 보는 실험은 panic을 일으킬 수 있으므로 우선 simulator에서 수행합니다. RTL 계층 신호를 관찰하는 방식과 firmware의 읽기 code를 추가하는 방식의 차이를 보고서에 명시하십시오.

4. 주소 공간과 세 가지 스택 개념

4.1 셸 이미지의 메모리 지도

주소 범위/주소	크기	용도
0x00000000 - 0x00003FFF	16 KiB	커널·셸 코드, 상수, 전역 데이터, 셸 task stack
0x00004000 - 0x00005FFF	8 KiB	앱 업로드 임시 버퍼와 앱 실행 영역
0x00006000 - 0x00007FFF	8 KiB	부팅·idle 문맥의 커널 스택 예약 영역
0x10000000	레지스터	UART TX 데이터 쓰기
0x10000004	레지스터	UART TX ready 읽기
0x10000008	레지스터	UART RX 데이터 읽기, FIFO 소비
0x1000000C	레지스터	UART RX ready 읽기
0x10001000	레지스터	mtime 하위 32비트
0x10001004	레지스터	mtimecmp 하위 32비트
0x80100000 - 0x80101FFF	8 KiB	MiniFS RAM disk

firmware/linker_shell.ld는 커널의 .text, .data, .bss를 하위 16 KiB 안에 배치하고 _stack_top=0x8000을 정의합니다. 마지막 RAM byte는 0x7FFF이지만, 아래로 자라는 빈 스택의 초기 SP는 그 다음 주소인 0x8000이어도 됩니다. 첫 stack allocation이 SP를 낮추기 때문입니다.

주소 공간 4 GiB가 모두 RAM이라는 뜻은 아닙니다. 32비트 주소로 표현할 수 있는 범위 안에서 decoder가 일부 구간만 장치에 연결합니다. RAM disk의 높은 시작 주소 0x80100000은 많은 실제 RAM을 사이에 배치했다는 뜻이 아니라, 별도 장치를 구분하기 쉬운 주소를 선택한 것입니다.

linker의 MEMORY 선언도 FPGA resource를 만들지 않습니다. linker는 코드·데이터 배치를 검사하고 RTL은 실제 storage를 제공합니다. 둘이 일치해야 실행됩니다. _bss_end <= 0x4000 assertion은 하위 정적 영역이 앱 창과 겹치지 않도록 검사하지만, 실행 중 stack이 넘쳐 흐르는 문제까지 정적으로 막아 주지는 않습니다.

4.2 부팅 스택과 셸 task 스택은 다르다

부팅 시 boot.S가 설정하는 sp=0x8000은 kernel_main()과 초기 idle 문맥이 사용합니다. 반면 셸 task 스택은 다음 배열의 한 행입니다.

```
#define STACK_WORDS 256
static uint32_t stacks[NTASK - 1][STACK_WORDS]
    __attribute__((aligned(16)));
```

한 task의 스택은 $256 \times 4 = 1024$ 바이트입니다. 셸 구성은 NTASK=2이므로 idle을 제외한 한 개의 배열 스택을 둡니다. 이 배열은 .bss에 있으므로 하위 16 KiB 영역에 들어갑니다.

현재 확인한 mini_shell.elf에서는 stacks=0x1840, _bss_end=0x1C40입니다. 셸 스택은 [0x1840, 0x1C40)이고 초기 SP는 0x1C40입니다. 이 숫자는 컴파일 결과의 관측값이며 소스가 바뀌면 달라집니다. 고정된 계약은 배열 크기와 linker 경계입니다.

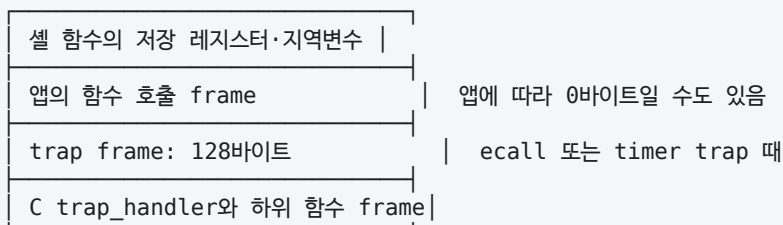
초기 셸 frame의 주소는 예제 symbol 기준 $0x1C40 - 0x80 = 0x1BC0$ 입니다. `task_sp[1]` 에는 이 frame 시작 주소가 저장됩니다. 첫 복원에서 `sp=0x1BC0` 을 선택하고 마지막에 128을 더하면 `sp=0x1C40` 으로 셸 함수를 시작합니다. “task_sp에는 언제나 함수가 보는 현재 SP가 저장된다”고 생각하면 128바이트 차이를 설명할 수 없습니다.

idle stack은 배열 `stacks` 의 slot 0이 아닙니다. `task_sp[0]` 은 첫 interrupt 때 부팅 stack 위에 만들어진 frame을 가리킵니다. 배열은 `NTASK-1` 개만 선언되므로 slot 1은 `stacks[0]` 에 대응합니다. task 번호, 배열 인덱스, 실제 stack 주소를 구분하면 `task_create`의 `slot-1` 도 자연스럽게 이해됩니다.

4.3 앱이 사용하는 스택

`run` 은 새 스택을 만들거나 SP를 `0x6000` 으로 옮기지 않습니다. 셸 task가 일반 함수 호출 방식으로 앱에 진입하므로 앱은 셸의 1 KiB 스택을 이어서 사용합니다. 앱이 syscall을 발생시키면 trap frame과 C handler의 stack frame도 이 스택에 놓입니다.

높은 주소: 셸 stack top



낮은 주소: 스택 성장 방향 ↓

8 KiB 앱 영역은 코드·포함된 데이터용 공간입니다. 앱이 8 KiB 전용 스택을 가진다는 뜻이 아닙니다. 큰 지역 배열, 깊은 재귀, 긴 함수 호출 체인은 1 KiB의 공유 스택을 넘을 수 있으며 현재는 guard page나 stack overflow trap이 없습니다.

스택에는 함수의 반환 주소가 항상 자동 저장되는 것은 아닙니다. JAL/JALR는 먼저 ra register를 갱신하고, compiler가 필요하다고 판단할 때 SW로 stack에 저장합니다. leaf 함수는 stack을 전혀 사용하지 않을 수도 있지만, trap은 언제 발생할지 모르므로 최소 128바이트와 handler 호출 여유를 별도로 고려해야 합니다.

스택 사용량을 분석할 때는 `task_shell` → `shell_run` → `app_main`이라는 C의 호출 구조와 compiler의 inline 결과를 함께 보아야 합니다. 최적화로 일부 함수가 합쳐져 symbol이 없어질 수 있습니다. 실습 확장에서는 GCC의 stack 사용량 보고와 ELF prologue의 `addi sp, sp, -N` 을 대조하고, 최대 호출 깊이에 trap 비용을 더해 예산을 잡을 수 있습니다. 큰 지역 buffer 하나만 보지 말고 동시에 살아 있는 frame의 합을 평가해야 합니다.

4.4 실행 이미지와 저장 파일의 수명 비교

동일한 앱이 세 곳에 존재할 수 있습니다. host의 `.bin`, MiniFS의 APP1 파일, unified RAM의 실행 image입니다. 하나를 바꾸거나 지워도 다른 복사본이 자동으로 모두 바뀌지는 않습니다.

동작	host .bin	MiniFS hello.app	실행 RAM 0x4000
make hello-app	새로 생성/갱신	변화 없음	변화 없음
upload 성공	읽기만 수행	header+payload로 교체	업로드 임시 buffer 내용
run hello.app	사용하지 않음	읽기만 수행	payload만 다시 복사
앱이 .data 수정	변화 없음	변화 없음	해당 byte 변경

동작	host .bin	MiniFS hello.app	실행 RAM 0x4000
rm hello.app	변화 없음	table entry 해제	이전 실행 byte가 남을 수 있음
PL 재구성	변화 없음	초기화	초기 펌웨어 이미지 상태

파일 삭제 후에도 실행 RAM에 machine code가 남을 수 있지만 `run` 은 파일부터 읽으므로 삭제한 이름으로 정상 재실행하지 못합니다. 반대로 host `.bin` 을 지워도 이미 MiniFS에 저장된 파일은 보드 전원이 유지되는 동안 `run` 할 수 있습니다. **파일 저장과 실행 image의 수명을 분리하는 것이 loader 설계의 기본입니다.**

현재는 별도 permission이 없으므로 임의의 함수 pointer가 실행 RAM 잔여 내용을 호출하는 것까지 막지는 않습니다. 그러나 정상 셸 인터페이스의 실행 경로는 항상 파일 검증을 거칩니다. interface의 동작 규칙과 모든 machine instruction에 대한 보호는 다른 수준의 보장입니다.

4.5 static 배열이 실제 FPGA 스택이 되는 과정

stacks는 처음부터 특별한 하드웨어 스택인 것이 아니라, 링커가 주소를 배정한 RAM 배열입니다. OS가 `sp`를 그 공간으로 옮기면 CPU가 그 배열을 스택으로 사용합니다. C 변수 이름을 다른 이름으로 바꾸어도 동작 원리는 같습니다. CPU는 C의 배열 이름을 알지 못하고 주소와 `load/store` 명령만 봅니다.

출처: `firmware/kernel.c`의 스택 선언. 주석을 제외한 핵심 코드입니다.

```
#define STACK_WORDS 256
static uint32_t stacks[NTASK - 1][STACK_WORDS]
    __attribute__((aligned(16)));
```

코드 요소	줄별 의미와 메모리에 미치는 영향
<code>#define STACK_WORDS 256</code>	전처리 상수이며 256바이트가 아니라 256개 word를 뜻합니다.
<code>uint32_t</code>	원소 하나가 4바이트이므로 한 행은 1,024바이트입니다.
<code>[NTASK - 1]</code>	부팅 스택을 사용하는 idle을 제외한 태스크 수입니다. 셸 모드에서는 한 행, 일반 모드에서는 세 행입니다.
파일 범위 <code>static</code>	정적 저장 기간을 가지며 이름은 이 C 번역 단위 안에서만 연결됩니다. 함수 진입 때 생성하거나 반환 때 없애는 지역 스택 변수가 아닙니다.
명시적 초기값 없음	정적 객체의 초기값은 0입니다. 현재 도구 체인은 이 배열을 <code>.bss</code> 에 배치합니다.
<code>aligned(16)</code>	시작 주소를 16의 배수로 정렬합니다. 16바이트만 확보한다는 뜻이 아니며, 배열 크기 자체는 변하지 않습니다.

주소를 결정하는 것은 linker script와 입력 객체의 배치입니다. `firmware/linker_shell.ld`의 관련 부분은 다음과 같습니다.

```
.bss (NOLOAD) : ALIGN(4) {
    _bss_start = .; *(.bss .bss.*) *(COMMON)
    . = ALIGN(4); _bss_end = .;
} > KRAM
```

첫 줄은 `.bss` 출력 구간을 선언합니다. 둘째 줄은 시작 위치를 기록하고 입력 객체의 미초기화 정적 데이터를 모읍니다. 셋째 줄은 끝을 정렬하여 `_bss_end`를 기록합니다. 마지막의 `> KRAM`은 커널용 메모리 영역에 배치하라는 뜻입니다.

입력 배열이 요구하는 16바이트 정렬도 함께 반영됩니다.

NOLoad 구간도 실행 중 RAM을 차지합니다. 다만 파일에 배열 크기만큼 0을 전부 담아 전송하는 방식이 아니라, 부팅 코드가 `_bss_start` 부터 `_bss_end` 직전까지 0을 기록합니다. 따라서 “HEX 파일에 1 KiB의 스택 데이터가 없으니 스택도 없다”는 해석은 잘못입니다. 이 공간은 heap의 `malloc()` 으로 확보한 것도 아닙니다.

다음 명령은 주소를 확인하기 위한 읽기 전용 검사입니다.

```
riscv64-unknown-elf-nm -n build/firmware/mini_shell.elf
riscv64-unknown-elf-readelf -SW build/firmware/mini_shell.elf
```

이 문서 갱신 시 확인한 기존 ELF에는 다음 symbol이 있습니다. 이는 보드 RAM을 실시간으로 읽은 결과나 이번에 새로 빌드한 결과가 아니라, 디스크에 존재하는 ELF의 배치 정보입니다.

```
00001830 B _bss_start
00001840 b stacks
00001c40 B _bss_end
00008000 A _stack_top
```

nm의 소문자 `b`는 local BSS symbol을 뜻합니다. `stacks[0][255]`의 시작 주소는 $0x1840 + 255 * 4 = 0x1C3C$ 이며 그 원소의 마지막 바이트는 `0x1C3F`입니다. 끝 주소 `0x1C40`은 배열 밖의 첫 위치입니다. C에서는 배열 끝의 다음 위치를 나타내는 포인터를 만들 수 있지만 그 위치의 원소를 읽거나 써서는 안 됩니다. 스택은 먼저 `sp`를 낮춘 뒤 저장하므로 초기 `sp`로 이 주소를 사용할 수 있습니다.

실제 저장 장치는 `rtl/rv32_soc.v`의 다음 배열입니다.

```
reg [31:0] mem [0:MEM_WORDS-1];
```

CPU 주소 `0x1840`은 word 인덱스 $0x1840 / 4 = 0x610$ 에 해당합니다. 따라서 태스크 스택의 첫 word도 `mem[0x610]`이라는 일반 프로그램/데이터 RAM 위치입니다. 이 프로젝트에는 주소 변환 MMU가 없으며 이 주소를 호스트 Linux의 RAM 주소나 ZCU104 ARM 프로세서의 DDR 주소로 해석해서는 안 됩니다.

4.6 sp와 저장 명령을 주소로 따라가기

다음은 배열 끝을 초기 `sp`로 사용했을 때의 교육용 명령 예입니다. 특정 함수의 실제 역어셈블을 그대로 인용한 것은 아닙니다.

```
addi sp, sp, -16
sw ra, 12(sp)
```

명령	주소 계산	의미
<code>addi sp, sp, -16</code>	$0x1C40 - 16 = 0x1C30$	기존 RAM 안에서 함수용 16바이트를 예약합니다.
<code>sw ra, 12(sp)</code>	$0x1C30 + 12 = 0x1C3C$	배열의 마지막 word 위치에 <code>ra</code> 값을 기록합니다.

하드웨어가 별도 stack memory로 전환되는 것은 아닙니다. `sp`도 `x2`라는 일반 레지스터이고, `sw`는 다른 주소에 쓸 때와 같은 데이터 메모리 경로를 사용합니다. ABI가 `x2`를 스택 포인터로 사용하기로 약속했고 compiler와 OS가 그 약속을 지키는 것입니다.

16바이트 정렬은 호출 규약을 지키기 위한 조건입니다. 이것이 1 KiB 용량 초과를 방지하지는 않습니다. 스택이 낮은 경계 0x1840 아래로 자라면 주변 정적 데이터를 손상할 수 있습니다. 현재 core에는 이 배열의 경계를 검사하는 stack 전용 보호 장치가 없습니다.

4.7 slot에서 1을 빼는 이유

`uint32_t *top = &stacks[slot - 1][STACK_WORDS];` 에서 slot은 태스크 번호이고 첫 번째 배열 첨자는 별도로 확보한 스택의 번호입니다. 두 번호가 다른 이유는 idle 태스크가 이미 준비된 부팅 스택을 사용하기 때문입니다. idle도 실행 문맥은 필요하지만 stacks 배열의 한 행을 추가로 배정받지는 않습니다.

태스크 slot	일반 모드 역할	실제 스택	저장된 frame 포인터
0	idle	부팅 시 sp=0x8000으로 시작한 스택	task_sp[0]
1	첫 번째 작업 태스크	stacks[0]	task_sp[1]
2	두 번째 작업 태스크	stacks[1]	task_sp[2]
3	세 번째 작업 태스크	stacks[2]	task_sp[3]

셸 모드는 slot 0인 idle과 slot 1인 shell만 있습니다. 따라서 stacks는 한 행이고, 셸의 스택은 stacks[0]입니다. task_sp는 모든 태스크를 위한 포인터 배열이므로 NTASK개이고, stacks는 idle을 제외한 실제 저장 공간이므로 NTASK-1개라는 차이가 있습니다.

```
static struct frame *task_sp[NTASK];
static uint32_t stacks[NTASK - 1][STACK_WORDS]
    __attribute__((aligned(16)));
```

첫 선언은 주소를 담는 슬롯을 만들고, 둘째 선언은 실제 스택 데이터가 놓일 공간을 만듭니다. task_sp[slot]은 태스크 번호를 그대로 쓰지만 stacks[slot - 1]은 부팅 스택을 사용하는 idle을 제외한 번호를 씁니다. 현재 task_create는 slot 1부터 NTASK-1까지를 생성하는 용도이며 slot=0을 전달하면 stacks[-1]을 계산하므로 잘못입니다. 함수가 자동으로 범위를 보정해 주는 것은 아닙니다.

4.8 STACK_WORDS가 256인 이유와 크기 변경의 조건

256은 RISC-V ISA나 ABI가 정한 태스크 스택 크기가 아닙니다. 현재 프로젝트에서 제한된 FPGA RAM 안에 태스크당 1 KiB를 예약하기 위해 선택한 값입니다. uint32_t 하나가 4바이트이므로 256 word가 1,024바이트입니다. 16바이트 정렬 조건과 전체 스택 용량은 다른 설계 항목입니다.

STACK_WORDS	태스크당 크기	일반 모드 세 행의 합	셸 모드 한 행의 합
256	1 KiB	3 KiB	1 KiB
512	2 KiB	6 KiB	2 KiB
1024	4 KiB	12 KiB	4 KiB

예약량의 식은 $(NTASK - 1) * STACK_WORDS * \text{sizeof}(\text{uint32_t})$ 입니다. 크기를 늘리면 .bss가 커지고 그 뒤의 배치가 바뀔 수 있습니다. 셸 구성에서는 코드·전역 데이터·태스크 스택을 합쳐 커널용 하위 16 KiB 안에 들어가야 하므로 linker의 경계 검사와 ELF 배치를 다시 확인해야 합니다. 매크로 수정만으로 이미 FPGA에서 실행 중인 이미지가 바뀌지는 않으므로 펌웨어 재빌드와 보드 반영도 필요합니다.

충분한 크기는 코드가 실제로 요구하는 최대 동시 사용량으로 판단합니다. 예를 들어 함수의 `volatile uint32_t buffer[200]` 배열 자체가 800바이트이며, 호출자 공간과 trap frame 128바이트, C handler의 스택을 더하면 1 KiB를 초과할 수 있습니다. 현재는 경계 보호가 없어 초과 시 주변 데이터를 손상할 수 있습니다. trap frame이 전체의 12.5%라는 계산만으로 나머지 87.5%를 모두 앱 지역변수에 배정해서는 안 됩니다. 실제 사용 예는 8.12절에서 추적합니다.

5. 리셋에서 셸 프롬프트까지

5.1 구성 시 초기화와 CPU reset의 차이

FPGA 구성 시 `rv32_soc`의 초기화 코드가 프로그램 RAM과 RAM disk를 0으로 초기화하고 `MEM_HEX`를 프로그램 RAM에 적재합니다. 합성에서는 이 초기값이 FPGA 구성 데이터에 반영됩니다. FPGA가 켜질 때 호스트의 파일 시스템에서 `.hex`를 다시 읽는 것은 아닙니다.

CPU reset은 PC·CSR·주변 상태를 초기화하지만 RAM disk 내용을 지우는 RTL 경로는 없습니다. `boot.S`는 `.bss`를 지우며, `fs_init()`은 RAM disk의 superblock이 유효하면 기존 파일을 유지합니다. 이 차이는 실습에서 매우 중요합니다.

동작	프로그램 실행 상태	MiniFS 파일
screen 접속·종료	CPU는 계속 실행	유지
SW20 CPU reset	부팅 코드부터 다시 실행	유효한 파일 시스템이면 유지
PL 비트스트림 재다운로드	구성 초기 상태부터 실행	초기화됨
전원 차단	실행 중단	휘발성 RAM 내용 소실

CPU reset은 모든 `.data`를 원본 이미지로 되돌리는 일반적인 프로그램 재적재와도 다릅니다. 현재 부팅 코드는 `.bss` 초기화만 수행하므로 앞으로 수정 가능한 초기값 전역변수를 추가할 때는 warm reset 정책도 검토해야 합니다.

초기화 주체도 구분해야 합니다. RTL `initial`의 RAM 초기값은 FPGA configuration 경로의 일이고, `boot.S`의 zero loop는 RISC-V CPU가 SW 명령으로 수행하는 일입니다. 후자는 CPU reset 때마다 다시 실행되지만 전자는 CPU reset만으로 반복되지 않습니다. 이 차이가 `.bss`는 지워지고 RAM disk는 유지되는 동작을 만듭니다.

`fs_init()`이 기존 파일을 보존하는 판단은 superblock 네 word가 맞는지에 달려 있습니다. table 전체의 모든 범위나 checksum을 검증하는 mount 과정은 아닙니다. 따라서 reset 후 magic이 맞다고 손상된 모든 metadata가 복구되는 것은 아닙니다. 현재 실습의 reset 보존은 정상적으로 기록된 파일 시스템을 전제로 합니다.

5.2 boot.S의 초기화 순서

리셋 PC는 `0x00000000`입니다. linker가 `_start`를 그 위치에 배치합니다.

```
PC=0 → _start
1. sp ← _stack_top
2. [_bss_start, _bss_end)를 0으로 채움
3. mtvec ← trap_entry
4. kernel_main 호출
```

스택을 먼저 준비해야 C 함수가 지역변수와 반환 주소를 저장할 수 있습니다. `.bss`를 지워야 `ticks`, `cur`, `task pointer`와 배열이 정상 초기 상태가 됩니다. `mtvec`를 먼저 설정해야 이후 interrupt나 `ecall`이 올바른 handler로 이동합니다.

zero loop의 `bgeu t0,t1,2f`는 시작 주소가 끝 주소에 도달하면 끝낸다는 unsigned 주소 비교입니다. `sw zero,0(t0)` 뒤 `addi t0,t0,4`로 한 word씩 지우며, `j 1b`는 앞에 있는 숫자 label 1로 되돌아갑니다. `2f`의 `f`는

forward, 1b의 b는 backward입니다. linker가 끝을 4바이트 경계로 맞추므로 word 반복이 경계를 안전하게 처리할 수 있습니다.

mtvec는 일반 함수 포인터와 비슷한 주소를 담지만 CPU가 trap 때 직접 참조하는 CSR입니다. call kernel_main 이전에 설정해 두면 이후 예외 진입 경로가 정의됩니다. 다만 처음 몇 명령이 실패하는 경우까지 완전히 복구하는 boot 구조는 아닙니다. 초기 SP·정렬·메모리 이미지가 옳다는 기본 전제 위에서 다음 환경을 단계적으로 구성합니다.

코드 해부 5-A. boot.S의 모든 실행 줄 따라가기

출처: [firmware/boot.S](#), 원본 13 - 32행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

13 |     .option norvc
14 |     .section .text.start, "ax", @progbits
15 |     .globl _start
16 | _start:
17 |     la    sp, _stack_top
19 |     la    t0, _bss_start
20 |     la    t1, _bss_end
21 | 1:
22 |     bgeu  t0, t1, 2f
23 |     sw    zero, 0(t0)
24 |     addi  t0, t0, 4
25 |     j     1b
26 | 2:
27 |     la    t0, trap_entry
28 |     csrw  mtvec, t0
30 |     call  kernel_main
31 | 3:
32 |     j     3b

```

원본 행	한 줄씩 읽는 동작과 의미
13	압축 명령 생성을 사용하지 않도록 assembler에 알립니다. 현재 CPU는 32비트 instruction만 인출합니다.
14	부팅 코드를 실행 가능한 .text.start section에 둡니다. linker가 이 section을 reset 위치에 배치합니다.
15	다른 object와 linker가 _start symbol을 볼 수 있게 합니다.
16	reset 진입 label입니다. label 자체가 실행되는 명령어는 아닙니다.
17	linker가 정한 stack top 주소를 sp에 넣습니다. 메모리에 저장된 값을 LW하는 것이 아니라 주소를 구성하는 pseudo-instruction입니다.
19	0 초기화할 .bss 시작 주소를 t0에 준비합니다.
20	끝 주소를 t1에 준비합니다. 끝 주소 자체는 초기화 범위에 포함하지 않습니다.
21	반복문의 숫자 label입니다. 아래 1b는 이 label로 되돌아옵니다.
22	unsigned 주소 비교로 t0가 끝에 도달했는지 확인합니다. 참이면 앞으로 있는 label 2로 갑니다.
23	현재 word를 0으로 씁니다. CPU가 실제 SW를 반복하며 RAM을 지우는 단계입니다.
24	다음 4바이트 word로 주소를 진행시킵니다.
25	반복문 처음으로 돌아갑니다. j는 별도 새로운 ISA 명령이 아니라 jump pseudo-instruction입니다.
26	초기화 완료 위치입니다.
27	trap_entry의 링크 주소를 준비합니다. trap.S 파일 이름을 하드웨어가 검색하는 것이 아닙니다.
28	준비한 주소를 mtvec CSR에 씁니다. 이후 trap의 PC 목적지가 정해집니다.

원본 행	한 줄씩 읽는 동작과 의미
30	C 커널 함수로 일반 호출합니다. 이 시점에는 SP·0 초기화 상태·trap vector가 준비되어 있습니다.
31	커널이 예상과 달리 반환했을 때의 정지 지점입니다.
32	자기 자신으로 분기하여 임의 메모리를 실행하지 않게 합니다.

세 책임을 나누어 기억하십시오. FPGA 구성은 초기 machine code를 놓고, boot.S는 C 실행 환경을 만들고, kernel_main은 task와 timer를 준비합니다.

5.3 가짜 초기 trap frame으로 셸 시작하기

kernel_main()은 fs_init() 다음에 task_create(1, task_shell)을 실행합니다. task_create()는 셸 스택 꼭대기에 128바이트 frame을 만들고, mepc 슬롯에 task_shell 주소를 넣습니다. ra 슬롯에는 task 함수가 반환했을 때 머물 task_exit 주소를 넣습니다.

이 frame은 실제 interrupt가 저장한 것은 아닙니다. 그러나 형식이 같으므로 trap 복귀 코드를 재사용할 수 있습니다. 초기 timer interrupt에서 scheduler가 이 frame을 선택하면 복원 코드의 mret가 최초로 task_shell에 진입합니다.

이 방법은 scheduler가 “처음 시작”과 “중단 후 재개”를 동일한 restore 함수로 처리하게 합니다. 처음에는 mepc를 entry 주소로 만든 인공 frame을, 다음부터는 trap.S가 저장한 실제 frame을 사용합니다. task마다 별도 시작용 jump 코드를 많이 두지 않아도 됩니다.

ra=task_exit는 task entry가 실수로 반환했을 때 잘못된 주소로 가지 않도록 하는 최소 처리입니다. 이 함수는 현재 무한 loop일 뿐 자원 해제나 task 제거를 수행하지 않습니다. 앱의 return은 이 경로가 아니라 shell_run의 호출 지점으로 돌아갑니다. task의 최초 entry return과 앱 함수 return을 구분해야 합니다.

코드 해부 5-B. task_create가 첫 복귀용 frame을 만드는 방법

출처: [firmware/kernel.c](https://github.com/riscv-mini-shell/firmware/blob/master/kernel.c), 원본 254 - 266행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

254 | static void task_create(int slot, void (*entry)(void))
255 | {
256 |     uint32_t *top = &stacks[slot - 1][STACK_WORDS];
257 |     struct frame *f = (struct frame *)((uint8_t *)top - sizeof(struct frame));
258 |     int i;
259 |     for (i = 0; i < 31; i++)
260 |         f->x[i] = 0;
261 |     f->mepc = (uint32_t)entry;
262 |     REG(f, 1) = (uint32_t)task_exit;
263 |     REG(f, 2) = (uint32_t)top;
264 |     task_sp[slot] = f;
265 | }
266 |

```

원본 행	한 줄씩 읽는 동작과 의미
254	task slot 번호와 최초 실행 함수의 주소를 받습니다. entry를 여기서 일반 함수 호출하지는 않습니다.
255	앞에서 선언한 함수·블록의 본문을 시작합니다.
256	해당 task stack 배열의 끝 바로 다음 주소를 얻습니다. slot 1은 stacks[0]에 대응하며 이 pointer 자체는 역참조하지 않습니다.

원본 행	한 줄씩 읽는 동작과 의미
257	byte pointer로 바꾼 뒤 128바이트를 빼 frame 공간을 예약합니다. uint32_t pointer 상태로 128을 빼면 512바이트가 되므로 cast가 중요합니다.
258	초기화 loop용 변수를 선언합니다.
260	x1부터 x31에 대응하는 31개 슬롯을 순회합니다.
261	초기 register 값을 0으로 설정합니다. 이 줄은 아직 실제 CPU register를 바꾸지 않고 RAM frame을 작성합니다.
262	MRET가 처음 선택할 PC를 entry 함수 주소로 정합니다.
263	task 함수가 반환할 경우의 ra를 task_exit로 준비합니다.
264	원래 SP를 기록합니다. 실제 복원은 frame base에 128을 더하는 trap.S 규칙을 사용합니다.
265	scheduler가 이 frame을 찾을 수 있도록 task_sp 배열에 pointer를 등록합니다.
266	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

새 task는 아직 실행된 적이 없지만 저장된 task와 같은 RAM 형식을 갖습니다. 따라서 첫 timer에서 이 frame을 선택하면 기존 복원 코드만으로 entry에 진입합니다.

5.4 timer를 켜 뒤 나타나는 두 메시지

kernel_main()은 mini shell boot를 출력하고 다음 tick을 예약합니다. mie.MTIE=1, mstatus.MIE=1로 timer interrupt를 허용한 뒤 idle loop에 머뭙니다. 첫 tick이 쉘 task를 시작시키면 쉘이 다음을 출력합니다.

```
Mini Shell ready. Type help.
rv>
```

이 메시지는 출력된 시점의 UART 데이터입니다. 나중에 터미널을 접속해도 자동으로 재전송되지 않습니다. Enter를 보내면 빈 명령행 처리 후 새 프롬프트가 출력되므로 현재 쉘 상태를 확인할 수 있습니다.

timer에는 두 단계 enable이 있습니다. mie의 MTIE는 machine timer라는 원인을 허용하고, mstatus의 MIE는 전역 허용입니다. 실제 조건은 MIE && MTIE && timer_irq입니다. compare를 먼저 미래로 잡은 뒤 enable을 켜야 초기화 도중 원치 않는 interrupt에 들어갈 가능성을 줄일 수 있습니다.

부팅 문자열은 kernel_main에서 직접 UART 함수를 호출하고, 쉘 프롬프트는 task가 SYS_PUTC를 반복 호출하는 경로로 출력됩니다. 화면에는 연속된 두 문자열처럼 보어도 실행 문맥과 진입 경로가 다릅니다. 첫 메시지만 보이고 두 번째가 보이지 않는다면 timer enable, 초기 frame, MRET 경로를 의심하는 근거가 됩니다.

5.5 최초 task 시작의 PC·SP 변화표

다음 표는 관측한 symbol 주소를 사용한 개념 추적입니다. kernel_main의 정확한 현재 SP는 compiler frame과 interrupt 시점에 따라 달라지므로 KSP로 표시합니다. task stack top과 trap vector는 현재 산출물의 값입니다.

시점	PC/실행 위치	SP와 frame	의미
reset 해제	_start=0x0	register reset 값에서 시작	아직 C 실행 환경 없음
la sp, _stack_top 이후	boot.S	SP=0x8000	boot stack 준비

시점	PC/실행 위치	SP와 frame	의미
task_create 완료	kernel_main	task_sp[1]=0x1BC0	셀 최초 frame 미리 생성
첫 timer trap 직전	idle loop	SP=KSP	boot/idle 문맥 실행 중
trap frame 저장	trap_entry=0x40	frame=KSP-128	idle register와 PC 보존
schedule 반환	trap.S	a0=0x1BC0	slot 1 선택
frame 복원 완료	MRET 직전	SP=0x1C40	셀의 초기 SP 준비
MRET 직후	task_shell entry	SP=0x1C40	셀 함수의 prologue 시작

실제 task_shell이 시작하면 compiler가 자신의 frame 크기만큼 SP를 더 낮출 수 있습니다. 따라서 C 본문의 SP가 항상 0x1C40인 것은 아닙니다. 최초 entry에서의 SP, 함수 prologue 이후의 SP, trap 진입 이후의 SP를 서로 다른 관측 시점으로 나누어 기록하십시오.

이 구조에서 첫 시작도 MRET를 사용한다는 점이 중요합니다. 셀 함수가 일반 `call task_shell`로 시작되는 것이 아니라 scheduler가 선택한 artificial frame을 복원하여 시작됩니다. 이후 앱 호출은 셀 함수 내부의 일반 JALR이므로 두 시작 경로를 비교해 볼 수 있습니다.

5.6 entry와 task_exit의 주소를 프레임에 넣는 이유

출처: `firmware/kernel.c`의 `task_create()`. 아래는 함수의 핵심 동작을 주석 없이 발췌한 것입니다.

```
static void task_create(int slot, void (*entry)(void))
{
    uint32_t *top = &stacks[slot - 1][STACK_WORDS];
    struct frame *f = (struct frame *)((uint8_t *)top - sizeof(struct frame));
    int i;
    for (i = 0; i < 31; i++)
        f->x[i] = 0;
    f->mepc = (uint32_t)entry;
    REG(f, 1) = (uint32_t)task_exit;
    REG(f, 2) = (uint32_t)top;
    task_sp[slot] = f;
}
```

코드 줄	역할
함수 선언	entry 는 인수도 반환값도 없는 함수를 가리키는 포인터입니다. slot 은 생성할 태스크 번호입니다.
top = ...	slot 1에 대해 stacks의 0번 행 끝 다음 주소를 구합니다. 아직 CPU의 실제 sp를 바꾸지 않습니다.
struct frame *f = ...	top에서 128바이트를 빼 초기 프레임 위치를 정합니다. uint8_t *로 바꾸었으므로 뺄셈 단위는 바이트입니다.
int i 와 반복문	x1부터 x31까지의 초기 저장 슬롯을 모두 0으로 채웁니다.
f->mepc = ...	최초 실행할 함수의 주소를 메모리 속 mepc 슬롯에 기록합니다. 이 대입 자체는 CSR 쓰기가 아닙니다.
REG(f, 1) = ...	x1, 즉 ra의 초기 저장값을 task_exit 주소로 바꿉니다.
REG(f, 2) = ...	원래 sp에 해당하는 기록용 슬롯을 준비합니다. 현재 복원 코드는 이 슬롯을 직접 읽어 sp로 복원하지 않습니다.
task_sp[slot] = f	스케줄러가 선택할 수 있도록 프레임의 주소를 기록합니다. 프레임 전체를 복사하지 않습니다.

`task_create(1, task_shell)`을 호출하면 `entry`는 `task_shell`을 가리킵니다. `task_shell`처럼 괄호 없는 함수 이름을 인수로 쓰면 함수 포인터가 전달됩니다. `task_shell()`처럼 괄호를 붙이면 지금 함수를 호출하는 표현식이므로 의미가 다릅니다.

`uint32_t`는 `<stdint.h>`의 32비트 부호 없는 정수형입니다. `(uint32_t)entry`는 새로운 함수나 변수를 정의하는 문법이 아니라, 함수 포인터 값을 프레임의 32비트 슬롯에 담기 위한 명시적 형 변환입니다. 현재 RV32 도구 체인과 주소 모델에서 사용하는 방식이며, 다른 플랫폼에서도 항상 성립하는 이식 가능한 함수 포인터 표현이라고 일반화해서는 안 됩니다. 특히 64비트 포인터를 이렇게 변환하면 주소가 잘릴 수 있습니다.

초기 `mepc`는 태스크가 시작할 곳을, 초기 `ra`는 태스크 함수가 반환할 때 갈 곳을 정합니다. 두 주소는 서로 다른 목적을 가집니다. 확인한 ELF 예에서는 `task_shell=0x258`, `task_exit=0x164`입니다. 이 주소들은 링커의 배치 결과이므로 코드 변경 후에는 다시 확인해야 합니다.

5.7 태스크 함수가 return하면 task_exit으로 가는 과정

일반 함수의 호출과 반환은 다음 의사 명령으로 나타낼 수 있습니다.

```
call function
# function이 반환하면 이 지점에서 계속 실행
```

호출은 `ra`에 복귀 주소를 준비합니다. 피호출 함수가 다른 함수를 호출하여 `ra`를 덮어쓸 필요가 있다면 compiler는 원래 복귀 주소를 보존하고 함수 종료 전에 복원합니다. 보통 이때 스택을 사용합니다. 함수의 끝에서는 다음 의사 명령으로 돌아갑니다.

```
ret
# 실제 명령 의미: jalr x0, 0(ra)
```

`ret`은 “C의 호출자를 찾아라”라는 고수준 동작이 아니라 `ra`를 목적지로 하는 점프입니다. 반면 새 태스크는 일반 `call`이 아니라 프레임 복원과 `mret`으로 시작합니다. `mret`은 `mepc`로 이동하고 `interrupt` 상태를 복원하지만 `ra`에 호출자 주소를 만들어 주지는 않습니다. 따라서 `task_create`가 미리 설정한 `ra`가 태스크 함수의 반환 목적지가 됩니다.

```
초기 frame: mepc = task_shell, ra = task_exit
             ↓ 레지스터 복원과 mret
task_shell 실행
             ↓ 태스크 entry 함수가 반환하는 경우
함수의 epilogue와 ret
             ↓ ra에 들어 있는 주소로 점프
task_exit 실행
```

태스크를 생성한 `task_create`로 돌아가는 것이 아닙니다. `task_create`는 초기 상태를 만든 뒤 이미 반환한 함수이며, 태스크 `entry`를 일반적인 함수 호출로 실행한 호출자가 아닙니다. 또한 앱의 `return`은 다릅니다. 앱은 셀에서 함수 포인터로 호출되므로 정상 반환하면 셀의 호출 지점으로 돌아갑니다.

5.8 무한 루프인 task_exit의 목적과 한계

출처: firmware/kernel.c의 실제 종료 대기 구현입니다.

```
static void task_exit(void)
{
    for (;;) {
    }
}
```

첫 줄은 프로젝트 내부의 함수 이름을 선언합니다. `for (;;)`는 조건 없이 반복하며 본문에서 아무 작업도 하지 않습니다. 따라서 호출자에게 정상 반환하지 않습니다. 이는 완료된 태스크가 잘못된 주소로 떨어져 실행되는 것을 방지하는 최소 처리입니다. `ra`를 초기화하지 않고 0으로 두었다면, 태스크의 `ret`이 부팅 코드 주소 0으로 이동할 수도 있습니다.

현재 `task_exit`은 태스크를 제거하는 종료 서비스가 아니라, 반환한 태스크의 실행을 정해진 위치에 머무르게 하는 함수입니다. 태스크를 `EXITED`로 표시하거나 스케줄링 대상에서 제외하거나 스택을 회수하지 않습니다. 일반 모드에서는 `timer`가 활성화되어 있으면 다른 태스크로 전환하지만, 다시 이 태스크의 차례가 오면 무한 루프를 재개하여 시간을 소비합니다.

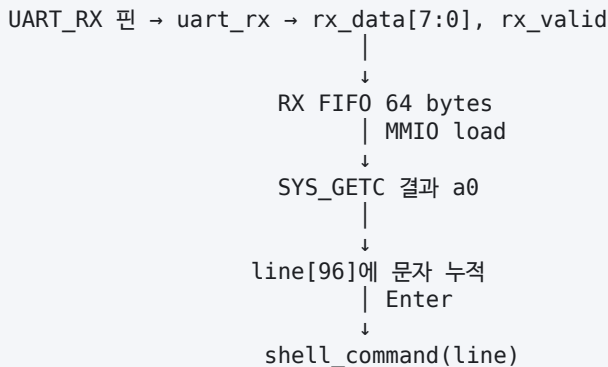
셸 모드는 `schedule()`이 `cur=1`로 셸을 다시 선택합니다. 따라서 셸 태스크가 반환하여 이 루프에 들어가면 `timer trap` 자체는 발생할 수 있어도 다시 같은 루프로 복귀하며 프롬프트 처리는 재개되지 않습니다. “무한 루프에서도 인터럽트는 가능하다”는 설명이 “셸이 항상 복구된다”는 뜻은 아닙니다.

실제 종료 기능을 설계하려면 종료 시스템 콜, 태스크 상태, 실행 가능한 태스크 선택, 마지막 실행 가능 태스크가 없을 때의 `idle` 처리가 필요합니다. 현재 스택 위에서 종료 처리를 하는 동안 그 스택을 다른 태스크에 즉시 재사용해서는 안 됩니다. 이것은 후속 설계 방향이며 현재 구현된 기능은 아닙니다.

6. UART에서 명령행까지

6.1 serial bit, byte, command line의 세 단계

UART RX 핀에는 한 번에 1비트만 들어옵니다. `uart_rx`가 start bit, 8개의 data bit, stop bit를 받아 한 바이트로 만듭니다. `rv32_soc`는 이 바이트들을 FIFO에 보관합니다. C의 `task_shell()`은 바이트들을 문자열로 모아 명령어로 해석합니다.



사용자가 `ls`와 `Enter`를 누르면 보드에는 보통 `0x6C`, `0x73`, `0x0D` 순서가 도착합니다. 세 byte는 각각 독립된 UART frame이며, 문장 전체에 대해 하나의 start bit가 붙는 것이 아닙니다. 셸이 command line을 완성하는 시점은 마지막 CR/LF를 읽은 뒤입니다.

UART는 수신과 송신이 별도 선으로 구성되므로 full-duplex 전송이 가능합니다. 키 입력을 받으면서 echo를 보낼 수 있지만 OS의 처리 속도와 FIFO 용량은 여전히 제한입니다. physical full-duplex라는 사실이 무한히 빠른 입력을 안전하게 받아 준다는 뜻은 아닙니다.

6.2 uart_rx와 uart_tx는 어디에 있는가

두 모듈의 인스턴스는 `zcu104_top`에 있습니다. `rv32_soc`에는 CPU가 접근하는 레지스터와 FIFO가 있고, top이 이 바이트 인터페이스를 물리적인 직렬 송수신 회로에 연결합니다.

```
uart_rx #(.CLOCK_HZ(12_500_000), .BAUD(115_200)) serial_rx (
    .clk(cpu_clk), .rst(rst), .rx(UART_RX),
    .data(rx_data), .valid(rx_valid)
);
```

CPU는 UART bit timing을 직접 세지 않습니다. RX 회로는 입력 핀을 동기화한 뒤 start bit 중량을 확인하고 각 bit 중량을 표본화합니다. 정상적인 stop bit가 확인되면 data를 확정하고 valid를 한 클록 동안 올립니다.

설정은 115200-8-N-1입니다. 12.5 MHz에서 정수 분주값은 108이므로 회로의 nominal bit period는 108클록이며 실제 baud는 약 115741입니다. 이는 이 RTL의 분주식으로 계산한 값입니다. CPU 한 사이클은 80 ns이지만 UART 한 바이트에는 start/stop을 포함해 약 86 μ s가 걸립니다.

RX 상태 전이를 클록 관점에서 보면 IDLE은 high인 선에서 low를 기다리고, START는 반 bit 뒤에도 low인지 확인합니다. BITS는 한 bit 간격으로 0번 bit부터 7번 bit까지 채우고, STOP은 다음 bit가 high인지 확인합니다. 짧은 low 잠음이 반 bit 뒤 사라졌다면 false start로 IDLE로 돌아갑니다. stop bit가 틀리면 현재 구현은 valid를 올리지 않습니다.

TX는 {stop=1, data[7:0], start=0} 의 10비트 frame을 shift하는 상태 기계입니다. ready는 busy의 반대이며, SoC에서 생성한 등록된 data/valid가 다음 clock sampling으로 전달됩니다. ready가 낮을 때 firmware가 무조건 써도 CPU가 자동으로 기다리는 버스가 아니므로, software의 polling 규약이 송신 손실을 막는 데 필요합니다.

6.3 FIFO와 polling

RX FIFO는 64바이트를 저장하며 read/write pointer로 비었는지 가득 찼는지를 구분합니다. CPU가 다른 작업을 하는 동안 도착한 몇 글자를 잠시 보관할 수 있습니다. 가득 찼는데 같은 사이클에 pop도 하지 않으면 새 바이트는 저장되지 않습니다. 현재는 RTS/CTS나 overflow 통지 레지스터가 없습니다.

```
#define UART_RX_DATA (*(volatile uint32_t *)0x10000008u)
#define UART_RX_READY (*(volatile uint32_t *)0x1000000cu)
```

SYS_GETC handler는 ready의 bit 0이 1이면 data를 읽고, 비어 있으면 -1을 돌려줍니다. FIFO가 비었는지 먼저 검사해야 데이터 값 0과 “문자 없음”을 혼동하지 않습니다. RX 데이터 읽기는 다음 원소로 read pointer를 이동시키는 부작용이 있습니다.

현재 RX는 interrupt 방식이 아닙니다. 셸이 SYS_GETC를 반복 호출하는 polling 방식이며, 입력이 없으면 SYS_YIELD를 호출합니다. timer interrupt는 별도 경로로 계속 동작합니다.

현재 간단한 버스에는 별도 read-enable 신호가 없고 RX pop은 주소 일치·FIFO 상태·trap/reset gating으로 생성됩니다. 펌웨어는 RX 데이터 주소를 read 전용으로 사용합니다. 향후 범용 버스로 확장할 때는 유효한 load transaction과 pop을 명시적으로 연결하는 편이 적절합니다.

read/write pointer는 7비트이고 배열 인덱스에는 하위 6비트를 사용합니다. 하위 6비트가 같아도 상위 wrap bit가 다르면 한 바퀴 차이가 있으므로 full입니다. 7비트 전체가 같으면 empty입니다. 원소 64개에 주소 bit 6개만 필요하지만 포인터에는 추가 상태 bit 하나가 필요한 이유입니다.

예를 들어 write pointer가 64, read pointer가 0이면 둘 다 배열의 index 0을 가리키지만 FIFO에는 64바이트가 있습니다. CPU가 하나를 읽는 edge에 새 byte도 들어오면 같은 사이클의 pop을 이용해 새 byte를 받을 수 있습니다. 약 11520byte/s에서 64바이트는 약 5.6 ms 분량이므로, 긴 handler나 긴 출력이 입력 소비를 막으면 overflow 위험을 분석해야 합니다.

코드 해부 6-A. RX FIFO의 입력·소비가 일어나는 줄

출처: [rtl/rv32_soc.v](#), 원본 75 - 89행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
75 | reg [7:0] rx_fifo [0:63];
76 | reg [6:0] rx_wr = 0, rx_rd = 0;
77 | wire rx_empty = (rx_wr == rx_rd);
78 | wire rx_full = (rx_wr[6] != rx_rd[6]) && (rx_wr[5:0] == rx_rd[5:0]);
79 | wire rx_pop = (daddr == 32'h1000_0008) && !rx_empty && !core_trap && !rst;
80 | always @(posedge clk) begin
81 |     if (rst) begin rx_wr <= 0; rx_rd <= 0; end
82 |     else begin
83 |         if (uart_rx_valid && (!rx_full || rx_pop)) begin
84 |             rx_fifo[rx_wr[5:0]] <= uart_rx_data;
85 |             rx_wr <= rx_wr + 1'b1;
86 |         end
87 |         if (rx_pop) rx_rd <= rx_rd + 1'b1;
88 |     end
89 | end
```

원본 행	한 줄씩 읽는 동작과 의미
75	64개의 8비트 원소를 가진 hardware FIFO 저장소입니다. C의 명령행 buffer와 다릅니다.
76	읽기·쓰기 pointer를 각각 7비트로 둡니다. 하위 6비트는 배열 위치, 상위 비트는 한 바퀴 차이를 구분합니다.
77	두 pointer 전체가 같으면 empty입니다.
78	하위 위치는 같지만 상위 비트가 다르면 full입니다. 단순한 배열 index 비교만으로는 empty와 구분할 수 없습니다.
79	RX_DATA 주소에 접근하고 데이터가 있으며 trap·reset이 아닐 때 pop합니다. 별도 load-enable이 없는 현재 버스의 제한도 함께 읽어야 합니다.
80	FIFO의 저장과 pointer 갱신은 rising edge에서 수행합니다.
81	reset은 FIFO pointer를 비웁니다. 배열의 모든 byte를 지우지 않아도 논리적으로 empty가 됩니다.
82	정상 동작의 입력과 출력 처리입니다.
83	새 byte가 유효하고 공간이 있거나 같은 edge에 pop할 때 저장을 허용합니다.
84	쓰기 pointer의 하위 6비트로 선택한 슬롯에 수신 byte를 씁니다.
85	쓰기 pointer를 한 칸 진행합니다. 고정 폭 덧셈이 wrap을 처리합니다.
86	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
87	읽기가 확정되면 읽기 pointer를 한 칸 진행합니다. trap이면 79행에서 차단되어 byte가 유실되지 않습니다.
88	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
89	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

FIFO가 full인데 pop도 없으면 입력 byte는 저장되지 않습니다. 이 회로에 무한 버퍼나 자동 재전송이 있는 것이 아니므로 host 속도와 CPU 소비 지연을 함께 고려해야 합니다.

6.4 한 줄을 만드는 코드

`task_shell()`의 line buffer는 96바이트입니다. 끝의 NUL을 위해 한 바이트를 남기므로 일반 명령행은 최대 95문자입니다. 인쇄 가능한 ASCII 문자를 받으면 buffer에 추가하고 `sys_putc()`로 다시 출력합니다. 이 동작이 키 입력이 화면에 보이는 echo입니다.

Enter에 해당하는 CR(0x0D) 또는 LF(0x0A)를 받으면 문자열 끝에 NUL을 넣고 `shell_command()`를 호출합니다. CR 바로 뒤에 오는 LF는 `last_cr`로 무시하여 CRLF 입력이 두 번의 Enter로 처리되지 않도록 합니다. Backspace(0x08)와 Delete(0x7F)는 직전 문자 하나를 지웁니다.

출력 쪽에서 `\n`은 셸 모드의 `uart_putc()`가 `\r\n`으로 바꿉니다. LF는 줄을 내리고 CR은 가로 위치를 맨 앞으로 돌립니다. CR이 없으면 터미널 설정에 따라 `rv>`가 오른쪽으로 밀리는 현상이 나타날 수 있습니다.

line buffer는 입력을 기록하는 배열이고 FIFO는 아직 CPU가 읽지 않은 byte를 보관하는 hardware queue입니다. 두 buffer는 위치와 역할이 다릅니다. line이 길어졌다고 FIFO 크기가 늘어나지 않으며 FIFO가 비었어도 아직 Enter가 오지 않았다면 line에는 미완성 명령이 남아 있을 수 있습니다.

95문자를 넘으면 overflow를 세워 Enter에서 `line too long`을 출력합니다. 일부 문자만 잘린 위험한 명령을 자동 실행하지 않도록 한 처리입니다. `word()`는 공백을 NUL로 바꾸며 같은 line 배열 안의 token pointer를 반환합니다. 따옴표, escape, pipe, 변수 확장, command history를 처리하는 일반 shell 문법은 구현하지 않았습니다.

코드 해부 6-B. CRLF 출력 변환을 실제 C로 읽기

출처: [firmware/kernel.c](#), 원본 96 - 111행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

96 | static void uart_putc_raw(char c)
97 | {
98 |     while ((UART_READY & 1u) == 0u) {
100 |     }
101 |     UART_TX = (uint8_t)c;
102 | }
104 | static void uart_putc(char c)
105 | {
106 |     #ifdef SHELL_MODE
108 |         if (c == '\n') uart_putc_raw('\r');
109 |     #endif
110 |     uart_putc_raw(c);
111 | }
```

원본 행	한 줄씩 읽는 동작과 의미
96	장치로 문자 하나를 내보내는 내부 함수입니다. newline 변환은 하지 않습니다.
97	앞에서 선언한 함수·블록의 본문을 시작합니다.
98	ready의 bit 0이 1이 될 때까지 MMIO를 반복해서 읽습니다. volatile 접근이므로 C의 일반 상수처럼 사라지면 안 됩니다.
100	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
101	byte pointer로 정의된 UART_TX에 문자를 씁니다. CPU의 store와 SoC의 TX data/valid 경로로 이어집니다.
102	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
104	외부에서 사용하는 문자 출력 함수입니다.
105	앞에서 선언한 함수·블록의 본문을 시작합니다.
106	C 전처리 시 SHELL_MODE가 정의된 빌드에만 다음 변환을 넣습니다.
108	LF를 보내기 직전에 CR을 추가합니다. 터미널의 가로 위치를 첫 칸으로 되돌리기 위한 문자입니다.
109	조건부 컴파일 구간을 닫습니다.
110	원래 요청 문자도 출력합니다. newline이라면 최종 wire 순서는 CR, LF입니다.
111	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

rv> 가 오른쪽으로 밀리던 현상은 instruction이나 scheduler가 바뀐 것이 아니라 터미널 제어문자의 조합 문제였습니다. 일반 OS 빌드와 셸 빌드의 출력 정책 차이도 #ifdef에서 확인할 수 있습니다.

코드 해부 6-C. 한 줄 입력을 완성하는 task_shell

출처: [firmware/kernel.c](#), 원본 509 - 535행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

509 | static void task_shell(void)
510 | {
511 |     char line[96];
512 |     uint32_t used = 0;
513 |     int ch, overflow = 0, last_cr = 0;
514 |     sys_puts("Mini Shell ready. Type help.\nr> ");
515 |     for (;;) {
516 |         ch = sys_fs_call(SYS_GETC, 0, 0, 0);
517 |         if (ch < 0) { sys_yield(); continue; }
518 |         if (ch == '\n' && last_cr) { last_cr = 0; continue; }
519 |         last_cr = (ch == '\r');
520 |         if (ch == '\r' || ch == '\n') {
521 |             sys_putc('\n');
522 |             if (overflow) sys_puts("line too long\n");
523 |             else { line[used] = 0; shell_command(line); }
524 |             used = 0; overflow = 0;
525 |             sys_puts("rv> ");
526 |         } else if (ch == 8 || ch == 127) {
527 |             if (used) { used--; sys_puts("\b \b"); }
528 |         } else if (ch >= 32 && ch <= 126) {
529 |             if (used < sizeof(line) - 1u && !overflow) {
530 |                 line[used++] = (char)ch;
531 |                 sys_putc((char)ch);
532 |             } else overflow = 1;
533 |         }
534 |     }
535 | }
```

원본 행	한 줄씩 읽는 동작과 의미
509	셸 task의 실행 함수입니다. 반복해서 byte를 받고 명령행으로 조립합니다.
510	앞에서 선언한 함수·블록의 본문을 시작합니다.
511	C stack에 96바이트 line buffer를 둡니다. 끝 NUL 때문에 실제 입력 한도는 95문자입니다.
512	현재 저장한 문자 수를 0으로 시작합니다.
513	반환 byte, overflow 상태, 직전 CR 여부를 관리합니다. ch는 -1도 표현해야 하므로 int입니다.
514	안내와 첫 prompt를 출력합니다.
515	셸의 입력 loop입니다.
516	SYS_GETC를 요청합니다. UART 핀을 직접 sample하는 것이 아니라 OS 서비스로 FIFO byte를 받습니다.
517	입력이 없으면 yield하고 다음 반복으로 갑니다. 현재 셸 정책에서는 같은 task가 다시 선택됩니다.
518	CR 직후의 LF는 같은 Enter의 나머지로 보고 버립니다.
519	이번 byte가 CR인지 다음 반복을 위해 기록합니다.
520	CR 또는 LF이면 명령행 끝 처리로 들어갑니다.
521	화면을 다음 줄로 보냅니다. 실제 TX는 앞 코드의 CRLF 변환을 거칩니다.
522	입력 길이가 넘쳤으면 잘린 명령을 실행하지 않고 오류를 출력합니다.
523	정상 길이라면 끝에 NUL을 넣고 parser에 전달합니다. 사용자가 입력한 byte 배열이 여기서 C 문자열이 됩니다.
524	다음 입력을 위해 길이와 overflow를 초기화합니다.
525	명령 처리가 끝난 후 새 prompt를 출력합니다. run한 앱이 반환하지 않으면 이 줄에도 도달하지 않습니다.

원본 행	한 줄씩 읽는 동작과 의미
526	Backspace 또는 Delete이면 수정 동작을 선택합니다.
527	저장 문자가 있을 때만 길이를 줄이고 화면에서도 직전 글자를 지웁니다.
528	일반 입력은 인쇄 가능한 ASCII 범위로 제한합니다.
529	NUL 한 자리와 overflow 상태를 검사합니다.
530	문자를 저장한 후 used를 증가시킵니다.
531	방금 받은 글자를 echo합니다.
532	공간이 부족하거나 이미 overflow이면 오류 상태를 유지합니다.
533	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
534	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
535	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

예를 들어 `ls` 와 `CR`은 `line[0]='l', line[1]='s', line[2]=0`을 만듭니다. 이후 `shell_command`가 실행되고 `used`가 0으로 돌아갑니다. FIFO는 입력 대기열이고 `line`은 이미 소비한 byte의 해석용 buffer입니다.

6.5 built-in command와 외부 앱

`help`, `ls`, `cat`, `write`, `rm`, `upload`, `run`은 셸 펌웨어에 미리 포함된 built-in 명령입니다. `shell_command()`의 문자열 비교 분기로 선택됩니다. `hello.app`은 나중에 업로드하는 파일이며, `run`이라는 built-in 명령이 이 파일을 실행합니다.

일반 명령행 buffer의 95문자 제한이 앱 크기 제한은 아닙니다. `upload` 명령행을 해석한 후에는 별도의 수신 반복문이 hex stream을 처리하기 때문입니다.

`write hello.txt Hello World`에서 첫 token은 명령, 두 번째는 이름이며 뒤의 문자열이 파일 내용입니다. 현재 `write`는 파일 전체 교체이고 `append`가 아닙니다. `cat`은 text로 의미를 해석하지 않고 읽은 byte들을 UART로 보내므로 binary 앱 파일을 cat하면 제어문자나 읽기 어려운 값이 나올 수 있습니다. 앱 검사는 cat 출력 대신 크기·header·host binary를 이용합니다.

`run hello.app`도 parser 단계에서는 문자열 명령입니다. `run` 분기가 선택된 뒤에야 loader가 파일 형식을 검사합니다. 따라서 이름 확장자 `.app` 자체가 실행 가능 여부를 결정하지 않습니다. header와 길이 검사를 만족해야 하며, 실행 진입 주소는 현재 고정 ABI로 결정됩니다.

코드 해부 6-D. word가 공백을 NUL로 바꾸는 이유

출처: [firmware/kernel.c](https://github.com/riscv/riscv-mini-shell/blob/master/firmware/kernel.c), 원본 328 - 337행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

328 | static char *word(char **cursor)
329 | {
330 |     char *p = *cursor, *start;
331 |     while (*p == ' ') p++;
332 |     start = p;
333 |     while (*p && *p != ' ') p++;
334 |     if (*p) *p++ = 0;
335 |     *cursor = p;
336 |     return start;
337 | }
```

원본 행	한 줄씩 읽는 동작과 의미
328	cursor의 주소를 받습니다. caller의 pointer 위치까지 갱신하려고 char**를 사용합니다.
329	앞에서 선언한 함수·블록의 본문을 시작합니다.
330	현재 위치를 p에 복사하고 token 시작 pointer를 준비합니다.
331	앞쪽 공백을 건너웁니다.
332	첫 비공백 위치를 반환할 token의 시작으로 기억합니다.
333	NUL 또는 다음 공백을 만날 때까지 이동합니다.
334	공백을 발견했다면 그 자리에 NUL을 쓰고 p를 다음 byte로 진행합니다. *p++ 는 현재 위치에 쓴 뒤 pointer를 증가시키는 표현입니다.
335	caller가 다음 token을 읽을 위치를 갱신합니다.
336	원래 line 배열 안의 token 시작 주소를 반환합니다. 새 문자열을 할당하지 않습니다.
337	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

run hello.app은 같은 line 배열 안의 run\0hello.app\0처럼 분리됩니다. 파일 이름 pointer는 이 buffer의 일부이므로 parser가 buffer를 다시 쓰기 전에 현재 명령 처리를 마쳐야 합니다.

코드 해부 6-E. cat와 write가 파일 서비스로 이어지는 줄

출처: [firmware/kernel.c](#), 원본 464 - 482행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

464 | } else if (str_eq(cmd, "cat")) {
465 |     name = word(&p);
466 |     if (!*name) { sys_puts("usage: cat <file>\n"); return; }
467 |     off = 0;
468 |     for (;;) {
469 |         n = sys_fs_read_at(name, buf, sizeof(buf), off);
470 |         if (n < 0) { sys_puts("file not found\n"); break; }
471 |         if (n == 0) { sys_putc('\n'); break; }
472 |         for (i = 0; i < n; i++) sys_putc(buf[i]);
473 |         off += (uint32_t)n;
474 |     }
475 | } else if (str_eq(cmd, "write")) {
476 |     name = word(&p);
477 |     while (*p == ' ') p++;
478 |     if (!*name) { sys_puts("usage: write <file> <text>\n"); return; }
479 |     sys_fs_call(SYS_FS_CREATE, (uint32_t)name, 0, 0);
480 |     n = sys_fs_call(SYS_FS_WRITE, (uint32_t)name, (uint32_t)p, str_len(p));
481 |     if (n < 0) sys_puts("write failed\n");
482 |     else { sys_puts("written\n"); }

```

원본 행	한 줄씩 읽는 동작과 의미
464	문자열 비교로 cat built-in 분기를 선택합니다. CPU opcode가 cat인 것은 아닙니다.
465	다음 token을 파일 이름으로 얻습니다.
466	이름이 없으면 사용법을 출력하고 명령 처리를 끝냅니다.
467	파일 내부 offset을 0부터 시작합니다.
468	부분 읽기를 반복합니다. buffer는 이 함수 앞부분에서 64바이트로 선언되어 있습니다.
469	현재 offset에서 최대 buffer 크기만큼 읽습니다.

원본 행	한 줄씩 읽는 동작과 의미
470	오류 -1이면 파일을 읽을 수 없다고 알리고 loop를 끝냅니다.
471	0이면 EOF입니다. 준비꿈을 출력하고 끝냅니다.
472	읽은 n개 byte만 UART로 보냅니다. buffer 전체나 NUL까지 보내는 방식이 아닙니다.
473	다음 읽기는 실제 읽은 byte 수만큼 진행한 위치에서 시작합니다.
474	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
475	write built-in 분기로 들어갑니다. cat과 다른 명령 처리입니다.
476	파일 이름을 분리합니다.
477	파일 내용 앞의 공백을 건너뜀니다. 이 구현은 따옴표·escape를 해석하는 일반 shell이 아닙니다.
478	이름이 없으면 기록하지 않습니다.
479	새 파일 생성을 시도합니다. 이미 있는 파일이면 이후 write로 교체하기 위해 여기의 실패를 무조건 중단 조건으로 쓰지 않습니다.
480	남은 문자열의 주소와 NUL 제외 길이를 FS_WRITE에 전달합니다.
481	쓰기 결과가 음수이면 실패를 출력합니다.
482	그 외에는 성공 메시지를 출력합니다. 기존 내용 뒤에 붙이는 append가 아니라 전체 교체입니다.

write hello.txt Hello World에서 파일 이름과 내용은 같은 line 배열의 서로 다른 위치입니다. cat의 read buffer는 별도 stack 공간이며 syscall이 그곳에 읽은 byte를 채웁니다.

6.6 문자 C의 serial frame을 손으로 그리기

문자 C의 ASCII 값은 0x43, 이진수는 01000011입니다. UART는 least significant bit부터 보내므로 data bit의 시간 순서는 1, 1, 0, 0, 0, 0, 1, 0입니다.

시간 →	idle		start		d0	d1	d2	d3	d4	d5	d6	d7		stop		idle
선의 값	1		0		1	1	0	0	0	0	1	0		1		1

각 bit는 nominal 108개의 12.5 MHz clock 동안 유지

RX의 shift register는 도착한 bit를 shift[bit_no]에 기록하므로 시간 순서가 LSB first여도 최종 data는 0x43이 됩니다. memory의 little-endian word 순서와 UART bit 전송 순서는 다시 다른 층위입니다. 하나는 multi-byte 객체의 주소 순서이고 다른 하나는 한 byte의 serial wire 순서입니다.

start bit를 찾은 뒤 매 bit 중앙을 sample하는 이유는 경계 근처의 전환을 피하기 위해서입니다. host와 FPGA의 baud가 정확히 같지는 않아도 한 frame 안에서 sample 위치가 data 안정 구간에 남으면 통신할 수 있습니다. 현재 회로는 간단한 단일 sample 방식이며 복잡한 oversampling·majority vote 수신기는 아닙니다.

7. ECALL과 운영체제 서비스의 경계

7.1 시스템 콜 번호는 OS가 정한다

RISC-V의 ECALL은 실행 환경에 예외를 발생시키는 명령입니다. 문자 출력, 파일 생성 등 서비스의 종류는 CPU의 opcode가 아니라 OS가 정한 레지스터 규약으로 전달합니다. 이 프로젝트는 a7에 서비스 번호, a0부터 인수를 넣습니다.

번호	서비스	입력	결과
1	SYS_PUTC	a0 : 문자	문자 출력
2	SYS_YIELD	없음	scheduler 선택 문맥으로 복귀
3	SYS_GETTICK	없음	a0 : ticks
4	SYS_FS_CREATE	a0 : 이름 주소	a0 : 0 또는 -1
5	SYS_FS_WRITE	a0 : 이름, a1 : 데이터, a2 : 크기	a0 : 기록 크기 또는 -1
6	SYS_FS_READ	a0 : 이름, a1 : 버퍼, a2 : 용량	a0 : 파일 크기 또는 -1
7	SYS_FS_DELETE	a0 : 이름	a0 : 0 또는 -1
8	SYS_FS_LIST	없음	UART 목록 출력, a0=0
9	SYS_GETC	없음	a0 : 0 - 255 또는 -1
10	SYS_FS_READ_AT	a0 : 이름, a1 : 버퍼, a2 : 용량, a3 : offset	a0 : 읽은 크기, EOF 0, 오류 -1

이는 이 Mini OS만의 ABI입니다. Linux의 syscall 번호와 호환되지 않습니다. 함수 호출에서 쓰는 표준 레지스터 규약과 OS별 syscall 번호 규약도 구분해야 합니다.

포인터 인수는 문자열 자체가 아니라 메모리 주소입니다. SYS_FS_WRITE에 a0=name, a1=data를 넘기면 handler는 같은 주소 공간에서 그 위치를 읽습니다. 현재는 user/kernel 주소 변환이나 안전한 copy-in 검사가 없으므로 caller가 올바른 주소와 길이를 제공해야 합니다. 나중에 U-mode로 나누면 이 전제가 달라집니다.

반환값 -1은 register에서는 0xFFFFFFFF라는 bit pattern입니다. C stub이 이를 int로 해석하므로 ch < 0으로 입력 없음·오류를 판단합니다. 정상 UART byte 0xFF는 zero-extension한 255로 반환하므로 -1과 다릅니다. API의 signed 반환형을 잘못 unsigned로 바꾸면 error 검사가 달라질 수 있습니다.

7.2 한 문자를 출력하는 앱 코드

```
static void putc_sys(char c)
{
    register uint32_t a0 __asm__("a0") = (unsigned char)c;
    register uint32_t a7 __asm__("a7") = 1u;
    __asm__ volatile ("ecall" : "+r"(a0) : "r"(a7) : "memory");
}
```

register ... __asm__("a0")는 inline assembly operand를 지정된 레지스터에 연결합니다. "+r"은 입력과 출력 operand, "r"은 입력 레지스터 operand입니다. volatile은 compiler가 이 assembly의 부작용을 무시하여

제거하지 못하게 하며, "memory" clobber는 주변 메모리 접근의 compiler 재배치를 제한합니다. 하드웨어 cache를 지우는 명령이라는 뜻은 아닙니다.

현재 handler는 frame을 통해 일반 레지스터를 복원하고 서비스 결과가 필요한 경우 저장된 a0 를 수정합니다. 이 구현 규약을 이용하여 앱이 ecall 이후에도 필요한 레지스터 값을 유지할 수 있습니다.

compiler는 일반 C 함수의 호출 규약만으로 inline assembly가 어떤 메모리를 바꾸는지 알 수 없습니다. 따라서 operand와 clobber 선언은 compiler와 작성자 사이의 계약입니다. "memory" 는 syscall이 pointer가 가리키는 메모리에 영향을 줄 수 있음을 알리는 보수적인 방법입니다. 단순 출력 stub도 같은 형태를 사용해 예제를 일관되게 구성했습니다.

이 stub에 a0 만 output으로 선언해도 되는 이유는 현재 OS가 다른 일반 register를 frame에서 복원한다는 구현 계약 때문입니다. 모든 OS의 ECALL에 같은 clobber 규칙을 그대로 적용할 수 있는 것은 아닙니다. 자신이 호출하는 runtime의 ABI와 compiler가 보는 operand 계약이 서로 일치해야 합니다.

7.3 하드웨어 trap 진입

M-mode에서 ecall 을 decode하면 cause 11의 동기 예외가 됩니다. core와 CSR 회로가 다음을 수행합니다.

```
mepc ← ecall 명령어 자신의 PC
mcause ← 11
MPIE ← 이전 MIE
MIE ← 0
PC ← mtvec
```

일반 레지스터를 RAM에 저장하는 일은 아직 하지 않습니다. 그 작업은 trap 진입점의 어셈블리 코드가 수행합니다. trap이 선택된 사이클에는 core가 일반 명령의 register/CSR/memory write를 막습니다.

ECALL 은 정상 완료된 명령처럼 retire되는 것이 아닙니다. 예외를 발생시킨 명령 주소가 mepc 에 저장되고, 소프트웨어가 서비스를 처리한 뒤 다음 명령으로 돌아가기로 결정합니다. 이 의미는 [RISC-V Machine-Level ISA의 Environment Call 설명](#)과 일치합니다. 소스 주석의 "completed"라는 표현은 서비스 처리 후 다음 명령으로 진행한다는 의도로 읽히, ISA의 retire 의미와 혼동하지 않아야 합니다.

mtvec 에는 trap_entry 라는 함수 이름이 아니라 linker가 정한 주소 값이 들어갑니다. 현재 관측 ELF에서 이 주소는 0x40 입니다. hardware는 symbol 이름을 모른 채 PC를 이 값으로 바꿉니다. 이후 addi sp,sp,-128 부터의 assembly가 순차 실행되어 frame을 만듭니다.

trap 진입 직후 일반 register는 아직 caller의 값을 유지합니다. 그래서 가장 먼저 t0를 임시로 덮어쓰면 원래 t0가 사라질 수 있습니다. trap.S는 원래 register들을 먼저 저장하고, x5/t0를 안전하게 저장한 뒤 그 register를 mepc 읽기 등에 사용합니다. 이 순서는 C handler보다 아래 단계의 정확성 조건입니다.

7.4 왜 f->mepc += 4인가

handler는 CAUSE_ECALL_M이면 저장된 frame의 mepc 를 4 증가시킵니다. 현재 코어는 32비트 고정 길이 명령만 사용하므로 다음 명령 주소가 현재 주소 +4입니다. 이 증가가 없으면 mret 이후 같은 ecall 을 다시 실행하여 서비스를 무한 반복합니다.

timer interrupt의 경우에는 증가시키지 않습니다. 이 core는 interrupt를 받아들이는 사이클의 현재 명령을 commit하지 않으므로, 그 PC로 돌아가 해당 명령을 실행해야 합니다.

예를 들어 앱의 ECALL 주소는 0x4014 이고 이어지는 LBU는 0x4018 입니다. frame의 값을 0x4018 로 바꾸어도 즉시 CPU PC가 바뀌지는 않습니다. trap.S가 나중에 frame에서 이 값을 읽어 CSR mepc에 쓰고, MRET가 그 CSR을 선택할 때 제어가 이동합니다. RAM의 frame PC, CSR mepc, 현재 hardware PC는 서로 다른 저장 위치입니다.

timer와 ECALL이 같은 사이클에 보이는 경우 현재 core의 우선순위는 timer입니다. ECALL 자체는 아직 처리되지 않았으므로 timer handler는 PC를 증가시키지 않고 돌아옵니다. 이후 ECALL이 다시 decode되어 syscall을 수행합니다. 각 handler가 “이미 처리한 명령인가”를 구분해야 중복 서비스나 명령 누락을 피할 수 있습니다.

코드 해부 7-B. trap_handler의 timer와 ECALL 분기

출처: [firmware/kernel.c](https://github.com/riscv/riscv-isa-sim/blob/master/src/asm_trap.S), 원본 185 - 205, 227 - 230행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

185 | struct frame *trap_handler(struct frame *f)
186 | {
187 |     uint32_t cause = csr_read_mcause();
189 |     if (cause == CAUSE_MTIMER) {
190 |         ticks++;
191 |         timer_rearm();
192 |         return schedule(f);
193 |     }
195 |     if (cause == CAUSE_ECALL_M) {
196 |         f->mepc += 4;
197 |         switch (REG(f, 17)) {
198 |             case SYS_PUTC:
199 |                 uart_putc((char)REG(f, 10));
200 |                 return f;
201 |             case SYS_YIELD:
202 |                 return schedule(f);
203 |             case SYS_GETTICK:
204 |                 REG(f, 10) = ticks;
205 |                 return f;
... (원본 206-226행 생략)
227 |             case SYS_FS_READ_AT:
228 |                 REG(f, 10) = (uint32_t)fs_read_at((const char *)REG(f, 10),
229 |                 (void *)REG(f, 11), REG(f, 12), REG(f, 13));
230 |                 return f;

```

원본 행	한 줄씩 읽는 동작과 의미
185	assembly가 frame pointer를 a0로 넘겨 호출하는 C handler입니다. 반환값도 frame pointer입니다.
186	앞에서 선언한 함수·블록의 본문을 시작합니다.
187	현재 CSR mcause를 읽어 hardware가 기록한 trap 원인을 얻습니다.
189	machine timer interrupt 원인과 비교합니다. syscall 번호와 비교하는 곳이 아닙니다.
190	처리한 timer 횟수를 증가시킵니다. 지나간 시간 전체를 자동 계산하는 것은 아닙니다.
191	compare를 미래로 옮겨 level IRQ의 원인을 해제합니다.
192	scheduler가 선택한 frame을 반환합니다. timer 경로에는 mepc+4가 없습니다.
193	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
195	M-mode ECALL 원인이면 서비스 처리로 들어갑니다.
196	RAM frame의 복귀 PC를 다음 instruction으로 옮깁니다. CSR mepc 자체는 restore 때 갱신됩니다.
197	중단된 caller의 a7 슬롯에서 서비스 번호를 읽습니다.
198	문자 출력 서비스의 분기입니다.

원본 행	한 줄씩 읽는 동작과 의미
199	저장된 a0의 문자 값을 UART 출력 함수에 넘깁니다.
200	같은 frame을 반환하여 출력 요청자에게 돌아갑니다.
201	yield 서비스의 분기입니다.
202	다음 실행 문맥은 scheduler에게 맡깁니다.
203	tick 조희 서비스의 분기입니다.
204	저장된 a0 슬롯을 ticks로 바꿉니다. 현재 C 함수의 a0에만 값을 넣어서는 안 됩니다.
205	수정된 같은 frame을 반환합니다.
227	뒤쪽의 파일 부분 읽기 서비스 분기입니다. 중간 FS case들은 이 발췌에서 생략했습니다.
228	이름 pointer를 저장된 a0에서 얻어 fs_read_at에 넘기고, 반환 int를 a0 슬롯에 기록할 준비를 합니다.
229	a1·a2·a3 슬롯에서 destination·capacity·offset을 가져옵니다.
230	복원 코드가 수정된 a0를 caller에 전달하도록 같은 frame을 반환합니다.

return f 는 MRET가 아닙니다. 먼저 일반 C 함수에서 trap.S로 반환하고, trap.S가 register·mepc를 복원한 다음 MRET를 실행합니다.

7.5 파일 읽기 syscall의 인수·결과 추적

loader가 payload 첫 60바이트를 읽는 순간을 보겠습니다. 이름 문자열은 셀의 line buffer 안에 있고, 그 주소를 NAME 이라고 표기합니다. 다음 값들이 ECALL 직전 register에 준비됩니다.

register	값	해석
a7 / x17	10	SYS_FS_READ_AT
a0 / x10	NAME	파일 이름 문자열 주소
a1 / x11	0x4000	복사할 destination buffer
a2 / x12	60	최대 읽을 byte 수
a3 / x13	12	파일 header 다음부터 읽기

trap.S가 이 값을 frame에 저장한 뒤 handler가 REG(f,17) 을 읽어 서비스를 선택합니다. fs_read_at(name,buffer,60,12) 가 RAM disk에서 실행 RAM으로 byte를 복사하고 60을 반환하면, handler는 frame의 a0 슬롯에 60을 씁니다. 복원 후 앱/셀의 a0는 NAME pointer가 아니라 결과 60입니다.

ECALL 전 a0 = 파일 이름 pointer
 handler 중 f->x[9] = 60으로 변경
 MRET 후 a0 = 60, C stub의 return 값도 60

argument register가 return register로 재사용되므로, caller가 이름 pointer를 나중에도 필요로 하면 compiler가 다른 register나 stack에 보존합니다. syscall wrapper의 +r(a0) 선언은 바로 이 입력·출력 재사용을 compiler에 알려 줍니다. pointer의 내용이 반환값으로 바뀌는 것이 아니라 pointer를 담았던 register가 바뀌는 것입니다.

코드 해부 7-A. 파일 부분 읽기 wrapper의 register 계약

출처: [firmware/kernel.c](https://github.com/riscv/riscv-isa-sim/blob/master/src/asm/asm.h), 원본 304 - 313행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
304 | static int sys_fs_read_at(const char *name, void *buf, uint32_t cap, uint32_t off)
305 | {
306 |     register uint32_t a0 __asm__("a0") = (uint32_t)name;
307 |     register uint32_t a1 __asm__("a1") = (uint32_t)buf;
308 |     register uint32_t a2 __asm__("a2") = cap;
309 |     register uint32_t a3 __asm__("a3") = off;
310 |     register uint32_t a7 __asm__("a7") = SYS_FS_READ_AT;
311 |     __asm__ volatile ("ecall" : "+r"(a0) : "r"(a1), "r"(a2), "r"(a3), "r"(a7) :
"memory");
312 |     return (int)a0;
313 | }
```

원본 행	한 줄씩 읽는 동작과 의미
304	이름·destination·최대 읽기 수·파일 offset을 받는 C wrapper입니다.
305	앞에서 선언한 함수·블록의 본문을 시작합니다.
306	이름 문자열의 주소를 a0에 넣습니다. 문자열 자체를 register 한 개에 담는 것이 아닙니다.
307	결과 byte를 쓸 buffer 주소를 a1에 넣습니다.
308	최대 byte 수 cap을 a2에 넣습니다.
309	파일 시작부터의 byte offset을 a3에 넣습니다. RAM 주소가 아닙니다.
310	이 OS가 정의한 SYS_FS_READ_AT 번호 10을 a7에 넣습니다.
311	ECALL을 실행합니다. a0의 +r은 입력과 출력, 나머지 r은 입력, memory는 compiler에 알리는 메모리 부작용 경계입니다.
312	복원된 a0를 int로 해석하여 읽은 크기·EOF 0·오류 -1을 C caller에 반환합니다.
313	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

이름 pointer를 넣었던 a0가 결과 register로 재사용됩니다. C caller가 다음에도 이름을 필요로 한다면 compiler가 다른 위치에 보존하며, OS는 저장된 frame의 a0 슬롯을 수정합니다.

8. Trap frame과 셸 모드의 스케줄링

8.1 frame의 정확한 메모리 형식

trap.S와 C의 struct frame은 정확히 일치해야 합니다.

```
struct frame {
    uint32_t mepc;
    uint32_t x[31];
};
#define REG(f, n) ((f)->x[(n) - 1])
```

frame 시작으로부터 offset	저장 내용
0	mepc
4	x1 / ra
8	trap 이전 x2 / sp 기록
12	x3 / gp
40	x10 / a0
68	x17 / a7
124	x31 / t6

총 크기는 $4 + 31 \times 4 = 128$ 바이트입니다. x0는 항상 0이므로 저장하지 않습니다. REG(f, 10)은 x[9]이므로 정확히 a0의 슬롯을 가리킵니다.

실제 memory 주소는 frame_base + 4*n으로 x_n 슬롯을 계산합니다. 예를 들어 frame이 0x1BC0이면 a0는 0x1BE8, a7은 0x1C04입니다. C의 x[9]는 struct 시작부터 4바이트 mepc를 지난 뒤 9개 word를 더한 위치이므로 같은 offset 40을 얻습니다. 배열 index와 architectural register 번호의 -1 관계를 이렇게 계산할 수 있습니다.

frame에 a0를 저장한 뒤 C handler의 인수 a0는 frame pointer로 바뀝니다. 원래 task의 a0와 handler의 현재 a0가 잠시 다른 의미를 갖는 것입니다. handler는 REG(f, 10)을 통해 저장된 task 값을 읽고 반환 결과도 그 슬롯에 써야 합니다. 현재 register a0에만 값을 넣고 frame을 수정하지 않으면 restore에서 이전 값으로 덮일 수 있습니다.

8.2 저장·C 호출·복원

trap_entry는 SP를 128바이트 낮추고 일반 레지스터와 mepc를 저장합니다. 이후 a0=sp로 설정하고 C의 trap_handler(frame*)를 호출합니다. C handler의 반환값 a0는 복원할 frame의 주소입니다.

```
mv    a0, sp
call  trap_handler
mv    sp, a0
```

반환된 frame이 같은 주소이면 기존 문맥으로 돌아갑니다. 다른 task의 frame이면 mv sp, a0 이후 그 task의 레지스터를 복원하므로 context switch가 됩니다. 마지막으로 mepc를 CSR에 쓰고 mret를 실행합니다.

SP는 저장된 x2 슬롯을 `lw sp, ...` 로 복원하지 않습니다. frame 주소를 SP로 선택한 다음 마지막에 128을 더하여 원래 SP를 얻습니다. 따라서 x2 슬롯은 디버깅용 기록이며, frame 배치 자체가 SP 복원 규칙입니다.

일반 함수 호출에서는 caller-saved register를 caller가 필요할 때만 저장하면 됩니다. interrupt는 어느 명령 사이에서든 발생할 수 있으므로 task가 그 시점에 어떤 register를 쓰는지 handler가 알 수 없습니다. 그래서 이 trap entry는 x1 - x31을 넓게 보존합니다. 이것이 C 함수의 보통 prologue보다 저장량이 큰 이유입니다.

C handler에서 다른 frame을 반환할 때 기존 stack에 올라간 handler frame은 C의 epilogue가 먼저 정리합니다. 그 후 assembly의 `mv sp, a0` 가 실행됩니다. 아직 C 함수가 사용하는 도중 임의로 SP를 바꾸는 방식이 아니므로 C 호출 규약을 유지하면서 context를 교체할 수 있습니다.

코드 해부 8-A. trap 진입: 원래 값을 잃지 않고 저장하기

출처: [firmware/trap.S](#), 원본 23 - 28, 54 - 63행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

23	trap_entry:
24	addi sp, sp, -128
25	sw x1, 4(sp)
26	sw x3, 12(sp)
27	sw x4, 16(sp)
28	sw x5, 20(sp)
	... (원본 29-53행 생략)
54	sw x31, 124(sp)
56	addi t0, sp, 128
57	sw t0, 8(sp)
58	csrr t0, mepc
59	sw t0, 0(sp)
61	mv a0, sp
62	call trap_handler
63	mv sp, a0

원본 행	한 줄씩 읽는 동작과 의미
23	mtvec가 가리키는 assembly 진입 label입니다. CPU는 일반 CALL 없이 이 위치로 제어를 옮깁니다.
24	중단된 task의 SP를 128바이트 낮춰 frame을 예약합니다. 별도 kernel stack으로 바꾸지 않습니다.
25	원래 ra를 frame offset 4에 저장합니다. 나중의 C 호출이 ra를 바꿔도 돌아갈 수 있습니다.
26	x3/gp를 offset 12에 보존합니다. x2는 SP를 이미 바꿨기 때문에 별도 방식으로 다룹니다.
27	x4/tp를 offset 16에 보존합니다.
28	x5/t0를 먼저 보존합니다. 뒤에서 임시 계산에 t0를 쓰기 위한 선행 조건입니다.
54	생략한 동일 패턴의 SW들 뒤에 x31까지 저장합니다. 모든 일반 register의 원래 값이 frame에 확보됩니다.
56	새 SP에 128을 더해 trap 직전 SP를 복원 계산합니다. 여기서 t0를 바꾸어도 원래 값은 offset 20에 남아 있습니다.
57	계산한 원래 SP를 offset 8에 기록합니다. 현재 구현에서는 정보용 슬롯입니다.
58	hardware CSR mepc를 일반 register t0로 읽습니다.
59	재개 PC를 frame offset 0에 저장합니다.
61	C 함수의 첫 인수 a0에 frame 시작 주소를 넣습니다. task의 원래 a0는 이미 frame에 있습니다.
62	C trap_handler를 일반 호출합니다. 이 CALL이 사용하는 ra와 task의 ra를 구분합니다.
63	handler가 반환한 frame pointer를 새 SP로 선택합니다. 다른 frame이면 이후 복원이 다른 task의 값으로 바뀝니다.

29 - 53행의 SW는 x6 - x30을 각각 offset 4×register 번호에 저장하는 반복 구조입니다. 핵심 순서는 원래 값 보존 → 임시 register 사용 → C 호출 → 복원할 frame 선택입니다.

코드 해부 8-B. trap 복귀: frame에서 CSR과 register로

출처: [firmware/trap.S](#), 원본 65 - 70, 95 - 98행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

65 | lw    t0,    0(sp)
66 | csrw  mepc, t0
67 | lw    x1,    4(sp)
68 | lw    x3,   12(sp)
69 | lw    x4,   16(sp)
70 | lw    x5,   20(sp)
    | ... (원본 71-94행 생략)
95 | lw    x30, 120(sp)
96 | lw    x31, 124(sp)
97 | addi  sp, sp, 128
98 | mret

```

원본 행	한 줄씩 읽는 동작과 의미
65	선택된 frame offset 0에서 재개 PC를 읽습니다. 선택 frame이 바뀌었다면 이 PC도 다른 task의 값입니다.
66	읽은 PC를 hardware mepc에 씁니다. MRET가 RAM frame을 직접 읽지 않는 이유입니다.
67	선택된 task의 ra를 복원합니다. app의 최종 ret에 필요한 셀 복귀 주소일 수 있습니다.
68	gp를 원래 값으로 복원합니다.
69	tp를 원래 값으로 복원합니다.
70	임시로 사용했던 t0도 저장 값으로 복원합니다. mepc 쓰기가 먼저 끝났으므로 더 이상 PC 전달용으로 필요하지 않습니다.
95	같은 패턴의 중간 LW 뒤에 x30을 복원합니다.
96	마지막 일반 register x31을 복원합니다.
97	128바이트 frame 예약을 해제하여 trap 이전 SP로 돌아갑니다. offset 8의 SP를 LW하지 않습니다.
98	PC를 mepc로 선택하고 MIE를 MPIE에서 복원합니다. 일반 ret와 다른 특권 instruction입니다.

71 - 94행은 x6 - x29의 복원입니다. 하드웨어의 MRET 한 줄과 소프트웨어의 여러 LW가 함께 있어야 전체 context 복귀가 완성됩니다.

8.3 일반 OS와 셸 모드의 scheduler 차이

일반 OS는 idle, A, B, C 네 슬롯을 round-robin으로 선택합니다. 셸 모드의 코드는 다음처럼 항상 slot 1을 고릅니다.

```

task_sp[cur] = f;
#ifdef SHELL_MODE
    cur = 1;
#else
    if (++cur >= NTASK) cur = 0;
#endif
return task_sp[cur];

```

첫 tick은 idle에서 셸로 전환합니다. 이후 셸의 timer interrupt나 yield는 셸 문맥을 다시 선택합니다. 따라서 셸 모드에서 A/B/C가 계속 출력되거나 idle과 10 ms씩 번갈아 실행된다고 설명하면 실제 코드와 맞지 않습니다.

입력이 없을 때 `yield`를 호출하더라도 셸이 다시 선택되므로 현재 구현은 busy polling 성격을 가집니다. 에너지 절약형 sleep이나 입력 대기 queue는 구현되어 있지 않습니다. CPU가 빠르게 FIFO를 비울 수 있다는 장점과 불필요한 실행량이라는 비용을 함께 관찰할 수 있습니다.

일반 OS의 C task가 문자 하나를 출력한 뒤 yield하면 자신의 time slice를 자발적으로 끝냅니다. A/B가 계속 출력하는 것과 비교하면 C의 문자 수가 적을 수 있습니다. 공평한 round-robin 선택 횟수와 같은 실행 명령 수·UART 문자 수는 동일한 지표가 아닙니다. 셸 모드는 입력 응답을 위해 이 데모 정책과 별도의 선택 규칙을 사용합니다.

셸 slot의 frame pointer는 trap마다 현재 SP에 따라 달라질 수 있습니다. 앱이 더 깊은 함수를 호출한 시점의 interrupt는 더 낮은 stack 위치에 frame을 만듭니다. `task_sp[1]`은 고정된 초기 frame 주소만 영원히 가리키는 것이 아니라, 가장 최근에 중단된 문맥을 가리키도록 갱신됩니다.

코드 해부 8-C. scheduler가 실제로 바꾸는 것은 무엇인가

출처: [firmware/kernel.c](https://github.com/riscv-mini-shell/firmware/blob/master/kernel.c), 원본 156 - 166행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

156 | static struct frame *schedule(struct frame *f)
157 | {
158 |     task_sp[cur] = f;
159 | #ifdef SHELL_MODE
160 |     cur = 1;
161 | #else
162 |     if (++cur >= NTASK)
163 |         cur = 0;
164 | #endif
165 |     return task_sp[cur];
166 | }
```

원본 행	한 줄씩 읽는 동작과 의미
156	현재 trap frame을 받아 다음에 복원할 frame을 돌려주는 함수입니다.
157	앞에서 선언한 함수·블록의 본문을 시작합니다.
158	현재 slot의 마지막 중단 위치를 기록합니다. register를 복사하는 것이 아니라 이미 저장된 frame의 pointer를 보관합니다.
159	C 전처리에서 셸 구성의 선택 규칙을 고릅니다.
160	셸 모드에서는 항상 slot 1을 선택합니다. 앱마다 새로운 slot을 만드는 코드가 아닙니다.
161	일반 OS 구성의 선택 규칙으로 이어집니다.
162	현재 slot을 하나 증가시켜 끝을 넘었는지 검사합니다. 나머지 연산 없이 round-robin을 만듭니다.
163	끝을 넘으면 idle slot 0으로 돌아갑니다.
164	빌드별 선택 분기를 닫습니다.
165	다음 task의 저장 frame 주소를 반환합니다. 실제 SP 변경과 register 복원은 trap.S가 이어서 수행합니다.
166	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

`cur` 변경만으로 CPU register가 순간적으로 바뀌지 않습니다. scheduler는 선택, assembly는 복원, MRET는 재개라는 세 책임을 나누어 수행합니다.

8.4 frame에 모든 CSR이 들어 있지는 않다

이 frame은 일반 정수 레지스터와 mepc 만 저장합니다. mstatus, mcause, 모든 CSR을 task별로 저장하는 범용 context가 아닙니다. 현재의 단일 M-mode, 비중첩 timer handler, 동일한 interrupt 정책에서 쓰는 최소 구조입니다. 사용자 모드나 중첩 interrupt, task마다 다른 CSR 상태를 지원하면 저장·복원 계약을 확장해야 합니다.

현재 mstatus의 MPIE는 hardware에 하나만 존재하고 task별 frame에는 복사하지 않습니다. handler가 MIE를 다시 켜지 않고 모든 task를 같은 정책으로 실행하기 때문에 간단한 restore가 가능합니다. 이 전제를 바꾸면 다른 task의 interrupt 상태를 실수로 물려주는지 검토해야 합니다.

중첩 trap을 허용하면 새 trap이 mepc와 mcause를 다시 덮어씁니다. 바깥 trap의 PC는 frame에 저장되어 있어도 cause, privilege 상태, stack 선택, 재진입 가능한 C 함수의 조건까지 따져야 합니다. mscratch를 이용한 전용 kernel stack 전환은 가능한 확장 방법이지만 현재 trap.S는 이를 사용하지 않습니다.

8.5 stack과 두 종류의 frame을 구분하기

stack은 태스크의 작업 공간 전체이고, trap frame은 그 공간 안에 저장하는 CPU 실행 상태의 기록입니다. trap frame 외에도 compiler가 함수 호출을 위해 만드는 함수 스택 프레임이 있습니다. 이름에 frame이 들어가더라도 두 자료구조의 형식과 목적은 다릅니다.

구분	생성 주체	담기는 내용	크기와 수명
태스크 stack	정적 배열과 OS의 sp 설정	함수 프레임, trap frame, 호출 중 임시 데이터	현재 태스크당 1 KiB 예약
함수 stack frame	compiler가 생성한 진입·복귀 코드	필요한 지역변수, 저장 레지스터, 인수 등	함수와 최적화에 따라 다르며 0바이트일 수도 있음
trap frame	trap.S 또는 최초 task_create	mepc와 x1 - x31의 저장 슬롯	현재 128바이트, 복원 때 해제

현재 C 구조체는 반드시 다음 순서로 읽어야 합니다. mepc가 맨 앞이고 x 배열이 뒤입니다. 순서를 바꾸어도 크기는 128바이트일 수 있지만 trap.S의 offset과 맞지 않아 다른 값을 복원하게 됩니다.

```
struct frame {
    uint32_t mepc;
    uint32_t x[31];
};
```

둘째 줄은 offset 0의 재개 주소이고, 셋째 줄은 offset 4부터 시작하는 레지스터 슬롯입니다. f->x[0]은 x0가 아니라 x1입니다. 상수 0인 x0는 저장하지 않습니다. 모든 지역변수가 스택에 들어가는 것도 아닙니다. compiler가 레지스터에만 보관하거나 최적화로 제거할 수 있습니다.

현재 단일 hart에는 태스크마다 별도의 물리 레지스터 세트가 없습니다. A가 a0=10을 가지고 실행하다가 B가 같은 a0를 99로 바꾸면 A의 값은 사라집니다. 그래서 A를 멈출 때 레지스터 값을 RAM의 frame에 저장하고 B의 frame에서 값을 읽어 실제 레지스터를 바꿉니다. 다시 A를 선택하면 A의 저장값을 복원합니다. 이 원리가 context switch입니다.

8.6 trap 전후의 스택과 프레임 수명

다음은 주소 관계를 이해하기 위한 예입니다. trap 직전 sp가 0x1C20 이라고 가정합니다. 최초 태스크의 top인 0x1C40 과 다른 것은 이미 함수가 스택을 사용하고 있기 때문입니다.

```

높은 주소 0x1C40   태스크 스택의 top
                  함수가 사용 중인 공간
0x1C20   trap 직전 sp
          trap frame 128바이트
0x1BA0   trap 진입 후 sp와 frame 시작
          C handler와 하위 함수의 스택 사용 공간
낮은 주소 방향으로 성장

```

출처: firmware/trap.S 의 실행 순서를 설명하기 위해 필요한 명령만 발췌했습니다. 생략된 레지스터 저장·복원 명령을 제외하고 실행 가능한 완전한 handler로 사용하면 안 됩니다.

```

addi sp, sp, -128
# 일반 레지스터 저장 코드 생략
addi t0, sp, 128
sw    t0, 8(sp)
csrr  t0, mepc
sw    t0, 0(sp)
mv    a0, sp
call  trap_handler
mv    sp, a0
# mepc 및 일반 레지스터 복원 코드 생략
addi sp, sp, 128
mret

```

명령	주요 역할
addi sp, sp, -128	예의 sp를 0x1BA0으로 내려 frame 공간을 확보합니다.
addi t0, sp, 128	저장된 원래 t0가 안전해진 뒤, trap 직전 sp인 0x1C20을 계산합니다.
sw t0, 8(sp)	x2 기록 슬롯에 원래 sp를 보관합니다.
csrr t0, mepc	하드웨어가 CSR에 기록한 재개 주소를 읽습니다.
sw t0, 0(sp)	읽은 주소를 이 태스크 소유의 RAM 프레임에 보존합니다.
mv a0, sp	C 호출 규약의 첫 인수로 frame 포인터를 전달합니다.
call trap_handler	C handler를 호출합니다. 원래 태스크의 ra는 이미 프레임에 저장되어 있습니다.
mv sp, a0	C가 반환한 frame 포인터를 선택합니다. 다른 태스크의 주소일 수도 있습니다.
마지막 addi sp, sp, 128	선택한 태스크의 trap 이전 sp를 복구합니다. x2 슬롯을 lw하는 방식이 아닙니다.
mret	선택한 mepc로 이동하고 CSR의 interrupt 허용 상태를 복원합니다.

하드웨어는 trap 진입 시 mepc·mcause·mstatus 등을 갱신하지만 일반 레지스터를 RAM에 자동으로 저장하지는 않습니다. 이 저장은 trap.S의 책임입니다. handler의 C 함수 프레임은 저장된 trap frame보다 낮은 주소에 생길 수 있으므로 128바이트만 남았다고 안전한 것이 아닙니다.

복원이 끝나면 frame의 바이트가 RAM에 남아 있을 수 있지만 그 위치는 다시 사용할 수 있는 스택 공간입니다. “해제”는 반드시 0으로 지운다는 뜻이 아닙니다. 또한 최초의 인공 frame은 top 바로 아래에 있지만, 이후 trap frame은 trap 발생 당시 sp 아래에 생깁니다. 매번 고정 주소 0x1BC0에 덮어쓰는 구조가 아닙니다.

8.7 panic에 들어가면 trap_handler가 끝나지 않는 이유

맞습니다. 현재 panic은 의도적으로 반환하지 않습니다. 다만 trap_handler가 매번 마지막까지 실행되어 panic을 호출하는 것은 아닙니다. 다음은 firmware/kernel.c의 제어 흐름을 요약한 코드이며 syscall별 본문은 생략했습니다.

```
if (cause == CAUSE_MTIMER) {
    ticks++;
    timer_rearm();
    return schedule(f);
}
if (cause == CAUSE_ECALL_M) {
    f->mepc += 4;
    /* 각 syscall case에서 return f 또는 return schedule(f) */
}
panic(cause, f->mepc);
return f;
```

timer 경로는 원인을 재예약하고 선택된 frame을 반환합니다. ECALL 경로는 재개 주소를 다음 명령으로 옮긴 뒤 각 case에서 반환합니다. 알 수 없는 syscall 번호도 default에서 a0에 -1을 넣고 반환합니다. 반면 illegal instruction과 그 밖의 처리하지 않는 trap 원인은 마지막 panic까지 도달합니다. syscall 서비스 번호와 mcause의 trap 원인 번호를 구분해야 합니다.

panic은 PANIC cause= 뒤에 원인을 8자리 16진수로, pc= 뒤에 저장 주소를 출력하고 다음 코드에 머뭙니다.

```
for (;;) {
}
```

panic의 무한 루프 때문에 trap_handler는 반환하지 않으며, 뒤의 return f와 trap.S의 복원 코드 및 mret도 실행되지 않습니다. 마지막 return f;는 현재 경로에서 도달 불가능한 문장입니다. 비복귀 속성을 명시하는 개선안으로 `__attribute__((noreturn))`을 사용할 수 있지만, 여기서는 기존 펌웨어를 변경하지 않고 실제 동작을 설명합니다.

예를 들어 illegal instruction에서 같은 mepc로 그냥 복귀하면 같은 잘못된 명령이 다시 예외를 발생시킵니다. 무조건 mepc에 4를 더해 건너뛰는 것도 정답이 아닙니다. 수행하지 못한 연산의 결과가 이후 코드에 필요할 수 있기 때문입니다. 현재 OS에는 오류 난 태스크만 종료하고 정상 문맥을 복구하는 정책이 없어, 진단 정보를 남기고 정상 실행을 중단합니다.

8.8 task_exit과 panic의 무한 루프가 다른 이유

차이를 결정하는 것은 반복문의 모양이 아니라 그곳에 도달했을 때의 interrupt 상태와 scheduler 정책입니다. rtl/rv32_csr.v는 trap 진입 시 다음과 같이 갱신합니다.

```
mstatus_r[7] <= mstatus_r[3]; // MPIE <- 이전 MIE
mstatus_r[3] <= 1'b0;        // MIE <- 0
```

첫 줄은 이전 interrupt 허용 상태를 보존합니다. 둘째 줄은 handler 수행 중 timer interrupt의 중첩 진입을 막습니다. 현재 handler는 panic에 도달하기 전에 MIE를 다시 켜지 않습니다. 정상 복귀라면 MRET가 MPIE를 MIE로 복구하지만 panic에서는 그 명령까지 도달하지 않습니다.

상황	timer에 의한 전환	결과
일반 모드 task_exit, MIE와 MTIE 활성화	가능	다른 태스크도 실행하지만 종료한 태스크의 차례마다 루프가 재개됩니다.
셀 모드 task_exit, MIE와 MTIE 활성화	trap은 가능	scheduler가 다시 셀을 선택하므로 같은 루프로 돌아갑니다.

상황	timer에 의한 전환	결과
trap_handler에서 호출한 현재 panic	불가능	MIE=0이고 MRET도 실행되지 않아 정상 태스크 실행이 중단됩니다.
interrupt를 꺼 둔 상태의 task_exit	불가능	timer만으로는 루프에서 벗어나지 못합니다.

timer counter는 계속 증가하고 timer_irq나 mip.MTIP가 1일 수도 있습니다. 그러나 요청이 존재하는 것과 CPU가 요청을 받아들이는 것은 다릅니다. 현재 RTL의 허용 식은 `mstatus.MIE && mie.MTIE && timer_irq`이므로 MIE=0이면 timer trap을 받지 않습니다.

CPU clock이나 FPGA 전체가 정지하는 것은 아닙니다. CPU는 루프 명령을 계속 실행하며 OS의 유용한 진행과 스케줄링이 중단됩니다. MIE=0은 모든 동기 예외까지 막는다는 뜻도 아닙니다. 이 설명은 정상적인 panic 출력과 반복 명령이 실행되는 경우의 timer 차단을 뜻합니다.

진단할 때는 먼저 UART의 cause와 PC를 기록하고 해당 빌드 ELF의 objdump 결과와 대조합니다. 원인을 확인하지 않고 재시작만 하면 고장 위치를 잃기 쉽습니다. 현재 시스템에서 정상 진행을 다시 시작하려면 원인 수정 후 재실행하거나 reset 등의 외부 개입이 필요하며, 단순히 무한 루프를 return으로 바꾸는 것은 복구 정책이 아닙니다.

8.9 trap_entry와 schedule은 누가 실행시키는가

질문을 단계별로 나누면 하드웨어의 trap 진입과 소프트웨어의 함수 호출을 구분할 수 있습니다. 먼저 `firmware/boot.S`가 목적지를 등록합니다.

```
la t0, trap_entry
csrw mtvec, t0
```

첫 줄은 linker가 결정한 trap_entry의 주소를 t0에 준비합니다. 둘째 줄은 그 주소를 mtvec CSR에 씁니다. 두 줄이 trap_entry를 지금 호출하는 것은 아닙니다. 앞으로 trap이 발생했을 때 이동할 목적지를 설정하는 작업입니다.

현재 `rtl/rv32_core.v`에는 다음 PC 갱신 경로가 있습니다.

```
else if (take_trap) pc <= csr_mtvec;
```

허용된 timer interrupt나 ECALL, illegal instruction 예외가 선택되면 CPU 하드웨어가 PC를 mtvec로 바꿉니다. 동시에 CSR 회로가 mepc·mcause·mstatus를 갱신합니다. 현재 Direct 모드에서는 모든 trap이 같은 진입점으로 들어갑니다. trap_entry는 일반 call 명령으로 호출되는 함수가 아니라, CPU가 trap을 받아 PC를 변경하여 실행하는 어셈블리 진입점입니다. 따라서 trap 진입 자체는 ra를 복귀 주소로 덮어쓰지 않습니다.

반면 trap_entry가 상태를 저장한 다음에는 `call trap_handler`라는 일반 함수 호출을 사용합니다. C handler가 원인을 읽고 필요한 경우 schedule을 호출합니다. 다음은 실제 소스에서 스케줄러를 사용하는 두 경로의 발췌입니다.

```
if (cause == CAUSE_MTIMER) {
    ticks++;
    timer_rearm();
    return schedule(f);
}
```

조건문은 timer trap인지 확인하고, 다음 두 문장은 tick 횟수와 다음 interrupt 시점을 갱신합니다. 마지막 문장은 schedule이 선택한 frame 포인터를 C handler의 반환값으로 그대로 전달합니다. 기존 timer 요청을 처리하지 않으면

복귀 직후 같은 요청을 다시 받을 수 있으므로 먼저 재예약합니다.

```
case SYS_YIELD:
    return schedule(f);
```

이 case는 ECALL 처리 switch 안에 있습니다. 태스크가 sys_yield를 호출하여 a7에 SYS_YIELD를 넣고 ECALL을 실행한 경우입니다. ECALL 공통 경로에서 이미 `f->mepc += 4`를 수행하므로 나중에 재개하면 ECALL 다음 명령부터 실행합니다. timer 경로에서는 같은 방식으로 4를 더하지 않습니다.

```
timer interrupt 또는 sys_yield의 ECALL
→ CPU가 PC를 mtvec로 변경
→ trap_entry가 레지스터 저장
→ call trap_handler
→ 원인에 따라 schedule(f)
→ 선택한 frame 포인터 반환
→ trap.S가 복원하고 mret
```

SYS_PUTC처럼 현재 태스크로 그대로 돌아가는 서비스는 보통 `return f`를 사용하여 `schedule`을 거치지 않습니다. 또한 `schedule` 호출이 반드시 다른 태스크로 바뀐다는 뜻은 아닙니다. 일반 모드는 순환 선택을 하지만 셸 모드는 `cur=1`로 셸을 다시 선택합니다. 하드웨어가 스케줄러의 C 이름을 알고 호출하는 것도 아닙니다. 태스크 선택 정책은 소프트웨어에만 있습니다.

8.10 C 함수의 반환 포인터가 어셈블리 복원으로 연결되는 과정

`trap_handler`는 `struct frame *`를 반환합니다. C 문법의 `return` 값이 어셈블리에서 보이지 않는 것은 아닙니다. RV32 호출 규약에 따라 이 포인터 값은 a0에 놓입니다. 호출 전의 첫 번째 인수도 a0에 전달하므로 같은 레지스터의 의미가 호출 경계에서 바뀝니다.

출처: firmware/trap.S의 인수 전달·호출·반환값 사용 명령입니다.

```
mv a0, sp
call trap_handler
mv sp, a0
```

명령	실행 전후의 의미
<code>mv a0, sp</code>	현재 frame 시작 주소를 첫 C 인수로 전달합니다. 원래 태스크의 a0는 이미 frame 안에 저장되어 있습니다.
<code>call trap_handler</code>	C 함수가 trap을 처리합니다. 정상 반환 시 a0에는 선택된 frame 주소가 있습니다.
<code>mv sp, a0</code>	이후 복원의 기준 주소를 반환된 frame으로 바꿉니다. 이때 원래 C 함수의 임시 스택은 정상 epilogue에서 정리된 상태입니다.

예를 들어 A의 frame 주소를 FA, B의 주소를 FB라고 하면 다음과 같습니다. 이는 주소 관계를 나타내는 기호이지 실제 코드의 symbol 이름은 아닙니다.

시점	sp	a0
A 상태 저장 후 인수 설정 완료	FA	FA
C handler가 B를 선택하고 반환	FA	FB

시점	sp	a0
mv sp, a0 이후	FB	FB
B의 x10 슬롯 복원 이후	FB	B의 저장된 a0 또는 syscall 결과

반환된 frame 포인터는 어느 CPU 문맥을 복원할지 지정합니다. 프레임 전체를 복사하지 않고 sp를 그 프레임으로 옮긴 뒤 같은 복원 코드를 실행합니다. 같은 태스크를 선택하면 FA가 그대로 반환되므로 동일한 코드가 일반 trap 복귀와 context switch를 모두 처리합니다.

이후 복원의 핵심 명령은 다음과 같습니다. 중간에 다른 레지스터 복원은 설명을 위해 생략했습니다.

```
lw    t0, 0(sp)
csrw  mepc, t0
lw    x1, 4(sp)
# 다른 레지스터 복원 생략
lw    x10, 40(sp)
# 다른 레지스터 복원 생략
addi  sp, sp, 128
mret
```

첫 두 줄은 RAM의 frame.mepc를 읽어 실제 CSR에 기록합니다. 셋째 줄은 선택한 태스크의 ra를 복원합니다. lw x10, 40(sp)는 a0를 태스크용 값으로 돌려놓습니다. 반환 포인터를 담았던 a0를 덮어써도 frame 주소는 이미 sp에 있으므로 문제가 없습니다. a0를 먼저 복원하고 나서 mv sp, a0를 하면 잘못된 주소를 선택하게 된다는 점에서 순서가 중요합니다.

마지막 addi는 frame 공간을 해제하여 선택한 태스크의 이전 sp를 복구합니다. 이어서 MRET가 mepc로 이동합니다. C의 return은 trap.S의 call 다음으로, MRET는 중단되었거나 새로 시작할 태스크로 이동하므로 서로 다른 두 단계의 복귀입니다.

8.11 C handler라는 용어의 의미

이 문서에서 C handler는 C 언어로 작성된 처리 함수를 뜻하며 구체적으로 kernel.c의 trap_handler를 가리킵니다. handler는 사건 처리 코드라는 일반 용어이지 별도의 CPU 장치나 C의 예약어가 아닙니다. compiler는 이 C 함수를 다른 함수와 마찬가지로 RISC-V 기계어로 변환합니다.

부분	작성 언어	담당 작업
trap_entry	어셈블리	C 코드를 안전하게 호출할 수 있도록 태스크 레지스터 저장
trap_handler	C	mcause 판별, syscall 서비스, timer 처리, 필요한 스케줄링
trap.S 복귀 부분	어셈블리	선택한 frame의 레지스터 복원과 MRET

어셈블리가 상태를 먼저 저장하지 않은 채 보통의 C 함수로 진입하면 함수 진입 코드가 레지스터를 바꾸어 원래 태스크 상태를 잃을 수 있습니다. 반대로 서비스 번호 판별이나 파일 시스템 동작까지 전부 어셈블리로 쓸 필요는 없습니다. 현재 설계는 정확한 상태 보존은 어셈블리, 처리 정책은 C에 맡깁니다.

“C handler의 스택”은 별도 전용 RAM을 뜻하지 않습니다. trap_handler라는 보통의 C 함수를 실행하기 위해 compiler가 만든 함수 프레임을 뜻합니다. 현재는 태스크의 trap frame 아래에 이 공간도 함께 놓입니다. C handler의 지역 포인터 f가 가리키는 trap frame과 C handler 자신의 함수 프레임을 혼동하지 않아야 합니다.

8.12 스택 사용 용도별 실제 코드와 역어셈블 예

이 절의 수치는 문서 작성 시 디스크에 존재하는 build/firmware/mini_shell.elf를 읽어 확인했습니다. 현재 소스를 새로 빌드하거나 보드의 실행 상태를 측정한 결과는 아닙니다. 재컴파일하면 최적화와 인라인에 따라 숫자가 달라질 수 있습니다. 다음 명령으로 자신의 ELF를 확인할 수 있습니다.

```
riscv64-unknown-elf-objdump -d build/firmware/mini_shell.elf
```

지역변수와 입력 버퍼

kernel.c의 task_shell은 다음 지역 데이터를 선언합니다.

```
char line[96];
uint32_t used = 0;
int ch, overflow = 0, last_cr = 0;
```

line은 UART에서 받은 명령행을 저장하는 96바이트 배열입니다. 입력 문자를 받을 때 `line[used++] = (char)ch`로 배열을 채웁니다. used는 다음 저장 위치, ch는 받은 문자, 나머지 값은 긴 입력과 줄 끝 처리를 위한 상태입니다. 이러한 스칼라 값은 compiler가 레지스터에 보관할 수도 있으므로 C 선언 크기를 모두 더하는 것만으로 스택 사용량을 구할 수 없습니다.

확인한 task_shell의 기계어에서 다음과 같은 함수 진입 코드를 볼 수 있습니다. 주소와 기계어 열은 생략하고 명령만 옮겼습니다.

```
addi sp, sp, -256
sw ra, 252(sp)
sw s0, 248(sp)
sw s1, 244(sp)
# 나머지 저장 레지스터 명령 생략
```

첫 줄은 256바이트의 함수 프레임을 확보합니다. 다음 세 줄은 그 안의 높은 offset에 복귀 주소와 저장 레지스터를 기록합니다. 따라서 지역 배열이 96바이트라는 사실과 함수 프레임이 256바이트라는 사실은 모순이 아닙니다. 프레임에는 다른 필요 데이터와 저장 레지스터, 정렬 공간 등이 함께 들어갑니다. compiler의 인라인으로 소스의 여러 함수가 하나의 기계어 함수에 합쳐질 수도 있습니다. 이 256바이트에 line의 96바이트를 다시 더하면 중복 계산입니다.

함수 호출을 위한 ra와 s0 보존

kernel.c의 uart_puts는 다음과 같이 문자열을 출력합니다.

```
static void uart_puts(const char *s)
{
    while (*s != '\0')
        uart_putc(*s++);
}
```

큰 지역 배열은 없지만 다른 함수를 호출합니다. 확인한 ELF에서 함수의 진입과 복귀 명령은 다음과 같습니다. 가운데의 문자 읽기와 호출 루프는 생략했습니다.

```
addi sp, sp, -16
sw ra, 12(sp)
sw s0, 8(sp)
mv s0, a0
# 문자 읽기와 uart_putc 호출 루프 생략
lw ra, 12(sp)
lw s0, 8(sp)
addi sp, sp, 16
ret
```

명령	주별 설명
첫 addi	정렬된 16바이트 프레임 확보합니다.
sw ra, 12(sp)	uart_puts를 호출한 곳으로 돌아갈 주소를 보존합니다. 내부 uart_putc 호출이 ra를 갱신하기 때문입니다.
sw s0, 8(sp)	호출 규약에서 보존을 요구하는 s0의 기존 값을 저장합니다.
mv s0, a0	첫 인수인 문자열 포인터를 이 함수의 작업용 s0에 보관합니다.
두 lw	호출자의 ra와 s0 값을 되살립니다.
마지막 addi와 ret	공간을 해제하고 복원한 ra로 돌아갑니다.

두 레지스터 값은 8바이트지만 프레임은 16바이트입니다. 호출 경계의 정렬 조건을 함께 충족하기 때문입니다. 지역 배열이 없다고 스택을 쓰지 않는 것은 아닙니다.

태스크 상태를 저장하는 trap frame

8.6절의 trap.S는 sp에서 128을 빼고 mepc와 x1부터 x31의 저장 슬롯을 채웁니다. 이는 임의 시점에 중단된 태스크 상태를 보존하는 공간입니다. 일반 함수의 호출 규약만 믿고 일부 임시 레지스터를 버리면 중단된 태스크가 쓰던 값을 잃을 수 있습니다. x0는 상수이므로 제외하고, x2는 원래 sp 값을 기록하지만 현재 복원은 frame 주소에 128을 더하는 방식입니다.

C handler와 그 하위 함수의 공간

확인한 trap_handler의 함수 진입 부분은 다음과 같습니다.

```
addi sp, sp, -32
sw ra, 28(sp)
sw s0, 24(sp)
sw s1, 20(sp)
sw s2, 16(sp)
sw s3, 12(sp)
sw s4, 8(sp)
```

128바이트 trap frame을 이미 만든 뒤 call trap_handler를 실행했으므로, 이 32바이트는 추가 공간입니다. 첫 줄은 공간 확보, 다음 여섯 줄은 C 함수의 복귀 주소와 사용하는 보존 레지스터를 저장합니다. trap frame의 ra는 태스크가 함수에서 돌아갈 주소이고, C handler 프레임의 ra는 trap.S의 call 다음으로 돌아갈 주소입니다. 같은 이름의 레지스터라도 서로 다른 시점의 값입니다.

handler가 호출하는 함수가 자기 프레임을 만들면 그것도 추가됩니다. 다만 모든 소스 함수가 별도 프레임을 가진다고 가정해서는 안 됩니다. compiler가 함수를 인라인하거나 tail call로 바꿀 수 있으므로 실제 call과 sp 조정 명령을 함께

확인해야 합니다. 위 `uart_puts`의 16바이트도 모든 `trap` 경로에서 무조건 더해지는 비용은 아닙니다.

8.13 하나의 스택에서 사용량을 합산하는 예

아래는 기존 ELF의 프레임 크기와 스택 `top`을 사용한 계산 예입니다. 셸이 자기 함수 프레임만 사용 중인 순간에 `timer trap`이 발생한다고 가정합니다. 다른 하위 함수가 실행 중인 경우에는 그만큼 추가해야 합니다.

단계	이번 단계의 추가 공간	sp 값	누적 사용량
셸 최초 진입 직전	0	0x1C40	0
<code>task_shell</code> 의 진입 코드	256바이트	0x1B40	256바이트
<code>trap_entry</code> 의 저장	128바이트	0x1AC0	384바이트
<code>trap_handler</code> 의 진입 코드	32바이트	0x1AA0	416바이트

```

높은 주소
0x1C40  태스크 스택 top
        task_shell 함수 프레임 256바이트
0x1B40  trap 직전 sp
        trap frame 128바이트
0x1AC0  trap frame 시작
        trap_handler 함수 프레임 32바이트
0x1AA0  C handler 진입 코드 이후 sp
        하위 함수 호출 시 낮은 주소 방향으로 추가 사용
0x1840  태스크 스택 배열의 시작 경계
낮은 주소

```

남은 공간은 이 가정에서 $1024 - 416 = 608$ 바이트입니다. 416바이트는 특정 순간의 사용량이지 최대 스택 사용량의 보장이 아닙니다. 동시에 살아 있는 함수 프레임과 `trap` 처리 경로를 합산해야 합니다. 가장 큰 지역 배열 하나만 보거나 모든 함수의 크기를 무조건 합산하는 방법은 모두 부정확합니다. 순차적으로 호출되어 이미 반환한 함수의 공간은 재사용할 수 있지만, 아직 반환하지 않은 호출자의 공간은 계속 살아 있습니다.

동일 태스크로 복귀하는 `timer` 경로에서는 C handler가 자기 32바이트를 정리하고, `trap.S`가 128바이트를 해제한 뒤 셸의 `sp=0x1B40`으로 돌아갑니다. 셸 함수가 계속 실행 중이므로 `top=0x1C40`까지 모두 되돌리는 것이 아닙니다. 다른 태스크를 선택했다면 마지막 복원은 그 태스크의 frame과 스택을 기준으로 진행됩니다.

이 예에서 처음 태스크 생성 때의 인공 frame 128바이트를 또 더하지 않습니다. 최초 `MRET` 이전에 그 frame은 이미 복원·해제되었고, 이후 함수 프레임과 `trap frame`이 같은 RAM을 다시 사용하기 때문입니다. 정적 예약 크기, 현재 사용량, 과거에 사용했던 공간을 구분해야 정확하게 계산할 수 있습니다.

9. MiniFS: 실행 파일을 보관하는 구조

9.1 RAM disk의 배치

9.1.1 RAM disk는 RAM을 파일 저장 공간으로 사용하는 방식이다

RAM disk는 메모리의 일정 영역을 디스크처럼 사용하는 것입니다. 현재 프로젝트에서는 SD card나 SSD의 저장장치 명령을 보내지 않습니다. CPU가 정해진 메모리 주소에 load/store하면 SoC의 별도 RAM 배열에 접근하고, MiniFS가 그 바이트들을 파일 시스템 형식으로 해석합니다.

`rv32_soc.v`의 실제 저장소는 `reg [31:0] ramdisk [0:2047]` 입니다. 32비트 word 2048개이므로 $2048 \times 4 = 8192$ 바이트 = 8 KiB 입니다. CPU에서 보이는 시작 주소는 `DISK_BASE = 0x80100000` 이고 마지막 바이트 주소는 `0x80101FFF` 입니다. 주소 값이 크다고 그 앞의 모든 주소에 실제 RAM이 있는 것은 아닙니다. address decoder가 이 8 KiB 범위를 해당 배열에 연결합니다.

MiniFS는 이 8 KiB를 512바이트 block 16개로 나눠 관리합니다. block은 파일 시스템의 논리적 관리 단위이고, RTL 배열의 4바이트 word와는 다릅니다. 한 block 안에는 $512 / 4 = 128$ 개 word가 들어 있습니다. hardware가 반드시 512바이트씩 한 번에 읽고 쓴다는 뜻이 아닙니다. 현재 software는 이름·파일 내용을 byte로, 일부 metadata를 32비트 word로 접근합니다.

```
전체 RAM disk: 16 blocks × 512 bytes = 8192 bytes
RTL 저장소:   2048 words × 4 bytes = 8192 bytes
한 FS block:  128 words × 4 bytes = 512 bytes
```

프로그램·stack을 담는 mem과 파일을 담는 ramdisk는 서로 다른 배열입니다. 특히 현재 instruction fetch는 mem만 읽습니다. 따라서 RAM disk에 저장한 앱을 실행하려면 loader가 내용을 프로그램 RAM의 `0x4000`으로 복사해야 합니다. RAM disk에 파일이 존재하는 것과 CPU의 실행 주소에 코드가 준비된 것은 서로 다른 상태입니다.

9.1.2 superblock·file table·file data는 서로 다른 질문에 답한다

세 영역의 의미를 먼저 구분하면 주소 계산을 이해하기 쉽습니다. superblock은 파일 시스템 전체의 설명, file table은 각 파일의 설명, file data는 실제 파일 내용입니다. 앞의 두 영역에 저장하는 설명 정보를 metadata라고 부릅니다.

영역	답하는 질문	현재 담는 정보
superblock	이 공간은 어떤 형식의 파일 시스템인가?	magic, version, block 수, block 크기
file table	어떤 파일이 있고 그 내용은 어디에 있는가?	이름, 유효한 바이트 수, 시작 block 번호
file data	그 파일의 실제 바이트는 무엇인가?	문자, APP1 header, 기계어, 기타 binary 데이터

예를 들어 `hello.txt`라는 이름과 크기 5는 file table에 있고, 문자열 `Hello`의 다섯 바이트는 file data 영역에 있습니다. superblock에 `hello.txt`의 이름을 기록하는 것이 아닙니다. 반대로 file table의 `name[16]` 안에 파일 내용 `Hello`를 넣는 것도 아닙니다.

저장 형식은 `minifs.h`의 상수·구조체와 `minifs.c`의 주소 계산으로 정해집니다. 다음 표의 offset은 RAM disk 시작을 0으로 보았을 때의 바이트 거리이며, 끝 주소는 해당 영역에 포함되는 마지막 바이트입니다.

block	byte offset	CPU 주소	내용
0	0x000 - 0x1FF	0x80100000 - 0x801001FF	superblock 예약 공간, 512바이트
1 - 2	0x200 - 0x5FF	0x80100200 - 0x801005FF	file table 예약 공간, 1024바이트
3 - 15	0x600 - 0x1FFF	0x80100600 - 0x80101FFF	file data, 6656바이트

즉 $512 + 1024 + 6656 = 8192$ 입니다. file data가 block 3에서 시작하는 이유는 block 0을 superblock에, block 1 - 2를 file table에 먼저 예약했기 때문입니다. 주소 0x80100000 자체에 “여기부터 파일 내용”이라는 hardware 의미가 있는 것이 아니라, MiniFS가 그 시작 부분을 metadata로 사용하기로 정한 것입니다.

9.1.3 superblock의 네 word를 바이트 단위로 읽기

현재 superblock은 앞의 16바이트만 의미 있는 field로 사용합니다. 나머지 $512 - 16 = 496$ 바이트는 예약 상태입니다. 실제 field가 16바이트라고 해서 file table이 바로 offset 16에서 시작하지는 않습니다. file table은 superblock에 예약한 전체 block을 건너뛴 offset 512에서 시작합니다.

C 접근	disk offset	저장 word	의미
header[0]	+0, 0x000	0x3153464D	magic: MiniFS 형식을 식별하는 표지 MFS1
header[1]	+4, 0x004	0x00000001	version: 현재 저장 형식의 버전 1
header[2]	+8, 0x008	0x00000010	block count: 전체 block 수 16
header[3]	+12, 0x00C	0x00000200	block size: block 하나의 크기 512바이트

header는 volatile fs_u32 *이므로 header[1]은 시작 주소의 +1바이트가 아니라 +4바이트에 있습니다. CPU 주소는 각각 0x80100000, 0x80100004, 0x80100008, 0x8010000C 입니다. 한 header 원소가 32비트라는 C 자료형 정보가 이 간격을 결정합니다.

little-endian 방식에서는 word의 가장 낮은 8비트를 가장 낮은 byte 주소에 저장합니다. 그래서 앞 16바이트를 byte 순서로 읽으면 다음과 같습니다.

```
disk offset 0x000:  4D 46 53 31  01 00 00 00
disk offset 0x008:  10 00 00 00  00 02 00 00
                  block count block size
```

처음 네 byte 4D 46 53 31은 ASCII 문자 M F S 1입니다. 32비트 숫자로 출력하면 역순으로 묶인 것처럼 보이는 0x3153464D이지만, 메모리의 낮은 주소부터 문자를 읽으면 정확히 MFS1입니다. magic은 데이터가 해당 형식처럼 보이는지 확인하는 표지이지, 전체 디스크의 checksum이나 암호학적 무결성 검사 값은 아닙니다.

version은 저장 규약을 바꿀 때 이전 형식과 구분하기 위한 값입니다. block count와 block size는 현재 코드가 예상하는 geometry, 즉 공간의 크기·분할 규칙을 확인합니다. 현재 fs_init()은 이 값을 읽어 임의의 크기 디스크에 맞춰 동적으로 재구성하지 않습니다. compile-time 상수 16·512와 같은지 검사합니다.

현재 superblock에는 파일 수, 빈 block 수, file table 주소, entry 크기 24, 최대 entry 수 32가 저장되지 않습니다. 이 정보는 코드의 상수와 구조체 정의에 들어 있습니다. 특히 구조체 크기나 table 배치를 바꾸면서 version을 그대로 두면, 예전 RAM 내용을 새 형식으로 잘못 읽을 수 있습니다. 저장 형식은 superblock 네 word만이 아니라 관련 코드 전체가 공유하는 약속입니다.

9.1.4 fs_init과 fs_format이 superblock을 사용하는 방식

fs_init() 은 header[0] - header[3]이 각각 magic·version·block count·block size와 일치하면 그대로 반환합니다. 일치하지 않으면 fs_format() 을 호출합니다. format은 파일 하나를 비우는 작업이 아니라 RAM disk 전체 8192바이트를 0으로 만들고 형식 정보를 다시 쓰는 작업입니다. 기존 파일은 유지되지 않습니다.

현재 format 순서는 전체 0 초기화 → version·block count·block size 기록 → magic 기록입니다. magic을 마지막에 쓰면 초기화 중간의 공간이 완성된 파일 시스템처럼 보일 가능성을 줄일 수 있습니다. 그러나 이 순서만으로 journal이나 원자적 transaction이 생기는 것은 아닙니다. 파일 쓰기 중 reset이 발생했을 때 metadata와 내용의 일관성을 완전히 보장하지 않습니다.

CPU reset은 RTL의 ramdisk 배열을 지우지 않으므로, 정상적인 superblock이 남아 있으면 재부팅 후 fs_init()이 기존 파일을 유지합니다. 반면 FPGA 재구성은 ramdisk 초기값을 다시 0으로 만들므로 이후 초기화에서 새 파일 시스템을 만듭니다. RAM 기반 저장소이므로 전원 종료 후 영구 보존도 보장하지 않습니다.

또한 네 word가 맞다고 모든 entry가 올바르다는 뜻은 아닙니다. 현재 init에는 name·size·start_block의 전체 범위 검사나 손상 복구 절차가 없습니다. 아래 배치·주소 예제는 정상 형식의 metadata를 전제로 합니다. 이 구분은 “형식을 인식한다”와 “파일 시스템 전체의 정합성을 검증한다”의 차이입니다.

코드 해부 9-B. fs_init과 fs_format의 한 줄씩 다른 책임

출처: [firmware/minifs.c](https://github.com/riscv-mini-shell/firmware/minifs.c), 원본 55 - 72행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
55 | int fs_format(void)
56 | {
57 |     fs_u32 i;
58 |     for (i = 0; i < FS_BLOCK_COUNT * FS_BLOCK_SIZE; i++) disk[i] = 0;
59 |     header[1] = FS_VERSION;
60 |     header[2] = FS_BLOCK_COUNT;
61 |     header[3] = FS_BLOCK_SIZE;
62 |     header[0] = FS_MAGIC;
63 |     return 0;
64 | }
66 | int fs_init(void)
67 | {
68 |     if (header[0] == FS_MAGIC && header[1] == FS_VERSION &&
69 |         header[2] == FS_BLOCK_COUNT && header[3] == FS_BLOCK_SIZE)
70 |         return 0;
71 |     return fs_format();
72 | }
```

원본 행	한 줄씩 읽는 동작과 의미
55	전체 RAM disk를 빈 MiniFS로 만드는 함수입니다. 기존 파일을 보존하는 작업이 아닙니다.
56	앞에서 선언한 함수·블록의 본문을 시작합니다.
57	초기화 byte index를 선언합니다.
58	16×512바이트 전부를 0으로 씁니다. CPU의 store 반복이므로 시간과 메모리 부작용을 갖습니다.
59	format version을 header의 두 번째 word에 기록합니다.
60	전체 block 수 16을 기록합니다.
61	block 크기 512를 기록합니다.
62	마지막에 magic을 기록합니다. 완성 표지를 뒤에 쓰는 순서지만 journal이나 transaction을 제공하지는 않습니다.
63	format 성공을 반환합니다.

원본 행	한 줄씩 읽는 동작과 의미
64	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
66	부팅 때 현재 RAM disk를 사용할 수 있는지 확인하는 함수입니다.
67	앞에서 선언한 함수·블록의 본문을 시작합니다.
68	magic과 version이 현재 코드가 기대하는지 확인합니다.
69	block 수와 크기도 일치해야 기존 파일 시스템으로 인정합니다.
70	네 조건이 맞으면 아무 byte도 지우지 않고 반환합니다.
71	맞지 않으면 전체 format을 수행합니다. 따라서 정상 CPU reset은 보존, 초기 0 상태는 새 format으로 연결됩니다.
72	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

header 네 word의 일치만 검사하므로 각 파일의 손상까지 검증하는 mount는 아닙니다. 파일 시스템 형식 확인과 데이터 무결성 검사를 구분해야 합니다.

9.1.5 file table은 파일을 찾아가는 metadata 배열이다

file table에는 크기가 같은 `struct file_entry` 32개가 연속으로 놓입니다. entry는 표의 한 행에 해당하며 “이 이름의 파일은 몇 바이트이고 어느 block에서 시작하는가”를 설명합니다. entry 하나의 24바이트 구성과 주소식은 9.2절에서 계산합니다.

table에 예약한 공간은 2개 block, 즉 1024바이트입니다. 실제 entry 배열은 $32 \times 24 = 768$ 바이트만 사용하므로 남은 256바이트는 현재 사용하지 않습니다. table의 실제 끝은 disk offset `0x4FF` 이고, 예약된 여백은 `0x500 - 0x5FF` 입니다. 다음 file data는 예약 공간까지 모두 지난 `0x600` 에서 시작합니다.

이 구조에서는 최대 파일 이름 수와 데이터 용량이 서로 다른 제한입니다. table slot이 32개이므로 빈 파일도 최대 32개까지 이름을 가질 수 있습니다. 하지만 data block은 13개뿐이므로 파일마다 최소 한 block이 필요한 비어 있지 않은 파일은 모두 한 block만 사용하더라도 최대 13개입니다. 큰 파일이나 연속 공간 부족 때문에 그보다 적은 수에서도 쓰기가 실패할 수 있습니다.

`ls` 는 사용 중인 entry의 이름과 size를 읽어 표시합니다. 데이터 영역의 문자 바이트를 처음부터 스캔해 파일 이름을 추측하지 않습니다. `cat hello.txt` 도 먼저 table에서 이름을 찾은 뒤 해당 entry의 위치·길이를 사용합니다. 따라서 metadata가 파일 내용에 접근하는 출발점입니다.

9.1.6 file data와 파일 내부 offset

file data는 파일의 실제 byte들을 저장하는 영역입니다. 현재 MiniFS는 한 파일을 연속된 block들에 배치합니다. entry의 `start_block` 은 RAM disk 전체를 기준으로 센 block 번호입니다. file data 영역 안에서 다시 0부터 센 번호가 아닙니다. 예를 들어 `start_block=3` 은 전체 disk의 네 번째 block, 즉 첫 data block을 뜻합니다.

block 번호 `b` 의 첫 byte 주소와 그 파일의 `j` 번째 byte 주소는 다음처럼 계산합니다. `b` 는 block 단위, `j` 는 byte 단위입니다.

```
block b의 시작 주소 = DISK_BASE + b × 512
파일의 byte j 주소 = DISK_BASE + start_block × 512 + j
정상 파일 byte 범위: 0 ≤ j < size
```

첫 data block 3은 $0x80100000 + 3 \times 0x200 = 0x80100600$ 에서 시작합니다. 이것을 `DISK_BASE + 512 + 24*i` 라는 entry 주소식과 구분해야 합니다. 앞의 식은 파일의 내용으로 가고, 뒤의 식은 파일을 설명하는 table 행으로

갑니다. i 번째 entry가 i 번째 data block에 배치된다는 규칙은 없습니다.

예를 들어 $\text{size}=600$, $\text{start_block}=3$ 인 파일을 가정하면 첫 512바이트는 block 3에, 나머지 88바이트는 block 4에 들어갑니다. 마지막 유효 byte는 $j=599$ 이므로 $0x80100600 + 599 = 0x80100857$ 입니다. 두 block의 예약 범위는 $0x80100600 - 0x801009FF$ 이지만 마지막 424바이트는 그 파일의 유효한 내용이 아닙니다.

텍스트 파일도 byte 배열입니다. 파일에 문자열 Hello 만 기록하면 논리 size는 5이고, C 문자열 끝의 NUL을 자동으로 추가하여 $\text{size}=6$ 으로 만들지 않습니다. binary 파일은 중간에 0x00이 얼마든지 들어갈 수 있으므로 읽기 종료를 NUL로 판단하면 안 됩니다. MiniFS는 entry의 size로 파일 끝을 판단합니다.

9.1.7 파일 시스템 주소를 RTL word index로 바꾸기

CPU는 byte 주소를 만들지만 ramdisk 배열 원소는 4바이트 word입니다. 이 범위에서 disk-relative offset을 O 라고 하면 $\text{word index} = O / 4$, $\text{byte lane} = O \% 4$ 입니다. RTL은 선택된 주소의 [12:2] 를 word index로 사용하고, core가 [1:0] 에 따라 byte를 고르거나 write strobe를 만듭니다.

대상	disk offset	RTL word index	의미
superblock magic	0	0	첫 header word
entry 0 시작	512	128	첫 파일 이름의 앞 4바이트
entry 0의 size	528	132	table 행의 +16바이트 field
entry 0의 start_block	532	133	table 행의 +20바이트 field
data block 3 시작	1536	384	첫 data word

따라서 testbench가 `ramdisk[384]` 에서 파일의 첫 문자를 검사하고, `ramdisk[128][7:0]` 에서 첫 entry의 사용 여부를 검사하는 것은 서로 다른 역할입니다. 전자는 내용, 후자는 이름의 첫 byte입니다. “파일이 지워졌다”를 확인할 때 내용이 0인지가 아니라 table의 이름 첫 byte가 0인지 확인하는 이유도 9.3절의 삭제 정책과 연결됩니다.

SoC hardware 자체는 superblock이나 file table을 해석하지 않습니다. `ramdisk[132]` 에 SW를 하는 것은 hardware 관점에서는 일반 RAM write이고, MiniFS 관점에서는 첫 파일의 size를 바꾸는 동작입니다. 같은 bit 저장에 software가 구조와 의미를 부여한 것입니다.

9.2 file table entry

9.2.1 entry 하나가 왜 24바이트인가

현재 구조체를 익숙한 고정폭 정수 이름으로 나타내면 다음과 같습니다. 실제 [minifs.h](#)는 `fs_u32` 를 사용하고, 현재 RV32 toolchain에서 이 자료형은 4바이트 unsigned integer입니다.

```
struct file_entry {
    char name[16];
    uint32_t size;
    uint32_t start_block;
};
```

field	entry 내부 offset	크기	담는 값
name[16]	+0 - +15	16바이트	파일 이름과 끝을 나타내는 NUL

field	entry 내부 offset	크기	담는 값
size	+16 - +19	4바이트	실제 파일 내용의 byte 수
start_block	+20 - +23	4바이트	내용이 시작하는 전체 disk block 번호

합계는 $16 + 4 + 4 = 24$ 바이트 = 0x18바이트입니다. 현재 ABI에서는 size와 start_block의 4바이트 정렬 조건을 각각 offset 16과 20에서 만족하고, 전체 크기 24도 4의 배수이므로 이 구조체에는 추가 padding이 필요하지 않습니다. 구조체는 언제나 field 크기만 더하면 된다는 일반 규칙이 아니라 **현재 field 순서와 ABI에서 성립하는 결과**입니다.

이름의 공간은 실제 이름 길이와 무관하게 항상 16바이트입니다. 이름이 a 여도 entry는 24바이트이고, hello.txt 여도 24바이트입니다. 따라서 entry 간 간격은 고정입니다. 이름에는 NUL이 포함되어야 하므로 표시 가능한 이름은 ASCII 기준 최대 15문자, 보다 정확하게는 NUL을 제외한 최대 15바이트입니다. UTF-8의 한 글자가 여러 byte라면 15글자와 같지 않을 수 있습니다.

이 형식은 현재 CPU의 little-endian 정수 배치와 C 구조체 layout에 기대고 있습니다. 다른 host 프로그램에서 디스크 이미지를 만든다면 field의 byte offset과 정수 byte 순서를 같은 규칙으로 직렬화해야 합니다. host의 임의 구조체 메모리를 그대로 복사하는 것만으로 호환성을 보장할 수 없습니다.

9.2.2 DISK_BASE + 512 + 24*i를 세 단계로 유도하기

entry 번호 i의 주소는 $DISK_BASE + 512 + 24*i$ 입니다. 이 식은 특별한 RISC-V 규칙이 아니라 “배열 시작 주소 + 앞선 원소들의 byte 수”입니다. entry 번호는 0부터 세므로 i=0은 첫 번째 파일 항목이고, 유효한 범위는 i=0 - 31입니다.

첫째, DISK_BASE는 전체 RAM disk의 시작인 0x80100000입니다. 이곳에는 file table이 아니라 superblock이 있으므로, 아직 파일 entry 위치에 도착한 것이 아닙니다.

둘째, +512는 superblock에 예약한 block 0 전체를 건너뛰는 거리입니다. 따라서 $DISK_BASE + 512 = 0x80100200$ 이 file table의 시작이고 entry 0의 시작입니다. superblock에서 사용하는 header가 16바이트라고 +16으로 바꾸면 table이 아니라 superblock 예약 영역을 가리키게 됩니다.

셋째, +24*i는 table의 시작에서 i개의 entry를 건너뛰는 거리입니다. entry 0에 도착하려면 아무것도 건너뛰지 않고, entry 1에는 24바이트, entry 2에는 48바이트를 건너뛸 것입니다. i가 하나 증가할 때마다 주소가 정확히 24바이트씩 증가합니다.

```
entry_address(i)
= RAM disk 시작 주소
+ superblock 예약 크기
+ 앞선 entry 개수 × entry 하나의 크기

= DISK_BASE + 512 + i × 24
= 0x80100000 + 0x200 + i × 0x18
```

여기서 512와 24는 십진수 바이트 수이고, 각각 16진수로는 0x200과 0x18입니다. +24를 +0x24와 혼동하면 간격이 24바이트가 아니라 36바이트가 됩니다. 또한 i는 파일 크기도 block 번호도 아니고 **file table 행 번호**입니다.

entry 번호 i	앞선 entry들의 크기 24*i	disk-relative 시작 offset	CPU 시작 주소
0	0	0x200	0x80100200
1	24	0x218	0x80100218
2	48	0x230	0x80100230

entry 번호 i	앞선 entry들의 크기 24*i	disk-relative 시작 offset	CPU 시작 주소
3	72	0x248	0x80100248
20	480	0x3E0	0x801003E0
21	504	0x3F8	0x801003F8
22	528	0x410	0x80100410
31	744	0x4E8	0x801004E8

마지막 entry 31은 0x801004E8 부터 24바이트를 사용하므로 끝 주소는 0x801004FF 입니다. 바로 다음 0x80100500 - 0x801005FF 의 256바이트는 table 예약 여백입니다. 산술식에 i=32를 넣어 주소가 계산된다고 유효한 33번째 파일 entry가 생기는 것은 아닙니다. software의 FS_MAX_FILES는 32입니다.

9.2.3 C pointer 연산에는 원소 크기가 이미 포함된다

minifs.c는 file table 시작을 다음과 같이 정의합니다.

```
static volatile struct file_entry *const files =
    (volatile struct file_entry *) (DISK_BASE + FS_BLOCK_SIZE);
```

DISK_BASE + FS_BLOCK_SIZE 는 정수 주소 계산이므로 먼저 0x80100200 을 만듭니다. 그 뒤 이 주소를 file_entry를 가리키는 pointer로 해석합니다. 이 선언 자체가 file table 메모리를 새로 할당하는 것은 아닙니다. 이미 SoC가 제공하는 RAM disk의 정해진 위치에 C 구조체 배열이라는 해석을 붙입니다.

files + i 또는 &files[i] 는 C pointer 연산이므로 자동으로 i * sizeof(struct file_entry) 만큼 이동합니다. 따라서 compiler가 사용하는 byte 주소는 자연스럽게 0x80100200 + 24*i 가 됩니다. source에 files[i] 라고만 적혀 있어도 그 안에 24바이트 간격이 포함되어 있습니다.

표현	이동 단위	i=1일 때의 주소
files + i	24바이트 entry	0x80100218
&files[i]	24바이트 entry	0x80100218
((volatile fs_u8 *)files) + 24*i	1바이트를 24*i개	0x80100218
files + 24*i	24바이트 entry를 24*i개	0x80100440, 의도와 다름

마지막 식은 24배를 두 번 적용하여 $24 \times 24 = 576$ 바이트 이동한 결과입니다. byte offset을 계산하는 식과 typed pointer의 원소 이동을 섞으면 이런 오류가 생깁니다. 같은 이유로 header+1은 4바이트, disk+1은 1바이트, files+1은 24바이트입니다. 자료형이 모두 다르기 때문입니다.

volatile 은 이 주소의 실제 읽기·쓰기를 compiler가 일반 임시 값처럼 생각하지 않도록 표현합니다. 뒤쪽의 const 는 files pointer 자체를 다른 주소로 바꿀 수 없다는 뜻입니다. file table 내용을 read-only로 만든다는 뜻이 아니므로 files[i].size = size 처럼 내용을 갱신할 수 있습니다.

현재 RV32I에는 MUL instruction이 없지만 24*i 계산은 가능합니다. compiler는 상수 곱을 shift·add로 만들거나 반복문에서 pointer를 24씩 증가시킬 수 있습니다. 예를 들어 (i << 4) + (i << 3) 은 정상 index 범위에서 $16*i + 8*i = 24*i$ 입니다. 정확한 instruction 선택은 최적화 결과에 따라 달라도 주소 계산의 의미는 같습니다.

코드 해부 9-A. disk·header·files가 같은 RAM을 다르게 읽는 줄

출처: [firmware/minifs.c](#), 원본 13 - 21행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

13 | #define DISK_BASE 0x80100000u
14 | #define DATA_FIRST_BLOCK 3u
15 | #define FS_MAGIC 0x3153464du
16 | #define FS_VERSION 1u
18 | static volatile fs_u8 *const disk = (volatile fs_u8 *)DISK_BASE;
19 | static volatile struct file_entry *const files =
20 |     (volatile struct file_entry *) (DISK_BASE + FS_BLOCK_SIZE);
21 | static volatile fs_u32 *const header = (volatile fs_u32 *)DISK_BASE;

```

원본 행	한 줄씩 읽는 동작과 의미
13	전체 RAM disk의 CPU 시작 주소입니다. 파일 내용 시작 주소와 구분합니다.
14	block 0 - 2는 metadata이므로 파일 내용은 block 3부터 할당합니다.
15	MiniFS 형식 식별 magic을 정의합니다. little-endian byte는 MFS1입니다.
16	현재 저장 형식의 version입니다.
18	같은 disk 시작을 1바이트 원소의 pointer로 봅니다. disk[j]의 실제 주소는 DISK_BASE+j입니다.
19	24바이트 file_entry 원소를 가리키는 고정 pointer 선언을 시작합니다.
20	file table은 시작에서 512바이트 뒤입니다. files[i]는 여기서 i×24바이트를 더한 주소가 됩니다.
21	disk 시작을 4바이트 word pointer로 봅니다. header[1]은 +1바이트가 아니라 +4바이트입니다.

세 pointer는 RAM을 새로 만드는 선언이 아닙니다. 이미 RTL이 제공하는 같은 주소 공간에 byte 배열·word 배열·구조체 배열이라는 서로 다른 C 해석을 붙입니다. const는 pointer 값의 변경을 제한하고 volatile은 대상 접근의 의미를 compiler에 알립니다.

9.2.4 entry 주소에서 size·start_block·파일 내용으로 이동하기

entry 시작 주소를 E라고 하면 이름은 E부터, size는 E+16부터, start_block은 E+20부터 저장됩니다. 이 숫자는 entry 내부 field offset이며, disk 시작 기준 offset과는 또 다른 기준입니다.

```

E = DISK_BASE + 512 + 24*i
name[j]의 주소      = E + j          (0 <= j < 16)
size field의 주소   = E + 16
start_block의 주소  = E + 20

```

예를 들어 i=20이면 E = 0x80100000 + 0x200 + 0x30 = 0x80100230 입니다. name의 주소 범위는 0x80100230 - 0x8010023F, size는 0x80100240 - 0x80100243, start_block은 0x80100244 - 0x80100247 입니다. 다음 entry 3이 0x80100248 부터 시작하므로 정확히 이어집니다.

이 entry의 size field에 5, start_block field에 3이 저장되어 있다고 가정해 봅니다. 0x80100244는 block 번호를 보관한 field의 주소이고, 3은 그 field에서 읽은 값입니다. 실제 내용의 첫 주소는 그 값을 이용해 다시 계산한 0x80100000 + 3*512 = 0x80100600 입니다. 파일의 다섯 번째 byte는 j=4이므로 0x80100604 입니다.

```

find_file(name) -> i = 2
                  |
entry E = 0x80100230
  +-- E+16 -> size = 5
  +-- E+20 -> start_block = 3

```

```

data_base = DISK_BASE + 3*512 = 0x80100600
byte[4]    = *(data_base + 4) @ 0x80100604

```

위 예제는 주소식의 의미를 보여 주는 가정이며, 첫 파일이 반드시 entry 2에 들어간다는 뜻은 아닙니다. 현재 fs_create()는 첫 번째 빈 entry를 선택하고, 데이터 allocator는 별도로 사용 가능한 연속 block을 찾습니다. 파일 이름이나 entry 번호만으로 data block을 예측하지 말고 start_block의 저장 값을 읽어야 합니다.

start_block은 pointer도 앱 entry 주소도 아닙니다. hello.app의 start_block이 3이어도 앱의 링크·실행 주소는 0x4000입니다. RAM disk의 내용 위치는 파일 시스템이 정하고 실행 RAM 위치는 앱 ABI와 loader가 정합니다.

9.2.5 24바이트 entry는 512바이트 block 경계에 걸칠 수 있다

512는 24로 나누어떨어지지 않습니다. $512 = 24 \times 21 + 8$ 이므로 block 1에 entry 0 - 20의 21개를 모두 넣으면 8바이트가 남습니다. 현재 MiniFS는 여기서 다음 entry를 강제로 새 block으로 옮기거나 padding을 넣지 않습니다. file table은 block 1 - 2에 걸친 하나의 연속 배열이기 때문입니다.

그 결과 entry 21은 $\text{DISK_BASE} + 512 + 24 \times 21 = 0x801003F8$ 에서 시작하며, block 1의 마지막 8바이트와 block 2의 처음 16바이트를 사용합니다.

entry 21 내부	CPU 주소 범위	속하는 block
name[0] - name[7]	0x801003F8 - 0x801003FF	1
name[8] - name[15]	0x80100400 - 0x80100407	2
size	0x80100408 - 0x8010040B	2
start_block	0x8010040C - 0x8010040F	2

이것은 현재 RAM 기반 구현에서 문제가 아닙니다. byte 주소는 연속되어 있고, size·start_block의 각 32비트 접근은 여전히 4바이트 정렬입니다. “entry가 FS block 경계를 가로지른다”는 것과 “LW/SW가 비정렬 주소를 접근한다”는 것은 다른 문제입니다.

만약 파일 시스템 아래에 512바이트 sector 단위 I/O만 제공하는 SD card driver를 넣는다면, 이런 entry를 읽기 위해 두 sector의 buffer를 다루는 계층이 필요할 수 있습니다. 그것은 미래 저장장치 계층의 책임입니다. 현재 배열에 block마다 임의의 빈칸을 넣으면 $24 \times i$ 라는 software의 주소 규약과 실제 배치가 어긋납니다.

9.2.6 첫 파일을 저장했을 때 metadata와 내용의 실제 값

새로 format되어 모든 byte가 0인 MiniFS에 hello.txt를 만들고 Hello 5바이트를 기록했다고 가정합니다. 현재 first-fit 규칙에 따라 첫 빈 entry 0과 첫 data block 3을 사용합니다. 이는 설명용 초기 조건이며 기존 보드의 파일을 지우라는 실행 지시가 아닙니다.

entry 주소는 $\text{DISK_BASE} + 512 + 24 \times 0 = 0x80100200$ 이고, 파일 이름은 그 위치에 저장됩니다. size=5는 0x80100210, start_block=3은 0x80100214에 저장됩니다. 실제 Hello는 0x80100600에서 시작합니다.

CPU 주소	낮은 주소부터 byte 4개	32비트 word로 읽은 값	의미
0x80100200	68 65 6C 6C	0x6C6C6568	이름의 hell
0x80100204	6F 2E 74 78	0x78742E6F	이름의 o.tx
0x80100208	74 00 00 00	0x00000074	이름의 t와 NUL·초기값

CPU 주소	낮은 주소부터 byte 4개	32비트 word로 읽은 값	의미
0x8010020C	00 00 00 00	0x00000000	이름 field의 남은 초기값
0x80100210	05 00 00 00	0x00000005	size=5
0x80100214	03 00 00 00	0x00000003	start_block=3
0x80100600	48 65 6C 6C	0x6C6C6548	내용의 Hell
0x80100604	6F 00 00 00	0x0000006F	내용의 o 와 뒤쪽 초기값

마지막 행 뒤쪽의 0들은 새로 format했다는 초기 조건 때문에 0입니다. fs_write()가 파일 뒤를 항상 0으로 채우거나 NUL을 추가하기 때문이 아닙니다. 삭제된 block을 재사용하는 경우에는 예전 byte가 남아 있을 수 있습니다. 정상 읽기는 size=5까지만 사용하므로 그 뒤 byte를 파일 내용으로 반환하지 않습니다.

이 상태에서 ls는 entry 0을 읽어 hello.txt 5 bytes를 출력하고, cat hello.txt는 start_block=3을 읽어 data 주소를 계산한 뒤 5바이트를 읽습니다. 따라서 이름의 소문자 h와 내용의 대문자 H는 서로 다른 주소에 있습니다. 파형에서 table 첫 word 0x6C6C6568 과 data 첫 word 0x6C6C6548 을 구분해 보면 이 차이가 명확합니다.

9.2.7 사용 중인 entry와 빈 파일 및 삭제의 차이

현재 사용 중 여부는 name[0] != 0으로 판단합니다. 별도의 valid bit나 inode bitmap은 없습니다. **size=0은 비어 있는 파일일 수 있으므로 빈 entry를 뜻하지 않습니다.** fs_create() 직후에는 이름이 있고 size=0·start_block=0인 정상 파일입니다. 내용이 없으므로 data block도 아직 필요하지 않습니다.

fs_create()는 size·start_block과 이름 뒤쪽을 먼저 준비하고 name[0]을 마지막에 기록합니다. 마지막 첫 글자가 공개되기 전에는 검색 함수가 빈 entry로 취급하도록 하는 순서입니다. 이 방식도 미완성 entry를 덜 노출하려는 단순 기법이지 멀티코어 동기화나 저장 매체 transaction을 대신하지는 않습니다.

fs_delete()는 name[0]=0으로 하고 size·start_block도 0으로 바꿉니다. 이름 field의 나머지 byte와 실제 data block을 모두 지우지는 않습니다. 그래서 debugger에서 예전 문자열 조각이 보이더라도 table의 첫 byte가 0이면 정상 API에서는 파일을 찾지 못합니다. 반대로 name[0]이 살아 있으면 내용 크기가 0이어도 ls에 파일이 나타납니다.

핵심 주소 관계를 다시 정리하면 disk 시작 → table 시작 → entry 시작 → field 주소 → field에서 읽은 block 번호 → 실제 data 주소입니다. 이 경로의 각 단계에서 무엇이 주소이고 무엇이 저장된 값인지, 단위가 byte인지 block인지 구분하면 DISK_BASE + 512 + 24*i 를 외우지 않고 다시 유도할 수 있습니다.

9.3 할당·읽기·삭제

파일 크기에서 필요한 block 수는 다음처럼 계산합니다.

```
blocks = (size + 511) >> 9
```

할당기는 table을 조사해 사용 중인 extent와 겹치지 않는 연속 구간을 찾습니다. 현재는 별도 free bitmap이 없습니다. **전체 빈 공간이 충분해도 연속 공간이 부족하면 쓰기에 실패할 수 있습니다.**

fs_read()는 버퍼에 파일 전체가 들어갈 때만 성공합니다. fs_read_at()은 offset부터 capacity만큼 부분 읽기를 허용하므로 셸의 cat 과 앱 로더가 64바이트 단위로 읽을 수 있습니다. EOF는 0, 없는 파일이나 오류는 -1입니다.

삭제는 table entry의 이름 첫 바이트를 0으로 하고 크기와 시작 block을 비웁니다. 데이터 block을 모두 0으로 지우지는 않습니다. 이후 할당기가 그 구간을 다시 사용합니다. 삭제된 바이트가 물리적으로 즉시 사라진다는 뜻은 아닙니다.

연속 할당의 단편화를 작은 예로 보겠습니다. data block 3, 5, 7이 비어 있고 다른 block은 사용 중이면 빈 공간은 총 3개 block이지만 2개 연속 block을 요구하는 파일은 들어갈 수 없습니다. 현재 allocator는 파일의 block 목록을 나누어 기록하는 구조가 아니므로 가장 긴 연속 빈 구간이 중요한 조건입니다.

기존 파일을 덮어쓸 때 allocator는 그 파일이 쓰던 구간을 재사용 후보로 취급합니다. 공간을 찾지 못하면 이전 metadata를 바꾸지 않고 실패하지만, 복사 중 전원·reset 문제까지 보장하는 transactional write는 아닙니다. `fs_read_at`은 매 호출마다 이름을 찾아 offset을 계산하므로 fd와 지속적인 file position을 관리하는 POSIX API와도 다릅니다.

코드 해부 9-C. 연속 block 할당의 겹침 검사

출처: [firmware/minifs.c](#), 원본 97 - 118행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

97 | static int find_extent(fs_u32 needed, int skip_file)
98 | {
99 |     fs_u32 first, i;
100 |     if (needed == 0) return 0;
101 |     if (needed > FS_BLOCK_COUNT - DATA_FIRST_BLOCK) return -1;
102 |     for (first = DATA_FIRST_BLOCK; first + needed <= FS_BLOCK_COUNT; first++) {
103 |         int free_run = 1;
104 |         for (i = 0; i < FS_MAX_FILES; i++) {
105 |             fs_u32 size, blocks, start;
106 |             if ((int)i == skip_file || !files[i].name[0]) continue;
107 |             size = files[i].size;
108 |             blocks = (size + FS_BLOCK_SIZE - 1u) >> 9;
109 |             start = files[i].start_block;
110 |             if (blocks && first < start + blocks && start < first + needed) {
111 |                 free_run = 0;
112 |                 break;
113 |             }
114 |         }
115 |         if (free_run) return (int)first;
116 |     }
117 |     return -1;
118 | }

```

원본 행	한 줄씩 읽는 동작과 의미
97	needed개의 연속 block을 찾습니다. 기존 파일을 덮어쓰면 그 파일의 block은 재사용할 수 있도록 skip_file을 받습니다.
98	앞에서 선언한 함수·블록의 본문을 시작합니다.
99	후보 시작 block과 table 순회 index를 선언합니다.
100	0바이트 파일은 데이터 block이 필요 없으므로 0을 반환합니다.
101	전체 data block 수보다 큰 요구는 즉시 거절합니다.
102	block 3부터 차례로 후보를 시험하되 후보 끝이 disk 끝을 넘지 않아야 합니다.
103	처음에는 후보 구간이 비어 있다고 가정합니다.
104	각 파일 entry와 충돌하는지 확인합니다.
105	현재 검사할 파일의 크기·block 수·시작을 담습니다.
106	덮어쓸 자기 파일이나 사용하지 않는 entry는 충돌 대상으로 보지 않습니다.
107	파일의 논리 byte 크기를 읽습니다.
108	512바이트 단위로 올림합니다. shift 9는 512로 나누는 효과입니다.
109	그 파일이 사용 중인 첫 block 번호를 읽습니다.

원본 행	한 줄씩 읽는 동작과 의미
110	두 반개구간 [first,first+needed), [start,start+blocks)가 겹치는지 검사합니다. 경계가 맞닿기만 한 것은 충돌이 아닙니다.
111	겹침을 발견했으므로 현재 후보를 사용할 수 없다고 표시합니다.
112	다른 파일을 더 확인할 필요 없이 이 후보 검사를 끝냅니다.
113	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
114	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
115	모든 entry와 겹치지 않았다면 첫 적합 후보를 반환합니다.
116	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
117	어떤 후보도 없으면 -1입니다. 빈 block 합계가 충분해도 연속 공간 부족으로 여기에 도달할 수 있습니다.
118	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

예를 들어 기존 구간 [4,6)과 후보 [6,8)은 $6 < 6$ 이 거짓이므로 겹치지 않습니다. 후보 [5,7)은 두 부등식이 모두 참이라 거절합니다. 이 두 경우를 손으로 계산하면 조건식의 <가 왜 (=가 아닌지 이해할 수 있습니다.

코드 해부 9-D. fs_write의 데이터 복사와 metadata 갱신 순서

출처: [firmware/minifs.c](#), 원본 120 - 136행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

120 | int fs_write(const char *name, const void *data, fs_u32 size)
121 | {
122 |     int slot = find_file(name);
123 |     fs_u32 i, blocks, off;
124 |     int first;
125 |     const fs_u8 *src = (const fs_u8 *)data;
126 |     if (slot < 0 || (size && !data) ||
127 |         size > (FS_BLOCK_COUNT - DATA_FIRST_BLOCK) * FS_BLOCK_SIZE) return -1;
128 |     blocks = (size + FS_BLOCK_SIZE - 1u) >> 9;
129 |     first = find_extent(blocks, slot);
130 |     if (first < 0) return -1;
131 |     off = (fs_u32)first * FS_BLOCK_SIZE;
132 |     for (i = 0; i < size; i++) disk[off + i] = src[i];
133 |     files[slot].start_block = (fs_u32)first;
134 |     files[slot].size = size;
135 |     return (int)size;
136 | }
```

원본 행	한 줄씩 읽는 동작과 의미
120	이름으로 찾은 파일 내용을 size바이트로 교체하는 함수입니다. append 위치 인수는 없습니다.
121	앞에서 선언한 함수·블록의 본문을 시작합니다.
122	이미 만들어진 파일 entry를 찾습니다. fs_create 없이 없는 이름에 쓰면 실패합니다.
123	복사 index, 필요한 block 수, disk 내부 byte offset을 준비합니다.
124	할당기의 성공 block 번호 또는 오류 -1을 받을 signed 변수입니다.
125	source를 byte pointer로 해석해 임의 길이 binary를 복사할 수 있게 합니다.
126	파일이 없거나, 0보다 큰 크기인데 데이터 pointer가 NULL이면 거절합니다.
127	요구 크기가 전체 data 영역을 넘는지도 검사합니다.
128	byte 크기를 필요한 block 수로 올립니다.

원본 행	한 줄씩 읽는 동작과 의미
129	자기 파일의 기존 영역도 재사용 후보로 허용하면서 연속 공간을 찾습니다.
130	공간을 찾지 못하면 복사·metadata 변경 전에 종료합니다.
131	block 번호를 byte offset으로 바꿉니다. disk가 이미 base pointer이므로 여기서 DISK_BASE를 다시 더하지 않습니다.
132	source에서 disk로 size바이트를 복사합니다. NUL에서 멈추는 문자열 복사가 아닙니다.
133	파일 entry가 새로 할당한 시작 block을 가리키게 합니다.
134	유효 파일 크기를 갱신합니다. 마지막 block의 남은 공간은 파일 내용이 아닙니다.
135	기록한 byte 수를 반환합니다.
136	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

공간 부족을 미리 발견하면 이전 파일을 보존하지만, 복사 도중 reset되면 데이터와 metadata가 어긋날 수 있습니다. 이 순서를 crash-safe write로 일반화하면 안 됩니다.

코드 해부 9-E. fs_read_at에서 파일 offset이 실제 주소로 바뀌는 줄

출처: [firmware/minifs.c](https://github.com/riscv/minifs.c), 원본 152 - 165행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

152 | int fs_read_at(const char *name, void *buffer, fs_u32 capacity, fs_u32 offset)
153 | {
154 |     int slot = find_file(name);
155 |     fs_u32 i, count, size, off;
156 |     fs_u8 *dst = (fs_u8 *)buffer;
157 |     if (slot < 0 || (capacity && !buffer)) return -1;
158 |     size = files[slot].size;
159 |     if (offset >= size) return 0;
160 |     count = size - offset;
161 |     if (count > capacity) count = capacity;
162 |     off = files[slot].start_block * FS_BLOCK_SIZE + offset;
163 |     for (i = 0; i < count; i++) dst[i] = disk[off + i];
164 |     return (int)count;
165 | }
```

원본 행	한 줄씩 읽는 동작과 의미
152	파일 내부 offset부터 최대 capacity바이트를 읽는 함수입니다. loader와 cat이 이 API를 공유합니다.
153	앞에서 선언한 함수·블록의 본문을 시작합니다.
154	이름으로 entry index를 찾습니다. 반환값은 block 번호가 아닙니다.
155	최종 읽기 수와 주소 계산용 변수를 선언합니다.
156	destination을 byte pointer로 해석합니다. loader에서는 0x4000+off를 가리킬 수 있습니다.
157	없는 파일이나 잘못된 NULL buffer 조건을 거절합니다. 모든 주소의 접근 권한을 검사하는 코드는 아닙니다.
158	metadata에 저장된 유효 크기를 읽습니다.
159	offset이 끝 이상이면 EOF인 0을 반환합니다. 아래 뽕셈이 unsigned underflow하지 않게 하는 순서이기도 합니다.
160	파일 끝까지 남은 byte 수를 계산합니다.
161	caller의 buffer 용량보다 많이 쓰지 않도록 읽기 수를 줄입니다.
162	전체 disk 기준 시작 block×512에 파일 내부 offset을 더합니다.

원본 행	한 줄씩 읽는 동작과 의미
163	disk base+off+i에서 dst+i로 byte를 복사합니다. 이 반복이 파일에서 실행 RAM으로 옮기는 실제 작업입니다.
164	실제로 복사한 byte 수를 반환합니다. 정상 EOF 0과 오류 -1은 서로 다릅니다.
165	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

start_block=3, offset=12, capacity=64, file size=72이면 count=min(60,64)=60입니다. 첫 source byte는 0x8010060C이고 destination을 0x4000으로 주었다면 그곳으로 APP1 payload가 복사됩니다.

코드 해부 9-F. 파일 삭제는 실제 byte 전체를 지우지 않는다

출처: [firmware/minifs.c](#), 원본 167 - 175행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

167 | int fs_delete(const char *name)
168 | {
169 |     int slot = find_file(name);
170 |     if (slot < 0) return -1;
171 |     files[slot].name[0] = 0;
172 |     files[slot].size = 0;
173 |     files[slot].start_block = 0;
174 |     return 0;
175 | }
```

원본 행	한 줄씩 읽는 동작과 의미
167	이름으로 파일을 삭제하는 함수입니다.
168	앞에서 선언한 함수·블록의 본문을 시작합니다.
169	삭제할 entry index를 찾습니다.
170	파일이 없으면 -1을 반환합니다.
171	name 첫 byte를 0으로 만들어 검색과 할당기가 빈 entry로 보게 합니다. 실제 data block에는 접근하지 않습니다.
172	논리 크기를 0으로 정리합니다.
173	시작 block 정보도 0으로 정리합니다.
174	삭제 성공을 반환합니다.
175	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

이후 할당기가 같은 block을 다른 파일에 재사용할 수 있습니다. debugger에서 예전 문자열이 남아 있는 사실과 정상 API로 그 파일을 읽을 수 있는지는 다른 질문입니다.

9.4 60바이트 앱이 ls에서 72바이트인 이유

호스트의 hello_app.bin은 순수 payload 60바이트입니다. 보드에 저장할 때 셸이 12바이트 APP1 header를 붙입니다. 따라서 MiniFS의 논리 파일 크기는 72바이트이며, 실제 할당은 한 block인 512바이트입니다.

payload	60 bytes
APP1 header	12 bytes
MiniFS file size	72 bytes
allocated disk space	512 bytes

앱 실행 영역은 8192바이트지만 전체 file data 공간은 6656바이트입니다. 여기에 header 12바이트를 제외하면 payload 최대값은 6644바이트입니다. 다른 파일이 있다면 가용 연속 공간에 따라 더 작은 앱도 실패할 수 있습니다.

file size와 할당 크기를 구분하면 앱 크기 제한도 정확히 설명할 수 있습니다. payload 6644 + header 12 = 6656바이트가 data block 13개를 전부 차지합니다. 이론적 최대 앱 하나를 저장하면 다른 비어 있지 않은 파일을 둘 공간은 남지 않습니다. 파일 table entry가 남아 있다는 사실과 data 공간이 남아 있다는 사실은 별개입니다.

현재 header는 파일 안에 저장되지만 실행 RAM에는 payload만 복사됩니다. 따라서 ls의 size, uploader JSON의 bytes, linker의 section size가 서로 다른 값을 표시할 수 있습니다. 실험 보고서에서는 “무엇의 크기인지” 단위를 함께 적어야 서로 모순되는 결과로 오해하지 않습니다.

9.5 두 파일을 저장하고 삭제하는 전체 예

비어 있는 FS에서 hello.app 72바이트와 notes.txt 600바이트를 차례로 기록한다고 가정합니다. 첫 파일은 한 block, 두 번째는 두 block이 필요합니다. 현재 first-fit 연속 할당에서는 다음과 같은 상태가 가능합니다.

entry	name	size	start_block	실제 data block
0	hello.app	72	3	3
1	notes.txt	600	4	4 - 5

hello.app의 첫 byte 주소는 0x80100600이고, 그 payload 첫 byte는 header 뒤인 0x8010060C입니다. 이 payload를 실행할 때는 RAM disk에서 그대로 fetch하지 않고 0x4000으로 복사합니다. notes.txt의 시작은 $0x80100000 + 4 \times 512 = 0x80100800$ 입니다.

rm hello.app은 entry 0을 비우지만 block 3의 byte를 모두 지우지 않습니다. 이어서 100바이트의 new.txt를 만들면 entry 0과 block 3을 재사용할 수 있습니다. 새 파일의 size가 100이면 읽기는 100바이트까지만 허용되므로 block 뒤에 남은 이전 byte는 정상 파일 내용에 포함되지 않습니다.

이 예는 file table이 “어떤 byte가 현재 파일에 속하는가”를 정의한다는 점을 보여 줍니다. RAM 안에 bit가 남아 있는 사실과 파일 시스템에서 접근 가능한 파일이라는 사실은 다릅니다. crash consistency나 기밀 삭제가 필요한 시스템에서는 이 단순 table 정책보다 더 많은 설계가 필요합니다.

9.6 fs_u8과 세 종류의 디스크 포인터

fs_u8은 특별한 CPU 명령이나 하드웨어 타입이 아니라 firmware/minifs.h에서 정의한 자료형 별칭입니다.

```
typedef unsigned int fs_u32;
typedef unsigned char fs_u8;
```

typedef는 기존 자료형에 이름을 붙입니다. 현재 RV32 환경에서 fs_u8은 1바이트, fs_u32는 4바이트이며 각각 부호 없는 값입니다. fs는 파일 시스템, u는 unsigned, 숫자는 의도한 비트 폭을 나타냅니다. 이는 현재 도구 체인의 크기를 전제로 한 이름이며 모든 C 플랫폼에서 unsigned int가 항상 32비트라는 의미는 아닙니다.

minifs.c는 같은 디스크 주소 공간을 다음 세 포인터로 해석합니다.

```
static volatile fs_u8 *const disk = (volatile fs_u8 *)DISK_BASE;
static volatile struct file_entry *const files =
    (volatile struct file_entry *) (DISK_BASE + FS_BLOCK_SIZE);
static volatile fs_u32 *const header = (volatile fs_u32 *)DISK_BASE;
```

표현식	접근 주소	용도
disk[i]	DISK_BASE + i	byte 단위 초기화·복사
header[i]	DISK_BASE + 4*i	superblock의 32비트 필드
files[i]	DISK_BASE + 512 + 24*i	파일 하나의 메타데이터 구조체

disk와 header는 같은 시작 주소를 다른 원소 크기로 바라봅니다. 포인터 형변환이 새 RAM을 만들거나 데이터를 복사하는 것은 아닙니다. 예를 들어 disk[4]는 1바이트이고 header[1]은 같은 주소에서 시작하는 4바이트 word입니다. files는 superblock 다음인 0x80100200에서 시작합니다.

원본 주석에 등장하는 “8 KiB program/stack memory”는 별도의 작은 메모리 구성을 가리키는 표현으로, 현재 셀의 프로그램 RAM 32 KiB와 혼동하면 안 됩니다. 현재 문서의 셀 구성에서는 프로그램 RAM 32 KiB와 별도 RAM disk 8 KiB를 구분합니다.

9.7 superblock의 의미와 초기화 판단

9.1절의 배치를 역할 중심으로 다시 읽으면, superblock은 개별 파일이 아니라 파일 시스템 전체를 설명하는 메타데이터입니다. file table은 각 파일을 설명하고 file data는 실제 내용을 보관합니다.

영역	예로 읽는 의미
Superblock	MiniFS 버전 1이며 512바이트 블록 16개로 이루어진 디스크이다.
File table	hello.txt는 11바이트이고 데이터가 특정 블록에서 시작한다.
File data	Hello World라는 실제 11바이트 내용이다.

superblock은 파일 시스템의 식별 정보와 기본 구조를 담고, 현재 MiniFS에서는 기존 RAM disk를 유지할지 포맷할지 판단하는 기준이 됩니다. 예약한 block 0은 512바이트이지만 현재 사용하는 필드는 앞의 16바이트뿐입니다.

필드	주소	현재 값과 해석
header[0]	0x80100000	0x3153464D, MiniFS magic
header[1]	0x80100004	1, 형식 버전
header[2]	0x80100008	16, 전체 블록 수
header[3]	0x8010000C	512, 블록당 바이트 수

magic은 형식을 식별하기 위한 고정값입니다. little-endian에서 0x3153464D가 메모리에 놓이는 순서는 4D 46 53 31 이고 ASCII로 읽으면 MFS1입니다. 전체 용량은 16×512=8192바이트입니다. 다만 현재 구현은 header에서 읽은 임의의 크기에 맞추어 동적으로 동작하지 않고, 컴파일 시 정한 상수와 일치하는지 검사합니다.

출처: minifs.c의 초기화 판단 코드입니다.

```
int fs_init(void)
{
    if (header[0] == FS_MAGIC && header[1] == FS_VERSION &&
        header[2] == FS_BLOCK_COUNT && header[3] == FS_BLOCK_SIZE)
        return 0;
    return fs_format();
}
```

조건문의 첫 줄은 식별값과 형식 버전을, 다음 줄은 블록 수와 크기를 검사합니다. 모두 맞으면 기존 데이터를 그대로 사용합니다. 그렇지 않으면 fs_format이 디스크 전체를 0으로 지우고 버전·블록 수·크기 다음에 magic을 마지막으로 기록합니다. 마지막 magic 기록은 다른 필드를 먼저 준비한 뒤 유효 표시를 남기는 순서이며 완전한 장애 복구나 원자성을 보장하는 것은 아닙니다.

현재 fs_init은 superblock이 맞지 않으면 오류만 반환하는 것이 아니라 전체 RAM disk를 포맷하여 기존 파일을 지웁니다. 실제 영구 저장장치로 확장할 때는 형식 불일치와 포맷 요청을 분리할 필요가 있습니다. 또한 네 필드가 맞다고 file table 전체가 정상이라는 뜻은 아닙니다. 디스크 밖을 가리키는 start_block, 중복 영역 같은 손상까지 이 비교로 검사하지 않습니다.

9.8 변수와 함수 앞의 static 및 private과의 비교

다음 두 선언은 모두 minifs.c의 파일 범위에 있습니다.

```
static volatile fs_u8 *const disk = (volatile fs_u8 *)DISK_BASE;
static int valid_name(const char *name);
```

파일 범위에서 static은 이름에 내부 연결을 부여합니다. 즉 현재 번역 단위 안에서 그 이름으로 연결하며, 다른 C 파일의 extern 선언이 이 심볼에 직접 연결되지는 않습니다. 보통 C 파일 하나와 include로 합쳐진 내용이 번역 단위를 이룹니다.

첫 선언의 키워드들은 서로 다른 일을 합니다.

요소	역할
static	disk라는 이름의 연결을 해당 번역 단위 내부로 제한
volatile	가리키는 메모리 접근을 컴파일러가 일반 메모리처럼 임의로 생략하지 않도록 표시
fs_u8 *	byte 단위 포인터
const	포인터 자체의 값을 변경하지 못하게 함

따라서 disk[0] = 0은 가능하지만 disk = other는 허용되지 않습니다. 메모리 내용을 상수로 만든 것이 아니라 주소를 담는 포인터를 const로 만든 것입니다. volatile은 잠금, 원자적 갱신, 메모리 보호를 대신하지 않습니다.

파일 범위 변수는 static을 쓰지 않아도 정적 저장 기간을 가지므로, 이 위치에서 핵심은 수명 연장이 아니라 내부 구현의 이름을 숨기는 것입니다. 반면 함수 내부의 static int count;는 함수가 반환한 뒤에도 저장 공간과 값을 유지한다는 차이가 있습니다.

static int valid_name(...)에서 static은 반환값에 붙는 속성이 아니라 함수의 연결을 제한하는 지정자입니다. int가 반환형이고, 호출마다 이름을 검사하여 0 또는 1을 반환합니다. 이전 반환값을 기억하거나 함수를 한 번만 실행하게 하는 의미가 아닙니다.

현재 valid_name은 NULL 포인터·빈 이름을 거부하고, 최대 16바이트 안에 NUL이 있는지 확인하며, slash와 공백을 거부합니다. 그 위에 있는 원본의 superblock 검사·포맷 설명 주석은 실제로 fs_init의 동작에 해당합니다. 주석의 위치보다 함수 본문의 조건문을 기준으로 역할을 구분해야 합니다.

C의 파일 범위 static은 클래스의 private처럼 내부 구현을 숨기는 데 쓰지만, 접근 경계는 클래스가 아니라 번역 단위입니다. MiniFS는 헤더에 fs_create·fs_read·fs_write 같은 API만 공개하고, disk·files·header와 valid_name 같은 보조 구현은 내부에 둡니다. 이것이 C의 모듈 단위 캡슐화입니다.

구분	C의 파일 범위 static	C++의 private 멤버
이름 접근 기준	해당 번역 단위	클래스 멤버와 friend
상태 구성	현재 MiniFS는 하나의 내부 상태 공유	비정적 데이터 멤버는 객체별로 존재 가능
하드웨어 보안	제공하지 않음	접근 지정자 자체는 하드웨어 보호가 아님

특히 현재의 신뢰된 M-mode 프로그램은 RAM 주소를 알면 직접 접근할 수 있습니다. static은 이름의 연결을 제한할 뿐 함수 주소의 전달이나 포인터를 통한 호출을 금지하지 않습니다. 다음 emit 예에서 이 차이가 실제로 드러납니다.

9.9 fs_create는 이름을 가진 빈 파일을 만든다

fs_create는 특정 이름을 file table에 등록하는 함수입니다. 9.2~9.3절의 원본 코드 해설을 호출자의 관점에서 요약하면 다음과 같습니다.

```
int result = fs_create("hello.txt");
```

이 호출은 이름 검사, 같은 이름의 기존 파일 검사, 빈 entry 검색을 수행합니다. 성공하면 name을 기록하고 size와 start_block을 0으로 초기화합니다. 현재 코드는 다른 필드를 준비한 뒤 name[0]을 마지막에 기록하여 사용 중인 entry로 표시합니다.

```
name      = "hello.txt"
size      = 0
start_block = 0
```

fs_create는 빈 파일의 메타데이터를 만들며, 실제 내용과 데이터 블록은 fs_write가 기록합니다. size=0인 상태의 start_block=0은 파일 내용이 superblock에 있다는 뜻이 아니라 데이터 블록을 아직 갖지 않는다는 표시입니다.

호출자는 생성 결과를 확인한 뒤 내용을 기록할 수 있습니다. 다음은 반환값을 확인하는 사용 예이며 기존 함수 정의 자체는 아닙니다.

```
int rc = fs_create("hello.txt");
if (rc == 0) {
    int written = fs_write("hello.txt", "Hello World", 11);
    if (written != 11) {
        /* 생성은 성공했지만 내용 기록은 실패했을 수 있다. */
    }
}
```

fs_create는 성공 시 0, 잘못된 이름·중복 이름·빈 entry 부족 시 -1을 반환합니다. fs_write의 성공 결과는 기록한 바이트 수이므로 서로 반환 규약이 다릅니다. 위 예는 파일이 이미 있으면 덮어쓰지 않습니다. 셸의 write 명령은 기존 파일 처리 정책이 별도로 있으므로 API 하나와 명령 전체를 구분합니다.

현재 RAM disk는 CPU reset으로 내용이 유지될 수 있지만 전원 차단이나 FPGA 재구성 후에도 보존되는 영구 저장장치가 아닙니다. 빈 파일을 만드는 것과 파일 내용의 영속성을 보장하는 것도 별개입니다.

9.10 emit은 함수 이름이 아니라 콜백 매개변수이다

`emit()`이라는 독립적인 함수 정의를 찾기보다 `fs_list`의 매개변수를 먼저 읽어야 합니다. 아래는 `minifs.c`의 실제 구현이며 주석만 생략했습니다.

```
void fs_list(void (*emit)(const char *name, fs_u32 size))
{
    fs_u32 i, j;
    char name[FS_NAME_BYTES];
    if (!emit) return;
    for (i = 0; i < FS_MAX_FILES; i++) {
        if (!files[i].name[0]) continue;
        for (j = 0; j < FS_NAME_BYTES; j++) name[j] = files[i].name[j];
        name[FS_NAME_BYTES - 1u] = '\0';
        emit(name, files[i].size);
    }
}
```

코드	주해 의미
함수 선언	<code>fs_list</code> 는 반환값이 없고, <code>emit</code> 이라는 함수 포인터 한 개를 인수로 받습니다.
<code>void (*emit)(...)</code>	파일 이름과 크기를 받고 <code>void</code> 를 반환하는 함수의 주소를 담습니다.
<code>i, j</code> 선언	entry 검색과 이름 byte 복사에 쓸 인덱스입니다.
name 배열	파일 이름을 임시로 복사할 16바이트 지역 버퍼입니다.
<code>if (!emit) return</code>	NULL 포인터이면 호출하지 않고 종료합니다.
바깥 for	고정된 최대 파일 수만큼 entry를 검사합니다.
name[0] 검사	빈 entry는 건너뜁니다.
안쪽 for	volatile 디스크의 이름을 일반 지역 버퍼에 복사합니다.
마지막 byte에 NUL 기록	전달할 문자열의 끝을 보장합니다.
emit 호출	전달받은 함수에 파일 하나의 이름과 크기를 넘깁니다.

`emit`은 `fs_list`가 인수로 받은 함수 포인터이며, 현재 실제 호출 대상은 `kernel.c`에 정의된 `fs_emit`입니다. `kernel.c`의 `SYS_FS_LIST` 처리에서 `fs_list(fs_emit);`로 주소를 넘깁니다. 여기서 괄호 없는 `fs_emit`은 함수 주소이고, `fs_emit(...)`처럼 인수와 괄호를 붙이면 지금 실행하는 함수 호출입니다.

출처: `kernel.c`의 출력 콜백입니다.

```
static void fs_emit(const char *name, uint32_t size)
{
    uart_puts(name);
    uart_puts(" ");
    uart_put_u32(size);
    uart_puts(" bytes\n");
}
```

첫 출력은 이름, 다음 출력은 구분용 공백, 셋째는 10진수 크기, 마지막은 단위와 줄 끝입니다. 예를 들어 hello.txt와 11을 전달하면 hello.txt 11 bytes 라는 한 줄이 나옵니다. 현재 도구 체인에서 fs_u32와 uint32_t는 이 콜백의 인수형으로 호환되며, 다른 플랫폼으로 옮기면 정의를 다시 확인해야 합니다.

```
kernel.c: fs_list(fs_emit)
           함수 주소 전달
           ↓
minifs.c: emit = 전달된 fs_emit 주소
           파일마다 emit(name, size)
           ↓
kernel.c: fs_emit(name, size)
           UART 출력
```

fs_emit이 static이어도 이 호출은 가능합니다. kernel.c가 자신의 내부 함수 주소를 직접 전달했기 때문입니다. minifs.c는 fs_emit이라는 외부 심볼을 찾아 연결하지 않고 전달된 주소로 호출합니다. 이처럼 함수를 인수로 전달해 필요한 순간에 호출하도록 하는 방식을 callback이라고 합니다.

fs_list는 어떤 파일이 있는지 찾고, 콜백은 그 정보를 어떻게 출력할지 결정합니다. 파일 시스템 순회와 UART 출력의 책임을 분리한 구조입니다. 다른 출력 함수를 전달하면 같은 목록 조회 코드를 재사용할 수 있습니다. 다만 현재 emit 호출은 동기적으로 진행하며, name은 fs_list의 지역 버퍼입니다. 콜백이 그 주소만 보관했다가 나중에 사용하면 버퍼가 덮어쓰이거나 수명이 끝날 수 있으므로 오래 보관하려면 내용을 복사해야 합니다.

10. C 프로그램에서 실행 파일까지

10.1 세 종류의 이미지 구분

프로젝트에서 “binary”나 “hex”라고 부르는 파일들이 모두 같은 역할을 하는 것은 아닙니다.

파일	내용	소비하는 주체
zcu104_mini_shell.bit	FPGA 논리·배선·메모리 초기값 구성 데이터	FPGA configuration 회로
mini_shell.elf	커널·셸 기계어와 ELF 메타데이터	호스트의 linker·검사 도구
mini_shell.hex	프로그램 RAM 초기화용 32비트 word 텍스트	RTL \$readmemh / 합성 도구
hello_app.elf	0x4000 에 링크된 앱과 ELF 메타데이터	호스트의 objcopy, objdump
hello_app.bin	앱의 기계어·상수·포함된 데이터 바이트	업로드 스크립트
hello_app.hex	앱 payload를 word 단위로 표현한 텍스트	셸 시뮬레이션 testbench
MiniFS의 hello.app	APP1 header + payload	보드의 shell_run()

.bit 는 RISC-V CPU가 실행하는 명령어 파일이 아닙니다. .bin 은 FPGA 회로를 구성하는 파일이 아닙니다. FPGA 회로를 먼저 구성하면 그 안의 CPU가 펌웨어를 실행하고, 그 펌웨어가 나중에 업로드한 앱을 읽어 실행합니다.

확장자가 파일 내용을 강제로 바꾸지는 않습니다. .elf 이름을 .bin 으로 바꿔 업로드하면 ELF header부터 payload로 취급되어 실행에 맞지 않습니다. 반대로 .bin 에 읽기 좋은 문자가 포함되어 있어도 전체가 text file이라는 뜻은 아닙니다. 생성 도구와 변환 단계를 따라 형식을 확인해야 합니다.

ELF의 file size에는 header와 symbol table 등이 포함되므로 raw image보다 훨씬 클 수 있습니다. FPGA 실행 RAM의 필요량은 ELF 파일의 host storage 크기와 같지 않습니다. 현재 앱에 필요한 실제 loadable byte는 objcopy의 raw payload와 linker section 배치로 판단합니다.

10.2 예제 C 프로그램

firmware/hello_app.c 의 핵심은 다음과 같습니다.

```
__attribute__((section(".text.app_entry")))
void app_main(void)
{
    const char *p = "Hello from loaded app!\n";
    while (*p) putc_sys(*p++);
}
```

putc_sys() 는 7장에서 설명한 ecall stub입니다. 문자열 끝의 NUL은 반복 종료 조건이고 출력하지 않습니다. 소스의 \n 은 OS 출력 경로에서 CRLF로 바뀝니다. 함수 끝에 도달하면 caller로 반환합니다.

app_main 이라는 이름 자체를 CPU가 인식하는 것은 아닙니다. compiler와 linker가 이 함수의 기계어를 약속된 첫 위치에 배치하고, loader가 그 주소를 호출합니다. 함수 이름은 호스트 ELF의 symbol로 남아 역어셈블과 디버깅에 쓰입니다.

const char *p 는 문자열을 가리키는 pointer 변수이고 문자열 자체는 image의 상수 영역에 있습니다. p++ 는 pointer가 가리키는 byte 주소를 1씩 증가시킵니다. 문자열 길이를 미리 저장하지 않아도 NUL을 만날 때까지

반복하므로, 실행 중 필요한 ISA는 주소 덧셈·byte load·조건 branch·ECALL입니다.

app_main에 main이라는 이름을 쓰지 않은 것은 별도 C startup을 연결하지 않기 때문입니다. hosted 환경의 main(argc,argv) 호출·종료 처리를 제공하는 runtime이 없으며, loader가 약속된 함수를 직접 호출합니다. 함수 이름과 entry section annotation이 함께 있어야 현재 build 계약을 만족합니다.

코드 해부 10-A. 독립 앱의 C 소스 전체를 한 줄씩 읽기

출처: [firmware/hello_app.c](#), 원본 2 - 16행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

2 | typedef unsigned int uint32_t;
4 | static void putc_sys(char c)
5 | {
6 |     register uint32_t a0 __asm__("a0") = (unsigned char)c;
7 |     register uint32_t a7 __asm__("a7") = 1u;
8 |     __asm__ volatile ("ecall" : "+r"(a0) : "r"(a7) : "memory");
9 | }
11 | __attribute__((section(".text.app_entry")))
12 | void app_main(void)
13 | {
14 |     const char *p = "Hello from loaded app!\n";
15 |     while (*p) putc_sys(*p++);
16 | }

```

원본 행	한 줄씩 읽는 동작과 의미
2	현재 RV32 ABI에서 32비트인 unsigned int에 이름을 붙입니다. 이 코드 자체가 CPU의 register 폭을 바꾸지는 않습니다.
4	문자 하나를 OS 서비스로 내보내는 내부 함수를 정의합니다.
5	앞에서 선언한 함수·블록의 본문을 시작합니다.
6	char의 부호와 무관하게 0 - 255의 byte 값으로 만들어 a0에 연결합니다.
7	이 Mini OS의 SYS_PUTC 번호 1을 a7에 준비합니다.
8	ECALL 예외를 발생시킵니다. compiler와 OS가 기대하는 operand·보존 계약을 inline assembly 제약으로 표현합니다.
9	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
11	app_main의 기계어를 특별한 입력 section에 둡니다. linker의 KEEP 규칙과 연결되는 표지입니다.
12	셀이 void(void)로 호출하는 entry 함수입니다. hosted C runtime의 main이 아닙니다.
13	앞에서 선언한 함수·블록의 본문을 시작합니다.
14	NUL로 끝나는 상수 문자열의 첫 byte를 p가 가리킵니다. 문자열은 payload 안에 있고 p 자체는 compiler가 register 등에 배치합니다.
15	현재 byte가 0이 아니면 출력하고 pointer를 한 칸 진행합니다. 끝 NUL은 출력하지 않습니다.
16	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

함수 끝에 도달하면 일반 함수 반환 규약으로 셀에 돌아갑니다. 이 작은 C 코드가 정확히 어떤 9개 instruction이 되었는지는 14장의 주소별 표와 연결해 보십시오.

10.3 make hello-app의 빌드 단계

10.3.1 toolchain은 서로 다른 일을 하는 도구들의 연결이다

toolchain은 C를 실행 가능한 byte로 만드는 도구들과 그 결과를 검사하는 도구들의 묶음입니다. 모든 명령이 compiler인 것은 아닙니다. `gcc`는 코드 생성·조립·링크를 조정하고, `objcopy`는 산출물 형식을 변환하며, `readelf·objdump·xxd`는 서로 다른 관점에서 결과를 보여 줍니다. 검사 도구로 파일을 열었다고 보드에서 앱이 실행되지는 않습니다.

도구	주로 받는 입력	만드는 결과 또는 보여 주는 정보
<code>make</code>	Makefile의 target·의존 관계	필요한 빌드 명령을 순서대로 실행
<code>gcc</code>	C, assembly, object와 옵션	preprocessing·compile·assemble·link를 지휘
<code>as</code>	assembly	instruction과 relocation을 담은 object
<code>ld</code>	object, linker script, library	최종 주소를 정한 ELF
<code>readelf</code>	ELF object 또는 executable	header, section, symbol, relocation 등의 구조
<code>objdump</code>	ELF/object 등	instruction 역어셈블, section byte 검사
<code>objcopy</code>	ELF/object 등	BIN 등 다른 형식의 파일
<code>xxd</code>	임의의 파일	실제 byte의 hex dump
<code>nm, size</code>	ELF/object	symbol 목록, section 크기
<code>bin2hex.py</code>	raw little-endian BIN	이 프로젝트용 32비트 word HEX 텍스트

GCC는 GNU Compiler Collection, ELF는 Executable and Linkable Format의 약자입니다. `readelf`는 ELF를 읽는 도구라는 이름이고, `objdump`와 `objcopy`는 각각 object를 표시하거나 복사·변환하는 도구입니다. `readelf·objdump·objcopy·nm·size`는 GNU Binutils에 속합니다. `xxd`는 별도의 byte 표시 도구이고, `bin2hex.py`는 이 저장소에서 작성한 Python 스크립트입니다.

현재 명령 이름 앞의 `riscv64-unknown-elf-`는 target toolchain을 구분하는 접두사입니다. 호스트 PC에서 실행되지만 결과는 RISC-V용인 **cross toolchain**입니다. `xxd`는 instruction을 해석하지 않고 byte를 표시하므로 RISC-V용 접두사가 필요하지 않습니다. 반대로 호스트용 `objdump`는 설치 구성에 따라 RISC-V 해독 기능이 없을 수 있어 여기서는 `cross objdump`를 사용합니다.

이번 설명의 실제 관측 환경은 GCC 9.3.0, GNU Binutils 2.34입니다. 다음 명령으로 자신의 도구를 확인할 수 있습니다. 버전이나 code가 달라지면 뒤의 주소·크기도 달라질 수 있으므로 예제 관측값과 ABI의 고정 규칙을 구분하십시오.

```
riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-readelf --version
riscv64-unknown-elf-objdump --version
riscv64-unknown-elf-gcc -dumpmachine
xxd -v
```

10.3.2 Makefile이 실제로 만드는 세 파일

```
hello_app.c + app.ld
  | GCC: compile + assemble + link
  ▼
build/firmware/hello_app.elf
  | objcopy -O binary
```

```

    ▼
build/firmware/hello_app.bin
  | bin2hex.py
    ▼
build/firmware/hello_app.hex

```

현재 [Makefile](#)의 hello-app target은 BIN과 HEX를 요구하고, BIN의 의존 파일로 ELF가 먼저 만들어집니다. GCC 한 명령이 compile·assemble·link까지 수행하므로 .s와 .o가 별도 파일로 남지 않아도 내부 단계는 존재합니다. 이 target은 .dis나 .map을 자동으로 만들지는 않습니다.

```

make hello-app
make -Bn hello-app

```

첫 명령은 필요한 빌드를 실행합니다. 두 번째는 이 target을 다시 만들 때 실행할 명령을 미리 확인하는 용도입니다. -B는 최신 시각과 무관하게 target을 다시 만드는 대상으로 취급하고, -n은 이 target의 recipe를 실제 실행하는 대신 출력합니다. 현재 hello-app 경로에는 재귀 Make recipe가 없어 compiler·objcopy를 실행하지 않고 명령행을 확인할 수 있습니다.

Nothing to be done for 'hello-app'.은 의존 파일보다 산출물이 최신이어서 다시 만들 일이 없다는 뜻입니다. 업로드나 실행 실패가 아닙니다. 실제 강제 재생성이 필요할 때는 make -B hello-app을 사용합니다. 이 명령은 앱 파일을 다시 만들며, 그 자체로 FPGA bitstream을 생성하거나 UART 업로드를 하지 않습니다.

Make는 파일 target과 prerequisite의 시각으로 재생성 여부를 판단합니다. 현재 ELF 규칙은 hello_app.c와 app.ld를 의존 파일로 지정하지만 compiler 버전·옵션 문자열 전체를 자동 추적하지 않습니다. 옵션만 바꾼 실험에서는 강제 재빌드 여부와 명령행을 함께 기록해야 이전 산출물을 새 결과로 오해하지 않습니다.

코드 해부 10-B. Makefile의 ELF·BIN·HEX 의존 관계를 줄별로 읽기

출처: [Makefile](#), 원본 181 - 191행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

181 | $(APP_ELF): firmware/hello_app.c firmware/app.ld | $(FW_BUILD)
182 |     $(RISC_V_GCC) $(CFLAGS_RV32I) -mno-relax -nostdlib -nostartfiles \
183 |         -Wl,--build-id=none,--no-relax -T firmware/app.ld -o $@ firmware/hello_app.c
185 | $(APP_BIN): $(APP_ELF)
186 |     $(RISC_V_OBJCOPY) -O binary $< $@
188 | $(APP_HEX): $(APP_BIN) scripts/bin2hex.py
189 |     $(PYTHON) scripts/bin2hex.py --max-bytes 8192 $< $@
191 | hello-app: $(APP_BIN) $(APP_HEX)

```

원본 행	한 줄씩 읽는 동작과 의미
181	ELF target은 C와 linker script를 일반 prerequisite로, 출력 directory를 order-only prerequisite로 둡니다. 수직 구분자 뒤의 directory는 먼저 필요하지만 directory 시각 변화만으로 ELF를 다시 만들지는 않습니다.
182	GCC driver를 실행해 ISA·ABI·코드 생성 옵션과 startup/library 제외 정책을 적용합니다. 끝 역슬래시는 명령행이 다음 줄로 이어진다는 뜻입니다.
183	linker 옵션과 script를 전달합니다. \$@는 현재 target인 ELF 경로입니다.
185	BIN을 만들려면 먼저 ELF가 있어야 한다고 선언합니다.
186	첫 prerequisite \$<인 ELF를 raw binary로 바꿔 target \$@에 씁니다.
188	HEX는 BIN과 변환 스크립트에 의존합니다. 스크립트가 바뀌어도 재생성이 필요합니다.

원본 행	한 줄씩 읽는 동작과 의미
189	원래 BIN을 32비트 word 텍스트로 변환하며 입력 크기를 검사합니다.
191	사용자가 요청하는 hello-app target을 BIN·HEX 준비에 연결합니다. 이 줄에는 UART 업로드나 FPGA 다운로드가 없습니다.

`make hello-app`은 이 의존 graph를 만족시키는 빌드 요청입니다. ELF가 최신이면 변환만 수행할 수 있고, 모두 최신이면 Nothing to be done이라고 나옵니다. 도구 옵션을 바꾼 실험에서는 실제 명령행과 강제 재생성 필요성도 확인해야 합니다.

10.3.3 C에서 ELF까지 중간 단계를 따로 남기는 실습

중간 파일의 차이를 보려면 Make의 기본 산출물과 구분되는 `build/toolchain_lab`에 단계별로 생성할 수 있습니다. 다음은 저장소 최상위에서 Bash로 수행하는 수업용 명령입니다. 현재 앱과 같은 code-generation 옵션을 배열에 모읍니다.

```
mkdir -p build/toolchain_lab
RV_APP_FLAGS=(
  -march=rv32i -mabi=ilp32 -O1
  -ffreestanding -fno-builtin -fno-pic
  -fno-asynchronous-unwind-tables
  -msmall-data-limit=0 -mno-relax
)
```

전처리·컴파일·조립을 차례로 멈추는 옵션은 `-E`, `-S`, `-c`입니다. 단계 선택의 기준은 [GCC Overall Options](#)에서 확인할 수 있습니다.

```
riscv64-unknown-elf-gcc "${RV_APP_FLAGS[@]}" -E \
  firmware/hello_app.c -o build/toolchain_lab/hello_app.i
riscv64-unknown-elf-gcc "${RV_APP_FLAGS[@]}" -S \
  build/toolchain_lab/hello_app.i -o build/toolchain_lab/hello_app.s
riscv64-unknown-elf-gcc "${RV_APP_FLAGS[@]}" -c \
  build/toolchain_lab/hello_app.s -o build/toolchain_lab/hello_app.o
```

`.i`는 include·macro 등이 처리된 C이고, `.s`는 compiler가 만든 assembly 텍스트입니다. `.o`는 assembler가 만든 relocatable ELF object이며 아직 최종 실행 주소를 모두 확정하지 않은 것입니다. `.s`는 일반적으로 전처리를 거치는 assembly 입력이라는 점도 구분하십시오. 현재 `boot.S`·`trap.S`처럼 사람이 작성한 입력과 compiler가 생성한 `hello_app.s`는 같은 단계의 출발점이 아닐 수 있습니다.

마지막으로 object와 linker script를 연결합니다. `-Map`은 수업용 map 파일을 추가로 남기는 옵션이고 기본 `hello-app` recipe에는 없습니다.

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -mno-relax \
  -nostdlib -nostartfiles -Wl,--build-id=none,--no-relax \
  -Wl,-Map=build/toolchain_lab/hello_app.map \
  -T firmware/app.ld \
  -o build/toolchain_lab/hello_app.elf build/toolchain_lab/hello_app.o
```

linker는 `.LC0` 같은 symbol의 최종 주소를 결정하고 object에 남아 있던 relocation을 해결합니다. `.s`를 읽는 것과 최종 `.elf`를 `objdump`로 읽는 것은 이 점에서 다릅니다. `.dis`라는 이름은 보통 역어셈블 결과를 저장한 텍스트 파일의 관례적인 확장자이지, CPU가 직접 실행하는 별도 기계어 형식이 아닙니다.

10.3.4 readelf로 ELF header 읽기

`readelf`는 instruction을 실행하는 도구가 아니라 ELF의 구조를 해석하는 도구입니다. 명령 이름은 `readelf`이며, 다음처럼 파일 header부터 확인합니다.

```
riscv64-unknown-elf-readelf -h build/firmware/hello_app.elf
```

현재 출력의 핵심 항목	관측값	읽는 방법
Class	ELF32	ELF 내부 주소·구조 형식이 32비트
Data	little endian	다중 byte 정수의 저장 순서
Type	EXEC	링크가 완료된 executable ELF
Machine	RISC-V	대상 instruction architecture
Entry point address	0x4000	ELF metadata에 기록된 시작 주소
OS/ABI	UNIX - System V	ELF의 ABI 표지이며 Linux 실행 가능성 보장은 아님

prefix가 `riscv64`여도 결과는 `ELF32`입니다. `target` 명령행에서 `RV32`와 `ILP32`를 선택했기 때문입니다. 또한 `Entry=0x4000`이라는 정보는 `host` 검사에 유용하지만 현재 `raw loader`가 이 ELF header를 읽는 것은 아닙니다. `loader`는 `BIN payload`를 고정 주소 `0x4000`에 놓고 그 위치를 호출합니다.

ELF header의 `Machine`이 `RISC-V`라고 이 교육용 core에서 모든 instruction이 실행된다고 보장하지도 않습니다. `M·C·F` 같은 추가 extension이나 미지원 CSR를 사용했지는 실제 code와 compiler 설정을 더 확인해야 합니다. header는 첫 번째 형식 검사이지 완전한 ISA 검증 결과가 아닙니다.

10.3.5 readelf의 section과 program header 구분

`-S`는 대문자 S로 section 목록, `-l`은 소문자 L로 program header를 표시합니다. `-W`는 긴 행을 넓게 출력하는 표시 옵션입니다. 이 옵션의 기본 의미는 [GNU readelf 문서](#)를 따릅니다.

```
riscv64-unknown-elf-readelf -W -S build/firmware/hello_app.elf  
riscv64-unknown-elf-readelf -W -l build/firmware/hello_app.elf
```

현재 `.text` 행에서 관측되는 핵심 숫자는 다음과 같습니다.

Name	Type	Addr	Off	Size	Flags	Align
.text	PROGBITS	00004000	001000	00003c	AX	4

`Addr`는 실행 시 사용할 주소 `0x4000`, `Off`는 ELF 파일 안의 offset `0x1000`, `Size`는 `0x3C=60`바이트입니다. 세 숫자는 서로 다른 기준입니다. `AX`는 메모리 할당 대상이며 executable 속성이 있다는 section metadata입니다. 현재 하드웨어에 실행 권한 검사가 생기는 것은 아닙니다.

`app.ld`는 함수의 instruction과 `.rodata` 문자열을 같은 output `.text`에 넣습니다. 따라서 `.text` 60바이트가 전부 instruction은 아닙니다. 현재는 code 36바이트와 NUL을 포함한 문자열 24바이트입니다. `.symtab`과 `.strtab` 등은 `host`가 symbol과 이름을 해석하는 metadata이며 raw payload의 일부가 아닙니다.

program header의 `LOAD` 항목은 ELF loader가 파일에서 메모리로 옮길 범위를 설명합니다. 이번 산출물에서는 `Offset=0x1000`, `VirtAddr=PhysAddr=0x4000`, `FileSiz=MemSiz=0x3C`입니다. section은 code·data·symbol 같은 구성 조각이고, segment는 load 관점의 범위입니다. 여기서는 `.text` 하나가 해당 `LOAD` 범위에 들어 있습니다.

일반 ELF에서 FileSiz보다 MemSiz가 크면 파일에 없는 메모리 초기화 영역이 있을 수 있지만, 현재 앱은 .bss를 허용하지 않고 두 값이 같습니다. LOAD의 Align=0x1000 같은 배치 속성도 이 ELF의 metadata일 뿐, 현재 CPU가 MMU나 4 KiB page를 구현했다는 뜻은 아닙니다.

10.3.6 symbol과 relocation으로 링크의 역할 확인하기

-s는 소문자 s로 symbol table, -r은 relocation, -A는 architecture-specific 속성을 표시합니다. 앞 절의 -S와 혼동하지 마십시오.

```
riscv64-unknown-elf-readelf -s build/firmware/hello_app.elf
riscv64-unknown-elf-readelf -r build/firmware/hello_app.elf
riscv64-unknown-elf-readelf -r -A build/toolchain_lab/hello_app.o
```

현재 최종 ELF의 app_main symbol은 Value=0x4000, Size=36, Type=FUNC, Bind=GLOBAL입니다. 여기서 Size 36은 함수 instruction의 byte 수이고 .text 전체의 60바이트와 다릅니다. GLOBAL은 linker가 다른 object의 참조와 연결할 수 있는 symbol 분류이지 OS의 전역 권한이나 사용자 접근 권한이 아닙니다.

단계별 실습에서 생성한 object에는 .LC0를 대상으로 하는 R_RISCV_HI20, R_RISCV_L012_I와 내부 branch 대상의 R_RISCV_BRANCH가 관측됩니다. 앞의 두 relocation은 문자열 주소를 LUI·ADDI의 immediate에 반영하는 작업과 연결됩니다. .o 단계의 위치만 보고 이미 0x4000에 배치되었다고 해석하면 안 됩니다.

최종 ELF에서는 relocation이 남아 있지 않다는 메시지가 나옵니다. 이것은 이번 정적 링크에서 주소 반영 작업을 마쳤다는 뜻입니다. 어느 주소로 옮겨도 실행 가능하다는 뜻은 아닙니다. 오히려 문자열 주소 0x4024가 instruction에 반영되어 있으므로 현재 BIN은 0x4000 배치를 전제로 합니다.

현재 app.ld는 .riscv.attributes를 버립니다. 그래서 object의 readelf -A에는 이번 toolchain의 rv32i2p0, stack alignment 16 등의 정보가 보여도 최종 ELF에서는 같은 출력이 없을 수 있습니다. 속성 section의 부재와 instruction의 부재는 다릅니다. compiler 버전·옵션과 최종 역어셈블을 함께 확인해야 합니다.

10.3.7 objdump로 최종 instruction 읽기

objdump는 저장된 기계어를 assembly 표현으로 되돌려 보여 줍니다. 원래 C 소스를 복원하는 도구는 아닙니다. 다음 명령은 실제 code 구간만 제한해서 표시합니다.

```
riscv64-unknown-elf-objdump -d -M no-aliases,numeric \
--start-address=0x4000 --stop-address=0x4024 \
build/firmware/hello_app.elf
```

옵션	현재 실습에서의 의미
-d	executable section의 instruction을 역어셈블
-M no-aliases,numeric	pseudo-instruction 대신 기본 표현, register는 x번호
--start-address	표시할 주소 범위의 시작
--stop-address	이 주소부터는 제외, 현재 문자열 시작 전까지 표시
-r	object의 relocation 정보를 함께 확인할 때 사용
-s	선택 section의 내용 byte 표시, symbol 목록 옵션이 아님
-S	debug 정보·source가 있을 때 소스와 역어셈블을 함께 표시

옵션 의미는 [GNU objdump 문서](#), 아래 숫자는 현재 앱의 실제 출력입니다.

```
4000: 000047b7 lui    x15,0x4
4004: 02478793 addi   x15,x15,36
4014: 00000073 ecall
4020: 00008067 jalr   x0,0(x1)
```

왼쪽은 instruction의 실행 주소, 가운데는 32비트 instruction word, 오른쪽은 해독 결과입니다. `numeric` 을 빼면 `x15·x10·x17` 대신 `a5·a0·a7` 같은 ABI 이름을 볼 수 있습니다. `alias`를 허용하면 마지막 JALR는 `ret` 로 표시될 수 있지만 기계어가 바뀐 것은 아닙니다.

stop 주소를 0x4024로 둔 이유는 현재 그 위치부터 문자열이기 때문입니다. `.rodata` 가 executable `.text` 에 합쳐져 있어 제한 없는 역어셈블은 문자열 일부도 instruction처럼 해석할 수 있습니다. `.text` 에 보이는 모든 줄이 실제 실행 경로라고 판단하지 마십시오. 다른 빌드에서는 `app_main` symbol 크기와 section 배치를 다시 확인해 범위를 정해야 합니다.

기본 Makefile에는 `-g` 가 없으므로 `objdump -S` 만 붙여도 원래 C 줄이 항상 나타나는 것은 아닙니다. debug 정보 생성 옵션 `-g`, 소스 파일의 접근 가능성, 최적화로 바뀐 대응 관계가 함께 영향을 줍니다. GCC의 `-S` 는 assembly 생성 후 멈춤이고 `objdump`의 `-S` 는 소스 혼합 표시이므로 도구 이름까지 포함해서 읽습니다.

10.3.8 objcopy로 ELF에서 raw binary 추출하기

`objcopy`는 C를 컴파일하지 않습니다. 이미 만들어진 ELF에서 loadable section의 byte를 raw image로 변환합니다. 아래에서 `-O` 는 대문자 O이며 output format을 binary로 지정합니다.

```
riscv64-unknown-elf-objcopy -O binary \
    build/firmware/hello_app.elf build/toolchain_lab/hello_app.bin
```

ELF header·symbol table·일반 debug metadata는 raw binary에 포함되지 않습니다. binary 추출은 load 주소를 기준으로 하며, 이번 예에서는 가장 낮은 내용 주소 0x4000이 BIN offset 0이 됩니다. 그래서 앞에 0x4000바이트의 빈 공간이 붙지 않습니다. section 간 간격이 있는 다른 ELF에서는 BIN 크기와 간격도 따로 점검해야 합니다. 형식 변환 규칙은 [GNU objcopy 문서](#)를 참고하십시오.

현재 앱을 같은 도구로 재빌드하면 ELF 파일은 4476바이트, BIN은 60바이트입니다. ELF에는 section header·symbol·파일 내부 정렬 간격 등이 있지만 BIN에는 실행 이미지의 code·상수 60바이트만 남기 때문입니다. ELF 크기 4476은 이 산출물의 관측값이고, 앞으로 도구나 debug 설정이 바뀌어도 같아야 하는 계약은 아닙니다.

raw BIN에는 시작 주소나 CPU 종류를 알려 주는 ELF header가 없습니다. `.bin`이라는 확장자만으로도 이를 알아낼 수 없습니다. APP1 loader가 단순한 것은 payload가 0x4000의 RV32I 함수라는 약속을 파일 밖에서 정했기 때문입니다. `objcopy`가 loader를 만들어 주거나 앱을 자동으로 실행하는 것은 아닙니다.

10.3.9 xxd로 byte 순서와 파일 offset 읽기

`xxd`는 파일 byte를 그대로 표시합니다. instruction이든 문자열이든 ELF header이든 같은 방식으로 보여 줍니다. `-g1` 은 byte 하나씩 구분하고 `-c16` 은 한 줄에 16바이트를 표시합니다. 주요 표시 옵션은 [xxd 매뉴얼](#)과 로컬의 `xxd -h` 에서 확인할 수 있습니다.

```
xxd -g1 -c16 build/firmware/hello_app.bin
xxd -g1 -s 0x14 -l 4 build/firmware/hello_app.bin
```

두 번째 명령의 `-s` 는 파일 offset 0x14에서 시작, `-l` 은 4바이트만 표시한다는 뜻입니다. 현재 출력은 다음과 같습니다.

```
00000014: 73 00 00 00 s...
```

왼쪽 00000014 는 BIN 파일 안의 byte offset이며 PC가 아닙니다. 가운데 네 byte를 little-endian word로 모으면 0x00000073, 즉 ECALL입니다. 오른쪽 s... 는 같은 byte를 문자로 표시한 보조 열일 뿐, ECALL이 문자 s를 출력한다는 뜻은 아닙니다. 0x73이 인쇄 가능한 ASCII s라 그렇게 보일 뿐입니다.

같은 instruction을 다른 도구에서 찾을 때 기준이 어떻게 바뀌는지 비교해 봅니다.

표현	현재 ECALL의 위치	계산 기준
ELF의 실행 주소	0x4014	.text 주소 0x4000 + 0x14
ELF 파일 내부 offset	0x1014	.text 의 Off 0x1000 + 0x14
BIN 파일 내부 offset	0x14	payload 첫 byte를 0으로 봄
word HEX의 위치	여섯 번째 줄	byte offset 20 / 4 = word index 5

따라서 `xxd -g1 -s 0x1014 -l 4 build/firmware/hello_app.elf` 도 같은 네 byte를 보여 줍니다. 이 숫자들은 이번 section 배치에 대한 값입니다. 다른 ELF에서도 무조건 0x1000을 더하는 규칙은 아닙니다. 먼저 `readelf`로 해당 section의 Addr와 Off를 확인해야 합니다.

`-g4` 는 네 byte를 묶어 보여 주는 표시 옵션이지 기본적으로 byte 순서를 뒤집는 옵션이 아닙니다. 첫 instruction의 bytes는 B7 47 00 00 이고, 일반 byte 묶음은 b7470000, little-endian word 값은 000047b7 입니다. word 관점의 표시를 원하면 다음처럼 `-e -g4` 를 사용할 수 있습니다.

```
xxd -e -g4 -l16 build/firmware/hello_app.bin
xxd -g1 -s 0x24 -l24 build/firmware/hello_app.bin
```

두 번째 명령은 현재 문자열의 24바이트를 표시합니다. 끝의 0A 00 은 newline과 NUL이고, NUL은 출력 문자가 아니라 C 반복문의 끝 표지입니다. `xxd -p` 는 주소·문자 열을 뺀 byte hex이고 `xxd -r -p` 는 그 표현에서 binary를 만들 때 사용합니다. 이는 아래의 32비트 word HEX 규약과 같지 않습니다.

10.3.10 bin2hex와 세 가지 hex 표현 구분하기

[bin2hex.py](#)는 BIN을 4바이트씩 읽고 `int.from_bytes(..., "little")` 로 word 값을 만든 뒤 8자리 hex와 newline을 씁니다. 길이가 4의 배수가 아니면 마지막 word를 0 byte로 채웁니다. 현재 60바이트 앱은 이미 4의 배수이므로 padding이 없습니다.

```
python3 scripts/bin2hex.py --max-bytes 8192 \
  build/firmware/hello_app.bin build/toolchain_lab/hello_app.hex
```

대상	첫 instruction의 표현	소비하는 쪽
raw BIN	byte B7 47 00 00	host uploader가 읽는 원본
UART ASCII hex	문자 b7470000	수신기가 byte 두 자리씩 복원
32비트 word HEX	한 줄 000047b7	\$readmemh 가 word로 읽음

현재 HEX의 첫 줄은 000047b7, 여섯 번째 줄은 00000073 입니다. 60바이트는 15개 word이고, 한 줄이 hex 문자 8개와 LF 1개라 HEX 텍스트 파일은 135바이트입니다. 135바이트는 프로그램 메모리를 135바이트 차지한다는 뜻이 아닙니다. 표현에 필요한 텍스트 크기일 뿐 실제 payload는 60바이트입니다.

이 HEX는 Intel HEX 형식이 아닙니다. 주소 record나 checksum record 없이 32비트 word를 한 줄씩 적는 현재 \$readmemh 입력 규약입니다. 원소 몇 번부터 적재하는가는 이를 읽는 RTL/testbench가 정합니다. 파일에 실행 주소 0x4000이 자동으로 들어 있는 것은 아닙니다.

단순히 xxd -p 출력을 이 word HEX 대신 사용하면 byte grouping과 endianness가 맞지 않을 수 있습니다. 반대로 word HEX를 byte hex로 간주해 xxd -r -p 에 넣으면 첫 byte 순서부터 달라집니다. 파일 확장자만 보지 말고 “byte 나열인가, word 숫자인가, 주소 record가 있는가”를 확인하십시오. 보드 업로드에는 원래의 hello_app.bin을 사용합니다.

--max-bytes 8192 는 변환 입력의 최대 크기 검사입니다. 현재 MiniFS에 넣을 수 있는 앱 payload 최대 6644바이트와는 다른 제한입니다. linker의 8 KiB 창, HEX 변환의 크기 검사, MiniFS의 공간·APP1 header 제한은 모두 각각 통과해야 합니다.

코드 해부 10-C. bin2hex.py가 endianness를 처리하는 실제 줄

출처: [scripts/bin2hex.py](#), 원본 20 - 31행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
20 | data = args.input.read_bytes()
21 | if len(data) > args.max_bytes:
22 |     raise SystemExit(
23 |         f"image is {len(data)} bytes, larger than {args.max_bytes}-byte memory"
24 |     )
26 | data += bytes((-len(data)) % 4)
27 | words = (
28 |     int.from_bytes(data[offset : offset + 4], "little")
29 |     for offset in range(0, len(data), 4)
30 | )
31 | args.output.write_text("".join(f"{word:08x}\n" for word in words), encoding="ascii")
```

원본 행	한 줄씩 읽는 동작과 의미
20	입력 파일을 텍스트가 아닌 raw bytes로 읽습니다.
21	사용자가 정한 최대 입력 크기와 비교합니다.
22	너무 크면 오류 메시지와 함께 종료합니다. 정상 output을 만들기 전에 수행하는 검사입니다.
23	실제 크기와 허용 크기를 오류에 표시합니다.
24	앞에서 여러 줄에 걸쳐 작성한 연결·호출·표현식을 닫습니다.
26	4의 배수가 되도록 0 - 3개의 0 byte를 덧붙입니다. 길이 5이면 3개, 길이 60이면 0개입니다.
27	word 값을 차례로 만드는 generator 표현식을 시작합니다.
28	네 byte를 little-endian 정수 하나로 해석합니다. B7 47 00 00은 숫자 0x000047B7이 됩니다.
29	byte offset을 0,4,8,... 순서로 진행합니다.
30	앞에서 여러 줄에 걸쳐 작성한 연결·호출·표현식을 닫습니다.
31	각 word를 소문자 8자리 hex와 LF로 표현하여 ASCII 파일로 씁니다. 08x 의 폭은 최소 폭이며 이 값은 32비트라 8자리 안에 들어갑니다.

이 함수는 instruction을 이해하지 않습니다. 문자열 byte도 똑같이 네 개씩 묶습니다. 따라서 HEX 변환 성공은 CPU가 그 값을 실행할 수 있다는 ISA 검증이 아닙니다.

10.3.11 nm·size와 raw binary의 보조 검사

nm은 symbol을 간단하게 보고 size는 section 크기를 요약하는 데 유용합니다. 다음은 파일을 수정하지 않는 검사 명령입니다.

```
riscv64-unknown-elf-nm -n build/firmware/hello_app.elf
riscv64-unknown-elf-nm -u build/firmware/hello_app.elf
riscv64-unknown-elf-size -A build/firmware/hello_app.elf
wc -c build/firmware/hello_app.bin
```

현재 nm -n은 주소순으로 00004000 T app_main을 보여 줍니다. T는 text 영역에 정의된 symbol이라는 표시이고, -u는 undefined symbol을 찾는 데 사용합니다. 이 앱의 최종 ELF에서는 미해결 symbol이 출력되지 않습니다. .o의 undefined symbol은 이후 링크할 대상일 수 있지만, 최소 앱의 최종 실행 파일에는 필요한 참조가 해결되어 있어야 합니다.

size -A는 현재 .text 60바이트를 보여 줍니다. wc -c는 파일 자체 byte 수를 셉니다. ELF에 대해 두 도구의 숫자가 달라도 모순이 아닙니다. section 내용 크기와 header·정렬·symbol을 포함한 전체 파일 크기를 구분하기 때문입니다. -A 등의 뜻은 도구별로 다르므로 readelf의 -A와 size의 -A를 같은 기능으로 외우지 않습니다.

BIN만 남았을 때도 instruction을 해석할 수 있지만 형식·architecture·표시 기준 주소를 사람이 알려 줘야 합니다.

```
riscv64-unknown-elf-objdump -D -b binary -m riscv:rv32 \
-M no-aliases,numeric --adjust-vma=0x4000 \
--start-address=0x4000 --stop-address=0x4024 \
build/firmware/hello_app.bin
```

-b binary는 raw 입력 형식, -m riscv:rv32는 해독 architecture, --adjust-vma는 표시 주소의 보정입니다. 이 옵션은 binary를 재배치하거나 내부 절대 주소를 고쳐 주지 않습니다. 잃어버린 함수 이름·section 구분도 자동 복구되지 않으므로 가능한 한 원본 ELF와 linker 정보를 함께 보관하는 것이 좋습니다.

10.3.12 도구별 검사를 하나의 실행 계약으로 묶기

검사는 ELF header → section·symbol → instruction → raw byte 순서로 좁혀 가면 이해하기 쉽습니다. 현재 앱에서는 다음 대응이 동시에 맞아야 합니다.

```
readelf: ELF32, RISC-V, entry = 0x4000
readelf: .text address = 0x4000, size = 0x3c
nm:      app_main = 0x4000
objdump: 0x4014 contains ECALL word 0x00000073
xxd:     BIN offset 0x14 contains 73 00 00 00
HEX:     word index 5 contains 00000073
```

이 검사는 “호스트 파일이 의도한 code와 배치인가”를 확인합니다. UART 수신, MiniFS 저장, loader의 복사, 실제 CPU의 fetch·trap·복귀가 정상인지는 이후 시뮬레이션·보드 실험의 역할입니다. ELF를 올바르게 만들었다는 사실과 FPGA에서 정상 실행했다는 증거는 구분해서 기록해야 합니다.

현재 9개 instruction·60바이트라는 수치는 이 source·도구·옵션 조합의 결과입니다. 새 앱을 만들 때 60바이트인지가 성공 기준은 아닙니다. ABI, entry 배치, 지원 instruction, 실행 창·파일 크기, 반환 동작이라는 계약이 기준입니다.

10.4 compiler 옵션의 의미

10.4.1 현재 명령행을 기능별로 분해하기

Makefile이 실행하는 앱 빌드 명령을 변수 없이 펼치면 다음과 같습니다. 별도의 `-E`, `-S`, `-c`가 없으므로 GCC driver가 최종 링크까지 진행합니다. 아래 예는 현재 기본 설정을 설명하는 것이며 이 문서 확장으로 Makefile의 옵션을 바꾼 것은 아닙니다.

```
riscv64-unknown-elf-gcc \  
-march=rv32i -mabi=ilp32 -O1 \  
-ffreestanding -fno-builtin -fno-pic \  
-fno-asynchronous-unwind-tables -msmall-data-limit=0 \  
-mno-relax -nostdlib -nostartfiles \  
-Wl,--build-id=none,--no-relax \  
-T firmware/app.ld \  
-o build/firmware/hello_app.elf firmware/hello_app.c
```

옵션들은 모두 같은 일을 하지 않습니다. 어떤 것은 instruction 선택, 어떤 것은 compiler의 환경 가정, 어떤 것은 최종 linker 동작에 영향을 줍니다. 한 단계의 설정을 다른 단계의 안전장치로 오해하지 않는 것이 중요합니다.

묶음	현재 옵션	정하는 약속
ISA·ABI	<code>-march=rv32i -mabi=ilp32</code>	사용할 명령과 자료형·호출 규칙
최적화	<code>-O1</code>	C의 의미를 유지하며 code 변환
실행 환경	<code>-ffreestanding -fno-builtin</code>	hosted C·표준 함수에 대한 가정 제한
주소 참조	<code>-fno-pic -msmall-data-limit=0</code>	고정 배치와 제한된 초기 register 환경
relaxation	<code>-mno-relax, --no-relax</code>	compiler/assembler·linker의 관련 변환 제한
보조 metadata	<code>-fno-asynchronous-unwind-tables, --build-id=none</code>	이 runtime이 사용하지 않는 정보의 생성 정책
링크 환경	<code>-nostdlib -nostartfiles -T firmware/app.ld</code>	startup·library·메모리 배치
출력 파일	<code>-o build/firmware/hello_app.elf</code>	결과 파일 경로

`-O1`의 O는 대문자 알파벳이고 `-o`는 소문자 출력 옵션입니다. compiler의 `-O1`과 objcopy의 `-O binary`도 서로 다른 기능입니다. 옵션 한 글자를 도구와 분리해서 외우면 혼동하기 쉽습니다.

10.4.2 march는 CPU가 해석할 수 있는 명령을 제한한다

`-march=rv32i`에서 rv는 RISC-V, 32는 기본 정수 register 폭 XLEN=32, i는 기본 정수 instruction 집합을 뜻합니다. 현재 앱의 산술·load/store·분기·함수 호출을 이 범위에 맞춰 생성하도록 요구합니다. FPGA의 clock 속도나 single-cycle 구조를 선택하는 옵션은 아닙니다. compiler는 instruction 의미를 목표로 하고, 그 명령을 몇 단계의 회로로 구현할지는 RTL의 책임입니다.

현재 core에는 M extension의 MUL/DIV, C extension의 16비트 compressed instruction, F/D extension의 부동소수점 register·instruction이 없습니다. `-march=rv32im`이나 `rv32ic`로 바꾸면 더 작은 code나 다른 instruction이 생성될 수 있지만, hardware가 그것을 지원하게 되는 것은 아닙니다. 소프트웨어 설정과 RTL 지원의 불일치는 illegal instruction 또는 잘못된 fetch로 이어질 수 있습니다.

또한 rv32i 설정이 C의 `*`와 `/` 연산자를 전부 금지하지는 않습니다. 상수 곱은 shift·add로 구현할 수 있고, 변수가 관련된 연산은 runtime helper를 호출할 수 있습니다. 중요한 것은 최종 실행되는 instruction과 필요한 지원 routine입니다. 실제 예는 10.4.6절에서 확인합니다.

CSR instruction은 현재 표준에서 Zicsr로 구분합니다. 최신 계열 도구에서는 CSR assembly를 사용하는 kernel에 `rv32i_zicsr` 같은 명시적 ISA 설정이 필요할 수 있지만, 설치된 GCC 9.3.0·Binutils 2.34 조합은 현재 저장소의 rv32i 설정으로 기존 CSR code를 처리합니다. 앱 `hello_app.c` 자체는 CSR instruction을 직접 사용하지 않고 ECALL을 사용합니다. 도구 업그레이드 시에는 새 문법의 허용 여부와 RTL의 구현 범위를 별도로 확인해야 합니다.

target 옵션의 기준은 [GCC RISC-V Options](#)이며, 이 교재의 실행 결과는 설치된 버전에서 관측한 것입니다. 옵션이 assembler에 수용되었다는 사실만으로 privileged ISA의 전체 동작을 구현했다는 결론을 내리지 않습니다.

10.4.3 mabi는 자료형과 함수 호출의 약속이다

`-mabi=ilp32`는 이 환경에서 `int·long·pointer`를 각각 32비트로 사용하는 ABI를 선택합니다. ISA가 “어떤 명령을 실행하는가”라면 ABI는 “함수 인수·반환값·register·stack·자료형을 어떻게 주고받는가”입니다. 이 둘은 관련되지만 같은 설정이 아닙니다.

현재 앱은 shell이 `void (*) (void)` 함수처럼 호출합니다. compiler가 만든 앱은 ABI에 맞춰 ra로 반환하고, 사용한 callee-saved register와 SP를 복원해야 합니다. trap.S의 context 저장과는 다른 일반 함수 호출의 계약입니다. SYS_PUTC의 서비스 번호 1은 이 ABI 자체가 정한 값이 아니라 Mini OS의 별도 syscall 규약입니다.

대상	현재 ILP32에서의 크기·의미
<code>char</code>	1바이트
<code>int, unsigned int</code>	4바이트
<code>long</code>	4바이트
<code>pointer</code>	4바이트
<code>long long</code>	8바이트, 필요하면 여러 instruction으로 처리
syscall 인수 register	현재 OS가 a0 - a3 등을 사용한다고 약속

32비트 CPU라고 모든 C 자료형이 4바이트가 되는 것은 아닙니다. 9장에서 `file_entry`의 두 정수 field가 각각 4바이트인 것은 현재 자료형과 ABI의 결과입니다. host에서 `pointer`가 8바이트라고 target의 `pointer`도 8바이트라고 추정하면 안 됩니다.

다음 검사는 프로그램을 실행하지 않고 compiler가 선택한 폭을 확인하는 방법입니다. 이 환경에서 `int·long·pointer`는 4, XLEN은 32, hosted 표지는 0으로 나옵니다.

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 \
  -ffreestanding -dM -E -x c /dev/null |
  rg ' __riscv_xlen|__SIZEOF_(INT|LONG|POINTER)|__STDC_HOSTED__ '
```

모든 object와 필요한 library는 호환되는 ISA·ABI로 준비해야 합니다. 최종 링크 명령에만 `rv32i·ilp32`를 지정한다고 이미 RV64나 다른 ABI로 만들어진 object가 자동으로 다시 컴파일되는 것은 아닙니다.

10.4.4 O1과 최적화에 따라 달라지는 instruction

`-O1`은 GCC 최적화 수준 1입니다. 사용하지 않는 계산 제거, 상수 처리, 일부 함수 inline 등으로 소스의 의미를 유지하면서 code 형태를 바꿀 수 있습니다. 어떤 변환이 적용되는지는 GCC 버전·target·source에 따라 달라집니다. 최적화 수준의 일반 의미는 [GCC Optimize Options](#)를 참고하십시오.

현재 hello_app에서는 첫 문자 H가 상수 72라는 사실을 이용한 `addi a0,zero,72`가 보이고, `putc_sys()`의 ECALL 경로가 `app_main` 안에 포함됩니다. 따라서 C 함수 개수와 ELF의 독립된 함수 symbol 개수가 항상 같지 않습니다. **소스 한 줄과 instruction 한 개도 일대일 대응하지 않습니다.**

동일 source와 나머지 옵션을 유지하고 최적화 수준만 바꿔 재빌드한 결과는 다음과 같습니다. 크기는 ELF 전체가 아니라 objcopy로 추출한 BIN payload 기준입니다.

옵션	현재 도구의 관측 payload 크기	이 결과에서 읽을 수 있는 것
-O0	172바이트	최적화하지 않은 형태는 더 많은 code를 필요로 함
-O1	60바이트	현재 Makefile의 기준 결과
-O2	60바이트	이 작은 예에서는 크기가 같음
-Os	60바이트	크기 지향 설정도 이 예에서는 같은 크기

이는 모든 프로그램에서 O2·Os가 O1과 같다는 뜻이 아닙니다. 파일 크기가 같다고 실행 시간이나 instruction 순서도 항상 같은 것은 아닙니다. 성능을 비교하려면 실제 실행 경로·UART 대기·trap 비용까지 측정해야 합니다. 또한 최적화로 함수 길이가 달라지면 문자열 주소와 ECALL 주소도 달라질 수 있으므로 14장의 숫자를 무조건 재사용하지 않습니다.

MMIO 접근과 shared 상태의 의미를 지켜야 할 때는 C의 volatile, inline assembly의 operand·clobber 규약도 올바르게 사용해야 합니다. 최적화를 끄는 것으로 잘못된 hardware 접근 규약을 근본적으로 해결할 수는 없습니다.

10.4.5 freestanding과 fno-builtin이 정하는 실행 환경

-ffreestanding은 완전한 hosted C 환경을 가정하지 않도록 합니다. 현재 앱에는 host OS의 argc/argv 전달, C startup, 자동 종료 처리, 완전한 표준 library가 없습니다. shell이 준비한 SP와 실행 환경 안에서 `app_main`을 직접 호출하므로 이 가정이 맞습니다.

GCC에서 freestanding 설정은 -fno-builtin도 함의합니다. Makefile은 의도를 명확히 하기 위해 두 옵션을 함께 적었습니다. builtin 가정을 끄면 일반 표준 함수 이름을 compiler가 특별한 의미로 인식해 변환하는 동작을 제한합니다. 이 관계는 [GCC C Dialect Options](#)에 설명되어 있습니다.

그러나 이 옵션들이 printf·memcpy·malloc 구현을 만들어 주는 것은 아닙니다. 현재 문자열 출력은 libc의 printf가 아니라 SYS_PUTC ECALL을 직접 요청하는 code입니다. -fno-builtin을 주었다고 printf를 호출할 수 있게 되는 것은 아니며, 구현이 없으면 링크 오류가 납니다.

반대로 freestanding이라고 compiler가 어떤 지원 함수도 호출하지 않는다는 보장은 없습니다. 구조체 복사나 compiler가 지원해야 하는 산술 연산 등에 별도 routine이 필요할 수 있습니다. 다음 절의 library 링크 정책까지 함께 읽어야 합니다. **-ffreestanding은 compile 환경 가정이고 -nostdlib는 link 입력 정책입니다.**

10.4.6 nostdlib·nostartfiles와 libgcc helper

-nostartfiles는 표준 startup object의 자동 사용을 생략하고, -nostdlib는 기본 startup과 표준 library의 자동 링크를 생략합니다. 따라서 이 구성에서는 두 옵션이 일부 중복되지만, startup도 library도 제공하지 않는다는 의도를 드러냅니다. -nostartfiles 하나만으로 모든 기본 library 링크가 사라지는 것은 아닙니다. 관련 구분은 [GCC Link Options](#)를 참고하십시오.

커널은 boot.S가 reset 후 stack과 .bss를 준비하고, 앱은 이미 실행 중인 shell의 stack과 CSR 상태를 물려받습니다. 현재 앱에 일반적인 crt0 startup을 붙이면 별도 entry·초기화·종료 경로를 요구할 수 있어 loader의 단순 함수 호출 계약과 맞지 않습니다. **이 옵션은 “초기화가 필요 없다”가 아니라 “필요한 초기화를 우리가 정의한다”는 뜻입니다.**

libc와 libgcc도 구분해야 합니다. libc는 printf·문자열·메모리 함수 같은 C library이고, libgcc는 compiler가 특정 연산을 구현할 때 사용하는 지원 routine을 포함합니다. 현재 `-nostdlib` 때문에 libgcc 역시 기본으로 링크되지 않습니다.

예를 들어 다음 함수만 RV32I object로 컴파일하면 이번 toolchain은 unsigned 나눗셈 helper `__udivsi3` 참조를 생성합니다.

```
unsigned quotient(unsigned a, unsigned b)
{
    return a / b;
}
```

그 object를 `nm -u`로 검사하면 `U __udivsi3`가 보입니다. 이것은 compiler가 DIV instruction을 생성했다는 뜻이 아니라, 나눗셈을 수행할 software routine의 주소가 필요하다는 뜻입니다. 링크 입력 어디에도 그 구현이 없으면 undefined reference로 실패합니다.

해결은 source 연산을 단순화하거나, 필요한 지원 routine을 직접 제공하거나, 현재 ISA·ABI에 맞는 library를 검토하여 명시적으로 링크하는 것입니다. 임의의 library를 붙이면 되는 것은 아닙니다. 앱 크기, 미지원 instruction, 추가 runtime 의존성도 확인해야 하며 이 교재의 기본 Makefile은 그대로 유지합니다.

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 \
-print-libgcc-file-name
```

현재 출력 경로에는 `rv32i/ilp32/libgcc.a`가 포함됩니다. 이는 해당 target 조합의 library 선택 경로를 확인하는 것이지 이미 앱에 링크되었다는 증거는 아닙니다. 최종 참조와 실제 포함 code는 ELF·map·역어셈블로 확인합니다.

10.4.7 fno-pic과 고정 실행 주소

PIC는 Position-Independent Code입니다. `-fno-pic`은 현재 앱을 위치 독립 code로 만들지 않는 정책입니다. **실제 배치 주소 0x4000은 이 옵션이 아니라 app.ld의 ORIGIN이 정합니다.** 두 설정이 함께 현재 고정 주소 ABI를 이룹니다.

현재 첫 두 instruction은 문자열의 주소 0x4024를 구성합니다. loader가 같은 byte를 0x5000에 복사했다고 instruction 내부의 0x4024 참조가 자동으로 0x5024로 바뀌지는 않습니다. 상대 branch 일부가 동작해도 절대 주소 참조 때문에 전체 프로그램이 올바르다고 할 수 없습니다.

PIC를 켜는 것만으로 임의 주소 앱 loader가 완성되는 것도 아닙니다. 생성 code의 주소 참조 방식, GOT 같은 보조 구조가 필요한지, relocation을 누가 적용하는지까지 계약을 정해야 합니다. 현재 APP1에는 relocation table이나 동적 linker 정보가 없습니다.

compiler의 PIC 설정과 linker가 PIE executable을 만드는 설정도 분리된 개념입니다. 이 환경의 실제 결과는 `readelf`에서 EXEC로 확인했습니다. 다른 toolchain의 기본 설정이 바뀌면 단순히 옵션 이름 하나를 보고 같은 결과라고 가정하지 말고 ELF 형식과 참조를 확인해야 합니다. code-generation 옵션의 일반 기준은 [GCC Code Generation Options](#)에 있습니다.

10.4.8 small-data와 relaxation을 함께 제한하는 이유

`gp=x3`는 ABI의 global pointer입니다. 어떤 RISC-V code 생성·링크 구성에서는 작은 전역 데이터를 gp 근처에 배치하여 짧은 주소 참조를 사용할 수 있습니다. 하지만 현재 boot.S와 앱 진입 경로는 그런 전역 pointer의 초기값을 준비하는 runtime을 제공하지 않습니다.

`-msmall-data-limit=0`은 작은 데이터를 별도 `small-data` 영역으로 분류하는 기준을 0으로 둡니다. “전역변수 크기를 0으로 만든다”거나 “stack을 작게 만든다”는 뜻이 아닙니다. 현재 제한된 startup 환경에서 `small-data` 배치에 의존하는 경로를 피하려는 설정입니다.

`relaxation`은 linker 등이 주소·거리 정보를 이용하여 `instruction sequence`를 더 짧거나 다른 참조 방식으로 바꾸는 최적화입니다. 그중 `gp-relative` 참조는 적절한 `gp` 초기화가 필요할 수 있습니다. 현재 빌드는 compiler/assembler 쪽의 `-mno-relax`와 linker 쪽의 `--no-relax`를 함께 사용합니다.

설정	주로 작용하는 단계	현재 목적
<code>-msmall-data-limit=0</code>	compiler의 data 배치 정책	<code>small-data</code> 의존 경로 줄이기
<code>-mno-relax</code>	GCC가 생성·조립하는 code의 <code>relaxation</code> 정책	관련 annotation·변환 허용 제한
<code>-Wl,--no-relax</code>	최종 linker	링크 시 <code>relaxation</code> 수행 제한

이것은 `gp`를 0으로 초기화하는 기능이 아닙니다. 이미 다른 설정으로 빌드된 object나 수동 assembly가 `gp`를 사용한다면 최종 링크 옵션만으로 안전해진다고 보장할 수 없습니다. 앱에 초기값 없는 특별한 register 의존성이 없는지 실제 결과를 보는 이유입니다.

현재 `hello_app`은 전역변수를 사용하지 않는 작은 예제여서 어떤 옵션을 제거해도 당장 결과가 달라지지 않을 수 있습니다. 그것은 옵션이 항상 불필요하다는 증거가 아닙니다. 새로운 앱·지원 함수에서 주소 참조 패턴이 달라질 수 있으므로 옵션의 목적과 startup의 책임을 함께 유지해야 합니다.

10.4.9 asynchronous unwind table은 timer interrupt와 다르다

unwind 정보는 debugger나 runtime이 호출 stack을 거슬러 올라갈 때 사용하는 metadata입니다. `-fno-asynchronous-unwind-tables`는 instruction 경계마다 정확한 stack 복원 정보를 제공하는 asynchronous unwind table의 생성을 끄는 옵션입니다. 현재의 작은 C 앱 runtime은 이 정보를 소비하지 않으므로 불필요한 보조 section을 줄이는 정책입니다.

이름에 asynchronous가 있다고 timer interrupt를 끄는 옵션은 아닙니다. timer의 허용은 `mie·mstatus`와 hardware 요청으로 정하고, trap frame 저장은 `trap.S`의 실제 instruction이 수행합니다. unwind table을 생략해도 함수 호출에 필요한 stack 사용이나 `ra` 저장이 자동으로 없어지는 것은 아닙니다.

또한 이 옵션 하나가 모든 종류의 exception·debug·unwind metadata를 모든 구성에서 완전히 없앤다는 뜻은 아닙니다. 다른 언어·옵션·library의 영향도 있으므로 section 목록을 확인해야 합니다. `-g`로 source line 정보를 남기는 것과도 목적이 다릅니다. 이 구분의 기준은 [GCC unwind 관련 옵션](#)입니다.

10.4.10 `-Wl·build-id·T`로 linker에 전달하는 정보

`-Wl,`은 GCC driver에게 “쉼표로 나눈 뒤의 항목들을 linker에 전달하라”는 표기입니다. 대문자 `W` 뒤에 소문자 `l`이 옵니다. 따라서 다음 옵션은 linker에게 두 개의 인수를 보냅니다.

```
GCC command: -Wl,--build-id=none,--no-relax
ld arguments: --build-id=none --no-relax
```

`--build-id=none`은 build-id note 생성을 하지 않도록 합니다. build-id는 빌드 산출물 식별을 돕는 metadata로, 현재 APP1의 byte 합 checksum과는 다릅니다. 이를 생략해도 UART 수신의 무결성 검사가 추가되거나 없어지는 것은 아닙니다. linker의 `build-id·map·script` 옵션은 [GNU ld Options](#)를 참고하십시오.

-T firmware/app.ld 는 기본 배치 대신 이 script를 사용하도록 합니다. script는 APP 영역의 시작 0x4000, 길이 8 KiB, app_main의 entry section 우선 배치, .bss 금지 등을 정의합니다. 파일 경로가 잘못되면 링크가 실패하고, 다른 script를 사용하면 빌드 자체는 성공하더라도 loader의 0x4000 계약과 어긋날 수 있습니다.

링커의 ASSERT는 이 최소 형식의 제한을 조기에 드러냅니다. 예를 들어 실제 사용되는 static volatile unsigned counter; 를 추가하면 .bss 공간이 필요하고 현재 script는 minimum app format does not support .bss 오류로 거절합니다. 0 초기화를 loader가 제공하지 않으므로 이 검사를 없애는 것만으로 문제를 해결해서는 안 됩니다.

-Wl, -Map=build/toolchain_lab/hello_app.map 은 symbol·입력 section이 어디에 배치되었는지 확인할 text map을 만듭니다. map은 CPU가 실행할 파일이 아니라 host의 분석 자료입니다. 새로운 함수·상수가 예상과 다른 위치에 들어갔을 때 source, map, readelf, objdump를 함께 보면 linker의 결정을 추적할 수 있습니다.

10.4.11 전처리 옵션과 SHELL_MODE를 구분하기

현재 앱 recipe에는 없지만 kernel·shell recipe에는 -DSHELL_MODE, -DTICK_CYCLES=125000u 같은 옵션이 있습니다. -D는 C/전처리 단계에서 macro를 정의합니다. -DSHELL_MODE는 해당 이름을 정의하여 #ifdef SHELL_MODE 분기를 선택하고, TICK_CYCLES 정의는 timer 예약에 사용할 상수를 결정합니다.

이것은 Verilog top의 SHELL_MODE parameter와 같은 저장 장소가 아닙니다. 이름을 같이 사용하지만 하나는 firmware를 만들 때, 다른 하나는 FPGA 회로를 elaboration·합성할 때 적용됩니다. 올바른 shell 구성은 두 빌드 경로가 서로 맞아야 합니다. 2장의 hardware/software 설정 구분과 연결됩니다.

-I는 header 검색 경로, -D는 macro 정의, -E는 전처리 후 중지, -S는 assembly 생성 후 중지, -c는 object 생성 후 중지입니다. -I로 directory를 추가한다고 library가 링크되는 것은 아니며, -D를 추가한다고 이미 생성된 object 내부의 상수가 바뀌는 것도 아닙니다. 해당 단계부터 다시 빌드해야 합니다.

10.4.12 debug 옵션과 실패 위치를 이용한 점검

소스와 instruction을 함께 보고 싶다면 수업용 별도 빌드에 -g를 추가할 수 있습니다. 예를 들어 10.3.3절의 compile 단계에서 동일한 code-generation 옵션에 -g를 추가하고, debug 정보를 가진 object를 링크한 뒤 objdump -d -S로 봅니다. -g는 최적화 수준과 독립적으로 사용할 수 있지만, 최적화된 변수·inline 함수는 소스와 일대일로 대응하지 않을 수 있습니다.

이번 앱을 -O1 -g로 검증했을 때 source 혼합 역어셈블이 가능했고 objcopy 후 payload는 기존의 60바이트와 같았습니다. 이것은 이 예제의 관측 결과입니다. debug section은 보통 raw payload에 포함되지 않지만, 모든 옵션 변경이 모든 프로그램의 instruction에 영향을 주지 않는다고 일반화하지 않습니다.

-Wall -Wextra는 수업용 추가 warning 검사에 유용합니다. 다만 현재 기본 CFLAGS에 들어 있는 옵션과 추가 실험 옵션을 구분해 기록하십시오. warning이 없다는 것은 ISA 지원·stack 여유·loader 규약까지 검증했다는 뜻이 아닙니다.

나타난 문제	실패한 계층	먼저 확인할 것
C 문법·자료형 오류	compile	source, include, macro 선택
CSR instruction을 허용하지 않음	assemble/ISA 설정	도구 버전, march의 extension 표기, RTL 지원
undefined reference to __udivsi3	link	compiler helper 구현과 library 정책
.bss 미지원 ASSERT	link/앱 형식	0 초기화가 필요한 전역 상태
앱 영역 초과	link/메모리 배치	section 크기와 app.ld의 8 KiB 창
HEX 변환 성공, 업로드 크기 거절	파일 시스템·프로토콜	8192바이트 검사와 6644바이트 payload 제한의 차이

나타난 문제	실패한 계층	먼저 확인할 것
실행 때 illegal instruction	CPU/runtime	최종 instruction, 주소·byte 순서, 미지원 CSR

마지막으로 옵션을 바꾸면 실제 명령행을 확인하고 재빌드한 뒤, ELF32·entry·section·instruction·BIN byte를 다시 검사합니다. 컴파일 성공은 전체 경로 중 한 단계의 성공입니다. UART 업로드와 FPGA 실행 검증은 그 뒤의 별도 단계이며 이 문서의 toolchain 실습만으로 대체되지 않습니다.

10.5 ELF에서 raw binary로 바뀌면서 사라지는 것

ELF에는 entry address, section 정보, symbol 등이 있습니다. `objcopy -O binary` 결과에는 그 메타데이터가 포함되지 않습니다. 메모리에 실릴 바이트들이 raw image로 남습니다. GNU 도구의 동작은 [objcopy 공식 문서](#)에서도 확인할 수 있습니다.

그래서 `hello_app.bin`의 첫 바이트만 보고 원래 링크 주소가 `0x4000`이었는지 일반적으로 복원할 수 없습니다. 현재 loader는 그 정보를 파일에서 읽는 대신 환경의 고정 규칙으로 알고 있습니다.

현재 ELF의 `.text` 주소는 `0x4000`이지만 raw `.bin` 앞에 16384바이트의 0이 자동으로 붙는 것은 아닙니다. 이 예제의 가장 낮은 loadable section부터 image가 시작하므로 파일 offset 0이 실행 주소 `0x4000`에 대응합니다. section 사이에 주소 간격이 있으면 raw image의 간격 처리도 확인해야 하지만, 현재 예제는 연속된 60바이트입니다.

APP1 header의 크기 field는 raw payload 크기만 기록합니다. entry, architecture, required privilege, relocation 정보는 header에 없습니다. loader가 단순한 이유는 이 정보를 모두 플랫폼의 고정 약속으로 둔 결과입니다. 파일 형식을 확장할수록 그 약속 일부를 metadata로 옮기고 검증하는 책임이 생깁니다.

11. 링커와 앱 ABI의 계약

11.1 ENTRY와 실제 첫 바이트는 별개다

앱 linker script의 주요 부분은 다음과 같습니다.

```
OUTPUT_ARCH(riscv)
ENTRY(app_main)
MEMORY { APP (rwx) : ORIGIN = 0x00004000, LENGTH = 8K }
SECTIONS
{
    . = ORIGIN(APP);
    .text : ALIGN(4) {
        KEEP(*(.text.app_entry))
        *(.text .text.*)
        *(.rodata .rodata.*)
    } > APP
    .data : ALIGN(4) { *(.data .data.*) } > APP
    .bss (NOLOAD) : ALIGN(4) { *(.bss .bss.*) *(COMMON) } > APP
    ASSERT(SIZEOF(.bss) == 0,
           "minimum app format does not support .bss")
}
```

ENTRY(app_main) 은 ELF의 진입점 정보를 지정합니다. 이 기능은 [GNU ld의 Entry Point 문서](#)에 설명되어 있습니다. 현재 raw loader는 ELF header를 읽지 않으므로, 이 선언만으로 올바른 실행이 보장되지는 않습니다.

실제 보장은 app_main 을 .text.app_entry 에 넣고, 그 section을 0x4000 의 첫 위치에 배치하는 데서 나옵니다. KEEP 는 section garbage collection을 사용하는 구성에서도 해당 입력 section을 유지하도록 하는 지시입니다. 현재 Makefile은 garbage collection 옵션을 쓰지 않지만 entry의 의도를 명시합니다.

output section의 순서가 raw image의 layout을 결정합니다. KEEP(*(.text.app_entry)) 뒤의 *(.text .text.*) 가 나머지 함수 코드를 모으고, 이어서 .rodata 를 같은 output .text 에 넣습니다. linker script를 읽을 때 input section 이름과 output section 이름을 구분하면 disassembly에 문자열이 섞이는 이유도 설명할 수 있습니다.

ENTRY 를 다른 symbol로 바꿔 ELF entry만 변경해도 현재 loader는 그 값을 읽지 않습니다. 이 실험으로 ELF metadata와 raw loader의 고정 규약을 구분할 수 있습니다. 진입 위치를 바꾸려면 파일 형식에 entry offset을 추가하고 검증하거나, entry section이 실제로 첫 byte에 놓이도록 유지해야 합니다.

코드 해부 11-A. app.ld의 각 줄이 loader와 맺는 계약

출처: [firmware/app.ld](#), 원본 3 - 18행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
3 | OUTPUT_ARCH(riscv)
4 | ENTRY(app_main)
5 | MEMORY { APP (rwx) : ORIGIN = 0x00004000, LENGTH = 8K }
6 | SECTIONS
7 | {
8 |     . = ORIGIN(APP);
9 |     .text : ALIGN(4) {
10 |         KEEP(*(.text.app_entry))
11 |         *(.text .text.*)
12 |         *(.rodata .rodata.*)
13 |     } > APP
14 |     .data : ALIGN(4) { *(.data .data.*) } > APP
15 |     .bss (NOLOAD) : ALIGN(4) { *(.bss .bss.*) *(COMMON) } > APP
16 |     ASSERT(SIZEOF(.bss) == 0, "minimum app format does not support .bss")
17 |     /DISCARD/ : { *(.comment) *(.riscv.attributes) }
18 | }
```

원본 행	한 줄씩 읽는 동작과 의미
3	출력 object architecture를 RISC-V로 정합니다. 실제 지원 instruction은 compiler 옵션과 code를 함께 확인해야 합니다.
4	ELF header의 entry symbol을 app_main으로 지정합니다. raw loader는 이 header를 읽지 않습니다.
5	APP 영역을 0x4000부터 8 KiB로 선언합니다. rwx는 linker의 영역 속성이며 FPGA에 접근 보호 회로를 만들지 않습니다.
6	입력 section을 출력 주소에 배치하는 규칙을 시작합니다.
7	앞에서 선언한 함수·블록의 본문을 시작합니다.
8	location counter를 APP 시작 주소로 설정합니다.
9	4바이트 정렬한 output .text를 시작합니다.
10	entry 전용 입력 section을 맨 앞에 둡니다. KEEP는 section 제거 옵션을 쓰는 경우에도 보존하려는 지시입니다.
11	나머지 code section을 모읍니다.
12	문자열·상수도 같은 output .text에 모읍니다. 이 때문에 역어셈블에서 데이터가 instruction처럼 표시될 수 있습니다.
13	.text 결과를 APP 영역에 배치합니다.
14	초기값 있는 .data도 APP에 놓습니다. 파일에 포함된 byte는 run 때 다시 복사됩니다.
15	.bss를 NOLOAD 영역으로 기술합니다. 메모리 요구를 나타내지만 초기값 byte를 일반 데이터처럼 저장하는 것이 아닙니다.
16	현재 loader가 .bss 초기화를 제공하지 않으므로 크기 0만 허용합니다. 이 제한을 지우는 것만으로 기능이 구현되지 않습니다.
17	이 최소 이미지에서 쓰지 않는 comment·architecture attribute section을 버립니다.
18	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

ENTRY(app_main) 과 첫 code 배치는 별개입니다. raw BIN offset 0에 app_main이 오게 하는 것은 section 순서이며, run은 그 약속을 믿고 0x4000을 호출합니다.

11.2 왜 다른 주소에 복사하면 안 되는가

현재 예제는 문자열 주소 0x4024를 명령어의 immediate로 구성합니다. payload 전체를 0x5000으로 옮겨 실행하면 코드 안의 문자열 참조는 여전히 0x4024를 가리킵니다. 상대 branch 일부가 맞게 동작해도 절대 주소를 포함한 참조는

틀릴 수 있습니다.

임의 주소 실행을 원하면 position-independent code와 그 실행 환경을 설계하거나, relocation 정보를 유지하고 loader가 적용해야 합니다. 현재 APP1 header에는 relocation table이 없습니다.

고정 주소 코드에는 함수·문자열·전역변수 주소가 여러 형태로 들어갈 수 있습니다. 명령어에 완성된 32비트 주소 한 개가 그대로 놓이는 경우만 생각하면 안 됩니다. LUI와 ADDI의 조합처럼 서로 떨어진 immediate field에 나뉘어 표현될 수도 있습니다. raw byte를 무작정 일정 값만큼 수정하는 relocation은 올바르지 않습니다.

현재 방식의 장점은 loader가 relocation type을 해석하지 않아도 된다는 것입니다. 비용은 앱 하나가 사용할 수 있는 위치가 고정된다는 점입니다. 여러 앱을 동시에 배치하려면 각각 다른 고정 주소로 링크하거나, position-independent 규칙·relocation·가상주소 중 하나 이상의 방법을 도입해야 합니다.

11.3 앱 시작 때 이미 준비된 것

앱은 boot.S 를 다시 실행하지 않습니다. 셸이 준비한 SP, CSR, timer, trap vector, UART, 파일 시스템 환경을 그대로 사용합니다. app entry의 함수형은 void app_main(void)이며 인수와 반환값을 정식 ABI로 전달하는 코드는 없습니다.

일반 함수 호출에서 ra는 반환 주소이고 sp는 stack pointer입니다. s0 - s11은 callee-saved이며 a·t 계열은 caller-saved입니다. 표준 정수 호출 규약은 16바이트 stack alignment를 사용합니다. 이 레지스터·정렬 규칙의 기준은 [RISC-V psABI의 Calling Convention](#)입니다. syscall 번호는 이 표준이 아닌 Mini OS 코드가 정합니다.

현재 앱은 셸로 정상 반환하려면 callee-saved 레지스터와 SP를 지켜야 합니다. 별도 syscall exit는 없습니다. return이 일반 함수 복귀 명령으로 변환되어 셸로 돌아옵니다.

caller-saved register는 앱이 바뀌도 셸 compiler가 호출 앞뒤에서 필요한 값을 처리합니다. callee-saved register는 앱 compiler가 사용하면 저장·복원해야 합니다. 이것은 “OS가 모든 앱 함수를 위해 register를 자동 보존한다”는 의미가 아닙니다. 일반 호출은 ABI로, 비동기 trap은 trap frame으로 보호하는 두 기구가 함께 작동합니다.

현재 gp를 초기화하는 startup은 앱에 없습니다. 그래서 small-data와 relaxation 관련 설정을 보수적으로 선택했습니다. tp 기반 TLS, C++ 전역 생성자, 동적 초기화, 환경변수, heap 제공도 이 최소 runtime의 계약에 포함되지 않습니다. 단순 C 함수에서 자연스럽게 쓰던 환경 의존 기능을 독립 앱에 가져올 때 이 초기화 책임을 점검해야 합니다.

11.4 .data와 .bss

초기값이 있는 .data는 raw image에 포함되면 실행 RAM으로 함께 복사됩니다. 따라서 동일 앱을 다시 run하면 파일에 저장된 초기 데이터가 다시 로딩될 수 있습니다.

반면 .bss는 일반적으로 파일에 초기값 바이트를 저장하지 않고 실행 전 0으로 초기화하는 영역입니다. 현재 APP1 형식은 .bss 주소·크기를 저장하지 않으며 loader도 이를 지우지 않습니다. 그래서 linker가 .bss 크기 0을 강제합니다.

```
static unsigned counter; /* 대개 .bss가 필요하므로 현재 앱 형식에서 거부 */
```

stack에 두는 지역변수까지 금지하는 것은 아닙니다. 다만 작은 공유 스택을 사용합니다. const char *p = "..."의 문자열은 이 linker에서 .rodata가 .text 출력 section에 합쳐져 payload에 포함됩니다.

예를 들어 초기값이 7인 수정 가능한 전역변수가 `.data`에 배치되면 파일에 그 7이 들어갑니다. 앱이 실행 중 9로 바꾸어도 MiniFS의 파일을 직접 수정하지 않았다면 다음 run의 재로딩으로 다시 7이 됩니다. 실행 RAM의 수정과 저장 파일의 수정은 독립적입니다.

미초기화 전역변수의 `.bss`는 “컴파일러가 0을 채운 byte를 파일에 저장한다”는 방식이 아닐 수 있습니다. 그래서 raw image 크기만큼 복사하는 loader로는 C가 기대하는 zero initialization을 일반적으로 제공할 수 없습니다. linker assertion은 이 환경의 제한을 link 시점에 드러내는 장치입니다. section을 임의로 바꾸어 assertion을 우회하기보다 runtime 규칙을 명확히 확장해야 합니다.

11.5 앱 간 연결과 외부 symbol

앱을 컴파일할 때 `kernel.c`와 다시 링크하지 않습니다. 커널 내부 함수 이름을 임의로 호출하면 linker는 그 정의를 찾을 수 없습니다. `syscall`은 커널 함수의 주소를 앱에 고정하지 않고 서비스를 이용하는 경로입니다.

현재는 동적 linker, shared library, symbol lookup table이 없습니다. 더 큰 라이브러리를 쓰려면 앱과 함께 정적으로 링크하고 크기·ISA·초기화 조건을 검토하거나 명시적인 OS 서비스로 제공해야 합니다.

`syscall`의 이점은 커널 내부 함수의 위치가 바뀌어도 서비스 번호와 의미가 유지되면 앱 호출 경로를 유지할 수 있다는 것입니다. 현재 `uart_putc`의 실제 주소는 build마다 바뀔 수 있지만 앱은 그 주소를 모르고 `a7=1`을 사용합니다. 이 경계가 binary interface의 중요한 사례입니다.

향후 서비스를 추가할 때는 번호뿐 아니라 인수 자료형·길이 단위·오류 값·register 보존 규칙도 문서화해야 합니다. kernel이 지원하지 않는 번호는 현재 -1을 반환합니다. `syscall` 이름을 C header로 공유하면 숫자의 오타를 줄일 수 있지만, 이미 빌드된 앱과 새 kernel의 호환성은 여전히 ABI 버전 정책으로 관리해야 합니다.

11.6 하위 함수를 호출하는 앱의 stack 예제

현재 hello 앱은 leaf 형태이지만 일반 앱은 여러 함수를 호출할 수 있습니다. 아래는 A를 세 번 출력하고 셀에 반환하는 호출 규약 설명용 assembly입니다. 기존 `hello_app.c`를 바꾸는 명령이 아니라 stack 사용을 읽기 위한 별도 예제입니다.

```
.option norvc
.section .text.app_entry,"ax",@progbits
.globl app_main
app_main:
    addi sp,sp,-16
    sw   ra,12(sp)
    sw   s0,8(sp)
    li   s0,3
.Lprint_again:
    call print_a
    addi s0,s0,-1
    bnez s0,.Lprint_again
    lw   s0,8(sp)
    lw   ra,12(sp)
    addi sp,sp,16
    ret

.section .text,"ax",@progbits
print_a:
    li   a0,65
    li   a7,1
    ecall
    ret
```

app_main은 셸이 전달한 ra를 stack의 +12에 저장합니다. call print_a가 ra를 내부 복귀 위치로 덮어써도 마지막에 저장된 셸 복귀 주소를 복원할 수 있습니다. s0는 callee-saved이므로 app_main이 loop counter로 사용한 뒤 원래 값을 복구합니다. 16바이트를 할당하여 호출 지점의 stack alignment를 유지합니다.

print_a 안의 ECALL은 새로운 function-call ra를 만들지 않습니다. trap handler가 사용하는 register들은 trap frame에서 복원되므로 print_a의 ret는 app_main의 loop로 돌아옵니다. 이후 app_main의 ret는 복원한 ra를 사용해 셸로 돌아옵니다. 같은 ret 라도 호출 깊이에 따라 가리키는 caller가 다릅니다.

SP가 S인 상태로 앱에 들어왔다면 일반 앱 frame은 [S-16, S) 에 있고, print_a의 ECALL frame은 그 아래 [S-144, S-16) 에 놓입니다. C handler frame은 더 아래로 자랍니다. stack 사용 예산을 “앱이 16바이트만 쓴다”로 끝내지 않고 비동기 처리 비용까지 더해야 하는 이유입니다.

11.7 커널 링커 스크립트 전체 읽기

앞의 11.1절은 별도로 적재할 앱의 linker script를 설명했습니다. 여기서는 셸과 커널 자체를 만드는 firmware/linker_shell.ld를 읽습니다. 두 파일은 같은 문법을 사용하지만 배치 영역과 진입점이 다릅니다. 아래 코드는 해당 파일의 전체 내용이며, 긴 줄만 읽기 쉽게 나누었습니다.

출처: firmware/linker_shell.ld. 줄바꿈 외에는 원본의 배치 규칙을 유지했습니다.

```
OUTPUT_ARCH(riscv)
ENTRY(_start)
/* 32 KiB RAM: kernel below 0x4000, flat app at 0x4000..0x5fff,
 * kernel stack grows down from 0x8000 and must not enter the app window. */
MEMORY { KRAM (rwx) : ORIGIN = 0x00000000, LENGTH = 16K }
SECTIONS
{
    . = ORIGIN(KRAM);
    .text : ALIGN(4) {
        KEEP(*(.text.start))
        *(.text .text.*)
        *(.rodata .rodata.*)
    } > KRAM
    .data : ALIGN(4) { *(.data .data.*) } > KRAM
    .bss (NOLOAD) : ALIGN(4) {
        _bss_start = .; *(.bss .bss.*) *(COMMON)
        . = ALIGN(4); _bss_end = .;
    } > KRAM
    _stack_top = 0x00008000;
    ASSERT(_bss_end <= 0x00004000,
        "shell kernel exceeds reserved lower 16 KiB")
    /DISCARD/ : { *(.comment) *(.riscv.attributes) }
}
```

링커 스크립트는 FPGA RAM을 만드는 코드가 아니라, RTL이 제공하는 주소 공간에 코드와 데이터를 배치하는 계약입니다. compiler와 assembler는 입력 오브젝트를 만들고, linker는 그 안의 section들을 모아 함수·변수·부팅용 symbol의 주소를 확정합니다. RAM의 실제 용량과 주소 디코딩은 여전히 RTL의 책임입니다.

주석의 전체 RAM은 32 KiB인데 MEMORY에는 16 KiB만 선언되어 있습니다. 모순이 아니라 커널 이미지가 사용할 정적 영역을 제한한 것입니다.

주소 범위	용도	이 스크립트의 역할
0x0000 - 0x3FFF	커널·셸 코드, 상수, 전역변수, 태스크 스택 배열	KRAM에 정적 section 배치
0x4000 - 0x5FFF	앱 업로드·실행 영역	커널의 정적 배치가 침범하지 않도록 제한

주소 범위	용도	이 스크립트의 역할
0x6000 - 0x7FFF	부팅·idle 스택용 영역	초기 sp용 심볼 0x8000을 제공

영역을 주석으로 설명하는 것과 실행 중 경계를 강제하는 것은 다릅니다. 이 파일에는 부팅 스택이 0x6000 아래로 내려가는지 검사하는 실행 코드가 없습니다.

11.8 아키텍처와 진입점 및 배치 문법

OUTPUT_ARCH와 ENTRY

`OUTPUT_ARCH(riscv)` 는 출력의 대상 아키텍처를 RISC-V로 지정합니다. 이것 하나가 RV32I 명령어 제한과 ABI까지 모두 결정하지는 않습니다. `compiler·linker`에 전달하는 `-march=rv32i`, `-mabi=ilp32` 같은 설정과 입력 객체의 형식도 일치해야 합니다.

`ENTRY(_start)` 는 ELF header의 진입점 주소를 `_start` symbol로 정합니다. 현재 `_start` 는 `boot.S`에 정의되어 있습니다. `ENTRY`는 ELF의 진입점 기록이며 CPU 리셋 PC를 변경하는 명령이 아닙니다. FPGA CPU는 리셋 시 하드웨어에 정해진 주소에서 시작합니다. 현재는 주소 0에 `.text.start`를 먼저 놓아 reset PC와 부팅 코드의 위치를 맞춥니다. HEX 초기화 이미지를 실행하는 CPU가 ELF header를 읽어 진입점을 찾는 것은 아닙니다.

MEMORY와 위치 카운터

문법 요소	구체적인 의미
KRAM	이 스크립트가 사용하는 메모리 영역 이름입니다. RTL 신호명이나 C 변수명이 아닙니다.
(<code>rxw</code>)	읽기·쓰기·실행 성격의 section을 위한 영역 속성입니다. CPU 접근 권한을 강제하는 MPU/PMP 설정이 아닙니다.
<code>ORIGIN = 0x00000000</code>	KRAM의 시작 주소입니다.
<code>LENGTH = 16K</code>	16×1024바이트이므로 끝의 다음 주소가 0x4000입니다.
SECTIONS	입력 section들을 출력 section과 주소에 배치하는 규칙을 묶습니다.
<code>. = ORIGIN(KRAM);</code>	현재 배치 위치를 KRAM 시작 주소로 설정합니다.
<code>> KRAM</code>	앞에서 만든 출력 section을 KRAM 영역에 넣습니다.

단독으로 쓰인 점 `.` 은 현재 출력 위치를 나타내는 location counter입니다. `.text` 라는 section 이름의 점과 구분해야 합니다. linker가 데이터를 배치하면 위치가 진행하며, 정렬 요구가 있으면 padding이 생길 수 있습니다.

`ALIGN(4)` 는 4바이트 경계를 뜻합니다. 현재 위치가 0x1831이라면 `. = ALIGN(4);` 실행 후 위치는 0x1834가 됩니다. 이미 4의 배수라면 그대로입니다. section 선언의 `: ALIGN(4)` 는 출력 section의 정렬 요구를 지정하고, 위치 카운터에 대한 대입은 그 지점의 위치를 실제로 앞으로 맞춥니다. 입력 객체가 16바이트 정렬을 요구하면 그 요구도 반영됩니다.

11.9 text와 data 및 bss의 줄별 해설

부팅 코드를 먼저 보존하고 배치하기

코드	하는 일
<code>.text : ALIGN(4) {</code>	4바이트 정렬의 출력 <code>.text</code> section을 시작합니다.
<code>KEEP(*(.text.start))</code>	모든 입력 객체의 <code>.text.start</code> 를 먼저 모으고 section 제거 대상에서 보존합니다.
<code>*(.text .text.*)</code>	나머지 일반 명령어 section과 <code>.text.trap</code> 같은 하위 이름들을 모읍니다.
<code>*(.rodata .rodata.*)</code>	문자열과 상수 테이블 등 읽기 전용 데이터를 같은 출력 section에 넣습니다.
<code>} > KRAM</code>	모든 결과를 KRAM에 배치합니다.

*가 괄호 밖에 있으면 입력 파일 전체를 선택하는 패턴이고 `.text.*` 안의 *는 section 이름의 나머지 부분을 선택합니다. KEEP은 section garbage collection이 사용될 때도 해당 입력 section을 버리지 않도록 합니다. 부팅 코드는 일반 함수의 호출 관계에서 참조되지 않더라도 필요하므로 이 의도가 중요합니다.

현재 `_start`가 `.text.start`의 처음에 있으므로 이 순서가 주소 0의 부팅 시작을 만듭니다. 이미 첫 규칙에 소비된 입력 section을 뒤의 `.text.*` 규칙이 중복 복사하는 것은 아닙니다. 또한 출력 `.text`에는 `rodata`도 합쳐지므로 해당 section의 모든 byte를 CPU 명령어로 해석하면 문자열까지 가짜 명령처럼 보일 수 있습니다.

초기값을 담는 data

`.data : ALIGN(4) { *(.data .data.*) } > KRAM`은 쓰기 가능한 초기화 데이터를 모읍니다. 예를 들어 사용되고 있는 전역 `uint32_t counter = 123;`은 보통 이 종류에 들어갑니다. 초기값 123에 해당하는 byte는 이미지에 포함되어 실행 전 RAM에 준비됩니다. 정확한 section은 compiler의 최적화와 속성에 따라 달라질 수 있습니다.

현재 스크립트는 별도의 ROM 적재 주소를 지정하여 `.data`를 RAM으로 복사하는 구조를 기술하지 않습니다. 프로그램 RAM에 초기 이미지를 반영하는 프로젝트이므로 일반적인 플래시 기반 MCU의 data 복사 루틴을 그대로 가정해서는 안 됩니다. 특히 CPU reset만 수행할 때 boot.S는 `.bss`를 지우지만 `.data` 초기값을 재복사하는 코드는 없다는 차이도 있습니다.

공간을 예약하고 부팅 때 지우는 bss

코드	하는 일
<code>.bss (NOLOAD) : ALIGN(4) {</code>	실행 중 메모리가 필요한 <code>.bss</code> 를 정의하되 일반 초기화 payload로 적재하지 않을 구간임을 표시합니다.
<code>_bss_start = .;</code>	현재 위치를 <code>.bss</code> 시작 symbol 값으로 정의합니다.
<code>*(.bss .bss.*)</code>	0으로 초기화되는 정적 데이터의 입력 section들을 모읍니다.
<code>*(COMMON)</code>	입력 객체에 COMMON으로 남은 미초기화 전역 객체도 이곳에 배치합니다.
<code>. = ALIGN(4);</code>	끝 위치를 4바이트 경계로 올립니다.
<code>_bss_end = .;</code>	끝의 다음 주소를 symbol로 기록합니다.

COMMON은 예를 들어 tentative definition을 compiler가 common symbol로 내보낸 경우를 처리합니다. 컴파일 옵션과 객체 형식에 따라 사용 여부가 달라지므로 모든 미초기화 변수가 반드시 COMMON이라는 뜻은 아닙니다.

NOLOAD는 RAM 공간이 없거나 자동으로 0이 된다는 뜻이 아닙니다. 현재 boot.S가 linker symbol로 범위를 알아내어 실제 0을 기록합니다. 따라서 부팅 코드가 빠지면 C가 요구하는 정적 객체의 0 초기화가 보장되지 않을 수 있습니다. 스택 배열 stacks도 .bss에 있으며 코드 크기가 달라지면 그 실제 주소도 달라질 수 있습니다.

_bss_start와 _bss_end의 대입은 별도 uint32_t 변수를 만들어 주소를 RAM에 저장하는 것이 아닙니다. linker가 심볼에 주소 값을 부여하는 것입니다. boot.S의 la는 이 주소 값을 레지스터에 준비합니다. 다음은 현재 초기화 루프의 동작을 나타내는 발췌입니다.

```
la    t0, _bss_start
la    t1, _bss_end
1:    bgeu t0, t1, 2f
      sw    zero, 0(t0)
      addi t0, t0, 4
      j     1b
2:
```

처음 두 줄은 시작과 끝을 준비합니다. bgeu는 현재 주소가 끝에 도달했으면 종료합니다. sw는 현재 word를 0으로 지우고 addi는 다음 word로 이동합니다. 1b는 뒤쪽이 아니라 코드상 앞서 나온 숫자 label 1로 돌아가라는 뜻이며, 2f는 앞으로 나오는 label 2를 뜻합니다. 지우는 범위는 시작 포함·끝 제외인 [_bss_start, _bss_end) 입니다.

11.10 스택 심볼과 링크 검사 및 제거 section

stack_top은 용량 예약 명령이 아니다

_stack_top = 0x00008000; 은 부팅 sp에 넣을 값입니다. KRAM의 16 KiB 범위 밖이더라도 symbol 값만 정의하는 것이므로 그 위치에 출력 section을 배치했다는 뜻은 아닙니다. boot.S의 la sp, _stack_top이 실행되어야 실제 CPU의 sp가 변경됩니다.

32 KiB RAM의 마지막 byte는 0x7FFF이고 초기 sp는 끝 다음인 0x8000입니다. 다음은 주소 계산을 보여 주는 교육용 예입니다.

```
addi sp, sp, -16    # 0x8000 -> 0x7FF0
sw    ra, 12(sp)    # 0x7FFC부터 4바이트 저장
```

첫 저장 전에 sp를 낮추므로 RAM 안에 기록됩니다. 이것은 부팅·idle용 스택이며 .bss의 셸 태스크 스택과 다릅니다. 이 symbol 한 줄이 자동으로 8 KiB를 확보하거나 lower bound를 검사하는 것은 아닙니다.

ASSERT가 보장하는 범위

ASSERT(_bss_end <= 0x00004000, "shell kernel exceeds reserved lower 16 KiB")는 링크 중 .bss 끝이 커널 경계를 초과하면 오류를 보고합니다. 끝 주소는 포함되지 않으므로 정확히 0x4000이면 조건을 만족합니다. MEMORY 영역의 용량 제한과 함께 정적 배치 오류를 찾는 데 사용합니다.

이 assertion 하나가 모든 미래 변경까지 검증하는 것은 아닙니다. 예를 들어 .bss 뒤에 별도의 section을 추가하면 그 영역도 검토해야 합니다. 현재 스택의 동적 성장, 무한 재귀, 앱의 잘못된 store는 이 식으로 검사할 수 없습니다. 또한 설명한 입력 패턴에 들어오지 않는 새 section이 생기면 linker의 별도 배치 결과를 ELF에서 확인해야 합니다.

DISCARD와 메타데이터

/DISCARD/ : { *(.comment) *(.riscv.attributes) } 는 입력 객체의 해당 section들을 출력에서 제외합니다. .comment에는 보통 compiler 식별 문자열이 들어가고, .riscv.attributes에는 ISA와 ABI 관련 속성 정보가 들어갑니다. CPU가 직접 실행할 명령은 아니지만 분석 도구에는 유용할 수 있으므로 모든 프로젝트에서 삭제해야 한다는 규칙은 아닙니다. 이 설정이 실제 생성된 명령어를 다른 ISA로 바꾸는 것도 아닙니다.

전체 연결을 정리하면 compiler·assembler가 입력 section을 만들고, linker가 이 스크립트에 따라 주소를 정하며, 이미지 변환 도구가 프로그램 RAM용 데이터를 만듭니다. 리셋 후 boot.S가 sp와 .bss, mtvec를 준비하여 kernel_main으로 넘어갑니다. 주소를 정하는 도구, 실제 초기화를 수행하는 명령어, 저장 공간을 제공하는 RTL을 구별해야 문제의 원인을 올바른 계층에서 찾을 수 있습니다.

기존 빌드 결과는 다음 읽기 전용 명령으로 검사할 수 있습니다.

```
riscv64-unknown-elf-readelf -h build/firmware/mini_shell.elf
riscv64-unknown-elf-readelf -SW build/firmware/mini_shell.elf
riscv64-unknown-elf-nm -n build/firmware/mini_shell.elf
```

첫 명령은 ELF 진입점, 둘째는 section 주소·크기·형식, 셋째는 _start·stacks·_bss_start·_bss_end·_stack_top 같은 심볼 값을 확인하는 데 사용합니다. 이것은 디스크의 빌드 결과 검사이며 보드에 같은 이미지가 적재되어 있다는 증명은 아닙니다.

12. UART 업로드 프로토콜과 APP1 형식

12.1 host와 board의 대화

업로드 스크립트는 처음 CR을 보내 현재 셸의 새 프롬프트를 받습니다. 이어서 파일 이름, payload 바이트 수, checksum을 command line으로 보냅니다.

```
host → board : CR
board → host : rv>
host → board : upload hello.app 60 000010dd + CR
board → host : send hex:
host → board : b747000093874702... (총 120개의 hex 문자)
board → host : uploaded
board → host : rv>
```

payload의 byte 수는 10진수입니다. checksum은 8자리 16진수이며 `sum(payload) & 0xFFFFFFFF`로 계산합니다. 명령 도움말의 `sum8`은 8자리 hex 표현을 가리키며 8비트 checksum을 뜻하지 않습니다.

처음 보내는 CR은 host와 board의 대화 경계를 맞추기 위한 동작입니다. 그 뒤의 `rv>`를 기다리면 셸이 command line을 받을 준비가 됐음을 확인할 수 있습니다. 하지만 이전에 중단된 upload가 payload를 기다리는 상태라면 CR은 공백으로 무시될 뿐 prompt를 만들지 못할 수 있습니다. 현재 protocol에는 상태를 강제로 되돌리는 독립 escape sequence가 없습니다.

host가 `send hex:`를 확인한 뒤 payload를 보내는 이유는 command parser와 payload receiver의 전환을 기다리기 위해서입니다. 데이터 전송이 끝난 뒤에도 `uploaded`와 `prompt`를 확인해야 실제 file write까지 끝났다고 판단할 수 있습니다. **host의 write 호출이 성공했다는 사실만으로 보드의 저장 완료가 보장되지는 않습니다.**

코드 해부 12-A. host uploader의 요청·대기·전송 순서

출처: [scripts/upload_app.py](#), 원본 70 - 86행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
70 | write_all(b"\r")
71 | until(b"rv> ")
72 | write_all(f"upload {args.name} {len(payload)} {checksum:08x}\r".encode("ascii"))
73 | until(b"send hex:\r\n")
74 | hex_bytes = payload.hex().encode("ascii")
75 | for pos in range(0, len(hex_bytes), 256):
76 |     write_all(hex_bytes[pos:pos + 256])
77 | reply = until(b"rv> ", seconds=60)
78 | if b"uploaded\r\n" not in reply:
79 |     raise RuntimeError(f"Upload failed: {reply!r}")
81 | app_reply = b""
82 | if args.run:
83 |     write_all(f"run {args.name}\r".encode("ascii"))
84 |     app_reply = until(b"rv> ", seconds=30)
85 |     if b"running\r\n" not in app_reply or b"returned\r\n" not in app_reply:
86 |         raise RuntimeError(f"App did not return: {app_reply!r}")
```

원본 행	한 줄씩 읽는 동작과 의미
70	현재 셸에 CR을 보내 빈 명령행 또는 새 prompt를 유도합니다.
71	prompt byte열을 수신할 때까지 기다립니다. FPGA가 명령을 받을 상태인지 확인하는 protocol 단계입니다.
72	파일 이름·10진수 길이·8자리 hex checksum을 하나의 ASCII 명령행으로 보냅니다.

원본 행	한 줄씩 읽는 동작과 의미
73	셸이 payload 수신 상태로 바뀌었다는 응답을 기다립니다.
74	BIN의 byte 순서를 그대로 유지하면서 byte당 두 개의 ASCII hex 문자로 변환합니다.
75	전송 문자열을 최대 256문자 조각으로 순회합니다. 조각별 board ACK가 생기는 것은 아닙니다.
76	현재 조각을 host serial 장치에 씁니다.
77	수신·검사·파일 저장 뒤 새 prompt를 기다립니다.
78	prompt만 왔다고 성공으로 판단하지 않고 uploaded 메시지도 확인합니다.
79	저장 완료 메시지가 없으면 host 오류로 보고합니다.
81	실행하지 않는 경우 결과는 빈 byte열로 둡니다.
82	--run이 지정된 경우만 실행 명령을 이어 보냅니다.
83	같은 파일 이름을 run 명령으로 요청합니다.
84	앱 반환 후 prompt를 기다립니다. 이 timeout이 FPGA 앱을 강제 종료하는 기능은 아닙니다.
85	호출 전·후 marker가 모두 있는지 검사합니다. 앱의 모든 계산 결과를 검증하는 조건은 아닙니다.
86	정상 복귀 증거가 없으면 오류를 보고합니다.

write_all 은 host의 부분 write를 반복 처리하고 until 은 수신 buffer에서 marker를 찾는 내부 함수입니다. OS write 성공과 MiniFS 저장 성공, 앱 반환 성공은 이렇게 서로 다른 응답으로 확인합니다.

12.2 왜 raw byte 대신 ASCII hex를 보내는가

byte 0xB7 은 문자 'b', '7' 두 개로 보냅니다. 보드는 첫 hex digit을 상위 4비트, 두 번째를 하위 4비트로 조합합니다. 공백, CR, LF, tab은 수신 parser가 건너뜁니다.

ASCII hex는 사람이 관찰하기 쉽고 작은 parser로 구현할 수 있지만 전송량이 두 배입니다. 115200 baud, 8-N-1의 이론적 byte rate는 약 11520바이트/초이므로 payload의 이상적인 전송률은 약 5760바이트/초입니다. 60바이트 payload는 120개의 UART 문자가 되어 약 10.4 ms가 필요하며, 명령·응답·OS 처리 시간은 별도입니다.

hex 수신기는 대문자 A - F도 허용하므로 b7 과 B7 은 같은 byte가 됩니다. 각 byte를 만들 때 상위 nibble을 읽은 뒤 하위 nibble을 기다리며, 그 사이의 허용 공백도 건너뜁니다. 따라서 줄을 나누어 보내는 것이 가능하지만, 빠진 hex digit 하나는 이후 byte 경계를 모두 어긋나게 할 수 있습니다.

현재 host는 hex stream을 최대 256문자 단위로 write하지만 이 단위가 board의 ACK packet은 아닙니다. 각 조각마다 확인을 받고 보내는 flow control이 없으므로 RX FIFO와 CPU 소비 속도에 의존합니다. 작은 예제의 성공을 더 큰 데이터 전송의 무조건적인 신뢰성으로 일반화하지 않아야 합니다.

12.3 두 가지 hex 표현은 바이트 순서가 다르다

첫 명령어 0x000047B7 을 예로 들어 봅니다.

표현	내용
.bin 의 바이트 순서	B7 47 00 00
UART ASCII hex stream	b7470000

표현	내용
\$readmemh 32비트 word 한 줄	000047b7

UART는 raw byte의 순서를 유지합니다. bin2hex.py는 4바이트를 little-endian 정수로 해석해서 한 word를 출력합니다. 따라서 .hex 텍스트를 그대로 upload payload로 붙여 넣으면 같은 데이터가 되지 않습니다. **호스트 스크립트에는 .bin을 전달해야 합니다.**

little-endian은 byte의 memory 순서를 말하고, hex 문자열에서 한 byte의 상위 digit을 먼저 쓰는 관례와는 다른 총위입니다. 0xB7 한 byte를 표현할 때는 항상 b7로 쓰지만, 32비트 word의 네 byte 순서는 낮은 byte부터 전송합니다. “little-endian이므로 문자열의 모든 글자를 뒤집는다”는 규칙은 없습니다.

bin2hex.py는 마지막 byte 수가 4의 배수가 아니면 0을 덧붙여 word를 완성합니다. 이 padding은 \$readmemh 표현을 위한 것이고 host uploader는 원래 .bin 길이를 사용합니다. 현재 60바이트 예제는 이미 4의 배수라 추가 padding이 없습니다.

12.4 셸의 업로드 임시 버퍼

shell_upload()는 앱 실행 창을 임시 버퍼로도 씁니다.

```
업로드 중의 unified RAM
0x4000 .. 0x400B : APP1 header를 만들 자리
0x400C ..      : UART에서 복원한 payload
```

payload를 전부 받은 뒤 byte sum을 검사하고, header를 작성합니다. 그 다음 SYS_FS_CREATE와 SYS_FS_WRITE로 header+payload 전체를 MiniFS에 저장합니다. 이 시점에는 파일이 저장된 것이며, 0x4000에는 아직 실행할 첫 명령어가 아니라 header가 있습니다.

재실행 시에는 shell_run()이 파일의 offset 12부터 payload만 읽어 0x4000에 배치합니다. 이 차이를 이해하지 못하면 header를 명령어로 실행하거나 앱을 12바이트 어긋난 주소로 실행하는 loader를 만들기 쉽습니다.

임시 buffer가 실행 창을 공유할 수 있는 이유는 업로드를 받는 동안 이전 앱이 실행 중이지 않기 때문입니다. 현재는 셸과 앱이 동기적인 함수 호출 관계이므로 앱이 반환한 후 다음 명령을 받습니다. 비동기 앱 실행으로 확장한다면 실행 중인 image를 upload가 덮어쓰지 않도록 별도의 buffer나 lifecycle 보호가 필요합니다.

60바이트를 받을 때 임시 payload는 0x400C - 0x4047에 놓고 header는 0x4000 - 0x400B입니다. 저장되는 전체 72바이트를 MiniFS에 복사한 뒤 run은 payload를 0x4000 - 0x403B로 다시 로딩합니다. 두 시점의 RAM 지도에서 시작 위치가 12바이트 다른 것을 직접 계산해 보십시오.

코드 해부 12-B. shell_upload의 buffer 구성·checksum·파일 저장

출처: [firmware/kernel.c](https://github.com/0x08-0x04/firmware/blob/master/kernel.c), 원본 399 - 420행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```

399 static void shell_upload(const char *name, uint32_t size, uint32_t checksum)
400 {
401     uint32_t *header = (uint32_t *)APP_BASE;
402     uint8_t *payload = (uint8_t *) (APP_BASE + APP_HEADER_BYTES);
403     uint32_t sum = 0, i;
404     int byte;
405     sys_puts("send hex:\n");
406     for (i = 0; i < size; i++) {
407         byte = receive_hex_byte();
408         if (byte < 0) { sys_puts("bad hex\n"); return; }
409         payload[i] = (uint8_t)byte;
410         sum += (uint32_t)byte;
411     }
412     if (sum != checksum) { sys_puts("checksum mismatch\n"); return; }
413     header[0] = APP_MAGIC;
414     header[1] = size;
415     header[2] = checksum;
416     sys_fs_call(SYS_FS_CREATE, (uint32_t)name, 0, 0);
417     byte = sys_fs_call(SYS_FS_WRITE, (uint32_t)name, APP_BASE,
418                       APP_HEADER_BYTES + size);
419     sys_puts(byte == (int)(APP_HEADER_BYTES + size) ? "uploaded\n" : "upload failed\n");
420 }

```

원본 행	한 줄씩 읽는 동작과 의미
399	parser가 검사한 이름·크기·checksum을 받아 파일을 만드는 함수입니다.
400	앞에서 선언한 함수·블록의 본문을 시작합니다.
401	실행 창 시작 0x4000을 header word 세 개를 쓸 pointer로 봅니다.
402	payload는 header 뒤 12바이트인 0x400C부터 받습니다. 아직 앱 실행 layout이 아닙니다.
403	누산 합과 byte index를 준비합니다.
404	수신 byte 또는 오류 -1을 받을 int입니다.
405	host에게 hex를 받을 준비가 되었음을 알립니다.
406	선언한 byte 수만큼 반복합니다.
407	ASCII hex 두 자리를 byte 하나로 복원하는 helper를 호출합니다.
408	유효한 hex가 아니면 오류를 출력하고 파일 기록 전에 돌아갑니다.
409	복원한 byte를 임시 payload buffer에 씁니다.
410	unsigned byte 값을 checksum에 더합니다.
411	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
412	전체 합이 다르면 MiniFS에 새 내용을 기록하지 않고 거절합니다.
413	header 첫 word에 APP1 magic을 기록합니다.
414	다음 word에 payload byte 수를 기록합니다.
415	세 번째 word에 검사한 합을 기록합니다.
416	파일 entry 생성을 시도합니다. 기존 이름이면 뒤의 write로 교체할 수 있습니다.
417	header 시작 주소부터 FS_WRITE로 저장합니다.

원본 행	한 줄씩 읽는 동작과 의미
418	저장할 크기는 payload만이 아니라 12+size입니다.
419	실제 기록 수가 전체 파일 크기와 같은지 확인하여 완료 또는 실패 메시지를 출력합니다.
420	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

업로드 직후 0x4000에는 APP1 header가 있습니다. 이 상태를 함수로 호출하면 header를 instruction으로 해석하게 됩니다. run 단계가 파일 offset 12부터 다시 복사하는 이유입니다.

12.5 APP1 header의 바이트 단위 해석

파일 offset	길이	값	의미
0	4	0x31505041	little-endian 바이트로 41 50 50 31, 즉 APP1
4	4	payload 크기	header를 제외한 바이트 수
8	4	byte sum	unsigned payload 바이트 합의 하위 32비트
12	가변	payload	고정 주소 0x4000 에 링크된 기계어·데이터

현재 60바이트 예제의 header와 첫 명령어는 다음과 같습니다.

```
41 50 50 31   3C 00 00 00   DD 10 00 00   B7 47 00 00 ...
└ APP1 ─┘    └ 60 ─┘    └ 0x10DD ─┘    └ first insn ┘
```

checksum은 전송 실수 일부를 찾는 용도입니다. byte 두 개를 서로 바꾸거나 합이 같은 값으로 바꾸면 검출하지 못할 수 있습니다. 암호학적 서명, 작성자 확인, 명령어 검증을 제공하지 않습니다.

checksum은 unsigned byte를 하나씩 더합니다. 예를 들어 0xFF는 -1이 아니라 255로 더해야 host의 `sum(payload)`와 같습니다. 합의 누산형이 `uint32_t`인 것은 파일 형식 규약을 분명히 하기 위한 것이며, 현재 최대 payload 6644바이트의 최대 합은 16942200이므로 실제 허용 크기 안에서는 32비트를 넘지 않습니다.

magic의 정수 값과 문자 순서도 직접 검산할 수 있습니다. 낮은 byte부터 41 50 50 31이므로 ASCII로 A P P 1이 됩니다. 이 12바이트 구조에는 alignment padding이 따로 끼지 않으며 세 `uint32_t`를 연속해서 저장합니다. 다른 host tool로 APP1을 만들 때는 host native endian 대신 little-endian을 명시해야 합니다.

12.6 업로드 실패와 현재 프로토콜의 범위

hex digit이 아니면 bad hex, 합이 다르면 checksum mismatch, 파일 쓰기가 실패하면 upload failed가 출력됩니다. host 스크립트에는 응답 timeout이 있지만 보드의 `receive_hex_byte()`에는 수신 timeout이나 취소 명령이 없습니다. payload 전송이 중단되면 셸이 나머지 바이트를 계속 기다릴 수 있습니다.

또한 host는 screen 등 다른 reader의 존재를 자동으로 확실히 차단하지 않습니다. 둘이 동시에 같은 serial port를 읽으면 각자 일부 응답을 가져갈 수 있습니다. 실습에서는 한 번에 한 프로그램이 UART를 사용하도록 합니다.

전송 오류가 날 때는 host 쪽 serial timeout과 board가 출력한 오류를 구분합니다. checksum mismatch는 보드가 정해진 수의 byte를 받아 합까지 계산했다는 증거입니다. timeout은 그 이전의 어느 단계에서도 생길 수 있으며 baud·연결·포트 중복·board state를 좁혀야 합니다.

파일 교체는 수신 checksum이 맞은 다음 진행되므로 합이 틀린 payload를 즉시 기존 file data에 쓰지는 않습니다. 하지만 실패 후 자동 재시도, 불완전한 입력 drain, 임시 파일을 사용한 commit은 구현하지 않았습니다. 신뢰할 수 있는 대량 전송을 설계할 때는 현재의 단순 성공 경로에 이러한 상태 처리를 더해야 합니다.

13. run 명령의 검증·복사·호출

13.1 실제 동작 순서

run hello.app 을 처리하는 shell_run() 은 다음 순서로 동작합니다.

1. 파일의 처음 12바이트를 읽고 APP1 magic을 검사합니다.
2. payload 크기가 4 이상이며 6644와 실행 창 크기 이내인지 검사합니다.
3. 파일 offset 12부터 64바이트 이하씩 읽어 APP_BASE+offset 에 복사합니다.
4. 복사한 payload의 byte sum을 계산합니다.
5. 기록된 합과 같은지, 선언된 payload 뒤에 불필요한 byte가 없는지 검사합니다.
6. running 을 출력하고 0x4000 을 함수처럼 호출합니다.
7. 앱이 반환하면 returned 를 출력합니다.
8. 명령 처리로 돌아간 셸이 rv> 를 출력합니다.

현재 SYS_EXEC 라는 별도 시스템 콜은 없습니다. run 로더 자체는 셸 C 코드에 있으며 파일 읽기에 FS syscall을 사용하고, 복사가 끝나면 직접 함수 포인터를 호출합니다. 장래의 OS 설계에서 process 생성 기능을 kernel syscall로 옮길 수 있지만, 그것은 다음 단계의 변경입니다.

검사의 순서도 의미가 있습니다. 크기를 먼저 검사해야 잘못된 size로 kernel 영역이나 stack에 복사하지 않습니다. 복사된 바이트의 합을 확인한 뒤 호출해야 전송·저장 중 일부 손상을 실행 전에 찾을 수 있습니다. 다만 형식 검증은 명령어의 동작 분석이 아니므로 문법적으로 APP1인 위험한 code를 막는 보호 장치로 해석하면 안 됩니다.

loader가 중간에 실패하면 running 은 출력하지 않고 함수 호출도 하지 않습니다. 이미 일부 payload가 실행 창에 복사되었더라도 다음 정상 run은 file에서 다시 덮어씁니다. loader의 실패 처리와 실행 창의 잔여 내용은 구분해야 하며, 현재는 실패 때 전체 창을 0으로 지우지는 않습니다.

코드 해부 13-A. run 전반부: header와 크기를 먼저 검증하기

출처: [firmware/kernel.c](#), 원본 422 - 435행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
422 | static void shell_run(const char *name)
423 | {
424 |     uint32_t header[3], size, sum = 0, off, chunk, i;
425 |     uint8_t extra;
426 |     uint8_t *app = (uint8_t *)APP_BASE;
427 |     int got;
428 |     if (sys_fs_read_at(name, header, sizeof(header), 0) != (int)sizeof(header) ||
429 |         header[0] != APP_MAGIC) {
430 |         sys_puts("not an app file\n"); return;
431 |     }
432 |     size = header[1];
433 |     if (size < 4u || size > APP_MAX_BYTES || size > APP_WINDOW_BYTES) {
434 |         sys_puts("invalid app size\n"); return;
435 |     }
```

원본 행	한 줄씩 읽는 동작과 의미
422	run built-in이 파일 이름을 넘겨 호출하는 loader 함수입니다. SYS_EXEC handler가 아닙니다.
423	앞에서 선언한 함수·블록의 본문을 시작합니다.

원본 행	한 줄씩 읽는 동작과 의미
424	header 세 word와 크기·합·복사 진행 상태를 C stack에 준비합니다. 파일 전체를 stack에 올리지는 않습니다.
425	파일 끝 검사용 byte 하나를 준비합니다.
426	payload를 놓을 고정 실행 주소를 byte pointer로 해석합니다.
427	FS 읽기의 실제 반환 크기 또는 오류를 받을 signed 변수입니다.
428	파일 offset 0에서 정확히 12바이트를 읽으려 합니다. 반환 수가 다르면 OR의 뒤 조건을 평가하지 않고 실패 분기로 갑니다.
429	header를 다 읽은 경우 APP1 magic을 검사합니다. 초기화되지 않은 header 값을 먼저 검사하지 않도록 순서가 중요합니다.
430	파일 없음·짧은 header·magic 불일치가 이 메시지로 모입니다. 이 분기에서는 앱을 호출하지 않습니다.
431	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
432	header의 payload 길이를 읽습니다. 이 시점에는 header가 정한 숫자이지 아직 검증된 실제 파일 길이가 아닙니다.
433	최소 instruction 4바이트, FS payload 한계, 실행 RAM 창 한계를 각각 검사합니다.
434	범위를 벗어나면 복사 전에 거절합니다. 이 검사는 loader 자신의 RAM 복사 범위를 보호합니다.
435	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

앱이 실행 중 어떤 주소에 store할지는 이 검사로 제한하지 못합니다. loader의 입력 범위 검증과 U-mode/PMP 같은 실행 중 격리는 서로 다른 기능입니다.

13.2 파일 offset과 목적지 주소

MiniFS 파일	unified 실행 RAM
offset 0: APP1 header	loader의 지역 header[3]
offset 12: payload[0]	0x4000
offset 13: payload[1]	0x4001
...	...
offset 12+N-1	0x4000+N-1

header는 실행 RAM 이미지의 앞에 남겨두지 않습니다. $N=60$ 이면 payload 마지막 byte는 0x403B 입니다. 파일 시스템의 block 위치와 실행 위치 사이에는 “내용을 복사한다”는 관계만 있습니다.

copy loop에서 `off` 는 payload 기준 offset, FS 읽기의 offset은 $12+off$ 입니다. 두 변수를 혼동하면 첫 12바이트를 건너뛰지 않거나, 반대로 payload 중간부터 읽는 문제가 생깁니다. 첫 iteration은 `off=0` , source file offset 12, destination 0x4000, chunk는 `min(size, 64)` 입니다.

payload가 150바이트라면 chunk는 64, 64, 22바이트이고 destination은 각각 0x4000, 0x4040, 0x4080입니다. 마지막 byte 주소는 $0x4000+149=0x4095$ 입니다. 64바이트 chunk는 stack에 큰 파일 buffer를 만들지 않고도 파일 전체를 처리하게 해 줍니다.

13.3 왜 마지막 byte를 한 번 더 읽는가

loader는 offset $12+size$ 에서 1바이트 읽기를 시도하고 EOF인 0을 기대합니다. 파일이 선언된 길이보다 짧으면 복사 중 truncated app, 길이보다 길거나 합이 틀리면 bad app checksum 으로 거절합니다. header와 실제 file size의 일관성을 검사하는 것입니다.

다만 파일이 CPU에 맞는 모든 명령으로 구성되었는지, 잘못된 주소에 store하지 않는지, 정상적으로 반환하는지까지 검사하지는 않습니다. APP1 header와 byte sum만으로 실행의 안전성이 보장되지 않습니다.

허용되는 file size는 정확히 `12+header.size` 입니다. header가 60을 선언했는데 file size가 71이면 마지막 read가 부족해 truncated, 73이면 EOF 확인에서 추가 byte가 발견됩니다. 선언된 60바이트는 맞아도 한 byte 값이 달라 합이 틀리면 checksum 오류입니다. 길이 검사와 내용 검사는 서로 보완합니다.

현재 최소 크기는 4바이트지만 payload 전체 크기가 반드시 4의 배수인지는 검사하지 않습니다. code 이후 문자열·데이터 길이는 임의 byte 수일 수 있기 때문입니다. entry에 최소 한 instruction이 있다는 크기 조건과 모든 code target이 올바르게 정렬되었다는 조건은 다릅니다.

13.4 함수 포인터 호출의 의미

```
((void (*)(void))APP_BASE)();
```

정수 주소 `0x4000` 을 “인수가 없고 반환값도 없는 함수의 주소”로 해석한 뒤 호출합니다. 이 코드는 해당 GCC·bare-metal 실행 환경에서 사용하는 구현 방식입니다. 임의의 C 환경에서 숫자 주소를 함수처럼 호출해도 된다는 뜻은 아닙니다.

현재 확인한 셸 ELF에서는 다음 명령어가 나왔습니다.

```
0x0A14: lui    a5, 0x4
0x0A18: jalr   ra, 0(a5)
0x0A1C: ...           # 앱에서 돌아온 뒤 실행
```

JALR 는 `ra=0x0A1C` 를 기록하고 PC를 `0x4000` 으로 변경합니다. core는 같은 명령어를 일반 함수 호출에도 사용합니다. 파일 실행을 위한 별도의 “EXEC instruction”은 필요하지 않습니다. 이 주소들은 현재 빌드 관측값이며 코드 수정·compiler 최적화에 따라 달라질 수 있습니다.

CPU에게 실행 파일의 개념을 추가하지 않아도 되는 이유를 이 한 줄에서 볼 수 있습니다. loader는 자료를 RAM에 쓰는 C 프로그램이고, 함수 포인터 호출은 기존 JALR로 구현됩니다. CPU는 loader가 어떤 이름의 파일을 읽었는지 몰라도 address `0x4000`의 bit pattern을 정상 instruction으로 decode합니다.

현재 함수형이 void이므로 앱의 a0에 남은 값을 exit status로 출력하지 않습니다. 반환 코드를 도입하려면 entry를 `int app_main(void)` 같은 계약으로 정하고 셸이 결과를 수집하도록 함께 바뀌어야 합니다. 표준 C의 main exit code 규칙이 이 함수 포인터 호출에 자동으로 생기지는 않습니다.

코드 해부 13-B. run 후반부: byte 복사에서 JALR 호출까지

출처: [firmware/kernel.c](#), 원본 436 - 450행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
436 |     for (off = 0; off < size; off += chunk) {
437 |         chunk = size - off;
438 |         if (chunk > 64u) chunk = 64u;
439 |         got = sys_fs_read_at(name, app + off, chunk, APP_HEADER_BYTES + off);
440 |         if (got != (int)chunk) { sys_puts("truncated app\n"); return; }
441 |         for (i = 0; i < chunk; i++) sum += app[off + i];
442 |     }
443 |     if (sum != header[2] ||
444 |         sys_fs_read_at(name, &extra, 1u, APP_HEADER_BYTES + size) != 0) {
445 |         sys_puts("bad app checksum\n"); return;
446 |     }
447 |     sys_puts("running\n");
448 |     ((void (*)(void))APP_BASE)();
449 |     sys_puts("returned\n");
450 | }
```

원본 행	한 줄씩 읽는 동작과 의미
436	payload offset 0부터 반복합니다. 다음 반복은 실제 선택한 chunk만큼 진행합니다.
437	아직 복사하지 않은 byte 수를 계산합니다.
438	한 번의 읽기를 최대 64바이트로 제한합니다.
439	파일의 12+off에서 실행 RAM의 0x4000+off로 읽습니다. source의 header만 건너뛰고 destination에는 header 공간을 만들지 않습니다.
440	요구한 chunk보다 적게 읽으면 선언 길이와 실제 파일이 다르므로 거절합니다.
441	실행 RAM에 복사된 byte를 합산합니다. 원래 host 파일이 아니라 실제 복사 결과를 확인합니다.
442	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
443	누산 합과 header의 기록을 비교합니다.
444	선언 payload 다음 위치에서 1바이트 읽어 EOF 0인지 확인합니다. 추가 byte도 허용하지 않습니다.
445	합 또는 끝 조건이 틀리면 실행하지 않고 돌아갑니다.
446	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
447	이전 검사를 통과한 뒤 running을 출력합니다. 아직 앱 정상 종료를 뜻하지 않습니다.
448	정수 주소를 void(void) 함수 pointer로 해석해 호출합니다. 현재 compiler는 일반 JALR 호출을 생성하며 셀의 SP와 ra 계약을 사용합니다.
449	앱 함수가 정상 반환했을 때만 도달합니다. MRET 뒤마다 실행되는 출력이 아닙니다.
450	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

size=1500이면 읽기 크기는 64·64·22, destination은 0x4000·0x4040·0x4080입니다. 마지막 ret가 셀의 호출 다음 위치로 돌아와야 returned가 출력됩니다.

13.5 반환 경로가 살아 있어야 하는 이유

앱이 ra를 훼손하거나 SP를 복구하지 않으면 정상 반환이 실패할 수 있습니다. 다른 함수를 호출하는 앱은 compiler가 필요한 prologue/epilogue와 ra 저장을 생성합니다. 수동 assembly로 앱을 작성할 때는 같은 규칙을 직접 지켜야

합니다.

현재 예제는 출력 stub이 inline되어 leaf 함수 형태이고, `ecall` 은 하드웨어에서 `ra` 를 바꾸지 않습니다. trap 내부 C 호출이 `ra` 를 사용해도 trap frame에서 원래 값이 복원됩니다. 따라서 예제의 마지막 `ret` 는 셸이 넣어 준 복귀 주소를 사용할 수 있습니다.

앱 내부에서 다른 함수를 JAL로 호출하면 `ra` 는 그 내부 복귀 주소로 바뀝니다. compiler는 앱이 끝난 뒤 셸로 돌아갈 주소가 필요하므로 stack 등에 기존 `ra` 를 보존하고 epilogue에서 복원합니다. 이를 수동 assembly로 작성한다면 save/restore를 빠뜨리지 않아야 합니다.

현재 예제의 `ecall` 은 `ra` 를 destination으로 지정하지 않지만 trap.S의 `call trap_handler` 는 `ra` 를 씁니다. 원래 앱의 `ra` 는 그 전에 frame offset 4에 저장되어 있으므로 syscall 뒤에도 셸 복귀 주소가 유지됩니다. 함수 호출 규약과 trap 저장 규약이 만나는 구체적인 사례입니다.

13.6 잘못된 앱 파일에 대한 검증 경로

아래 표는 현재 `shell_run()` 의 조건식을 기준으로 예상한 동작입니다. 모든 사례가 이미 자동 test에 포함되어 있다는 뜻은 아닙니다. 후속 negative test의 기대값으로 사용할 수 있습니다.

파일 상태	실패하는 조건	출력	앱 진입
이름이 없음	첫 12바이트 read 결과가 -1	not an app file	없음
file size가 8	첫 read 결과가 12보다 작음	not an app file	없음
첫 12바이트는 있으나 magic이 다름	header[0] 비교	not an app file	없음
magic은 맞고 payload size=3	최소 크기 검사	invalid app size	없음
size=60인데 payload가 59바이트	chunk read 크기 비교	truncated app	없음
size=60이고 합이 다름	누산 sum 비교	bad app checksum	없음
size=60, 합은 맞지만 trailing byte 존재	EOF read가 0이 아님	bad app checksum	없음
형식은 맞지만 unsupported instruction 포함	loader 검사는 통과 가능	실행 중 PANIC 가능	있음

마지막 행이 특별히 중요합니다. loader는 code의 의미를 증명하는 static analyzer가 아닙니다. 파일 형식이 일관되어도 앱이 잘못된 주소에 쓰거나 무한 loop에 들어갈 수 있습니다. 실행 전에 알 수 있는 구조적 조건과 실행 중에만 드러나는 동작을 나누어 검증해야 합니다.

부정 입력 test에서는 오류 문자열에 더해 kernel image와 file table이 의도치 않게 바뀌지 않았는지, PC가 앱 창으로 넘어가지 않았는지를 관측할 수 있습니다. 복사 도중 실패하면 실행 창 일부는 바뀔 수 있으므로 “모든 RAM이 불변”이라는 잘못된 기대 조건을 세우지 않아야 합니다.

14. 60바이트 앱의 실제 명령어 추적

14.1 명령어와 문자열의 분리

현재 산출물은 총 60바이트이며 앞 36바이트가 9개의 32비트 명령어, 뒤 24바이트가 문자열과 종료 NUL입니다.

주소	기계어 word	명령어	동작
0x4000	000047B7	lui a5,0x4	a5=0x4000
0x4004	02478793	addi a5,a5,36	문자열 주소 0x4024 구성
0x4008	04800513	addi a0,zero,72	첫 문자 H=0x48
0x400C	00178793	addi a5,a5,1	다음 문자 주소로 이동
0x4010	00100893	addi a7,zero,1	SYS_PUTC 선택
0x4014	00000073	ecall	문자 출력 요청
0x4018	0007C503	lbu a0,0(a5)	다음 문자 읽기
0x401C	FE0518E3	bne a0,zero,0x400C	NUL이 아니면 반복
0x4020	00008067	jalr zero,0(ra)	셀로 복귀, ret 의 실제 명령
0x4024 - 0x403B	문자열 byte	Hello from loaded app!\n\0	실행할 명령이 아닌 상수 데이터

compiler는 첫 문자가 상수 H임을 알고 첫 load 대신 immediate를 사용했습니다. 소스의 while 문과 pointer 증가가 기계어에서는 이런 배치가 된 것입니다. C의 한 줄과 한 명령어를 일대일로 대응시키면 이 최적화를 이해하기 어렵습니다.

첫 instruction 0x000047B7의 memory byte는 B7 47 00 00이며 fetch port가 이를 32비트 word로 decoder에 공급합니다. rd=15이므로 a5를 쓰고, U immediate 0x4000을 그대로 전달합니다. 다음 ADDI가 36을 더해 문자열 주소를 완성합니다. raw file, memory word, assembly operand의 세 표현을 같은 값으로 연결해 볼 수 있습니다.

이 표에서 9개는 static instruction 수입니다. 반복문의 0x400C - 0x401C 구간은 문자열의 각 문자마다 여러 번 수행됩니다. syscall handler와 trap.S 명령들은 다른 주소에 있으며 표의 9개에 포함되지 않습니다. 앱 파일이 짧은 사실이 전체 OS 작업량이 9클록이라는 뜻은 아닙니다.

14.2 첫 문자 H의 전체 이동

PC=0x4008에서 a0=72가 됩니다. 0x4010에서 a7=1, 0x4014에서 ECALL 예외가 발생합니다. trap frame의 a0 슬롯에는 0x48, a7 슬롯에는 1, mepc에는 0x4014가 저장됩니다.

C handler는 mcause=11을 확인하고 frame의 mepc를 0x4018로 바꿉니다. SYS_PUTC 분기에서 uart_putc('H')를 호출하고 TX ready를 기다린 뒤 0x10000000에 0x48을 씁니다. SoC는 TX data/valid를 만들고 uart_tx가 이를 start/data/stop bit로 직렬화합니다. PC의 터미널이 그 바이트를 문자 H로 표시합니다.

handler가 같은 frame을 반환하면 trap.S가 레지스터와 mepc를 복원하고 mret합니다. PC는 0x4018이 되고 다음 문자 e를 읽습니다. 이 시점에는 아직 앱에서 셀로 돌아온 것이 아닙니다. OS 서비스에서 앱으로 돌아온 것입니다.

첫 문자의 pointer는 ECALL 전에 이미 0x4025가 되어 있습니다. trap frame에 a5가 보존되므로 돌아온 뒤 LBU a0,0(a5)는 H 다음의 e를 읽습니다. 만약 handler가 a5를 복원하지 않으면 UART 출력 함수가 사용한 임시 값으로

pointer가 바뀌어 문자열 순회가 망가질 수 있습니다. 작은 예제가 register 보존 규칙도 시험하는 이유입니다.

MRET 이후의 앱 PC는 0x4018 이고 ra는 여전히 셸 복귀 주소입니다. 즉 복귀 방향을 결정하는 값이 두 종류 동시에 살아 있습니다. 파형에서는 CSR mepc와 register x1을 함께 보면 syscall 복귀와 최종 app return을 구분할 수 있습니다.

14.3 마지막 문자와 복귀

앱이 보내는 payload 문자열은 23문자입니다. 마지막 newline을 포함하며 끝의 NUL은 보내지 않습니다. newline syscall은 셸 출력 경로에서 CR과 LF 두 byte를 전송하므로 앱 메시지의 실제 UART 바이트 수는 24입니다. `running`, `returned`, 명령 `echo`와 `prompt`는 이 수에 포함하지 않습니다.

마지막 newline 이후 `lbu`가 NUL을 읽으면 `bne`가 성립하지 않습니다. 다음 PC는 0x4020 이고 `jalr zero,0(ra)`가 셸의 복귀 주소로 이동합니다. `rd=zero`이므로 새 반환 주소를 보존할 필요가 없습니다.

문자열 출력만 놓고 동적 앱 instruction 수를 계산해 볼 수 있습니다. 초기 설정 3개, 문자 23개마다 loop의 5개, 마지막 `ret` 1개이므로 $3 + 23 \times 5 + 1 = 119$ 개의 앱 instruction 시도가 정상 흐름에 나타납니다. 이 계산에는 `trap.S` handler, timer 중단에 따른 재시도와 UART polling 명령이 포함되지 않습니다. ECALL은 retire되지 않는 예외 명령이므로 이를 retirement counter 값과 동일시하지 않아야 합니다.

정상 반환 직후 셸은 `returned`를 출력하지만 OS가 앱의 모든 동작을 사후 검사한 것은 아닙니다. 앱이 형식과 ABI를 지켜 return한 사실을 관찰한 것입니다. 향후 앱별 결과 값을 검증하려면 지정된 출력·파일·메모리 signature 등을 추가로 검사해야 합니다.

14.4 역어셈블에서 보이는 가짜 명령어

앱 linker는 `.rodata`를 `.text` 출력 section에 합칩니다. 따라서 `objdump -d`가 문자열 영역까지 명령어로 해석하여 `c.flw`, `c.fld` 같은 뜻밖의 mnemonic을 보여 줄 수 있습니다. 이것만 보고 compiler가 압축 명령어나 부동소수점 명령을 앱에 생성했다고 판단하면 안 됩니다.

0x4020에서 반환하므로 정상 실행은 0x4024의 문자열을 instruction으로 fetch하지 않습니다. 데이터 주소, 제어 흐름, `xxd`의 ASCII 표시를 함께 보아야 합니다. 이 예제에서는 `--stop-address=0x4024`로 코드 부분만 역어셈블하면 혼동을 줄일 수 있습니다.

`objdump`는 binary byte와 section 정보를 보고 해석하며 C의 “이 위치는 문자열”이라는 원래 의도를 항상 충분히 아는 것은 아닙니다. `.text` 안에 상수를 모은 현재 script에서는 특히 code와 data의 경계를 직접 확인해야 합니다. 지원하지 않는 mnemonic이 보인다면 그 위치로 실제 제어가 도달하는지를 먼저 추적합니다.

반대로 “아마 데이터일 것”이라고 모든 이상한 instruction을 무시해서도 안 됩니다. 정상 branch의 target이 잘못되어 문자열로 떨어지면 실제 illegal instruction이나 예기치 않은 연산이 발생할 수 있습니다. ELF entry, label, branch target, 마지막 `ret`, memory dump를 묶어 판단해야 합니다.

14.5 한 명령어를 손으로 해석하기

0x00008067의 opcode는 1100111로 JALR, `rd`는 x0, `rs1`은 x1, immediate는 0입니다. 그래서 계산되는 target은 $(x1+0) \& \sim 1$ 입니다. x1의 ABI 이름이 `ra`이므로 어셈블러는 이 형태를 `ret`라는 pseudo-instruction으로 표현할 수 있습니다.

`ret` 와 `ecall` 은 서로 다른 복귀·진입 경로입니다. `ret` 는 일반 register `ra`, `mret` 는 CSR `mepc` 를 사용합니다. 이 둘을 구분하는 것이 다음 장의 중심입니다.

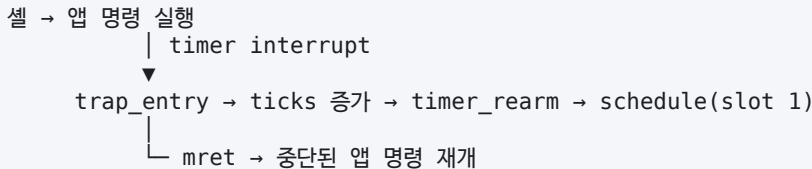
bit field를 계산하는 일반식은 `opcode=word&0x7F`, `rd=(word>>7)&31`, `funct3=(word>>12)&7`, `rs1=(word>>15)&31` 입니다. JALR에서는 상위 12비트를 signed immediate로 해석합니다. 이 방법으로 `0x000780E7` 을 해석하면 `rd=x1`, `rs1=x15`, `immediate=0`이므로 `jalr ra,0(a5)` 임을 확인할 수 있습니다.

같은 연습을 ECALL의 `0x00000073` 에 적용하면 opcode SYSTEM, funct3=0입니다. 그러나 SYSTEM의 funct3=0인 모든 bit pattern이 ECALL은 아니므로 core는 instruction 전체를 비교합니다. MRET는 `0x30200073` 으로 별도 일치 조건을 갖습니다. major opcode 확인만으로 명령의 유효성을 결정할 수 없는 사례입니다.

15. 실행 중 인터럽트와 세 종류의 복귀

15.1 앱 실행 중 timer가 발생하면

앱 실행 중에도 기존 timer 설정은 유지됩니다. 앱은 별도 task가 아니므로 scheduler의 `cur` 는 계속 셀 slot 1입니다. timer interrupt가 발생하면 앱의 현재 PC와 레지스터가 셀 stack에 저장됩니다. scheduler는 그 frame을 slot 1에 기록하고 다시 같은 slot을 선택합니다.



“timer가 있으니 앱과 셀 명령 입력이 동시에 진행된다”는 의미는 아닙니다. 앱이 반환할 때까지 셀의 명령 parser 함수는 호출 흐름상 대기합니다. 무한 반복 앱에서 셀로 강제로 돌아오는 Ctrl+C 처리도 현재 없습니다.

timer가 앱의 LBU를 중단하면 core는 destination write를 막고 그 LBU의 PC를 mepc에 저장합니다. handler를 마친 뒤 같은 LBU를 실행하므로 pointer와 읽기 순서가 유지됩니다. 일반 RAM read는 부작용이 없지만 UART RX load에는 FIFO pop이 있으므로 3장에서 설명한 trap gating이 필요합니다.

다른 task를 선택하지 않아도 trap save/restore 비용은 발생합니다. timer가 있다는 사실과 실질적인 병렬 처리, 앱의 강제 종료 기능은 각각 다릅니다. 현재 OS는 한 core에서 하나의 instruction stream을 실행하고 interrupt 때 잠시 handler로 이동합니다.

15.2 ECALL, 함수 호출, context switch 비교

구분	진입 원인	복귀 주소의 핵심	복귀 방법
일반 앱 함수 호출	셀의 JALR	ra	앱의 ret
syscall	앱의 ECALL 예외	frame의 mepc=ecall PC+4	MRET
timer 처리	timer level과 enable	frame의 중단된 PC	MRET
다른 task로 전환	scheduler가 다른 frame 반환	선택된 frame의 mepc	그 frame 복원 후 MRET

하나의 앱 실행에는 수십 번의 syscall trap 복귀가 포함될 수 있지만 앱 자체의 최종 함수 복귀는 한 번입니다. trap이 발생했다고 항상 다른 task로 전환하는 것도 아닙니다.

일반 함수의 return은 caller의 호출 지점으로 돌아가지만, task switch는 서로 관련 없는 함수 흐름의 중간으로도 이동할 수 있습니다. frame에 전체 register와 재개 PC가 있으므로 C 호출 관계에 의존하지 않고 다른 실행 문맥을 선택할 수 있습니다. 이 점이 ordinary call stack과 scheduler의 context 관리가 다른 이유입니다.

현재 task 전환에서도 페이지 테이블을 바꾸는 주소 공간 전환은 없습니다. 모두 같은 physical memory를 사용합니다. 일반적인 process context switch는 이러한 register 상태 이외에 주소 공간·권한·kernel bookkeeping도 다룰 수 있습니다. 현재 문서의 context switch 설명은 해당 최소 구현 범위로 읽어야 합니다.

15.3 timer 재설정과 interrupt masking

timer IRQ는 `mtime >= mtimecmp` 동안 1인 level입니다. handler는 다음 compare 값을 미래로 옮겨 IRQ 조건을 해제합니다. FPGA 설정의 `TICK_CYCLES=125000`은 12.5 MHz에서 약 10 ms입니다. handler가 현재 시각을 읽어 다음 시간을 정하므로 이상적인 고정 위상 tick과는 차이가 있을 수 있습니다.

trap 진입은 `MIE=0`으로 만듭니다. 현재 handler는 중간에 이를 켜지 않으므로 timer handler가 자신을 중첩 호출하지 않습니다. syscall 안에서 UART가 바쁠 때 기다리거나 파일을 복사하는 동안 timer 처리는 지연될 수 있습니다. IRQ는 level로 남지만 지나간 tick마다 독립적인 요청이 쌓이는 queue는 아닙니다.

MIE를 끄는 것은 timer counter를 멈추는 것이 아닙니다. mtime은 계속 증가하고 compare 조건은 계속 평가됩니다. handler가 길어지면 MRET 직후 pending timer를 바로 받을 수 있습니다. interrupt enable과 peripheral의 시간 진행을 별도로 그려 보면 “interrupt를 켜는데 시간이 왜 흘렀는가”라는 혼동이 사라집니다.

현재 `timer_rearm()`은 직전 예정 시각에 `TICK_CYCLES`를 더하지 않고 handler가 읽은 현재 시각에 더합니다. 처리 지연이 tick phase에 누적될 수 있고, 여러 주기가 지났더라도 ticks를 한 번만 올립니다. ticks는 정확한 wall-clock을 복원하는 완전한 clock source가 아니라 handler 진입 횟수라는 점을 기억해야 합니다.

코드 해부 15-A. timer의 counter·compare·level IRQ 연결

출처: [rtl/simple_timer.v](#), 원본 26 - 30, 43 - 53행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
26 | reg [63:0] mtime;
27 | reg [63:0] mtimecmp;
30 | assign irq = (mtime >= mtimecmp);
   | ... (원본 31-42행 생략)
43 | always @(posedge clk) begin
44 |     if (rst) begin
45 |         mtime <= 64'd0;
46 |         mtimecmp <= 64'h0000_0000_ffff_ffff;
47 |     end else begin
48 |         mtime <= mtime + 64'd1;
50 |         if (wr_en && !addr[2]) mtime[31:0] <= wr_data;
51 |         if (wr_en && addr[2]) mtimecmp[31:0] <= wr_data;
52 |     end
53 | end
```

원본 행	한 줄씩 읽는 동작과 의미
26	현재 시각을 세는 64비트 counter입니다.
27	다음 interrupt 기준을 담는 64비트 compare register입니다.
30	counter가 compare 이상인 동안 IRQ는 계속 1입니다. 단 한 클록의 pulse가 아닙니다.
43	timer 상태 갱신은 CPU와 같은 clock의 rising edge에서 일어납니다.
44	reset 조건을 먼저 선택합니다.
45	시각을 0으로 초기화합니다.
46	compare의 상위 word는 0, 하위 word는 최대값으로 시작합니다.
47	reset이 아닐 때의 정상 동작입니다.
48	counter 전체를 매 clock 1씩 증가시킵니다. MIE가 0이어도 이 줄은 계속 수행됩니다.

원본 행	한 줄씩 읽는 동작과 의미
50	MMIO write에서 주소 bit 2가 0이면 counter 하위 word를 씁니다. 같은 블록에서 뒤의 해당 bit write가 앞의 increment보다 우선합니다.
51	주소 bit 2가 1이면 compare 하위 word를 씁니다. 상위 32비트를 갱신하는 경로는 없습니다.
52	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.
53	앞에서 연 블록을 닫습니다. 이 구분자 자체가 새로운 CPU instruction이나 별도 동작을 추가하지는 않습니다.

C의 timer_rearm은 `MTIME_L0 + TICK_CYCLES`를 compare 하위 word에 씁니다. 짧은 실행에서는 미래 deadline을 만들지만 64비트 비교와 32비트 갱신의 불일치는 15.4 - 15.5절의 wrap 문제로 이어집니다.

15.4 장시간 동작의 timer 제한

timer 내부 counter는 64비트지만 MMIO는 하위 32비트만 제공합니다. 현재 `mtimecmp`의 상위 word를 갱신하는 인터페이스도 없습니다. 하위 word가 wrap하는 시간은 다음과 같습니다.

$$2^{32} / 12,500,000 = \text{약 } 343.6\text{초} = \text{약 } 5\text{분 } 44\text{초}$$

이 경계 부근에서는 compare 재설정이 의도대로 미래를 가리키지 않아 interrupt가 반복될 수 있고, 경계 이후에는 상위 word 불일치로 IRQ를 해제하지 못할 수 있습니다. 따라서 장시간 켜 놓은 셀의 응답 정지를 단순 UART 접속 문제로만 해석하면 안 됩니다. 수업 실습은 이 제한을 알고 짧게 수행하며, 장시간 사용을 위한 설계 확장은 상·하위 timer MMIO와 안전한 64비트 갱신 규약입니다.

상위 word를 노출하는 것만으로 64비트 접근이 한 번에 원자적으로 되는 것은 아닙니다. RV32 CPU는 보통 상·하위를 여러 load/store로 다루므로 read 사이의 rollover와 compare 갱신 중 일시적으로 지난 deadline이 되는 경우를 처리해야 합니다. high-low-high 재확인 같은 읽기 규약과 안전한 compare write 순서를 함께 설계해야 합니다.

현재 한계를 실험할 때는 elapsed time, 마지막 UART 응답, reset 여부를 함께 기록하십시오. 입력이 멈춘 뒤 screen만 재접속하는 것은 timer 상태를 바꾸지 않습니다. SW20 reset은 timer 상태를 다시 시작하지만 정상 RAM disk는 보존합니다. 원인을 명확히 하기 전에 재구성하면 파일과 실행 증거까지 없어질 수 있습니다.

15.5 64비트 비교와 32비트 재설정의 불일치 예

낮은 word가 끝에 가까운 순간 `mtime=0x00000000_FFFFFFF0`이라고 가정하고, 설명을 단순하게 하기 위해 다음 interval을 `0x200`으로 잡습니다. C가 낮은 word끼리 더하면 `0xFFFFFFFF+0x200`의 `uint32_t` 결과는 `0x00000100`입니다.

값	정상적인 64비트 deadline	현재 low-word만 쓸 때
compare high	0x00000001	기존 0x00000000 유지
compare low	0x00000100	0x00000100
compare 전체	0x00000001_00000100	0x00000000_00000100
현재 mtime과 비교	아직 미래	이미 과거

현재 timer는 64비트 unsigned 비교를 하므로 잘못 작은 compare 값에 대해 IRQ를 계속 올립니다. 낮은 word가 wrap하기 직전에도 이 상황이 생길 수 있으며, 이후 mtime high가 1이 되면 compare high 0으로는 미래 deadline을 표현할 수 없습니다. 단순히 32비트 덧셈이 wrap한다는 사실보다 전체 비교 폭의 불일치가 핵심입니다.

이 예에서 high word를 나중에 쓰는 기능을 추가하더라도 중간 상태에 IRQ가 순간적으로 걸리지 않도록 write 순서를 정해야 합니다. software와 peripheral 양쪽의 ABI를 함께 고치는 문제이며, 문서의 현재 구현에는 아직 그 변경이 적용되지 않았습니다.

16. ZCU104 실습 절차

16.1 호스트 명령과 rv> 명령 구분

`make`, `python3`, `screen`은 Linux 호스트 터미널에서 실행합니다. `ls`, `run hello.app`은 UART로 보이는 `rv>`에 입력합니다. 아래 code block은 어느 쪽 명령인지 문맥에 표시합니다. 프롬프트 문자열 자체를 입력할 필요는 없습니다.

UART 장치 경로는 현재 보드의 FT4232 채널 D입니다. 다른 케이블 일련번호나 USB 열거 순서에서는 달라집니다. `/dev/ttyUSB3`보다 `/dev/serial/by-id/...` 경로가 장치 식별에 유리합니다.

두 터미널을 쓰면 하나는 `host build·upload`용, 다른 하나는 UART `screen`용으로 역할을 정해 두는 것이 편리합니다. 다만 uploader가 실행되는 순간에는 `screen`이 포트를 놓아야 합니다. 별도 터미널 창을 사용한다는 사실이 별도 serial hardware를 사용하는 것은 아닙니다.

Linux shell의 `ls`는 PC의 directory를 보여 주고 `rv>`의 `ls`는 FPGA RAM disk를 보여 줍니다. 같은 문자열의 명령이지만 실행 주체와 파일 시스템이 다릅니다. `hello_app.bin`이 host에 존재하는 것과 `hello.app`이 MiniFS에 존재하는 것은 서로 자동 연결되지 않으며 UART upload가 그 사이를 잇습니다.

16.2 처음부터 준비하는 순서

먼저 UART를 사용하는 `screen`을 종료합니다. `screen` 화면에서 `Ctrl+A`를 누르고 손을 떼 다음 `K`, 확인 `y`를 누릅니다. 이미 detach했다면 `screen -ls`로 정확한 세션을 확인하고 `screen -S 세션ID -X quit`으로 그 세션만 종료합니다.

호스트에서 실행합니다.

```
cd /home/eulia/RISC-V
make hello-app
make vivado-zcu104-shell
make program-zcu104-shell
python3 scripts/upload_app.py \
  --file build/firmware/hello_app.bin --name hello.app --run
```

`make vivado-zcu104-shell`은 비트스트림 생성이 필요한 때 수행합니다. 이미 현재 소스의 `.bit`가 있으면 그 단계를 생략할 수 있습니다. 다운로드를 보드 상태를 초기화하므로 업로드보다 먼저 수행해야 합니다.

정상 응답의 예는 다음과 같습니다.

```
{
  "uploaded": "hello.app",
  "bytes": 60,
  "checksum": "000010dd",
  "ran": true,
  "app_output": "run hello.app\r\n\r\nrunning\r\n\r\nHello from loaded app!\r\n\r\nreturned\r\n\r\nrv> "
```

이 JSON은 호스트 스크립트의 보고서입니다. FPGA가 JSON을 출력하는 것은 아닙니다. `app_output` 필드에 실제 UART에서 수신한 앱 실행 응답이 들어 있습니다. 예제 크기·checksum은 소스와 compiler 결과에 따라 바뀔 수 있습니다.

처음 실습할 때는 각 단계의 결과를 하나씩 확인합니다. `make hello-app` 뒤에 ELF와 BIN이 생겼는지, Vivado 뒤에 `.bit`가 생겼는지, program 단계에서 올바른 장치를 선택했는지, upload report에 `uploaded`와 `ran`이 있는지를 봅니다. 앞 단계가 실패했는데 다음 명령을 계속 실행하면 오래된 산출물을 새 결과로 오해하기 쉽습니다.

UART가 여러 개 있으면 host uploader의 `--port`로 정확한 장치를 지정합니다. 현재 자동 선택은 해당 형태의 by-id 포트가 하나일 때를 전제로 합니다. board cable이 달라지면 안내문의 일련번호를 자신의 장치로 바꾸어야 하며, baud는 115200으로 맞춥니다.

16.3 UART에 재접속하여 실행

업로드 스크립트가 끝난 뒤 호스트에서 다음을 실행합니다.

```
screen /dev/serial/by-id/usb-Xilinx_JTAG+3Serial_46635-if03-port0 115200
```

빈 화면이면 Enter를 한 번 누릅니다. 이미 전송된 부팅 메시지와 프롬프트는 새 연결에서 다시 보이지 않을 수 있습니다. Enter가 새 프롬프트를 만들어 줍니다.

```
rv> ls
hello.app 72 bytes
rv> run hello.app
running
Hello from loaded app!
returned
rv>
```

이 단계에서 `make program-zcu104-shell`이나 `make test-zcu104-app`을 다시 실행하면 안 됩니다. 그 명령은 보드를 재구성하여 방금 저장한 RAM disk를 초기화합니다. 같은 파일의 재실행에는 `run hello.app`만 필요합니다.

성공한 uploader는 이미 프롬프트까지 읽고 종료합니다. 새 screen은 그 이후의 serial byte만 보므로 처음에 아무 글자가 없는 상황이 자연스럽습니다. Enter를 누르면 CR이 전달되고 빈 line 처리 후 prompt가 새로 출력됩니다. 이때 다시 boot banner가 안 나온다는 사실은 오히려 보드를 재시작하지 않았다는 상태와 맞습니다.

`run hello.app`을 여러 번 반복하면 매번 파일에서 payload를 다시 복사하고 호출합니다. 한 번 로딩된 함수를 그 자리에서 계속 호출하기만 하는 cache형 loader가 아닙니다. 이 동작은 앱의 `.data` 초기 상태를 다시 공급하는 효과도 가지지만 별도 heap·외부 장치 상태까지 초기화하지는 않습니다.

16.4 이미 셸이 실행 중일 때 앱만 바꾸기

히로·셸 펌웨어·메모리 배치를 바꾸지 않고 앱 C 코드만 바꿨다면 앱을 다시 컴파일하고 업로드하면 됩니다. 매번 FPGA를 합성할 필요가 없습니다.

```
# 먼저 screen을 종료한 뒤 호스트에서 실행
make hello-app
python3 scripts/upload_app.py \
  --file build/firmware/hello_app.bin --name hello.app --run
```

`--run`을 빼면 저장까지만 수행합니다. 같은 이름이 있으면 현재 업로드 경로는 파일 내용을 교체합니다. 이후 screen에서 Enter → `ls` → `run hello.app` 순서로 확인합니다.

새 메시지를 가진 앱을 같은 `hello.app` 이름으로 업로드하면 FS write가 그 파일을 교체합니다. 다른 이름으로 저장하면 table entry는 별도로 생기지만 실행 창은 여전히 하나입니다. `run first.app`, `run second.app` 을 차례로 실행할 수 있고, 두 파일을 동시에 실행하는 의미는 아닙니다.

앱 수정만으로 되는 범위에는 사용 ISA, entry 주소, syscall ABI, 크기·stack 제한을 유지한다는 조건이 붙습니다. 예를 들어 `.bss` 지원이나 새로운 syscall을 추가하면 앱만 교체해서는 안 되고 loader/kernel 변경을 반영한 셸 이미지를 먼저 만들어야 합니다.

16.5 screen의 종료·분리·중지

조작	의미	UART를 다른 프로그램이 써도 되는가
<code>Ctrl+A → K → y</code>	현재 serial window 종료; 유일한 window면 session 종료	해당 session이 종료됐으면 가능
<code>Ctrl+A → D</code>	화면만 detach	session이 포트를 계속 사용하므로 업로드 전 종료 필요
<code>screen -r 세션ID</code>	detached session 재접속	기존 session 사용
<code>screen -x 세션ID</code>	같은 session 화면 공유	별도 uploader와의 동시 사용 허용이 아님
<code>Ctrl+Z</code> 로 stopped된 foreground job	shell 작업 제어상 일시 정지	backend screen이 남을 수 있음

마지막 경우에는 원래 Linux shell에서 `fg %1` 등 해당 job을 foreground로 되돌릴 수 있습니다. 정확한 job 번호는 그 shell의 `jobs` 로 확인합니다. 화면 공유와 같은 serial device를 별도 프로세스가 중복해서 읽는 것은 구분해야 합니다.

`screen -x` 는 하나의 session이 이미 읽고 있는 화면을 여러 terminal에 보여 주므로 reader가 여러 개 생기는 것과 다릅니다. 반대로 별도의 screen을 같은 device로 열거나 uploader를 함께 실행하면 서로 응답을 나눠 읽을 수 있습니다. 이 경우 한쪽 화면에 일부 글자가 빠지거나 uploader가 marker를 찾지 못할 수 있습니다.

종료할 때는 `screen -ls` 에서 확인한 특정 session을 대상으로 합니다. detach는 UI 연결만 끊으므로 “터미널 화면에서 빠져나왔으니 포트가 비었다”는 판단이 틀릴 수 있습니다. serial port를 사용하는 주체를 하나로 유지하는 운영 절차가 현재 host tool의 중요한 전제입니다.

16.6 증상에서 계층을 찾아가기

증상	먼저 해석할 내용	확인·대응
새 screen이 빈 화면	과거 UART 출력은 재생되지 않음	Enter로 새 <code>rv></code> 요청
<code>ls</code> 가 빈 목록	현재 FS에 등록된 파일 없음	업로드 후 재구성 여부 확인
<code>not an app file</code>	파일 없음, header 읽기 실패, magic 불일치를 같은 문구로 보고	<code>ls</code> 로 존재 확인, <code>.bin</code> 재업로드
<code>checksum mismatch</code>	업로드 stream과 요청한 합이 다름	UART 동시 접근·누락 확인
<code>truncated app</code>	header가 선언한 크기만큼 읽지 못함	파일·header 길이 검토
<code>invalid app size</code>	실행 허용 범위 밖	payload 크기와 linker 결과 확인
PANIC cause=00000002	illegal instruction 예외	panic PC와 ELF 역어셈블 대조

증상	먼저 해석할 내용	확인·대응
몇 분 뒤 응답 정지	timer low-word 경계 등 가능	약 343초 제한 확인, SW20으로 재시작
uploader가 응답 timeout	포트·보드·셸 입력 상태 확인 필요	screen 점유, baud, 중단된 upload 상태 점검

SW20 reset은 유효한 RAM disk를 보존하도록 설계되어 있습니다. 전원 재인가나 PL 재다운로드는 파일을 지우므로 같은 대응으로 취급하지 않습니다.

문제 해결은 물리 연결·clock/reset → UART byte → 셸 parser → 파일 형식 → CPU 실행 순서로 좁히면 좋습니다. `rv>`가 보이면 최소한 CPU·TX와 셸 시작 경로는 상당 부분 작동한 것입니다. `ls`에 파일이 보이면 업로드 후 FS metadata가 남아 있다는 증거이고, `running`까지 나오면 loader의 호출 전 검사를 통과했다는 증거입니다.

반면 `returned`가 안 보인다면 앱 무한 loop, illegal instruction, stack/ra 손상, timer 제한 등을 고려해야 합니다. 마지막 메시지와 PC·panic 정보를 수집하면 원인 범위를 줄일 수 있습니다. 서로 다른 계층의 증상을 모두 재다운로드 하나로 처리하면 재현 가능한 원인 분석이 어려워집니다.

17. 검증 방법과 관측 가능한 증거

17.1 host artifact 검사

소프트웨어 단계에서는 ELF32, RISC-V machine, entry 0x4000, section 크기, .bss 부재를 검사합니다. objdump 로 앱 코드가 지원 명령으로 되어 있는지 보고 xxd 로 payload와 문자열을 확인합니다. magic과 checksum을 통과한다고 이 모든 성질이 자동 증명되는 것은 아닙니다.

현재 예제에서 확인한 값은 다음과 같습니다.

항목	관측값
ELF class / endian	ELF32 / little-endian
entry	0x4000
.text 크기	0x3C, 60바이트; 문자열 포함
명령 부분	0x4000 - 0x4023, 36바이트
payload byte sum	0x000010DD
MiniFS 저장 크기	header 포함 72바이트

같은 소스라도 compiler 버전과 최적화에 따라 instruction 배치가 달라질 수 있습니다. 따라서 문서의 0x4014 같은 값은 해당 예제 artifact에서 검산하고, 자신의 결과가 달라졌다면 symbol과 disassembly를 기준으로 새로운 관측 지점을 잡아야 합니다. 계약인 entry 0x4000과 관측값인 내부 branch 주소를 구분하십시오.

재현 기록에는 source revision 또는 변경 내역, GCC 옵션, ELF entry, BIN 길이·합, 사용 bitstream의 SHA-256을 함께 남기는 것이 유용합니다. checksum은 payload 전송 확인용이고 SHA-256은 실험한 큰 artifact의 식별용입니다. 두 값은 역할과 강도가 다릅니다.

17.2 RTL 시뮬레이션이 확인하는 것

```
make sim-shell
```

현재 tb_shell.v 는 serial bit waveform을 uart_rx 에 입력하고 실제 rv32_soc 를 실행합니다. 셸 명령과 앱 업로드 후 다음을 검사합니다.

관찰 대상	검사 의미
written, cat 결과, ls, removed	shell parser와 MiniFS 서비스 연계
ramdisk 내용과 table entry	파일 저장·삭제의 내부 상태
uploaded	hex 수신·검증·파일 저장 성공 경로
0x4000 <= pc < 0x6000	CPU가 앱 실행 영역에서 실제 instruction fetch
mem[4096] == app_words[0]	첫 기계어가 올바른 실행 위치에 로딩됨
앱 메시지와 returned	앱 수행 후 셸 호출 흐름으로 복귀

testbench의 TX는 `uart_tx_ready=1`로 두고 SoC의 byte-level valid/data를 관찰합니다. 따라서 이 테스트만으로 실제 UART TX baud와 물리 pin 연결까지 검증했다고 할 수 없습니다. RX는 직렬 waveform을 사용하고 TX는 빠른 byte interface로 확인하는 혼합형 테스트입니다.

시뮬레이션 펌웨어는 tick을 2000클럭으로 줄여 사용합니다. FPGA 이미지의 125000클럭과 다르므로 테스트 실행 시간과 interrupt 횟수를 보드와 그대로 비교하지 않습니다.

testbench가 앱 영역 PC를 관찰하는 것은 단순한 UART 문자열 검사보다 강한 증거입니다. firmware 안에 같은 문구가 우연히 들어 있어도 그 검사만으로 앱 실행을 입증할 수는 없기 때문입니다. 앱 첫 word 비교는 payload의 로딩 주소와 byte 순서를 추가로 확인합니다. 여러 독립적인 관측을 조합하면 한 종류의 거짓 양성을 줄일 수 있습니다.

`$readmemh`에 image보다 큰 배열을 지정하면 남은 word가 파일에 없다는 warning이 나올 수 있습니다. SoC는 먼저 RAM을 0으로 채우고 실제 image만 적재하므로 이를 무조건 오류로 해석할 필요는 없습니다. 그러나 누락된 image나 잘못된 path 같은 warning은 실제 실패 원인이므로 메시지와 testbench 조건을 함께 확인해야 합니다.

코드 해부 17-A. PASS를 구성하는 실제 testbench 검사 줄

출처: [tb/tb_shell.v](#), 원본 118 - 124행. 주석·빈줄 생략, 행 번호는 원본 기준입니다.

```
118 | wait(prompts >= 7);
119 | require_text("Hello from loaded app!",22);
120 | require_text("returned",8);
121 | if(!saw_app_pc) $fatal(1,"CPU never fetched from app RAM");
122 | if(dut.mem[4096] != app_words[0]) $fatal(1,"loaded app instruction mismatch");
123 | $display("PASS: UART RX, MiniFS upload, CPU app fetch/execute/return, shell commands");
124 | $finish;
```

원본 행	한 줄씩 읽는 동작과 의미
118	일곱 번째 prompt가 나타날 때까지 시뮬레이터가 기다립니다. CPU가 실행하는 instruction은 아닙니다.
119	수집한 TX byte log에서 예제 앱 문구를 찾습니다.
120	앱 호출 후 returned 출력도 확인합니다.
121	앱 RAM 범위 PC가 관측되지 않았다면 실패합니다. 문자열이 나왔다는 사실과 실제 앱 fetch를 구분하는 검사입니다.
122	실행 RAM 첫 word가 업로드할 앱의 첫 word와 같은지 비교합니다. 주소·endianness·복사를 함께 확인합니다.
123	위 조건을 통과한 뒤 PASS 메시지를 출력합니다. 확인하지 않은 보호·장시간 안정성까지 보장하는 문구로 해석하지 않습니다.
124	정상 완료로 시뮬레이션을 끝냅니다. 실제 보드에는 이 종료 system task가 합성되지 않습니다.

testbench의 `$fatal`, `$display`, `$finish`는 시뮬레이터의 동작입니다. firmware의 ECALL이나 Mini Shell 명령과 다른 계층이며, 이 문서 작성 자체가 새 보드 테스트를 수행했다는 뜻은 아닙니다.

17.3 실제 보드 검증

```
make test-zcu104-app
```

이 target은 `hello-app`을 준비하고, `test-zcu104-shell`을 통해 실제 PL을 재구성한 뒤 셸 파일 명령을 검사하고, 업로드 스크립트로 앱 저장·실행을 검사합니다. 셸 bitstream은 먼저 만들어져 있어야 하며 UART를 사용하는 terminal을 달아야 합니다.

2026-09-29 실제 보드 기록은 [ZCU104 안내](#)에 있습니다. `write/cat/ls/rm/ls` 검사는 176바이트의 응답을 수신하며 통과했습니다. 이어 60바이트 앱을 업로드하여 `running`, `Hello from loaded app!`, `returned`, `rv>`를 확인했습니다. 현재 확인한 당시 bitstream SHA-256은 다음과 같습니다.

```
14e8051181301004070c03b18de9537efd265c66a329f1dd85f4d84109b996dc
```

원본 UART capture는 `build/zcu104_shell/upload_capture.bin`입니다. 업로드를 다시 하면 이 파일은 새 capture로 바뀔 수 있으므로 실험 기록을 보관하려면 별도 이름으로 복사해야 합니다. 이 장의 보드 결과는 해당 시점의 기록이며, 문서 생성 자체가 새 보드 테스트를 뜻하지 않습니다.

실제 보드에서는 RTL 시뮬레이션이 생략한 pin 배치, UART baud, 클록 분주, 케이블·host serial 경로가 함께 검증됩니다. 반면 FPGA 내부의 모든 register나 RAM을 자동으로 관측하는 것은 아닙니다. UART 성공과 내부 PC 파형은 서로 다른 계층의 증거이므로 한쪽을 다른 쪽의 완전한 대체로 여기지 않습니다.

테스트 target이 PL을 재구성한다는 사실은 실험 설계에도 영향을 줍니다. 기존 파일의 reset 보존을 시험하려는데 매번 재구성 target을 실행하면 다른 조건을 시험한 것이 됩니다. reset 종류, 기존 파일 유무, 업로드 순서를 기록해야 파일 시스템 유지성에 대한 결론이 유효합니다.

17.4 현재 자동 검사의 범위

host uploader는 실행 응답에 `running`, `returned`, 프롬프트가 있는지 검사합니다. 예제 문구 자체의 일치까지 모든 앱에 대해 강제하지는 않습니다. 보드의 예제 메시지는 실제 capture로 확인했습니다. RTL testbench는 그 예제 문구도 검사합니다.

현재 검사는 정상 경로 smoke test이며 잘못된 header, checksum 충돌, 최대 크기, FIFO overflow, stack overflow, 장시간 timer wrap, 악성 코드 격리까지 포괄하지 않습니다. **교재에서 “PASS”는 어떤 관측 조건을 통과했는지와 함께 해석해야 합니다.**

추가 test를 설계한다면 normal path, boundary, invalid input을 나누는 것이 좋습니다. boundary에는 최소 4바이트, 마지막 허용 payload 크기, line buffer 경계, file table 32개가 들어갑니다. invalid input에는 없는 파일, 틀린 magic, truncated payload, 추가 trailing byte, 틀린 합이 들어갑니다. 각각 어떤 error와 메모리 변화가 기대되는지 먼저 정해야 합니다.

예를 들어 checksum 실패 test는 단지 오류 문자열뿐 아니라 PC가 앱 창에 진입하지 않는지도 확인할 수 있습니다. syscall 보존 test는 여러 register에 signature를 넣고 trap 후 유지 여부를 검사할 수 있습니다. 현재 자동 test가 이미 이런 모든 검사를 한다고 설명하지 않고, 후속 검증 항목으로 구분해야 합니다.

17.5 파형을 추가해 관찰할 신호

더 깊이 관찰하려면 다음 신호들을 testbench dump 대상으로 선택할 수 있습니다. 현재 `tb_shell.v` 자체에는 `$dumpfile/$dumpvars`가 없으므로 `make sim-shell`만으로 새 shell VCD가 생긴다고 기대하지 않습니다.

신호/상태	확인할 관계
<code>serial_in</code> , <code>rx_byte</code> , <code>rx_valid</code>	serial frame이 한 바이트로 바뀌는 시점
<code>dut.rx_wr</code> , <code>dut.rx_rd</code>	CPU가 입력을 소비하는 속도
<code>pc</code>	셀 → 앱 → trap → 앱 → 셀의 제어 흐름

신호/상태	확인할 관계
<code>dut.daddr</code> , <code>dut.dwstrb</code> , <code>dut.dwdata</code>	파일 저장과 실행 RAM 복사
<code>core.take_trap</code> , <code>CSR.mepc</code> / <code>mcause</code>	ECALL·timer 구분과 복귀 PC
<code>tx_valid</code> , <code>tx_byte</code>	syscall과 출력 byte의 대응

전체 RAM 배열을 모두 dump하면 VCD가 커질 수 있습니다. 처음에는 PC, trap, MMIO bus, `mem[4096]` 주변처럼 의미가 분명한 신호를 좁혀 관찰하는 것이 좋습니다.

파형은 rising edge를 기준으로 보십시오. combinational `rd_data`, `next_pc`가 잠시 변하는 동안 아직 register와 RAM은 바뀌지 않았을 수 있습니다. `take_trap=1` 인 edge에서 `strobe`가 0인지, 다음 PC가 `mtvec`인지, 이후 trap.S의 여러 SW가 frame을 만드는지를 순서대로 찾으면 제어와 data의 관계가 보입니다.

처음에는 `ecall` 한 번을 확대해 보고, 다음에는 전체 앱 실행 구간으로 축소하여 반복 구조를 확인합니다. serial waveform은 약 86μs 단위 byte, CPU는 80ns 단위 cycle이므로 같은 화면에서 두 시간 규모를 모두 읽으려 하면 복잡해집니다. 목적에 맞게 zoom과 관측 신호를 나누는 것이 중요합니다.

17.6 invariant를 이용한 검증 설계

invariant는 특정 입력 예제 하나보다 넓은 상황에서 유지되어야 하는 조건입니다. 다음은 현재 설계를 읽으며 만들 수 있는 assertion·관측 조건의 예입니다. 일부는 현재 testbench의 검사와 대응하고, 일부는 추가 검증 과제입니다.

조건	검사할 이유	현재 구현과의 관계
x0 read는 항상 0	기본 ISA 상태 보장	regfile의 조합 mux와 write 무시
trap 선택 edge에 <code>dmem_wstrb=0</code>	중단된 store의 중복 부작용 방지	core의 write gating
<code>core_trap</code> 이면 RX pop 없음	입력 byte의 손실 방지	SoC의 rx_pop 조건
정상 앱 fetch는 실행 창 안에서 시작	loader entry 계약	현재 test의 saw_app_pc와 연관
frame base는 16바이트 정렬 유지	C handler 호출 ABI	task stack과 128바이트 frame 크기
checksum/길이 실패 시 함수 호출 없음	검증 전 실행 방지	negative test로 보장 가능
앱 반환 뒤 SP와 callee-saved 값 유지	셀의 정상 실행 지속	별도 signature test로 보장 가능

assertion의 위치도 중요합니다. nonblocking assignment의 결과는 edge 처리 후 갱신되므로 testbench가 old state를 읽는지 new state를 읽는지 알고 작성해야 합니다. 단순히 signal 값이 한번 보였다는 사실과 명령이 해당 edge에 완료됐다는 조건을 구분합니다.

모든 조건을 한 번에 자동화할 필요는 없습니다. 우선 관측하려는 실패를 하나 정하고, 정상 상태에서 성립하는 조건을 쓴 뒤, 의도적으로 조건을 깨는 작은 변형에 test가 반응하는지 확인합니다. 검증 코드 역시 잘못 작성할 수 있으므로 test 자체의 민감도를 점검해야 합니다.

18. 현재 구현의 한계와 확장 설계

18.1 M-mode 앱의 권한

앱·셀·handler는 모두 M-mode이며 주소 공간도 같습니다. 앱은 현재의 하드웨어에서 kernel RAM, MMIO, RAM disk를 직접 접근하거나 CSR을 바꿀 수 있습니다. **syscall 사용은 소프트웨어의 약속이며 하드웨어가 강제하는 접근 경계가 아닙니다.**

APP1 검증은 파일 구조와 단순 합을 확인합니다. 파일이 유효한 형태라는 것과 코드가 안전하다는 것은 별개입니다. 수업에서는 작성 내용을 이해한 앱을 사용합니다. 임의 코드를 격리하려면 U-mode, 접근 보호, 잘못된 접근에 대한 trap, 별도 실행 문맥을 함께 설계해야 합니다.

RISC-V의 M-mode는 machine 수준의 제어 권한을 뜻합니다. 현재 앱이 ECALL을 쓴다고 그 외의 직접 MMIO나 CSR 접근이 hardware에서 막히는 것은 아닙니다. API를 사용하는 모범적인 코드와 권한이 제한된 코드는 다른 개념입니다. 교육 예제에서는 의도한 프로그램을 실행한다는 전제가 있습니다.

격리 설계에서는 잘못된 pointer를 사용하는 FS syscall도 고려해야 합니다. U-mode에서 직접 kernel memory를 못 쓰게 하더라도 handler가 검증 없이 user pointer를 따라가면 보호 경계를 우회할 수 있습니다. privilege, memory permission, syscall의 주소·길이 검사, fault 복구가 함께 설계되어야 합니다.

18.2 앱 종료와 오류 처리

정상 앱은 C 함수 반환으로 셀에 돌아옵니다. 무한 루프 앱은 셀 parser를 진행시키지 않으며 timer가 발생해도 같은 task로 복귀합니다. illegal instruction 등 처리하지 않는 예외는 panic()에서 멈춥니다. **앱 하나만 종료하고 셀을 복구하는 fault recovery는 없습니다.**

이를 확장하려면 셀 task와 앱 task를 분리하고, 앱에 대한 소유 stack과 상태를 기록하며, 오류 발생 시 어떤 자원을 반환하고 어느 문맥을 복구할지 정의해야 합니다. 단순히 panic()의 무한 루프를 return으로 바꾸는 것만으로는 올바른 복구가 되지 않습니다.

앱이 반환하지 않을 때 host uploader의 timeout은 host의 기다림을 끝낼 뿐 FPGA 앱을 종료하지 않습니다. 셀 parser가 앱 호출에 묶여 있으므로 host에서 새로운 명령을 보내도 즉시 처리된다는 보장이 없습니다. 현재 recovery는 reset에 의존할 수 있고, 이는 process kill과 다른 동작입니다.

task 종료를 설계한다면 RUNNABLE/RUNNING/EXITED 같은 상태, kernel이 소유한 stack, exit status, 기다리는 부모 문맥을 정의해야 합니다. 또한 종료한 task의 frame을 scheduler가 다시 선택하지 않도록 해야 합니다. 반환 PC만 셀로 바꾸는 임시 처리로는 손상된 SPL나 resource 상태까지 복구할 수 없습니다.

18.3 메모리 시스템의 제약

코어의 memory interface에는 wait-state handshake가 없습니다. 조합식 instruction/data read 때문에 이 RAM을 동기식 BRAM이나 DDR로 그대로 대체할 수 없습니다. BRAM 기반 multi-cycle 또는 pipeline 설계로 확장할 때는 읽기 지연, pipeline stall, load 완료, trap의 정확한 commit 경계를 함께 구현해야 합니다.

unmapped instruction fetch는 현재 SoC에서 NOP를 돌려주고, 여러 잘못된 접근이 access fault로 보고되지 않습니다. 보호·오류 모델이 완전하지 않으므로 PC가 잘못되면 항상 명확한 fault message가 나온다고 가정할 수 없습니다.

동기식 BRAM은 주소를 제시한 사이클과 데이터가 나오는 사이클이 다를 수 있습니다. 현재 LW는 같은 사이클의 data를 바로 rd_data에 사용하므로, 메모리만 바꾸면 이전 값이나 잘못된 값을 쓸 수 있습니다. multi-cycle 설계에서는 요청·대기·완료 상태를, pipeline에서는 memory 응답과 write-back 제어를 명시적으로 넣어야 합니다.

stall은 현재 instruction의 완료를 기다리는 microarchitecture 동작이고 context switch는 OS가 다른 register/PC 문맥을 선택하는 동작입니다. 둘 다 실행이 잠시 멈춰 보일 수 있지만 저장해야 할 상태와 발생 주체가 다릅니다. stall 중 interrupt를 언제 받을지까지 결정하면 precise trap 계약으로 다시 연결됩니다.

18.4 운영체제 기능별 다음 단계

확장 목표	필요한 설계 요소	현재 학습 내용과의 연결
.bss 지원	header의 영역 정보, 범위 검사, zeroing	boot.S의 .bss 초기화 재사용
앱 인수	argc/argv 규약, 문자열·포인터 배치, 수명	ABI와 stack frame
앱별 stack	호출 trampoline 또는 별도 task context	trap frame·SP 복원
여러 앱 동시 실행	task 생성·종료·상태·스케줄링	일반 OS round-robin
U-mode 격리	privilege 전환, PMP 등 보호, pointer 검증	CSR·trap 경계
ELF loader	ELF/segment 파싱, 범위·길이 검사, entry 확인, zero-fill	고정 APP1 로딩의 일반화
임의 주소 로딩	PIC 규약 또는 relocation 적용	linker와 절대 주소
SD 카드 유지 저장	block device driver, 저장 완료·오류 처리	MiniFS와 장치 계층 분리
장시간 timer	64비트 MMIO와 일관된 read/write 순서	interrupt 재설정
cache 추가	instruction/data 동기화와 FENCE.I	store된 코드의 fetch 가시성

이 표는 현재 구현되어 있다는 설명이 아니라 후속 설계 과제입니다. 특히 ELF loader를 추가한다고 자동으로 프로세스 격리나 Linux 호환성이 생기는 것은 아닙니다. 실행 파일 해석, runtime ABI, 권한, 장치 환경은 각각 맞아야 합니다.

확장 순서는 요구하는 관측 결과로 정하는 것이 좋습니다. “초기값 0의 전역변수를 쓰는 앱”이 목표면 .bss 부터, “앱이 멈춰도 셸이 살아 있기”가 목표면 별도 task와 복구부터 다루는 방식입니다. ELF loader는 배치 정보가 많아지는 문제를 해결하지만 CPU 보호나 scheduler 문제를 대신 해결하지 않습니다.

각 단계마다 기존 API를 유지할지 version을 바꿀지도 정해야 합니다. APP2처럼 magic/version을 달리하면 loader가 지원하지 않는 형식을 명확히 거절할 수 있습니다. 한 field의 의미를 조용히 바꾸면 오래된 앱이 형식 검사를 통과하고도 잘못 실행될 수 있습니다.

18.5 파일 시스템과 프로토콜의 후속 과제

MiniFS는 연속 할당과 작은 고정 table을 사용합니다. crash consistency, 권한, directory, 파일 descriptor, 동시 접근 잠금은 제공하지 않습니다. 현재 syscall handler 동안 timer가 마스킹되어 파일 작업이 하나씩 진행되지만, 나중에 선점 가능한 kernel이나 멀티코어를 도입하면 명시적인 동기화가 필요합니다.

업로드 프로토콜에는 취소, board-side timeout, packet sequence, 재전송, 강한 무결성 검사, 완전한 flow control이 없습니다. 큰 앱을 안정적으로 전송하려면 block 단위 ACK나 credit, checksum/CRC, 실패 시 임시 파일 처리 정책 등을 정해야 합니다.

SD 카드로 바꾸면 CPU의 load/store 한 번으로 disk byte가 즉시 제공되는 현재 전제가 사라집니다. block read/write 완료를 기다리는 driver와 buffer, 실패·재시도 정책이 필요합니다. 그 위에 같은 file table 개념을 유지할 수는 있지만 persistence가 생기면 중간 write와 전원 차단 of 일관성도 새로운 요구가 됩니다.

프로토콜의 ACK를 추가할 때는 “serial driver가 받았다”, “보드 FIFO가 받았다”, “파일에 commit했다” 중 어느 완료를 의미하는지 정의해야 합니다. 재전송으로 같은 block이 두 번 도착해도 결과가 같게 만드는 sequence 규약도 중요합니다. 현재 단순 byte stream은 이런 성질을 학습하기 위한 출발점입니다.

19. 수업 구성과 연습문제

19.1 4회 실습 수업 구성 예

회차	읽을 내용	실습	제출물
1	2 - 6장	top/parameter 연결, memory map, UART Enter 관찰	module 계층도와 byte 수신 경로
2	7 - 9장	syscall·trap frame·MiniFS layout 추적	ECALL 한 번의 PC·SP 변화표
3	10 - 15장	예제 ELF 분석, header 계산, 앱 실행	disassembly 주석과 파일/메모리 배치
4	16 - 18장	정상·오류 실험, 확장 설계	검증 기록과 설계 제안

보고서에는 실행한 명령, 관측한 주소·바이트, 관측값에서 도출한 결론을 연결하도록 합니다. UART 출력만 보고 메모리 보호까지 검증했다는 식의 범위 확대를 피합니다.

1회차에는 instruction과 data가 같은 RAM을 읽는 경로를 직접 그리게 하고, 2회차에는 ECALL 한 번에서 SP와 mepc가 언제 바뀌는지를 표로 제출하게 합니다. 3회차에는 앱의 첫 word·문자열·header의 서로 다른 위치를 실제 hex dump와 연결하고, 4회차에는 정상 결과 하나와 의도적으로 만든 오류 결과 하나를 비교합니다. 이렇게 하면 code를 읽는 능력과 실행 증거를 해석하는 능력을 함께 평가할 수 있습니다.

평가 예시는 구조·계약 설명 30%, 주소·명령어 계산 25%, 재현 가능한 실험 기록 30%, 한계·확장 분석 15%입니다. 장치 출력만 복사한 보고서보다 “어떤 조건을 바꾸었고 왜 이 결과를 기대했는가”를 설명하는 보고서에 높은 비중을 둡니다. 아래 문제는 정답 암기보다 실제 source와 연결하여 답하도록 구성했습니다.

19.2 기본 이해 문제

1. C의 `-DSHELL_MODE` 만 켜고 top의 memory size는 일반 OS 구성으로 두면 어떤 문제가 생길 수 있는가?
2. `SHELL_BUILD=0` 을 설정해도 셸 bitstream이 선택되는 이유는 무엇인가?
3. `hello_app.bin` 이 60바이트인데 `ls` 에는 72바이트로 보이는 이유를 설명하라. 실제 disk 할당 크기는 얼마인가?
4. `0x4001` 의 바이트를 저장할 때 unified RAM의 word 인덱스와 byte lane은 무엇인가?
5. 일반 함수 호출의 `ra` 와 trap의 `mepc` 를 구분하라.
6. `ecall` handler에서 저장된 `mepc` 에 4를 더하지 않으면 무엇이 반복되는가?
7. Enter를 누르면 비어 보이던 screen에 `rv>` 가 나타나는 이유를 RX와 TX 경로로 설명하라.
8. RAM disk의 파일 시작 주소를 그대로 함수 포인터로 호출하면 안 되는 이유는 무엇인가?

1 - 8번은 각 문장에 source 근거를 하나 이상 연결해 답하십시오. 예를 들어 1번에서는 단순히 “메모리가 부족하다”라고 쓰기보다 linker의 stack top과 RTL RAM 끝 주소를 비교해야 합니다. 5번은 두 register 이름을 나열하는 데서 끝내지 말고 누가 언제 쓰고 어떤 명령이 복귀에 사용하는지 적습니다.

오답에서 흔히 나타나는 혼동은 host 파일과 MiniFS 파일, 파일 offset과 CPU 주소, 앱 return과 MRET, screen detach와 종료입니다. 답안을 작성할 때 각 값의 소유 주체와 단위를 붙이면 많은 오류를 스스로 찾을 수 있습니다.

19.3 계산·코드 해석 문제

9. `STACK_WORDS=256` 이고 `trap frame`이 128바이트라면, `trap frame` 하나만으로 `stack` 전체의 몇 퍼센트를 사용하는가? 왜 남은 부분을 모두 앱 지역변수로 사용할 수 없는가?
10. `payload`가 `01 02 FE FF` 네 바이트인 실험용 파일의 APP1 header를 little-endian 바이트로 작성하라. 이 `payload`를 실제 실행할 필요는 없다.
11. `payload`가 1200바이트라면 APP1 파일 크기와 MiniFS block 수는 얼마인가?
12. 파일 table의 32 entry가 사용하는 공간과 두 block 예약 공간의 차이를 계산하라.
13. `hello_app.bin`의 첫 네 바이트 `B7 47 00 00`이 `.hex`에서는 왜 `000047b7`이 되는가?
14. timer 주기가 125000클록, CPU가 12.5 MHz일 때 tick 간격을 구하라. 하위 32비트 wrap 시간도 계산하라.
15. `.text`의 문자열을 `objdump`가 `c.flw`로 표시한다. 이 사실만으로 core가 F/C extension을 지원한다고 결론 내릴 수 있는가?

계산 문제는 최종 숫자뿐 아니라 식과 단위를 제시합니다. 11번에서는 `payload` 크기, header를 포함한 file size, 올림한 block 수, 할당 byte 수를 순서대로 써야 합니다. 13번에서는 byte 네 개를 word 하나로 조합하는 과정을 보여 주십시오. 14번에서는 Hz가 초당 cycle 수라는 점을 이용합니다.

문제 10의 임의 `payload`는 header 형식을 익히기 위한 데이터이며 실행 가능한 프로그램이라고 가정하지 않습니다. 형식상 최소 크기를 만족하는 것과 유효한 instruction으로 정상 종료하는 것은 다릅니다. 이 차이를 명시하는 것도 답안의 일부입니다.

19.4 실험 문제

16. 앱 문자열을 바꾸고 FPGA 재합성 없이 앱만 다시 업로드하라. 어떤 ABI·주소 계약이 그대로이기 때문에 가능한지 설명하라.
17. `--run` 없이 업로드한 뒤 screen에서 Enter, `ls`, `run`을 수행하라. host report와 board 출력을 구분하여 기록하라.
18. `write note.txt text`로 일반 텍스트 파일을 만들고 `run note.txt`를 입력하라. 어떤 검증에서 거절되는지 코드로 설명하라.
19. 앱을 업로드한 뒤 SW20 CPU reset을 수행하고 `ls`를 관찰하라. 이어 PL을 재다운로드하고 다시 비교하라. 두 번째 단계는 RAM 파일을 지우므로 실습 파일만 둔 상태에서 수행하라.
20. testbench에 PC와 trap 관측을 추가하여 `0x4014 → trap_entry → 0x4018` 경로를 찾고 timer trap과 구분하라.

실험 기록 양식에는 변경한 변수, 초기 상태, 실행 명령, 기대 결과, 실제 결과, 불일치 해석을 넣습니다. 19번의 reset 비교에서는 각 단계 전에 같은 파일이 존재하는지 확인하고, CPU reset과 PL configuration을 정확히 기록해야 합니다. 테스트 target이 내부적으로 재구성하는 경우를 놓치면 실험 조건이 달라집니다.

20번에서는 VCD의 단일 PC 값만 캡처하지 말고 ECALL 주소, `trap_entry`, 복귀 주소를 연속 시간으로 표시합니다. 같은 구간에서 `mcause`가 11인지 `0x80000007`인지 확인하면 `syscall`과 timer를 구분할 수 있습니다. 파형의 시간 축과 clock edge를 보고서를 읽는 사람이 확인할 수 있게 남기십시오.

19.5 설계 문제

21. APP1의 다음 버전에서 `.bss`를 지원하려면 어떤 header field와 검증 조건이 필요한가?

22. 앱 전용 stack을 도입하려고 SP만 바꾸면 어떤 문제가 생기는가? 셀의 복귀 SP와 ra를 어떻게 보존할지 설계하라.
23. UART RX FIFO overflow를 host가 감지하고 복구할 수 있도록 protocol을 설계하라.
24. 무한 반복 앱을 timer 기반으로 종료시키고 셀로 돌아오려면 scheduler와 task lifecycle을 어떻게 확장해야 하는가?
25. instruction cache를 추가한 후 간헐적으로 이전 앱이 실행된다면 loader와 CPU의 어떤 계약을 조사해야 하는가?

설계 답안은 새 자료구조만 제시하지 말고 기존 성공 경로와 실패 경로가 어떻게 바뀌는지 설명해야 합니다. 예를 들어 APP2에 bss_size를 추가하면 field 길이, 정수 overflow 검사, 실행 창 경계, zeroing 시점, 오래된 APP1 처리 정책까지 필요합니다. 추가하는 정보가 loader의 어떤 결정을 가능하게 하는지 연결하십시오.

여러 확장을 한꺼번에 구현하는 것보다 하나의 계약을 바꾸고 그 계약을 검증하는 작은 실험을 제안하는 것이 좋습니다. cache를 넣으면서 동시에 loader·scheduler·UART protocol을 전부 바꾸면 문제가 생겼을 때 원인을 분리하기 어렵습니다. 실험 가능성과 검증 조건도 설계 품질의 일부입니다.

19.6 ISA와 데이터패스 심화 문제

26. `0x000780E7` 에서 opcode, rd, funct3, rs1, immediate를 추출하고 PC가 `0x0A18` 일 때 다음 PC와 ra를 구하라. x15는 `0x4000` 이다.
27. RAM word `0x44332211` 의 offset 2에 `0xAA` 를 SB로 기록한다. write data, strobe, 결과 word를 구하라.
28. `rs1=0xFFFFFFFF` , `rs2=1` 일 때 SLT와 SLTU의 결과를 비교하고, branch의 BLT/BLTU와 연결하라.
29. immediate `0xFF0` 을 32비트로 확장하라. `addi sp,sp,imm` 에서 초기 SP가 `0x1C40` 이면 결과는 무엇인가?
30. PC `0x401C` 에서 `0x400C` 로 BNE한다. B immediate의 값, bit 12·11·10:5·4:1을 구하고 instruction field 배치를 설명하라.
31. `csrr t0,mcause` 와 `csw mie,t0` 의 실제 CSR 명령을 쓰고, rd가 x0인 경우 무엇이 버려지는지 설명하라.
32. UART TX store와 timer trap이 같은 사이클에 선택되었다. 왜 PC만 mtvec로 바꾸면 충분하지 않은가?
33. 46개 명령의 합계를 계열별로 다시 계산하라. `ret` , `li` , `csrr` 와 `syscall` 번호 10이 이 수에 어떻게 관련되는지 설명하라.
34. `SRA(0x80000000,1)` 과 `SRL(0x80000000,1)` 의 결과를 구하라. register shift amount 32는 어떻게 처리되는가?
35. 현재 9개 static instruction으로 이루어진 앱이 23문자를 출력할 때 앱 경로의 instruction 시도를 계산하고, 이 값으로 실행 시간을 바로 계산할 수 없는 이유를 설명하라.

26 - 31번은 bit 계산을, 32번은 edge에서의 부작용을, 33 - 35번은 ISA·ABI·동적 실행 비용의 구분을 평가합니다. 숫자를 compiler 출력과 대조할 수 있지만 먼저 손으로 계산한 식을 제시하십시오. 코드의 명령어 배치가 바뀐 경우 문제에서 주어진 주소를 기준으로 답하고 자신의 빌드 결과와 차이를 따로 적습니다.

19.7 CSR 이름과 trap 상태 심화 문제

36. CSR, MIE, MPIE, MTIE, MTIP의 영문 이름을 쓰고 각각 register 전체인지 bit인지 구분하라. `mie` 와 `mstatus.MIE` 의 주소·bit 위치를 설명하라.
37. `mstatus=0x88`, `mie=0x80`, `timer_irq=1`이다. PC=`0x400C`에서 timer trap이 선택되면 `mstatus·mepc·mcause·mip`는 어떤 값인가? handler가 timer를 미래로 재설정하고 같은 frame으로 MRET했을 때 값을 다시 구하라.

38. mstatus=0에서 ECALL을 실행했다. handler가 frame.mepc에 4를 더한 뒤 MRET하면 MIE·MPIE는 무엇인가? MIE=0인데 왜 trap에 진입할 수 있었는가?
39. `csrsi mie,7`, `csrsi mie,0x80`, `li t0,0x80`; `csrs mie,t0` 중 MTIE를 켜는 올바른 방법을 고르고 나머지의 문제를 설명하라.
40. `t0=0x41`일 때 `csrw mtvec,t0`를 수행했다. 현재 core에서 읽히는 값과 timer의 진입 PC는 무엇인가? Vectored를 지원하는 별도 CPU에서 같은 설정을 허용할 때와 비교하라.
41. `mcause=0x80000007`인데 `mip=0`이다. 모순인가? `csrw mcause,zero`와 `csrw mip,zero`로 현재 timer 요청을 해제할 수 있는지도 설명하라.
42. task를 바꿔도 `mhartid=0`인 이유와 `csrr t0,misa`가 `cause 2`를 일으키는 이유를 설명하라. 각각 software task 수, hardware hart 수, CSR 지원 목록과 연결하라.
43. `mscratch=0x1000`, `t0=0x2222`에서 `csrrw t0,mscratch,t0` 후 값을 구하라. 이 한 instruction으로 일반 register 전체나 stack이 자동 보존되는가?
- 36번은 명칭과 계층, 37 - 38번은 진입·복귀 상태, 39 - 40번은 mask·주소 계산, 41 - 43번은 CSR와 OS 자료구조의 역할 구분을 평가합니다. 모든 계산은 현재 M-mode 전용 RTL을 기준으로 하며, 별도 CPU라고 명시한 항목만 표준 비교 예입니다. 각 답안에 software가 쓰는 값과 hardware가 자동 갱신하는 값을 따로 표시하십시오.

19.8 스택과 종료 경로 심화 문제

44. stacks가 0x1840에서 시작하고 한 행이 256개의 uint32_t라면 마지막 원소 주소, 마지막 byte 주소, 초기 sp를 각각 구하라. 초기 sp가 배열 밖이어도 되는 조건은 무엇인가?
45. trap 직전 sp가 0x1C20이라면 frame 시작, mepc 슬롯, ra 슬롯, a0 슬롯의 주소를 구하라. 복원 때 x2 슬롯을 직접 읽지 않고도 원래 sp를 얻는 이유는 무엇인가?
46. `task_create(1, task_shell)`에서 `entry`의 값과 `entry()`의 차이를 설명하라. 최초 mepc와 ra가 각각 어떤 목적을 가지는가?
47. `task_exit`이 실행될 때 일반 모드와 셸 모드의 차이를 설명하라. interrupt가 가능하다는 사실만으로 프롬프트가 복구된다고 결론 낼 수 있는가?
48. panic에서 `timer_irq=1`인데 context switch가 일어나지 않는다. MIE, MTIE, MTIP를 이용해 설명하라. panic 뒤의 `return f`와 MRET는 실행되는가?
49. 함수 stack frame과 trap frame을 구분하고, 1 KiB 중 128바이트를 뺀 나머지를 모두 앱 지역변수로 사용하면 안 되는 이유를 설명하라.

19.9 호출 경계와 스택 사용량 확인 문제

50. `trap_entry`에 들어갈 때 일반 call이 실행되는가? `boot.S`, `mtvec`, RTL의 PC 갱신, mepc의 역할을 순서대로 설명하라.
51. `trap_handler` 호출 직전 a0와 호출 직후 a0는 각각 무엇인가? 왜 x10을 복원하기 전에 `mv sp,a0`가 필요한가?
52. 일반 모드 slot 3과 셸 모드 slot 1은 각각 stacks의 어느 행을 사용하는가? `task_sp`와 stacks의 원소 수가 다른 이유는 무엇인가?
53. 셸 함수 프레임 256바이트, trap frame 128바이트, C handler 프레임 32바이트일 때 `top=0x1C40`에서 sp 변화를 계산하라. 지역 배열 `line[96]`을 다시 더하면 왜 틀리는가?

54. `uart_puts`의 저장 `ra`, `trap frame`의 저장 `ra`, `trap_handler` 함수 프레임의 저장 `ra`가 각각 어떤 복귀를 위한 것인지 설명하라. C handler가 하드웨어 장치의 이름인지도 답하라.

20. 해설·용어·코드 찾아보기

20.1 기본 문제 해설

1번은 하드웨어와 소프트웨어 주소 계약 불일치입니다. 셸 linker의 `_stack_top=0x8000`, 앱 주소 `0x4000` 등은 8 KiB 구성의 실제 RAM 범위 밖입니다. 이미지 크기만 우연히 들어가더라도 스택과 실행 영역이 맞지 않습니다.

2번은 Tcl이 값이 아니라 변수 존재를 검사하기 때문입니다. C의 `#ifdef`도 정의 여부 검사이고 Verilog 조건식은 값 검사라는 차이가 있습니다.

3번은 `payload 60 + header 12 =` 논리 파일 72바이트이며 할당은 한 block 512바이트입니다. 4번의 word index는 4096, byte lane은 1이고 strobe는 `0010`입니다.

5번에서 `ra`는 JAL/JALR가 기록하는 일반 함수의 복귀 주소, `mepc`는 trap 시 CSR에 보존되는 PC입니다. 6번에서는 같은 ECALL을 다시 실행합니다. 7번에서는 CR을 RX가 수신하고 셸이 빈 command line을 처리한 뒤 TX로 새 prompt를 보내기 때문입니다.

8번에서 RAM disk는 현재 instruction port의 저장소가 아니고 file header도 붙어 있으며, 앱 내부 주소는 `0x4000` 기준으로 링크되어 있습니다. 세 이유를 모두 검토해야 합니다.

1번의 구체적인 실패 후보에는 첫 C 함수의 stack 접근, 앱 창의 store/fetch, 커널 image 범위 초과가 있습니다. 어느 것이 먼저 나타나는지는 image와 instruction 흐름에 따라 달라질 수 있으므로 원인을 하나의 UART 메시지로 단정하지 않습니다. 계약 불일치 자체가 핵심입니다.

5 - 6번은 세 저장 위치를 그려 답하면 명확합니다. 현재 PC는 실행 중 명령, CSR `mepc`는 trap 복귀용 hardware 상태, `frame.mepc`는 software가 보관·수정하는 값입니다. `ra`는 이와 독립된 일반 register입니다. `frame.mepc += 4`가 즉시 현재 PC를 바꾸는 것이 아니라 restore와 MRET를 거쳐 효력을 갖는다는 설명까지 포함하면 완전한 답입니다.

20.2 계산 문제 해설

9번은 $128/1024=12.5\%$ 입니다. 남은 공간에도 이미 존재하는 셸 함수 frame, 앱 호출 frame, handler와 하위 함수 frame이 함께 들어갑니다.

10번의 합은 $1+2+254+255=512=0x200$ 입니다. header는 다음과 같습니다.

```
41 50 50 31 04 00 00 00 00 02 00 00
```

11번은 파일 크기 1212바이트, 필요한 block 수는 올림한 3개이며 1536바이트가 할당됩니다. 12번은 실제 table 768바이트, 예약 1024바이트, 차이 256바이트입니다.

13번은 raw byte들을 little-endian 32비트 word로 해석한 결과입니다. 14번은 tick 10 ms, wrap 약 343.6초입니다. 15번은 데이터 바이트의 오해석일 수 있으므로 실제 PC 경로·section 배치와 함께 판정해야 합니다.

11번을 식으로 전개하면 $Nfile=1200+12=1212$, $blocks=(1212+511)>>9=3$, $Nalloc=3*512=1536$ 입니다. 마지막 block의 미사용 부분은 $1536-1212=324$ 바이트입니다. 이 여유가 생기는 현상을 allocation granularity에 따른 내부 단편화로 설명할 수 있습니다.

14번의 계산은 $125000 / 12500000 = 0.01 \text{ s}$, $4294967296 / 12500000 = 343.59738368 \text{ s}$ 입니다. compare 갱신 직전의 낮은 word wrap 때문에 문제가 정확히 한 순간에만 시작된다고 단정하기보다는 경계 부근의 지속적인 IRQ 가능성을 분석해야 합니다. timer의 전체 비교 폭과 MMIO 노출 폭이 다르다는 구조가 원인입니다.

20.3 설계 문제의 검토 기준

.bss 확장에서는 최소한 초기화 영역의 시작과 길이, 파일에 저장된 영역과 메모리에서만 필요한 영역의 구분, 정수 overflow와 앱 창 경계 검사가 필요합니다. 앱 stack 확장에서는 새 stack의 정렬뿐 아니라 기존 SP, 복귀 주소, callee-saved register, trap 발생 시 소유 문맥을 고려해야 합니다.

UART 확장에서는 sequence와 길이, ACK/재전송, timeout, 수신 buffer 용량을 연결해야 합니다. 앱 강제 종료는 현재 실행 상태를 버린 뒤 복원할 셀 문맥과 자원 소유 관계가 필요합니다. cache 문제에서는 copy 완료와 instruction fetch 동기화, cache invalidate 또는 일관성 처리, FENCE.I 지원을 조사해야 합니다.

21번에서 bss_start와 bss_size를 모두 파일에 저장할 수도 있고, payload 뒤에 연속 배치한다는 규칙으로 일부 field를 생략할 수도 있습니다. 어느 설계를 택하든 더하기의 overflow, 앱 창 초과, 파일에 없는 memory 요구량을 검증해야 합니다. loadable 영역과 zero-fill 영역이 stack을 침범하지 않도록 검사하는 것이 핵심입니다.

22번에서는 셀의 원래 SP를 어디에 보존하고 누가 복원하는지 명확해야 합니다. 새 stack에서 발생한 trap을 기존 task_sp가 어떻게 관리할지도 정합니다. 24번에서는 실행 취소 후 선택할 정상 셀 frame이 필요하며, 앱이 공유 kernel memory를 손상시킬 수 있는 현재 권한에서는 완전한 복구를 보장하기 어렵다는 한계도 언급해야 합니다.

16 - 20번의 실험 해설은 결과를 외우는 대신 조건을 비교하는 데 있습니다. 앱 문자열 수정은 ABI를 유지하면 앱 재업로드로 충분하고, 일반 text 파일 run은 header 검사에서 거절됩니다. 정상 FS에서 CPU reset은 파일을 보존하고 PL 재구성은 초기화합니다. ECALL 파형은 cause 11과 PC+4 복귀를, timer 파형은 cause의 최상위 bit와 같은 PC 재개를 근거로 구분합니다.

20.4 용어 정리

용어	이 문서에서의 의미
top module	합성 대상 회로의 최상위 구조
parameter	Verilog 회로 elaboration 시 정하는 설정
firmware	CPU가 부팅 후 실행하는 커널·셀 코드
bitstream	FPGA의 회로·메모리 초기 구성을 담는 데이터
ABI	바이너리 수준의 호출·레지스터·배치 약속
loader	파일을 검사하고 실행 메모리에 배치하는 코드
payload	APP1 header를 제외한 앱 이미지 바이트
MMIO	load/store 주소를 주변장치 register에 대응시키는 방식
CSR	Control and Status Register: 명령으로 접근하는 제어·상태 register
hart	Hardware Thread: 독립적으로 instruction을 실행하는 hardware 단위
MIE / MPIO	Machine Interrupt Enable / Machine Previous Interrupt Enable
MTIE / MTIP	Machine Timer Interrupt Enable / Machine Timer Interrupt Pending

용어	이 문서에서의 의미
pending / enable / cause	현재 요청 / 처리 허용 / 마지막 trap의 원인 기록
syscall	앱·task가 OS 서비스를 요청하는 약속된 호출 경로
trap	exception·interrupt에 의해 handler로 제어가 이동하는 사건
context	재개를 위해 보존하는 PC·register·stack 관련 상태
task	scheduler가 frame과 stack을 관리하는 실행 단위
relocation	최종 배치 주소에 맞춰 주소 참조를 수정하는 처리
.bss	실행 전 0 초기화가 필요한 정적 저장 영역
warm reset	FPGA를 재구성하지 않고 CPU 상태를 다시 시작하는 reset

용어를 함께 묶어 보면 경계가 더 잘 보입니다. ISA는 CPU 명령의 의미, microarchitecture는 그 명령을 구현하는 내부 회로 구조, ABI는 compiler와 runtime이 공유하는 호출 약속입니다. loader는 그 ABI에 맞는 image를 메모리에 놓고 entry로 제어를 넘기는 software입니다. 한 층의 변경이 다른 층의 계약을 깨뜨리는지 항상 확인합니다.

또 다른 묶음은 주소, offset, index입니다. 주소는 CPU address space 안의 위치이고, file offset은 파일 첫 byte에서의 거리이며, array index는 자료형 원소 수를 기준으로 한 위치입니다. APP_BASE+off, 12+off, mem[4096]은 관련되지만 단위와 기준점이 다릅니다. 디버깅 메모에 기준점을 함께 적는 습관이 도움이 됩니다.

20.5 코드 찾아보기

파일	중심 함수·모듈	읽을 때 확인할 질문
zcu104_top.v	zcu104_top	parameter가 RAM과 firmware에 어떻게 연결되는가?
build_zcu104.tcl	synth_design	top과 generic은 누가 지정하는가?
rv32_soc.v	mem, ramdisk, RX FIFO	파일 byte는 어떤 주소 디코더를 통과하는가?
rv32_core.v	LOAD/STORE/JALR/SYSTEM	byte 복사와 함수 호출이 어떤 명령으로 수행되는가?
rv32_csr.v	trap entry와 MRET	PC와 interrupt enable은 어떻게 보존되는가?
uart_rx.v	IDLE/START/BITS/STOP	한 byte의 완료는 언제 판단하는가?
uart_tx.v	ready/valid와 shift	문자 출력이 핀 waveform으로 어떻게 바뀌는가?
boot.S	_start	C 실행 전에 무엇을 준비하는가?
trap.S	trap_entry	frame 저장·선택·복원 순서는 무엇인가?
kernel.c	task_shell, shell_upload, shell_run, trap_handler	사용자 입력에서 앱 실행까지 어떻게 이어지는가?
minifs.c	fs_init, fs_write, fs_read_at	file offset은 disk 주소로 어떻게 바뀌는가?
linker_shell.ld	kernel memory layout	stack과 앱 창은 왜 겹치지 않아야 하는가?
app.ld	APP origin과 entry section	raw image의 첫 명령을 어떻게 보장하는가?
hello_app.c	app_main, putc_sys	앱이 커널 함수 주소 없이 출력하는 방법은 무엇인가?

파일	중심 함수·모듈	읽을 때 확인할 질문
upload_app.py	host protocol	파일·checksum·UART 응답을 어떻게 다루는가?
tb_shell.v	serial input, PC/메모리 검사	실행을 문자열 이외의 어떤 증거로 확인하는가?

읽기 순서는 처음에는 top → SoC → core, 다음에는 boot → trap → kernel, 마지막에 app linker → host uploader → loader로 잡을 수 있습니다. 각 경계에서 입력과 출력 signal 또는 인수·반환값을 표시하면 전체 code를 한 번에 외우지 않아도 경로를 복원할 수 있습니다.

line 번호는 code 변경 때 움직일 수 있으므로 파일 이름과 module/function 이름을 함께 사용합니다. `rg -n 'shell_run|trap_handler|task_shell' firmware/kernel.c`처럼 symbol로 찾으면 현재 코드 위치를 빠르게 확인할 수 있습니다. compiler가 inline한 함수는 ELF symbol에 없을 수 있어 source와 disassembly를 같이 보아야 합니다.

20.6 참고 자료와 문서의 범위

프로젝트의 구현 상세는 위 소스와 [MiniFS 안내](#), [앱 로더 안내](#), [ZCU104 안내](#)를 기준으로 합니다. 기존 [전체 설계 매뉴얼](#)은 기본 RV32I datapath와 초기 OS를 더 넓게 설명합니다.

표준 ISA의 명령 의미는 [RISC-V RV32I 명세](#), trap/CSR의 기준은 [Machine-Level ISA](#), 명령어 동기화는 [Zifencei](#)를 참고합니다. register 호출 규약은 [RISC-V psABI](#), ELF entry와 binary 변환은 [GNU ld ENTRY](#)와 [GNU objcopy](#)를 참고했습니다. 열람일은 2026-09-30입니다.

이 core는 교육용 최소 구현으로, 표준 privileged architecture의 모든 예외·권한·CSR 동작을 구현한 것은 아닙니다. 문서에서 “현재 구현”이라고 명시한 동작은 해당 RTL·펌웨어의 동작이고, “확장”은 아직 추가해야 할 설계입니다. 실험 결과의 주소·크기는 해당 산출물의 관측값과 고정 ABI 계약을 구분해서 사용해야 합니다.

CSR의 read-modify-write와 immediate 형태는 [RISC-V Zicsr 명세](#)를 함께 참고하십시오. 표준에서 정의된 기능과 현재의 간략화된 구현을 비교하는 것이 목적이며, 지원한다고 적힌 mnemonic 개수만으로 conformance를 주장하지 않습니다. 하드웨어에서 어떤 예외·권한·주소 검사를 생략했는지까지 읽어야 software의 적용 범위를 판단할 수 있습니다.

CSR 심화 절은 2026-10-01에 [공식 CSR 주소·명칭 목록](#), [Machine-Level ISA](#), [Zicsr](#)를 대조했습니다. 현재 mask, reset 값, trap 우선순위와 값 변화는 `rv32_csr.v·rv32_core.v·simple_timer.v·boot.S·trap.S·kernel.c`를 근거로 설명합니다. 표준 전체를 이 프로젝트가 지원한다는 의미는 아닙니다.

10.3·10.4절은 2026-10-01의 로컬 GCC 9.3.0·Binutils 2.34로 예제 앱을 다시 빌드한 결과를 사용합니다. 단계별 빌드와 Make 빌드의 BIN 일치, ELF·raw 파일의 ECALL 위치, 최적화 수준별 payload 크기, debug 정보와 raw payload의 구분, 나눗셈 helper 참조와 `.bss` 거부를 확인했습니다. 도구의 일반 옵션 의미는 해당 절의 GNU GCC·Binutils·xxd 원문 링크를 참고하고, 다른 버전에서는 실제 출력과 옵션 지원 여부를 다시 확인해야 합니다. 이 검사는 host toolchain 검증이며 새로운 보드 실행 시험을 뜻하지 않습니다.

이 확장판은 firmware나 RTL의 기능을 추가하지 않고 교재 설명을 확장한 것입니다. 새로운 확장 설계·실험 제안은 현재 구현·검증 결과와 구별되어 있습니다. PDF를 재생성할 때는 `make shell-guide-pdf`를 사용하고, 수정 후 페이지 수와 표·code block의 잘림을 확인하여 인쇄 교재의 형태를 유지합니다.

20.7 ISA 심화 문제 해설

26번에서 opcode는 0x67, rd는 1, funct3는 0, rs1은 15, immediate는 0입니다. 다음 PC는 0x4000, ra는 0x0A1C 입니다. rd에 쓰는 주소는 jump target이 아니라 caller의 다음 instruction 주소입니다.

27번의 write data는 0xAAAAAAAA, strobe는 0100, 결과 word는 0x44AA2211 입니다. 28번에서는 signed -1이 1보다 작아 SLT=1, unsigned 최대값은 1보다 커 SLTU=0입니다. 같은 입력이면 BLT는 taken, BLTU는 not taken입니다.

29번의 immediate는 -16, 32비트 표현은 0xFFFFF0 이고 SP는 0x1C30 이 됩니다. 30번의 offset은 -16이며 13비트 표현은 0x1FF0 입니다. bit 12=1, bit 11=1, bits 10:5= 111111, bits 4:1= 1000, bit 0=0입니다. 이 조각을 B형의 [31], [7], [30:25], [11:8] 에 각각 배치합니다.

31번은 CSRRS t0, mcause, x0 와 CSRRW x0, mie, t0 입니다. 후자에서 이전 CSR 값의 일반 register 반환을 버리지만 mie에 새 값을 쓰는 동작은 남습니다. 32번은 store가 이미 장치에 반영되면 같은 PC 재개 때 중복 출력할 수 있기 때문입니다. trap 선택 edge에 memory·register·CSR write와 RX pop 같은 부작용을 함께 억제해야 합니다.

33번은 $10+9+5+3+6+2+2+1+1+6+1=46$ 입니다. ret/li/csrr은 기존 명령으로 표현되는 assembler 문법이고 서비스 번호 10은 ECALL로 전달하는 OS 데이터입니다. 34번의 SRA 결과는 0xC0000000, SRL 결과는 0x40000000 입니다. register shift amount 32는 하위 5bit가 0이므로 shift하지 않습니다.

35번은 정상 앱 흐름에서 $3+23*5+1=119$ 개의 instruction 시도입니다. 이 수에는 handler의 수많은 저장·복원·polling 명령이 없고, timer에 의한 재시도도 없으며 ECALL은 retire되지 않습니다. 따라서 119를 그대로 clock 수나 retirement 수로 사용하면 틀립니다. dynamic trace를 기준으로 각 종류의 비용을 구분해야 합니다.

20.8 CSR 심화 문제 해설

36번의 CSR는 Control and Status Register입니다. MIE는 Machine Interrupt Enable, MPIE는 Machine Previous Interrupt Enable, MTIE는 Machine Timer Interrupt Enable, MTIP는 Machine Timer Interrupt Pending입니다. mie는 CSR 번호 0x304의 register 전체이고 MTIE는 그 bit 7입니다. mstatus.MIE는 CSR 번호 0x300의 bit 3이며 MPIE는 그 bit 7입니다. MTIP는 CSR 번호 0x344인 mip의 bit 7입니다.

37번의 trap 직후 mstatus=0x80, mepc=0x400C, mcause=0x80000007입니다. 원인을 아직 해제하지 않았으므로 mip=0x80이고, MIE가 0이 되어 irq_pending은 0입니다. timer를 충분한 미래로 예약하여 요청이 해제된 상태로 MRET하면 mstatus=0x88, PC=0x400C, mip=0입니다. 새 trap이나 별도 CSR write가 없으면 mepc와 mcause는 이전 값을 유지합니다.

38번은 trap 때 MPIE에 이전 MIE=0이 저장되고, MRET 뒤 MIE=0·MPIE=1, 즉 mstatus=0x80입니다. ECALL은 timer interrupt 허용 식과 독립된 동기 예외입니다. 따라서 MIE=0이어도 정상적으로 trap에 들어갑니다. 이 결과는 MRET가 무조건 interrupt를 켜는 오해를 반박합니다.

39번은 register mask 0x80을 사용한 csrs mie, t0가 올바릅니다. immediate 7은 bit 번호가 아니라 mask 0b00111이므로 bit 7에 영향을 주지 않습니다. 현재 mie에서 그 하위 비트들은 저장되지 않습니다. immediate 0x80은 5비트 범위를 초과하므로 assembler가 표현할 수 없습니다.

40번은 현재 mtvec 읽기 값이 0x40, timer 진입 PC도 0x40입니다. 현재 RTL이 MODE를 0으로 강제하기 때문입니다. 비교 대상 CPU가 BASE=0x40·MODE=1을 지원하고 허용한다면 cause 7의 interrupt는 0x5C로 갑니다. 두 동작을 같은 core의 관측 결과로 섞으면 안 됩니다.

41번은 모순이 아닙니다. mcause는 마지막 trap의 기록이고 mip는 현재 timer level입니다. compare를 재예약하면 mip는 0이 되지만 mcause는 남을 수 있습니다. mcause에 0을 쓰면 기록만 바뀌고, 현재 mip 쓰기는 무시됩니다. 요청 해제는 timer comparator의 조건을 바꿔야 합니다.

42번에서 모든 software task는 하나의 hardware hart를 공유하므로 mhartid는 0으로 유지됩니다. misa는 현재 CSR decoder에 없으므로 읽으려는 instruction이 illegal이 됩니다. 이름이 표준에 존재한다는 것, assembler가 인코딩할 수 있다는 것, 현재 hardware가 수행할 수 있다는 것은 세 가지 서로 다른 조건입니다.

43번의 결과는 t0=0x1000, mscratch=0x2222입니다. 32비트 값 하나씩 교환했을 뿐 다른 일반 register나 RAM stack은 바뀌지 않습니다. 전체 context 저장은 trap.S의 여러 SW와 software의 frame 관리가 담당합니다. scratch register 하나를 갖추었다고 stack 전환과 중첩 예외 처리가 자동 완성되지 않습니다.

20.9 스택과 종료 경로 심화 문제 해설

44번은 마지막 원소 시작 0x1C3C, 마지막 byte 0x1C3F, 초기 sp 0x1C40입니다. 끝 다음 포인터를 만드는 것은 가능하지만 그 위치를 역참조하면 안 됩니다. 함수나 trap 진입이 먼저 sp를 낮춰 배열 안의 유효 주소에 기록해야 합니다. 정렬과 배열 경계 보호는 별개의 문제입니다.

45번의 frame 시작과 mepc 슬롯은 0x1BA0, ra 슬롯은 0x1BA4, a0 슬롯은 0x1BC8입니다. x10의 offset은 40바이트입니다. trap 진입 때 정확히 128을 뺐으므로 선택한 frame 주소에 128을 더하면 그 태스크의 이전 sp가 됩니다. 다른 태스크를 선택하면 그 태스크의 frame을 기준으로 계산합니다.

46번의 entry는 task_shell의 함수 포인터이고 entry()는 그 함수를 호출하는 표현식입니다. task_create는 호출하지 않고 주소만 mepc 슬롯에 기록합니다. 초기 mepc는 시작 위치이고 초기 ra는 태스크 entry가 반환할 때 이동할 task_exit입니다. 주소를 uint32_t로 변환하는 표현식 자체는 함수 호출도 CSR 쓰기도 아닙니다.

47번은 일반 모드에서 timer가 허용되면 다른 태스크가 실행되지만 종료한 태스크도 여전히 선택됩니다. 셸 모드에서는 scheduler가 셸을 계속 선택하므로 task_exit 루프로 복귀합니다. interrupt 발생과 유용한 작업으로의 전환은 다른 조건입니다. 어느 모드에도 현재 EXITED 상태에 따른 제거 처리는 없습니다.

48번은 trap 진입으로 MIE=0이 되었기 때문입니다. MTIE=1, MTIP=1이어도 전역 허용이 꺼져 timer trap을 수락하지 않습니다. panic이 반환하지 않으므로 return f, frame 복원, MRET도 실행되지 않습니다. CPU는 loop 명령을 실행하며 timer 주변장치 자체는 계속 동작할 수 있습니다.

49번에서 함수 frame은 compiler가 필요한 데이터와 레지스터 보존을 위해 만들며 크기가 가변입니다. trap frame은 현재 trap.S가 mepc와 일반 레지스터 상태를 담는 고정 128바이트 기록입니다. 셸과 앱의 살아 있는 함수 frame, C trap_handler와 하위 함수의 사용량까지 같은 스택에 더해지므로 남은 896바이트를 모두 앱 지역변수에 줄 수 없습니다.

20.10 호출 경계와 스택 사용량 확인 문제 해설

50번은 일반 call이 아닙니다. boot.S가 trap_entry 주소를 mtvec에 기록하고 CPU가 trap을 선택하면 PC를 mtvec로 변경합니다. 중단 주소는 mepc에 기록합니다. 그 뒤 어셈블리가 레지스터를 저장하고 C trap_handler를 호출할 때 일반 call이 등장합니다.

51번에서 호출 전 a0는 현재 frame 주소인 첫 인수이고 호출 후에는 선택된 frame 주소인 반환값입니다. 반환값을 먼저 sp에 옮기면 그 뒤 a0를 태스크의 저장값으로 복원해도 frame의 위치를 잃지 않습니다. 순서를 뒤집으면 태스크의 데이터 값을 frame 주소로 오해할 수 있습니다.

52번은 각각 `stacks[2]`, `stacks[0]`입니다. `idle`은 부팅 스택을 쓰지만 `frame` 주소를 기억할 `task_sp[0]`은 필요합니다. 따라서 `task_sp`는 NTASK개, 별도 스택 배열의 행은 NTASK-1개입니다.

53번은 `0x1C40` → `0x1B40` → `0x1AC0` → `0x1AA0`이고 누적 416바이트입니다. `line`은 이미 `task_shell` 함수 프레임에 포함되므로 96을 더하면 중복입니다. 다른 하위 함수가 살아 있으면 추가 공간이 필요하므로 이 값은 최대 사용량의 증명이 아닙니다.

54번의 `uart_puts` 저장 `ra`는 문자열 출력을 요청한 호출자로 돌아가기 위한 값입니다. `trap frame`의 `ra`는 중단된 태스크가 사용하던 값이고, `trap_handler` 함수 프레임의 `ra`는 `trap.S`의 `call` 다음으로 돌아가기 위한 값입니다. `C handler`는 C 언어로 작성한 처리 함수라는 표현이며 현재 `trap_handler`를 뜻합니다. 별도 CPU 장치나 C 예약어가 아닙니다.